CS 380: ARTIFICIAL INTELLIGENCE

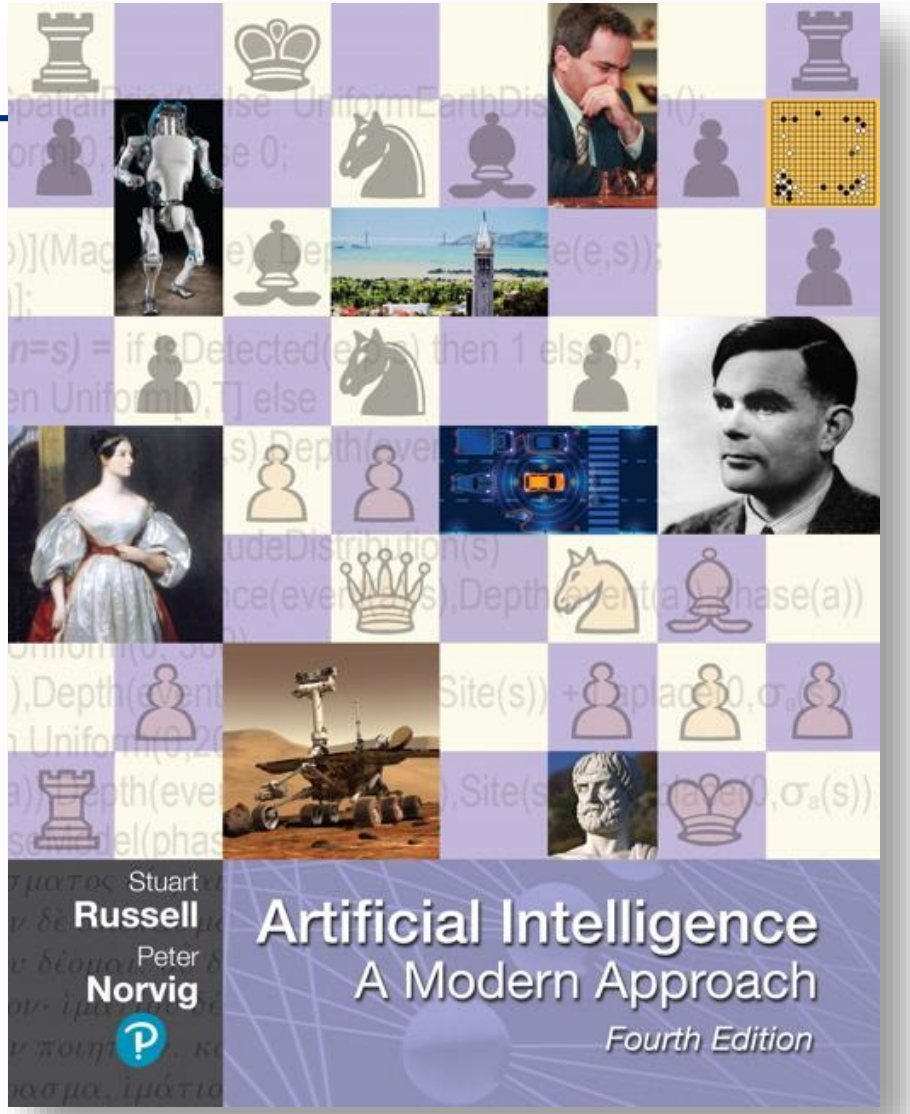
Lecture 1: Introduction

Welcome!

- About me
  - **Jenn Engimann**, Adjunct Professor, Computer Science
  - Email: < jae63@drexel.edu >
  - Office Hours: Thurs 10-11 on Zoom, or by appointment
- Our course TAs
  - **Binh Le** < bql23@drexel.edu >
  - **Don Enwerem** < de435@drexel.edu >
  - **Zaafir ul Hasan** < zuh23@drexel.edu >
  - CLC lab hours posted on Blackboard
- About this course
  - Syllabus, schedule, & resources on Blackboard

Textbook

- Russell & Norvig, 4th edition
  - 3rd is ok too
- A great standard introduction to AI



Classes

- We will have classes twice/week
  - Lecture will be recorded on Echo 360 and posted to Blackboard
- For lectures slides, many thanks to…
  - Dario Salvucci and Santi Ontañón for great material
  - Stuart Russell and Hwee Tou Ng for more great material linked from http://aima.cs.berkeley.edu/
  - The slides here are a conglomeration of all the above plus lots of new material and revisions

Homework Assignments

- 4 programming assignments
  - To be done individually – standard CS rules apply
- For each assignment, you should submit a ZIP file with all requested materials
  - ZIP… not RAR or any other format
  - You may include a README with additional info
  - Failure to follow these guidelines → reduced credit
  - Late submissions lose 33% per day late

Homework Assignments

- Our language of choice: Python 3
  - AI algorithms can be implemented in anything, of course
  - Python is one of the most commonly used languages, and has excellent associated AI/ML libraries
- We will assume that assignments should run on the equivalent of tux.cs.drexel.edu
  - Currently, tux uses Python 3.10.12
- Comment your code when/where needed
  - Not everywhere, just where it's needed
  - We want to understand what you wrote and why, to properly give you credit for what you did

Course Structure: Evaluation

- Homework assignments: 50%
- Midterm exam: 25%
- Final exam: 25%

Blackboard

- All materials will be posted on Blackboard
  - Including schedule, lecture slides, assignments, etc.
- Announcements will generally also be sent out via email

What is Artificial Intelligence?

What is Artificial Intelligence?

Me

Describe AI in Shakespearean English



ChatGPT

What is Artificial Intelligence?

Me

Describe AI in Shakespearean English

ChatGPT

O wondrous craft, born of man's hand and mind,
A creature not of flesh, yet swift and kind,
With circuits bright and logic keen it flies,
A mirror of our thoughts 'neath starry skies.

It learns, it speaks, with wisdom vast and true,
Yet lacks the heart, the soul that we pursue.
No joy, no grief, within its core may dwell,
It thinks, it acts, but cannot love or tell.

In our design, 'tis both a boon and bane,
A tool to ease our work, or bring us pain.
So wield it wise, for power in its might
Can turn to dark or lead to paths of light.

What is Artificial Intelligence?

- Origins (John McCarthy 1956):

We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.

What is Artificial Intelligence?

- Origins (John McCarthy 1956):

We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.

What is Artificial Intelligence?

- Origins (John McCarthy 1956):

We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.

What is Artificial Intelligence?

- Origins (John McCarthy 1956):

We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together **for a summer**.

Related Fields

▪ Philosophy	Logic, methods of reasoning, mind as physical system foundations of learning, language, rationality
▪ Mathematics & CS Theory	Formal representation and proof algorithms, computation, (un)decidability, (in)tractability, probability
▪ Economics	utility, decision theory
▪ Neuroscience	physical substrate for mental activity
▪ Psychology	phenomena of perception and motor control, experimental techniques, cognitive models
▪ Computer engineering	building fast computers, software vs. hardware
▪ Control theory	design systems that maximize an objective function over time
▪ Linguistics	knowledge representation, grammar

What is Artificial Intelligence?

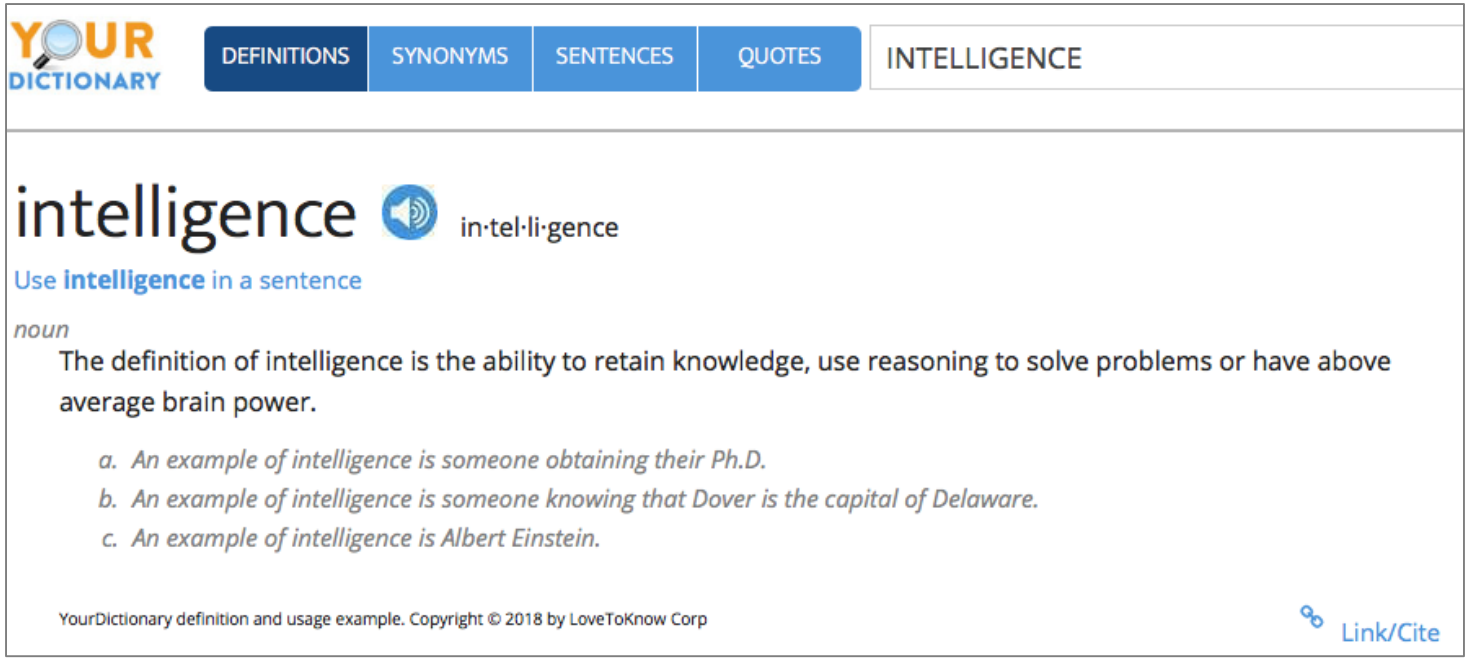
- Basically:
  - Replicating (human) intelligence in a machine.

What is Artificial Intelligence?

- Basically:
  - Replicating (human) intelligence in a machine.
- So, what is “intelligence”?

What is Artificial Intelligence?

- Basically:
  - Replicating (human) intelligence in a machine.
- So, what is "intelligence"?



What is Artificial Intelligence?

- Basically:
  - Replicating (human) intelligence in a machine.
- So, what is “intelligence”?
  - It's very hard to define “intelligence” rigorously
  - Any definition would be arbitrary:
    - We could list features of intelligence, such as: learning, language, knowledge, reasoning, understanding, etc. But any such list leaves things out.
  - Intelligence is often a term that we use to describe certain behaviors that we observe in humans and not in other entities

What is Artificial Intelligence?

- So, if we cannot define intelligence, then… What is AI? And how would we recognize it when we achieve it?

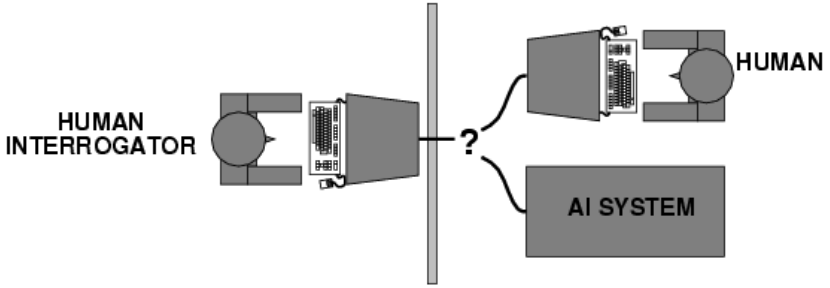
What is Artificial Intelligence?

- So, if we cannot define intelligence, then… What is AI? And how would we recognize it when we achieve it?

Thinking humanly	Thinking rationally
Acting humanly	Acting rationally

Acting Humanly: The Turing Test

- Turing (1950) "Computing machinery and intelligence":
- "Can machines think?" → "Can machines behave intelligently?"
- Operational test for intelligent behavior: the Imitation Game



- Turing predicted that by 2000, a machine might have a 30% chance of fooling a lay person for 5 minutes
- Anticipated many of the major arguments for & against AI that would appear over next 50 years
- Suggested major components of AI: knowledge, reasoning, language understanding, learning

Acting Humanly: The Turing Test

- Turing (1950) example:

Q: Please write me a sonnet on the subject of the Forth Bridge.

A : Count me out on this one. I never could write poetry.

Q: Add 34,957 to 70,764.

A: (Pause about 30 seconds and then give as answer) 105,621.

Q: Do you play chess?

A: Yes.

Q: I have K at [e1], and no other pieces. You have only K at [e3] and R at [h8]. It is your move. What do you play?

A: (After a pause of 15 seconds) R-[h1] mate.

Acting Humanly: The Turing Test

- Turing (1950) example:

Q: Please write me a sonnet on the subject of the Forth Bridge.

A : Count me out on this one. I never could write poetry.

Q: Add 34,957 to 70,764.

A: (Pause about 30 seconds and then give as answer) 105,621.

Q: Do you play chess?

A: Yes.

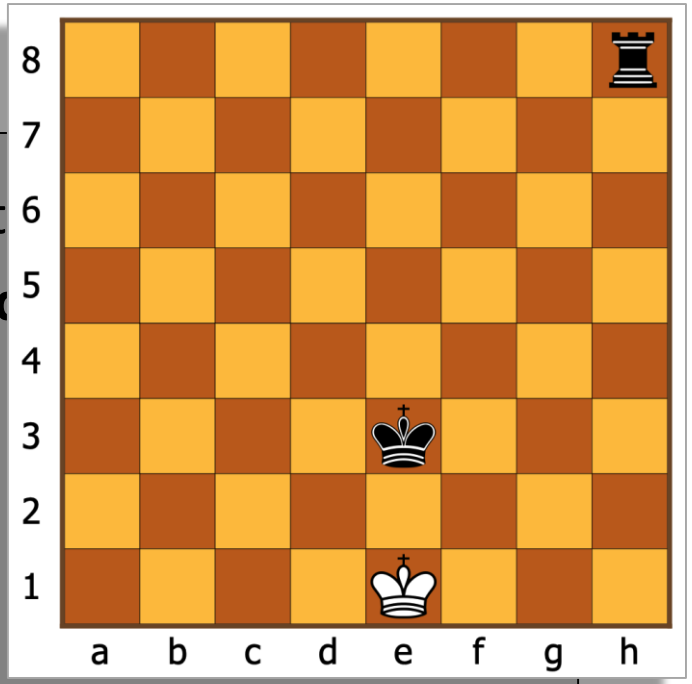
Correct answer: 105,721 !!

Q: I have K at [e1], and no other pieces. You have only K at [e3] and R at [h8]. It is your move. What do you play?

A: (After a pause of 15 seconds) R-[h1] mate.

Acting Humanly: The Turing Test

- Turing (1950) example:



Q: Please write me a sonnet on the subject

A : Count me out on this one. I never could

Q: Add 34,957 to 70,764.

A: (Pause about 30 seconds and then give

Q: Do you play chess?

A: Yes.

Q: I have K at [e1], and no other pieces. You have only K at [e3] and R at [h8]. It is your move. What do you play?

A: (After a pause of 15 seconds) R-[h1] mate.

Thinking Humanly: Cognitive Modeling

- 1960s "cognitive revolution": information-processing psychology
- Requires scientific theories of internal activities of the brain
- Validation of these theories would require:
  - Predicting and testing behavior of human subjects (top-down)
  - Direct identification from neurological data (bottom-up)
- Both approaches (roughly, Cognitive Science and Cognitive Neuroscience) are now distinct from AI
  - … but will they merge again??

Thinking Rationally: Laws of Thought

- Aristotle: What are correct arguments/thought processes?
- Several Greek schools developed various forms of logic: notation and rules of derivation for thoughts; may or may not have proceeded to the idea of mechanization
- Direct line through mathematics and philosophy to modern AI
- Problems:
  - Not all intelligent behavior is mediated by logical deliberation
  - What is the purpose of thinking?
    What thoughts should I have?

Acting Rationally: A Rational Agent

- Rational behavior: doing the “right thing”
- The “right thing” = that which is expected to maximize goal achievement, given the available information
- Doesn't necessarily involve thinking – e.g., a blinking reflex – but thinking should be in the service of rational action

Acting Rationally: A Rational Agent

- This course focuses on designing **rational agents**
- An agent is an entity that perceives and acts
- Abstractly, an agent is a function from percept histories to actions:
  - f: P* → A
- For any given class of environments and tasks, we seek the agent (or class of agents) with the best performance
- Caveat: computational limitations make perfect rationality unachievable
  - → design best program for given machine resources

What is Artificial Intelligence?

- So, if we cannot define intelligence, then… What is AI? And how would we recognize it when we achieve it?

Thinking humanly	Thinking rationally
Acting humanly	**Acting rationally**

What is Artificial Intelligence?

- In reality, AI encompasses many computational techniques to replicate any aspect of intelligence:
  - Automated reasoning
  - Knowledge representation
  - Learning
  - Natural language generation & understanding
  - Problem solving
  - Planning, prediction
  - Vision, perception
  - Action
- AI draws on many, many fields: computer science, mathematics, philosophy, economics, psychology, neuroscience, linguistics, control theory, and more

Brief History of AI

• 1943	McCulloch & Pitts: Boolean circuit model of brain
• 1950	Turing's "Computing Machinery and Intelligence"
• 1956	Dartmouth meeting: "Artificial Intelligence" adopted
• 1950s-60s	Early AI programs, including Samuel's checkers program, Newell & Simon's Logic Theorist, Gelernter's Geometry Engine
• 1966-73	AI discovers computational complexity Neural network research almost disappears
• 1969-79	Early development of knowledge-based systems
• 1980--	AI becomes an industry
• 1986--	Neural networks return to popularity
• 1990s	Probability, Agents (neural networks are back)
• 1995--	The emergence of intelligent agents
• 2000s	Human-level AI back on the agenda

What can AI do nowadays?

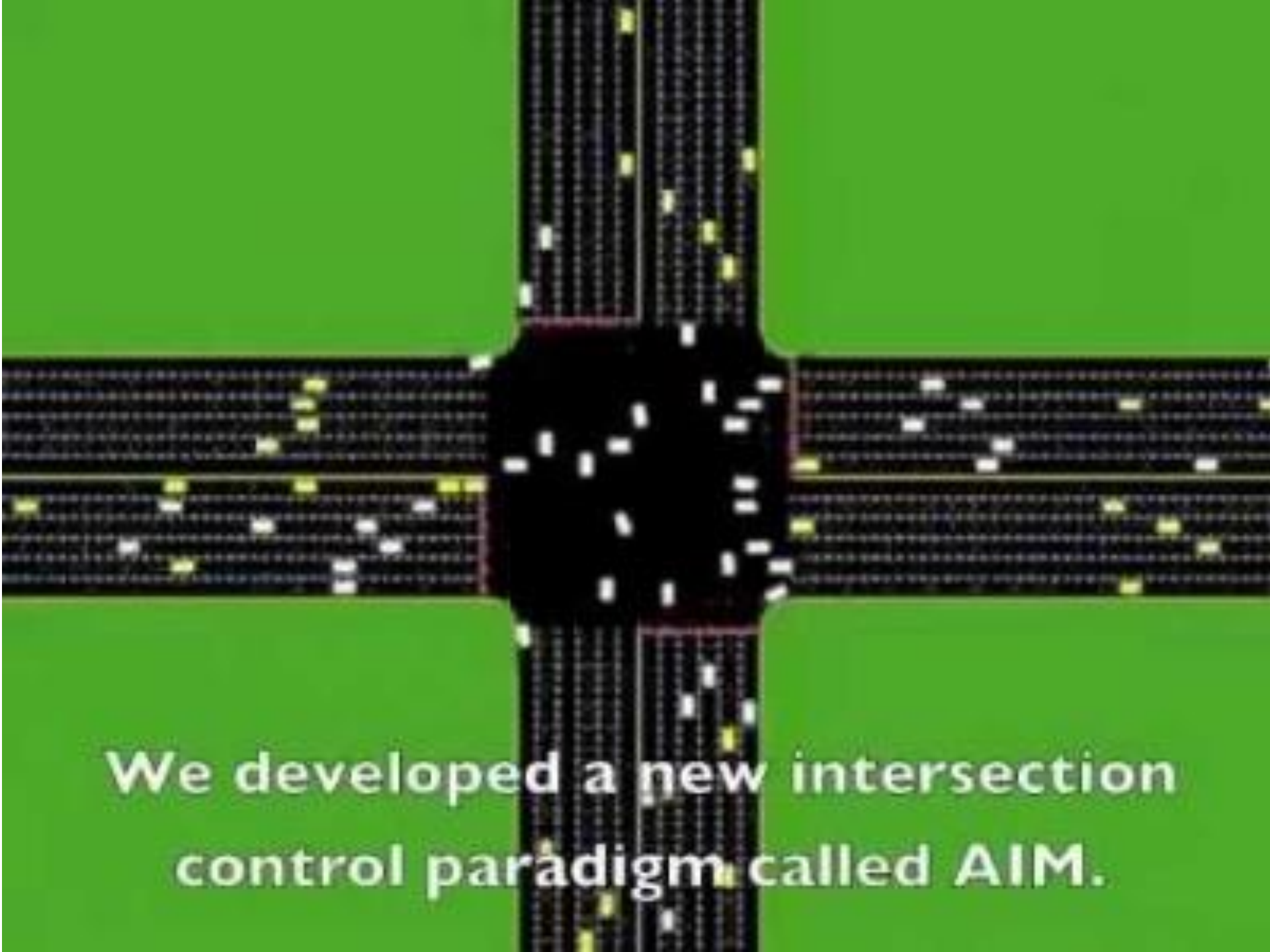
What can AI do nowadays?

- Play ping-pong?
  - https://www.youtube.com/embed/IXyKLDNzGGI



What can AI do nowadays?

- Play ping-pong? YES
- Autonomously drive a car?
  - You all know about autonomous driving
  - Where is the future headed?
    - https://www.youtube.com/embed/4pbAI40dK0A
    - https://www.youtube.com/embed/4SmJP8TdWTU


We developed a new intersection
control paradigm called AIM.



What can AI do nowadays?

- Play ping-pong? YES
- Autonomously drive a car? YES
- Play Soccer?
  - https://www.youtube.com/embed/aLy5pUsmpKE
  - https://www.youtube.com/embed/CJlTj4vmDZE


Carnegie Mellon


1

What can AI do nowadays?

- Play ping-pong? YES
- Autonomously drive a car? YES
- Play Soccer? KIND OF
- Play Chess/Go?
  - https://www.youtube.com/embed/_HCWBS6k8j0

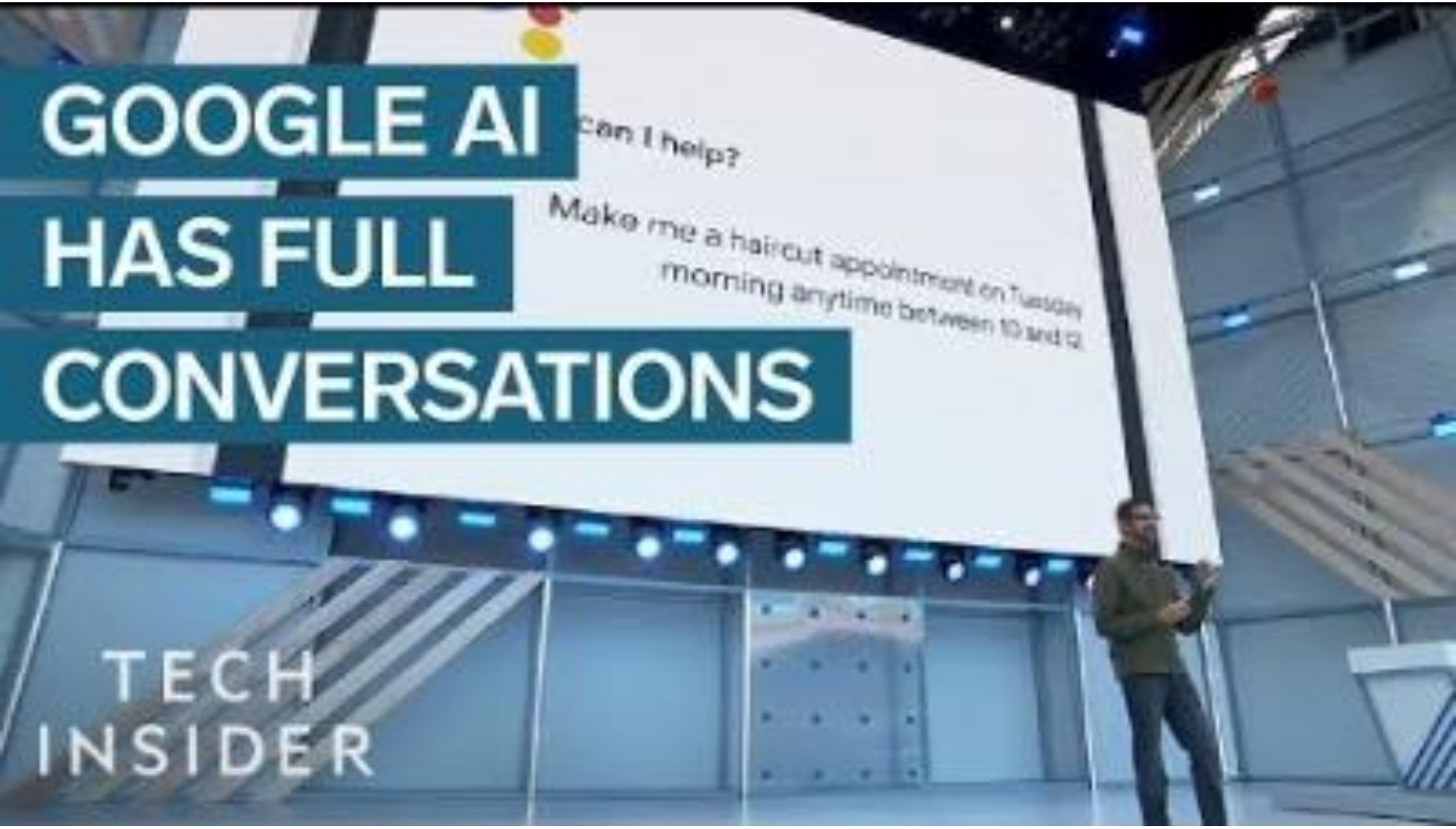

The Future of Go
KE JIE
02:54:50
ALPHAGO
02:57:25

What can AI do nowadays?

- Play ping-pong? YES
- Autonomously drive a car? YES
- Play Soccer? KIND OF
- Play Chess/Go? YES
- Accurately translate a text?
  - http://translate.google.com

What can AI do nowadays?

- Play ping-pong? YES
- Autonomously drive a car? YES
- Play Soccer? KIND OF
- Play Chess/Go? YES
- Accurately translate a text? KIND OF
- Have a conversation?
  - https://www.youtube.com/embed/lXUQ-DdSDoE


GOOGLE AI
HAS FULL
CONVERSATIONS
can I help?
Make me a haircut appointment on Tuesday
morning anytime between 10 and 12
TECH
INSIDER

What can AI do nowadays?

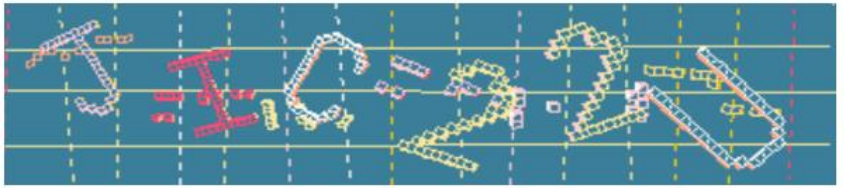
- Play ping-pong? YES
- Autonomously drive a car? YES
- Play Soccer? KIND OF
- Play Chess/Go? YES
- Accurately translate a text? KIND OF
- Have a conversation? KIND OF
- Understand an image?

What can AI do nowadays?













- Understand an image?

What can AI do nowadays?

- Play ping-pong? YES
- Autonomously drive a car? YES
- Play Soccer? KIND OF
- Play Chess/Go? YES
- Accurately translate a text? KIND OF
- Have a conversation? KIND OF
- Understand an image? KIND OF
- Play Jeopardy better than humans?
  - https://www.youtube.com/embed/P18EdAKuC1U


THINK
$24,000
$77,147
$21,600

What can AI do nowadays?

- Play ping-pong? YES
- Autonomously drive a car? YES
- Play Soccer? KIND OF
- Play Chess/Go? YES
- Accurately translate a text? KIND OF
- Have a conversation? KIND OF
- Understand an image? KIND OF
- Play Jeopardy better than humans? YES
- Create art?
  - https://www.youtube.com/embed/DJToDLBPovg

AN AVOCADO ARMCHAIR

AI-GENERATED IMAGES



OpenAI

SOTA TEXT-TO-IMAGE

Next Time…

- Rational Agents

CS 380: ARTIFICIAL INTELLIGENCE

Lecture 10: Machine Learning

Summary so far:

- Rational Agents
- Problem Solving
  - Systematic Search
    - Uninformed
    - Informed
  - Local Search
  - Adversarial Search
- Logic and Knowledge Representation
  - Predicate Logic
  - First-order Logic
- Natural-Language Processing
- Today: Machine Learning

Machine Learning

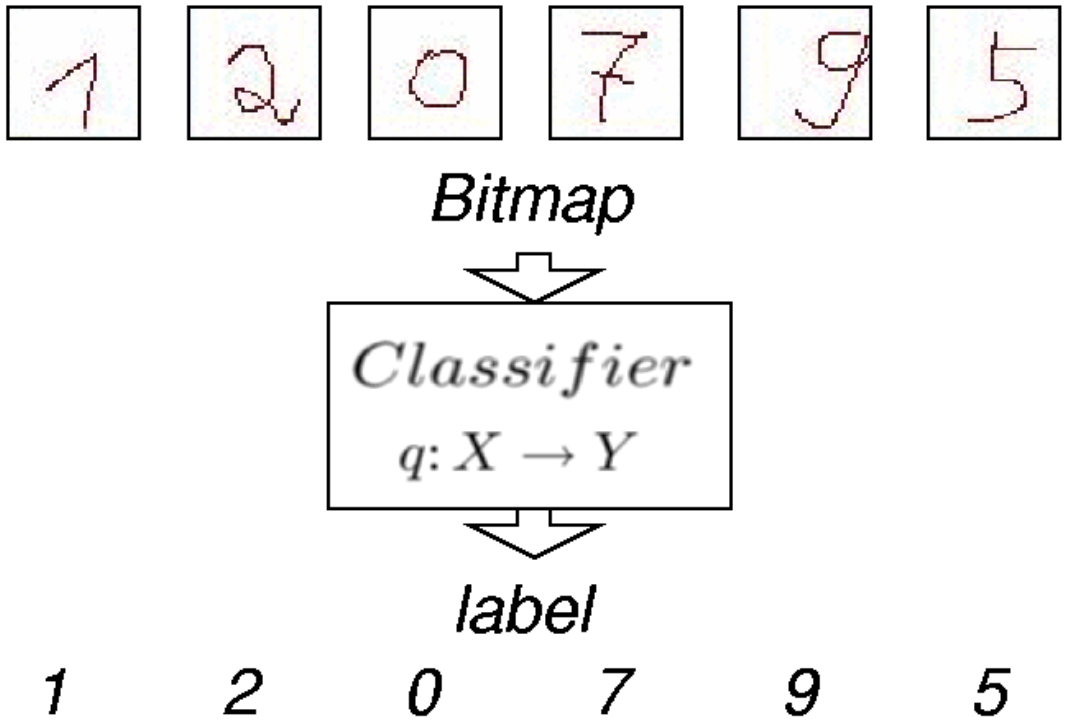
- Computational methods for computers to exhibit specific forms of learning. For example:
  - Learning from Examples:
    - Supervised learning
    - Unsupervised learning
  - Reinforcement Learning
  - Learning from Observation (demonstration/imitation)

How it works

- Feed input data into an algorithm
- Allow the algorithm to find patterns or relationships
- Algorithms builds a model to make predictions or decisions
- The model improves over time with more data

Examples

- **Supervised Learning**: learning to recognize writing



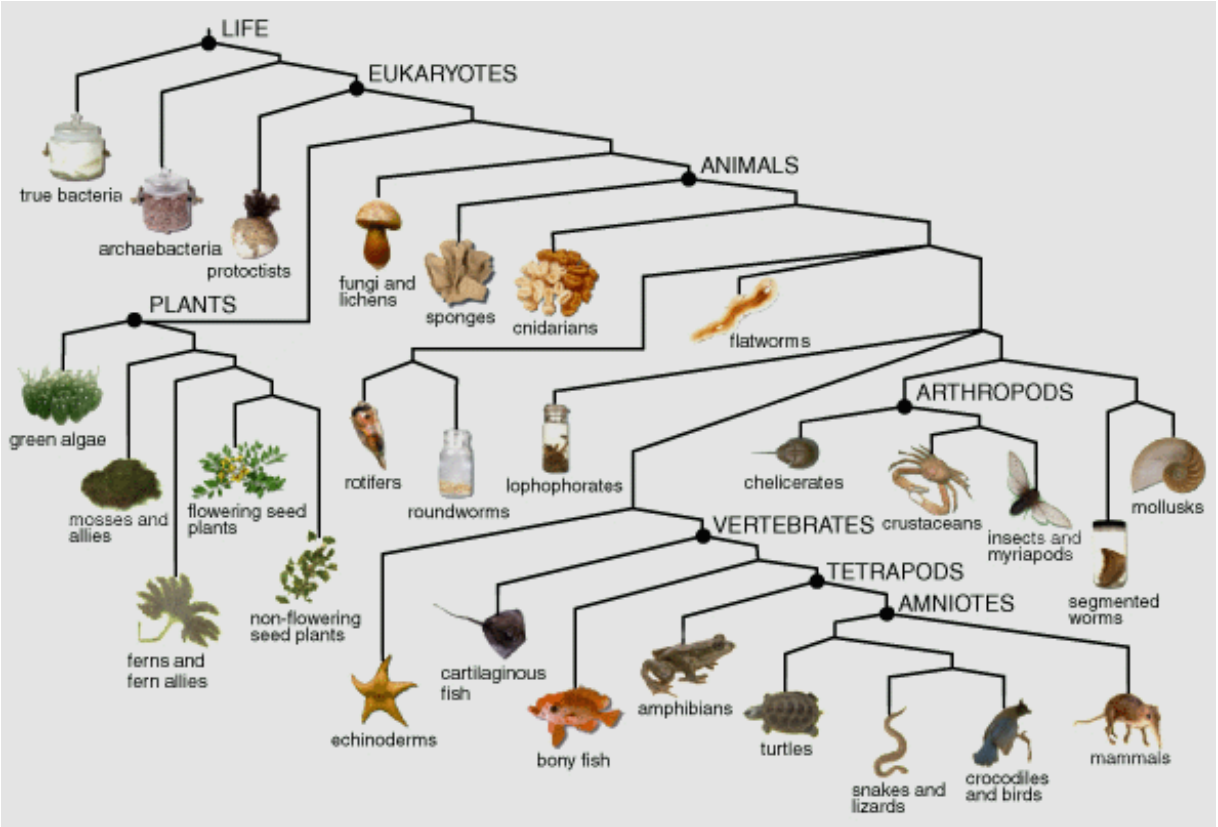
Examples

- **Supervised Learning**: image classification



Examples

- **Unsupervised Learning**: clustering observations into meaningful classes



Examples

- **Reinforcement Learning**: learning to walk



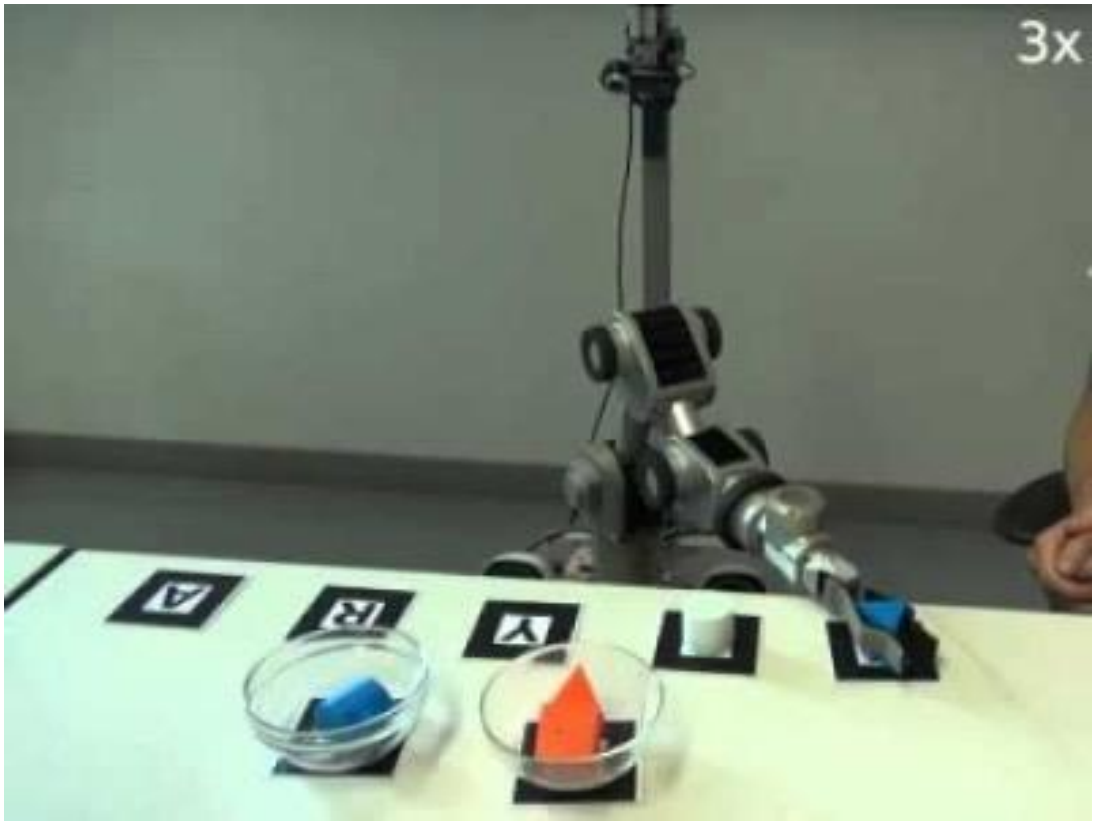
Examples

- **Learning from demonstration**: performing tasks that other agents (or humans) can do



Examples

- **Learning from demonstration**:
  - https://www.youtube.com/watch?v=3nSn1DGek4c

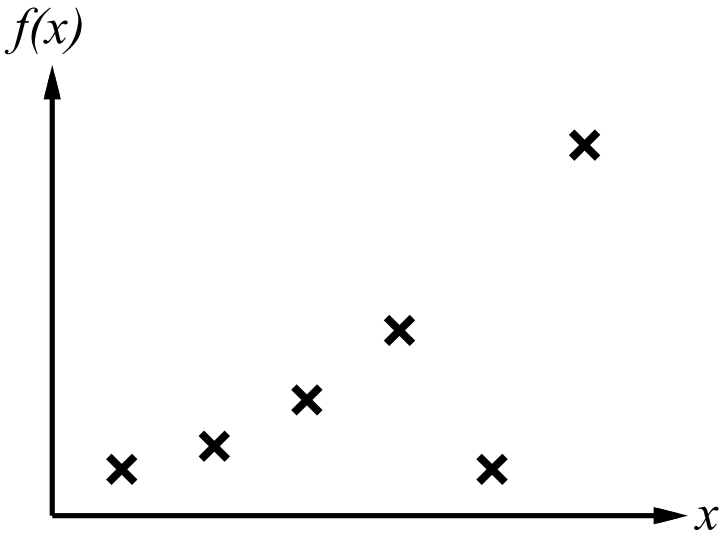


Inductive Learning

- Inductive learning tries to infer general concepts from simple examples
- Let's try to learn something in its simplest form: Learning a function from examples
  - **Desired**: f is a target function we want to learn
    — but we don't know what it is or how to compute it
  - **Given**: examples where an example = $< x , f(x) >$
  - **Problem**: find a hypothesis h such that $h \approx f$

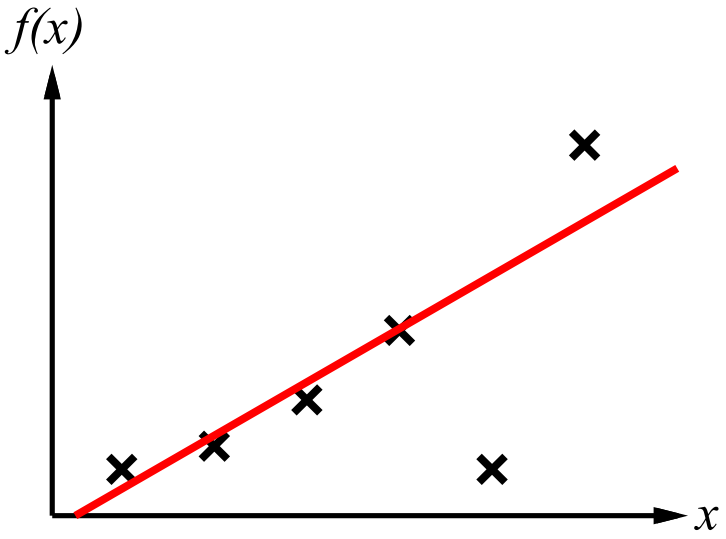
Inductive Learning

- Now, let's try to construct a hypothesis *h* such that $h \approx f$



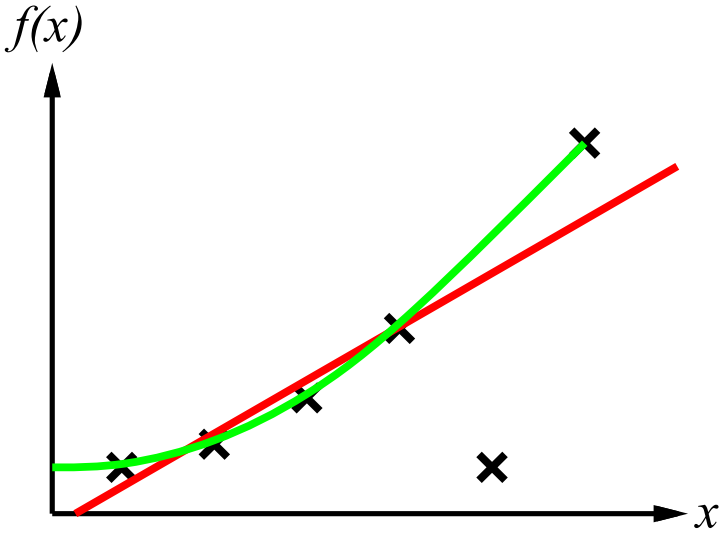
Inductive Learning

- Now, let's try to construct a hypothesis *h* such that $h \approx f$



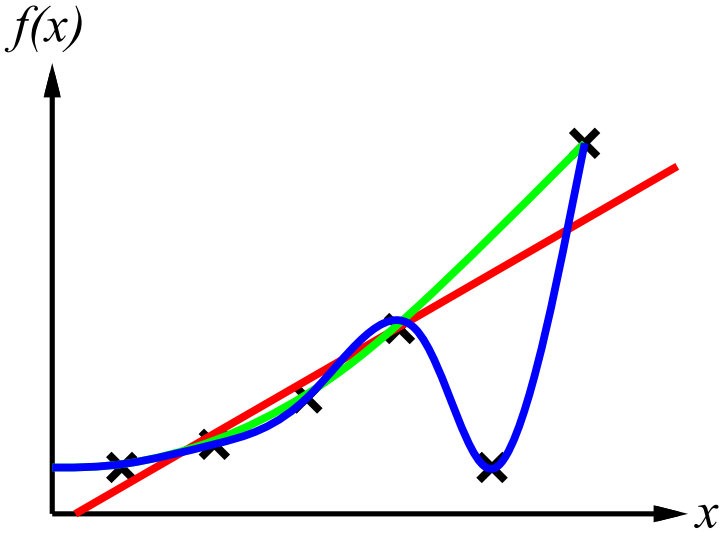
Inductive Learning

- Now, let's try to construct a hypothesis *h* such that $h \approx f$



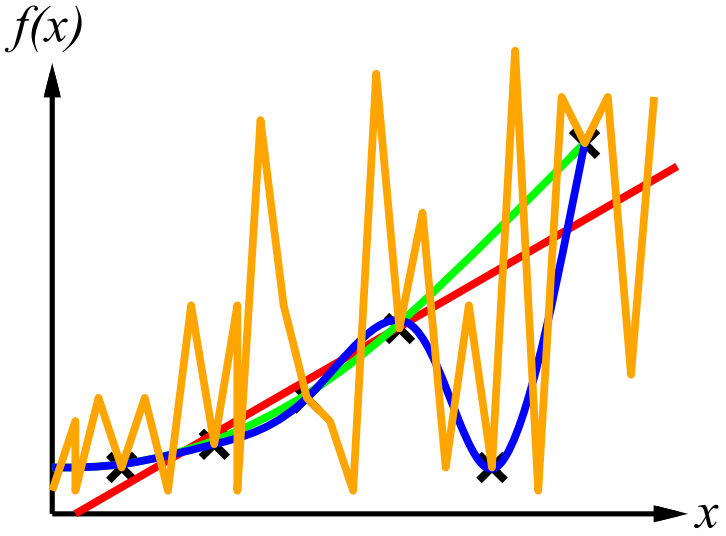
Inductive Learning

- Now, let's try to construct a hypothesis *h* such that $h \approx f$



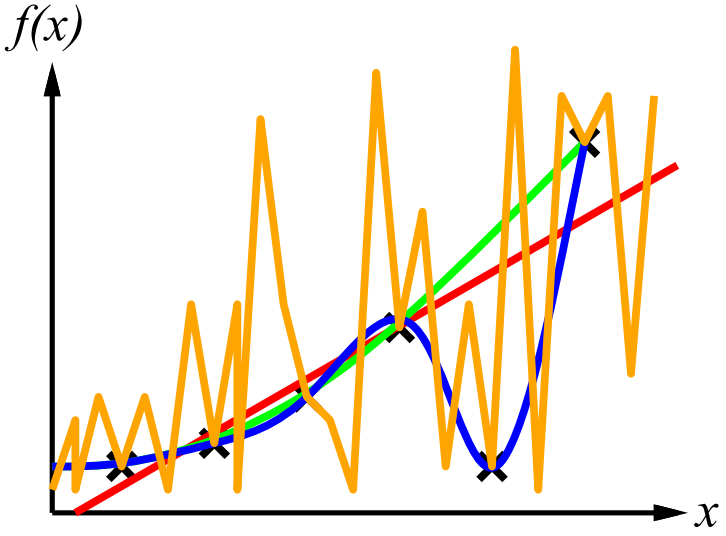
Inductive Learning

- Now, let's try to construct a hypothesis *h* such that $h \approx f$



Inductive Learning

- Now, let's try to construct a hypothesis *h* such that $h \approx f$





William of Occam
(1287-1347)

- **Occam's razor**: The simplest explanation tends to be the best one.

Attribute-Based Representations

- Items (or examples) described by attribute values (Boolean, discrete, continuous, etc.)
- Example: Will I wait for a table at a restaurant?
  - What factors (attributes) would affect your decision?

Attribute-Based Representations

- Items (or examples) described by attribute values (Boolean, discrete, continuous, etc.)
- Example: Will I wait for a table at a restaurant?
  - Do I have an **Alternative**?
  - Is there a **Bar**?
  - Is it **Friday**?
  - Am I **Hungry**?
  - Are there **Patrons** in the restaurant?
  - What is the **Price**?
  - Is it **Raining**?
  - Do I have a **Reservation**?
  - What **Type** of food is it?
  - What's the **Estimated** wait time?

  → I will wait, **True** or **False**

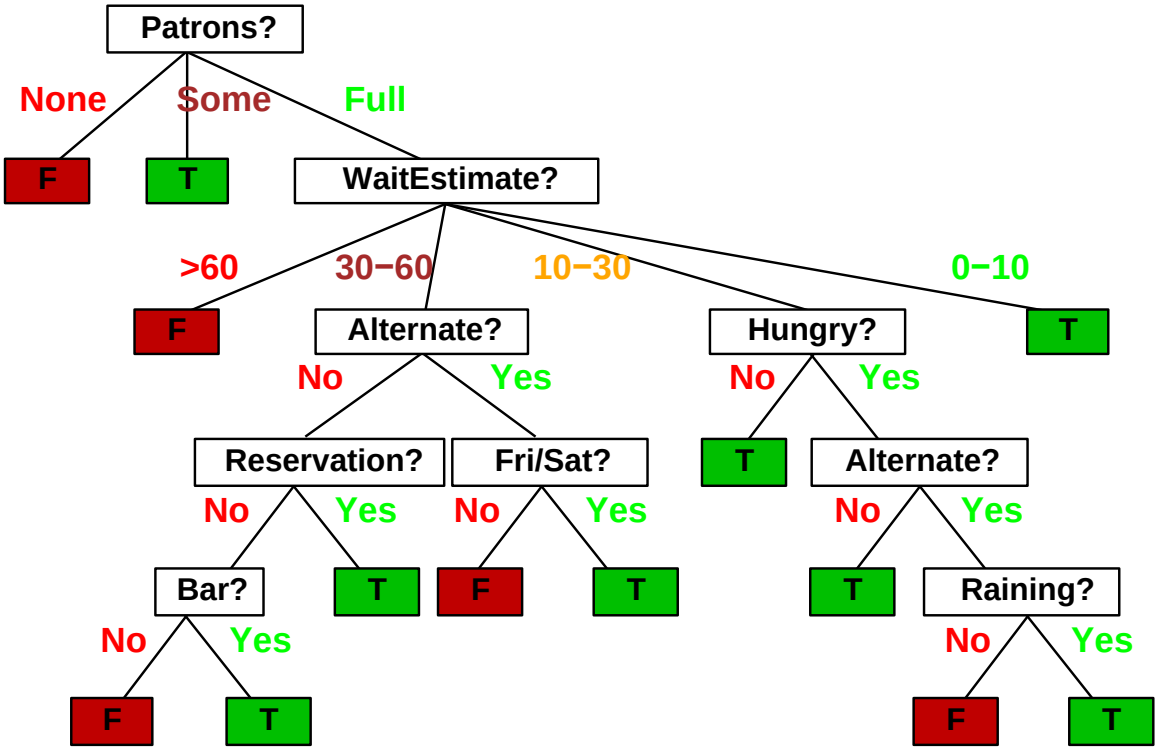
Attribute-Based Representations

- Items (or examples) described by attribute values (Boolean, discrete, continuous, etc.)
- Example: Will I wait for a table at a restaurant?

	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X1	T	F	F	T	Some	$$$	F	T	French	0–10	**T**
X2	T	F	F	T	Full	$	F	F	Thai	30–60	**F**
X3	F	T	F	F	Some	$	F	F	Burger	0–10	**T**
X4	T	F	T	T	Full	$	F	F	Thai	10–30	**T**
X5	T	F	T	F	Full	$$$	F	T	French	>60	**F**
X6	F	T	F	T	Some	$$	T	T	Italian	0–10	**T**
X7	F	T	F	F	None	$	T	F	Burger	0–10	**F**
X8	F	F	F	T	Some	$$	T	T	Thai	0–10	**T**
X9	F	T	T	F	Full	$	T	F	Burger	>60	**F**
X10	T	T	T	T	Full	$$$	F	T	Italian	10–30	**F**
X11	F	F	F	F	None	$	F	F	Thai	0–10	**F**
X12	T	T	T	T	Full	$	F	F	Burger	30–60	**T**

Decision Trees

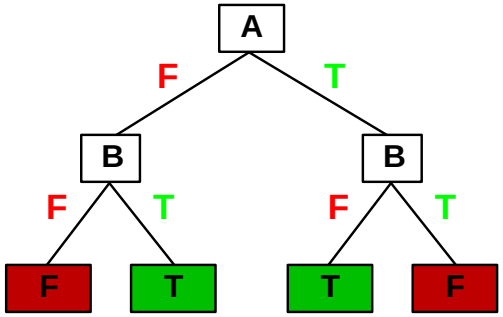
- Decision Tree asks questions about the attributes of a given example to provide an answer
- Here is the "true" tree for whether to wait:



Decision Trees

- Decision trees are expressive: they can express any function of the input attributes
- For example, for Boolean functions, we can convert *truth table row → path to leaf*

A	B	A xor B
F	F	F
F	T	T
T	F	T
T	T	F



- Trivially, there is a consistent decision tree for any training set w/ one path to leaf for each example (unless *f* nondeterministic in *x*)
- But it probably won't generalize to new examples!

Decision Trees

- How many distinct decision trees are there with *n* Boolean attributes?
  = number of Boolean functions
  = number of distinct truth tables with 2^n rows = 2^{2^n}
  – E.g., with 6 Boolean attributes, there are 18,446,744,073,709,551,616 trees
- How many purely conjunctive hypotheses are there? (e.g., Hungry ∧ ¬Rain)
  – Each attribute can be positive, negative, or omitted
  → 3^n distinct conjunctive hypotheses
- A more expressive hypothesis space…
  – increases chance that target function can be expressed
  – increases number of hypotheses consistent w/ training set
  – But may get worse predictions!

Decision-Tree Learning

- **Goal**: find a small tree consistent with examples
- **Idea**: (recursively) choose the "most significant" attribute as the root of a (sub)tree

```
function DTL(examples, attributes, default) returns a decision tree
    if examples is empty then return default
    else if all examples have the same classification then return the classification
    else if attributes is empty then return Mode(examples)
    else
        best ← Choose-Attribute(attributes, examples)
        tree ← a new decision tree with root test best
        for each value v_i of best do
            examples_i ← {elements of examples with best = v_i}
            subtree ← DTL(examples_i, attributes − best, Mode(examples))
            add a branch to tree with label v_i and subtree subtree
        return tree
```

ID3

- Developed by Quinlan (1986)
- ID3 = “Iterative Dichotomiser 3”
  - Create a decision tree that iteratively evaluates an item based on its attributes
  - Key idea: use *entropy* (or *information gain*) to make these decisions

ID3

- Consider how different attributes might split a set of training examples
  - Based on **Patrons**…

	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X1	T	F	F	T	Some	$$$	F	T	French	0–10	**T**
X2	T	F	F	T	Full	$	F	F	Thai	30–60	**F**
X3	F	T	F	F	Some	$	F	F	Burger	0–10	**T**
X4	T	F	T	T	Full	$	F	F	Thai	10–30	**T**
X5	T	F	T	F	Full	$$$	F	T	French	>60	**F**
X6	F	T	F	T	Some	$$	T	T	Italian	0–10	**T**
X7	F	T	F	F	None	$	T	F	Burger	0–10	**F**
X8	F	F	F	T	Some	$$	T	T	Thai	0–10	**T**
X9	F	T	T	F	Full	$	T	F	Burger	>60	**F**
X10	T	T	T	T	Full	$$$	F	T	Italian	10–30	**F**
X11	F	F	F	F	None	$	F	F	Thai	0–10	**F**
X12	T	T	T	T	Full	$	F	F	Burger	30–60	**T**

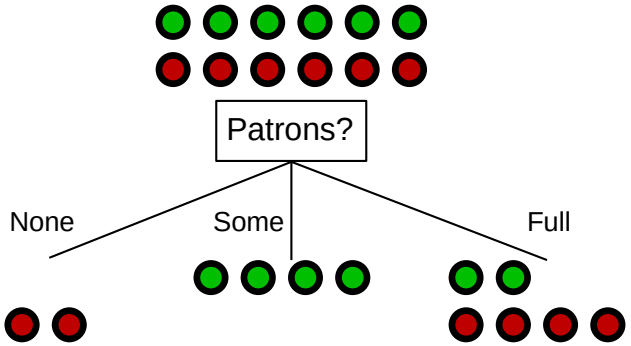
ID3

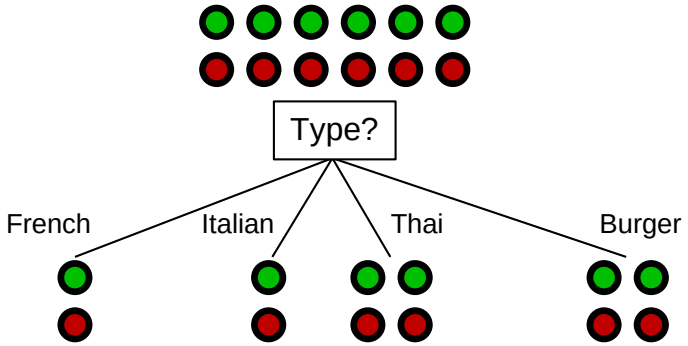
- Consider how different attributes might split a set of training examples
  - Based on **Type**…

	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X1	T	F	F	T	Some	$$$	F	T	French	0–10	**T**
X2	T	F	F	T	Full	$	F	F	Thai	30–60	**F**
X3	F	T	F	F	Some	$	F	F	Burger	0–10	**T**
X4	T	F	T	T	Full	$	F	F	Thai	10–30	**T**
X5	T	F	T	F	Full	$$$	F	T	French	>60	**F**
X6	F	T	F	T	Some	$$	T	T	Italian	0–10	**T**
X7	F	T	F	F	None	$	T	F	Burger	0–10	**F**
X8	F	F	F	T	Some	$$	T	T	Thai	0–10	**T**
X9	F	T	T	F	Full	$	T	F	Burger	>60	**F**
X10	T	T	T	T	Full	$$$	F	T	Italian	10–30	**F**
X11	F	F	F	F	None	$	F	F	Thai	0–10	**F**
X12	T	T	T	T	Full	$	F	F	Burger	30–60	**T**

ID3

- Consider how different attributes might split a set of training examples

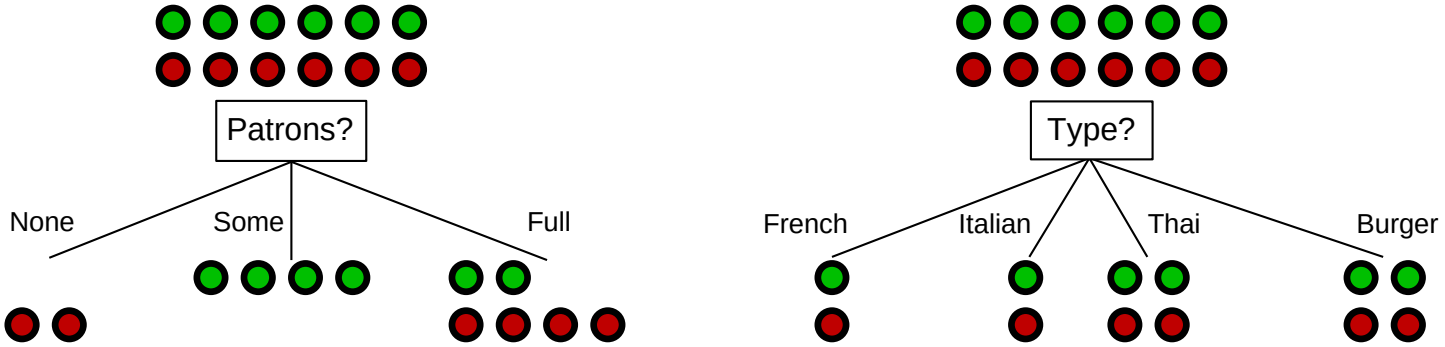




- Which of these attributes should we prefer?

ID3

- Consider how different attributes might split a set of training examples



- Which of these attributes should we prefer?
  - Better to have subsets that are all/mostly the same class
  - "**Patrons**" is a better choice here

Entropy

- "Information" is what answers questions in the context of decision trees
  - In essence, the more clueless I am about the answer initially, the more information is contained in the answer
- Scale
  - 1 bit = answer to Boolean question with prior $\langle 0.5, 0.5 \rangle$
  - Information in an answer when prior is $\langle P_1, \ldots, P_n \rangle$ is

$$H(\langle P_1, \ldots, P_n \rangle) = \sum_{i=1}^{n} -P_i \log_2 P_i$$

… also called the **entropy** of the prior

Entropy

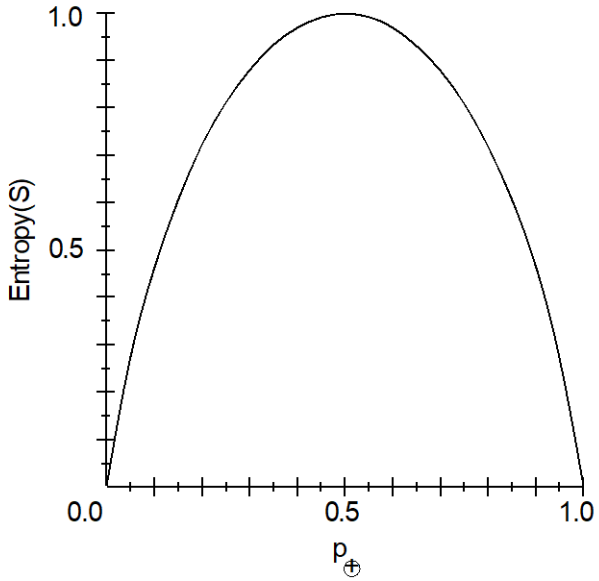
- Given a set of symbols *S*…
- Drawn from an alphabet *B*…
- **Entropy** is a measure of uncertainty of a variable
- Entropy = expected number of bits needed to encode the next symbol (in a sense, the amount of "surprise" over the outcome of the next event)
  - If *S* is drawn at random from *B*, entropy is maximal
  - If *S* is always the same symbol from *B*, entropy is minimal
    - we know which symbol will come next, so there's no new information

Entropy

- Entropy can be defined as:

$$H(\langle P_1, \ldots, P_n \rangle) = \sum_{i=1}^{n} -P_i \log_2 P_i$$

- Example for a binary variable:

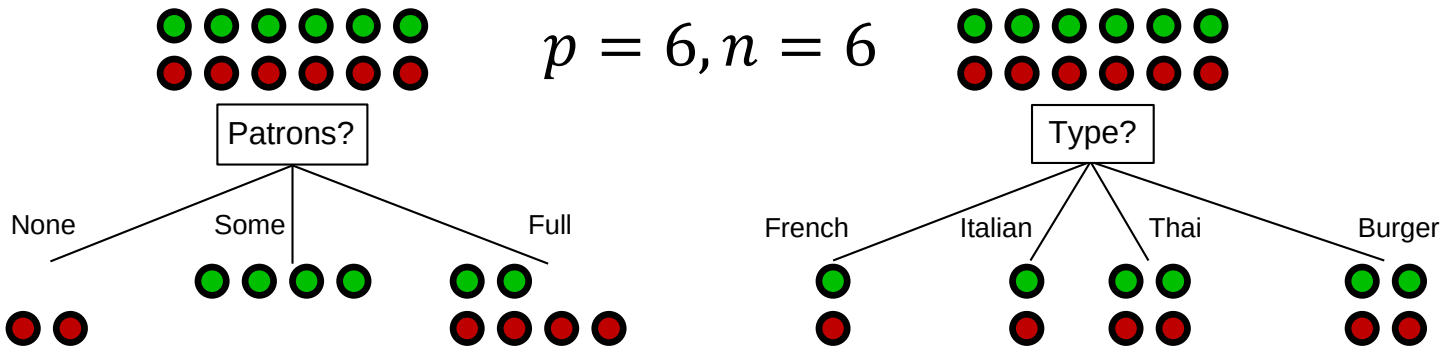


ID3

- An attribute splits the examples E into subsets E_i, each of which (we hope) needs less information to complete the classification
- Let E_i have p_i positive and n_i negative examples
  - $H(\langle \frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i} \rangle)$ bits needed to classify a new example
  - the **expected** number of bits per example over all branches is

$$\sum_i \frac{p_i + n_i}{p + n} H(\langle \frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i} \rangle)$$

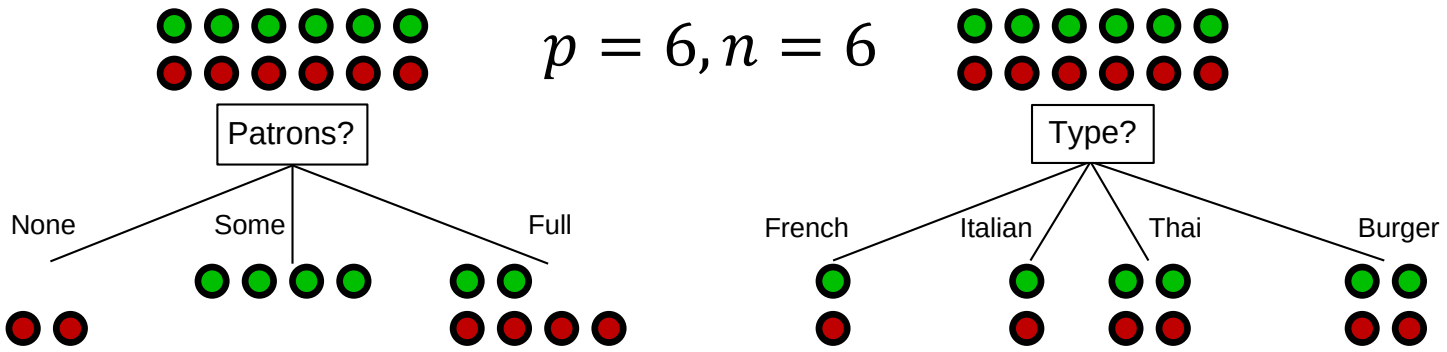
ID3



- For **Patrons**:
  - Subset E_1 has $p_1 = 0$ positive, $n_1 = 2$ negative
  - Subset E_2 has $p_2 = 4$ positive, $n_2 = 0$ negative
  - Subset E_3 has $p_3 = 2$ positive, $n_3 = 4$ negative

$$\sum_i \frac{p_i + n_i}{p + n} H\left(\left\langle \frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i} \right\rangle\right) = 0.459 \text{ bits}$$

ID3



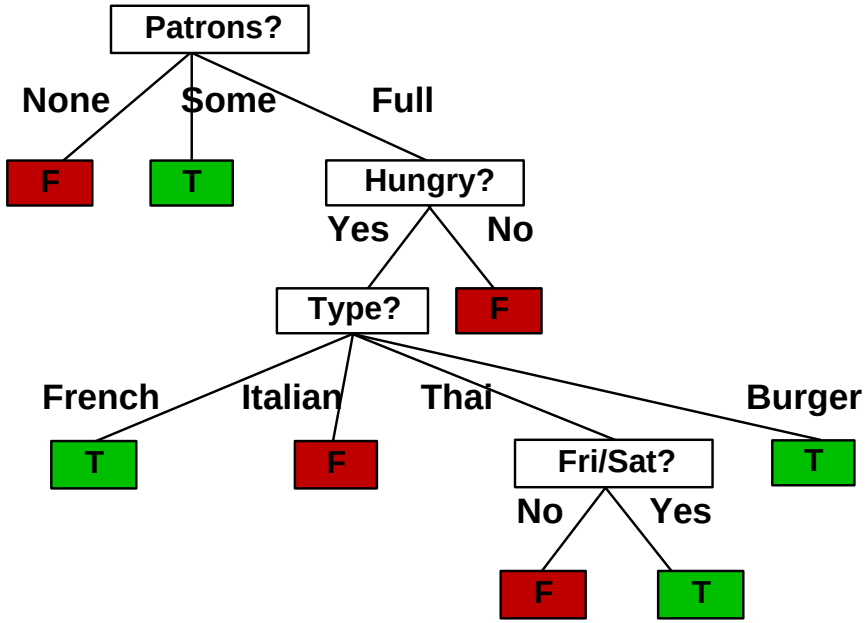
- For **Type**:
  - All subsets have $p_i = n_i$

$$\sum_i \frac{p_i + n_i}{p + n} H\left(\left\langle \frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i} \right\rangle\right) = 1 \text{ bit}$$

- So we choose the attribute that minimizes the resulting entropy, i.e., **Patrons**

ID3

- The ID3 tree after learning:



- A lot simpler than the full tree we saw earlier
- Why is simpler a good thing here?

ID3

- Alternatives to Entropy…
  - E.g., Information Gain is a common alternative
  - And there are others (Gain Ratio, GINI Index, RLDM, etc.)
- Alternative search strategies…
  - E.g., Decision Forests (instead of 1 deep tree, use multiple shorter trees and aggregate their decisions)

Decision Trees in Python

- You could go ahead and write code for decision trees from scratch
  - You'd have control over all elements of the system
- … but we're not planning that in our intro course
- Instead, let's use a common Python library to build a basic decision tree
  - Core library: **scikit-learn**
    - Note: This library doesn't use ID3, but rather CART, which builds on C4.5, which builds on ID3 — so not the same algorithm that we've discussed today, but same general idea
  - Also useful: **pandas** for data, **matplotlib** for plots

```
> python -m pip install scikit-learn pandas matplotlib
```

restaurant.csv

```
Alternative,Bar,Friday,Hungry,Patrons,Price,Rain,Reservation,Type,EstimatedWait,WillWait
T,F,F,T,Some,$$$,F,T,French,0-10,T
T,F,F,T,Full,$,F,F,Thai,30-60,F
F,T,F,F,Some,$,F,F,Burger,0-10,T
T,F,T,T,Full,$,F,F,Thai,10-30,T
T,F,T,F,Full,$$$,F,T,French,>60,F
F,T,F,T,Some,$$,T,T,Italian,0-10,T
F,T,F,F,None,$,T,F,Burger,0-10,F
F,F,F,T,Some,$$,T,T,Thai,0-10,T
F,T,T,F,Full,$,T,F,Burger,>60,F
T,T,T,T,Full,$$$,F,T,Italian,10-30,F
F,F,F,F,None,$,F,F,Thai,0-10,F
T,T,T,T,Full,$,F,F,Burger,30-60,T
```

decision_tree.py

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
import pandas as pd

# load data as pandas data frame
df = pd.read_csv('restaurant.csv')
print('\n' + str(df) + '\n')

# extract header to find feature and target column names
header = list(df)
features = header[:-1]
target = header[-1]

# convert text data to numeric data, saving text as classes
classes = {}
for col in header:
    classes[col] = sorted(set(df[col]))
    df[col] = [classes[col].index(x) for x in df[col]]

# extract feature and target data for training
# (for a real application, would split into train and test sets)
X = df[features]
y = df[target]

# create and train the decision tree
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X, y)

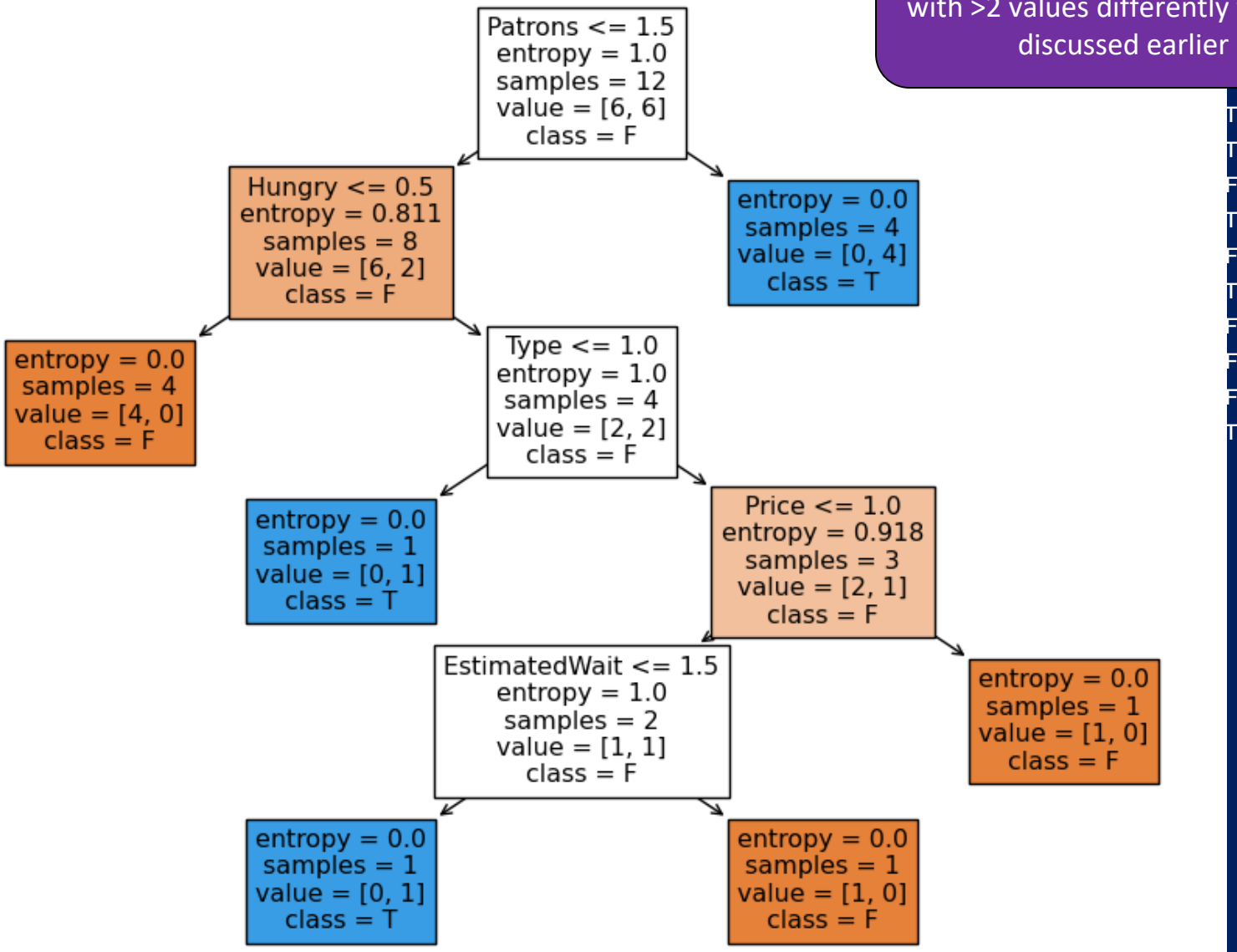
# plot the decision tree
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=features,
          class_names=classes[target])
plt.show()
```

```
> python decision_tree.py

   Alternative Bar Friday Hungry Patrons Price Rain Reservation     Type EstimatedWait WillWait
0            T   F      F      T    Some   $$$    F           T   French          0-10        T
1            T   F      F      T    Full    $     F           F     Thai         30-60        F
2            F   T      F      F    Some    $     F           F   Burger          0-10        T
3            T   F      T      T    Full    $     F           F     Thai         10-30        T
4            T   F      T      F    Full   $$$    F           T   French           >60        F
5            F   T      F      T    Some   $$     T           T  Italian          0-10        T
6            F   T      F      F    None    $     T           F   Burger          0-10        F
7            F   F      F      T    Some   $$     T           T     Thai          0-10        T
8            F   T      T      F    Full    $     T           F   Burger           >60        F
9            T   T      T      T    Full   $$$    F           T  Italian         10-30        F
10           F   F      F      F    None    $     F           F     Thai          0-10        F
11           T   T      T      T    Full    $     F           F   Burger         30-60        T
```

```
> python decision_tree.py

   Alte
0
1
2
3
4
5
6
7
8
9
10
11
```




```
T
T
F
T
F
T
F
F
F
T
```

Note: scikit-learn can only build a binary tree, so it handles features with >2 values differently than we discussed earlier

Overfitting

- What if almost all examples fit a clear pattern, but there's one that doesn't fit?

	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X1	T	F	F	T	Some	$$$	F	T	French	0–10	**F**
X2	T	F	F	T	Full	$	F	F	Thai	30–60	**F**
X3	F	T	F	F	Some	$	F	F	Burger	0–10	**F**
X4	T	F	T	T	Full	$	F	F	Thai	10–30	**F**
X5	T	F	T	F	Full	$$$	F	T	French	>60	**F**
X6	F	T	F	T	Some	$$	T	T	Italian	0–10	**T**
X7	F	T	F	F	None	$	T	F	Burger	0–10	**T**
X8	F	F	F	T	Some	$$	T	T	Thai	0–10	**T**
X9	F	T	T	F	Full	$	T	F	Burger	>60	**T**
X10	T	T	T	T	Full	$$$	F	T	Italian	10–30	**F**
X11	F	F	F	F	None	$	F	F	Thai	0–10	**F**
X12	T	T	T	T	Full	$	F	F	Burger	30–60	**F**

Overfitting

- What if almost all examples fit a clear pattern, but there's one that doesn't fit?

	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X1	T	F	F	T	Some	$$$	F	T	French	0–10	**F**
X2	T	F	F	T	Full	$	F	F	Thai	30–60	**F**
X3	F	T	F	F	Some	$	F	F	Burger	0–10	**F**
X4	T	F	T	T	Full	$	F	F	Thai	10–30	**F**
X5	T	F	T	F	Full	$$$	F	T	French	>60	**F**
X6	F	T	F	T	Some	$$	T	T	Italian	0–10	**T**
X7	F	T	F	F	None	$	T	F	Burger	0–10	**T**
X8	F	F	F	T	Some	$$	T	T	Thai	0–10	**T**
X9	F	T	T	F	Full	$	T	F	Burger	>60	**T**
X10	T	T	T	T	Full	$$$	F	T	Italian	10–30	**F**
X11	F	F	F	F	None	$	F	F	Thai	0–10	**F**
X12	T	T	T	T	Full	$	F	F	Burger	30–60	**F**

Overfitting

- What if almost all examples fit a clear pattern, but there's one that doesn't fit?

	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X1	T	F	F	T	Some	$$$	F	T	French	0–10	**F**
X2	T	F	F	T	Full	$	F	F	Thai	30–60	**F**
X3	F	T	F	F	Some	$	F	F	Burger	0–10	**T**
X4	T	F	T	T	Full	$	F	F	Thai	10–30	**F**
X5	T	F	T	F	Full	$$$	F	T	French	>60	**F**
X6	F	T	F	T	Some	$$	T	T	Italian	0–10	**T**
X7	F	T	F	F	None	$	T	F	Burger	0–10	**T**
X8	F	F	F	T	Some	$$	T	T	Thai	0–10	**T**
X9	F	T	T	F	Full	$	T	F	Burger	>60	**T**
X10	T	T	T	T	Full	$$$	F	T	Italian	10–30	**F**
X11	F	F	F	F	None	$	F	F	Thai	0–10	**F**
X12	T	T	T	T	Full	$	F	F	Burger	30–60	**F**

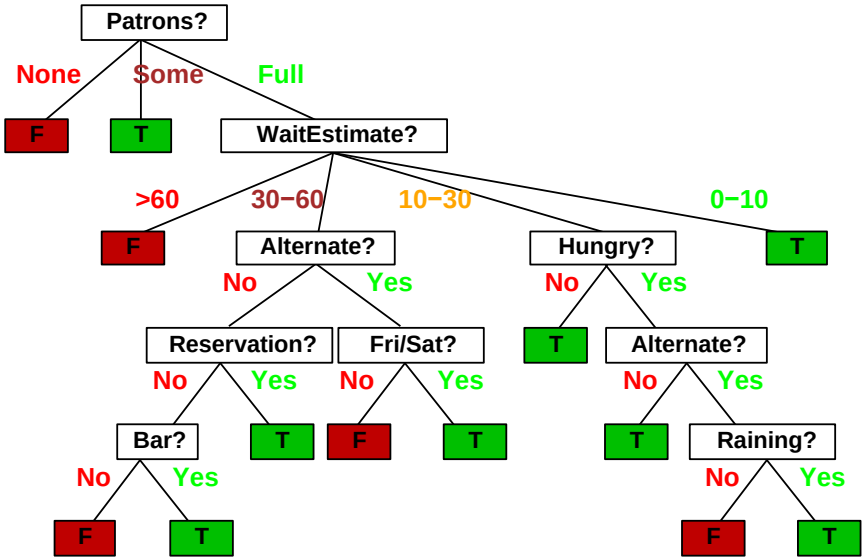
Overfitting

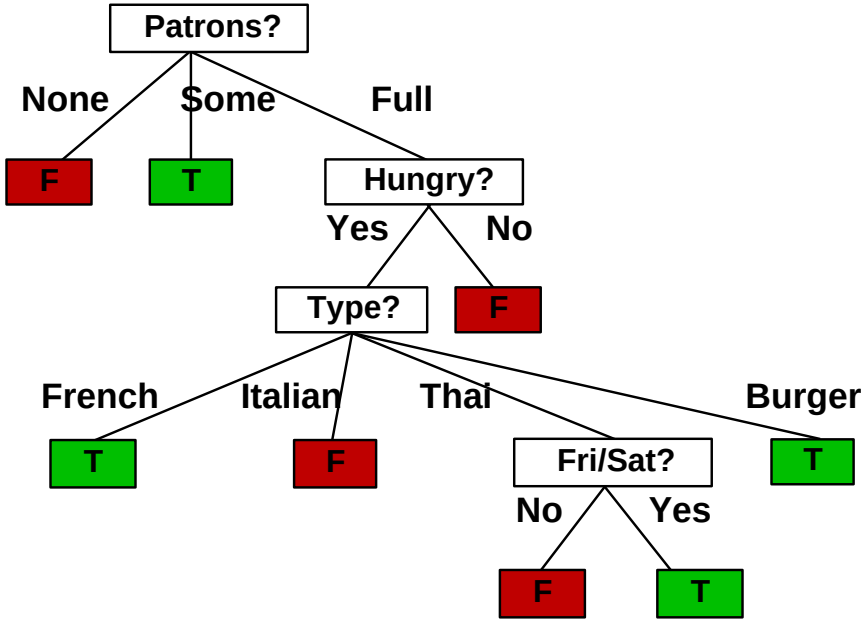
- What happens if we have an example in the training set that is noise? (e.g. that is mislabeled)
  - ID3 will try to force it into the decision tree!
- This is an example of **Overfitting** — when what you've learned does very well on training examples, but much less well on other examples
  - In other words, it doesn't generalize
  - One of the most critical concepts in machine learning!
- For ID3, some strategies exist to avoid this
  - e.g., prevent leaves that have very few examples
  - e.g., stop building the tree when there would be little information gain in the next level

Overfitting

- More generally, what can you do?
  - Use a **training set** and a **test set**
    - Say, reserve 90% for training and 10% for testing
  - Use cross-validation
    - Repeatedly split your set into different training vs. test sets
    - E.g., Leave-one-out cross-validation
  - By not training on the data you're testing, you're trying to ensure generality

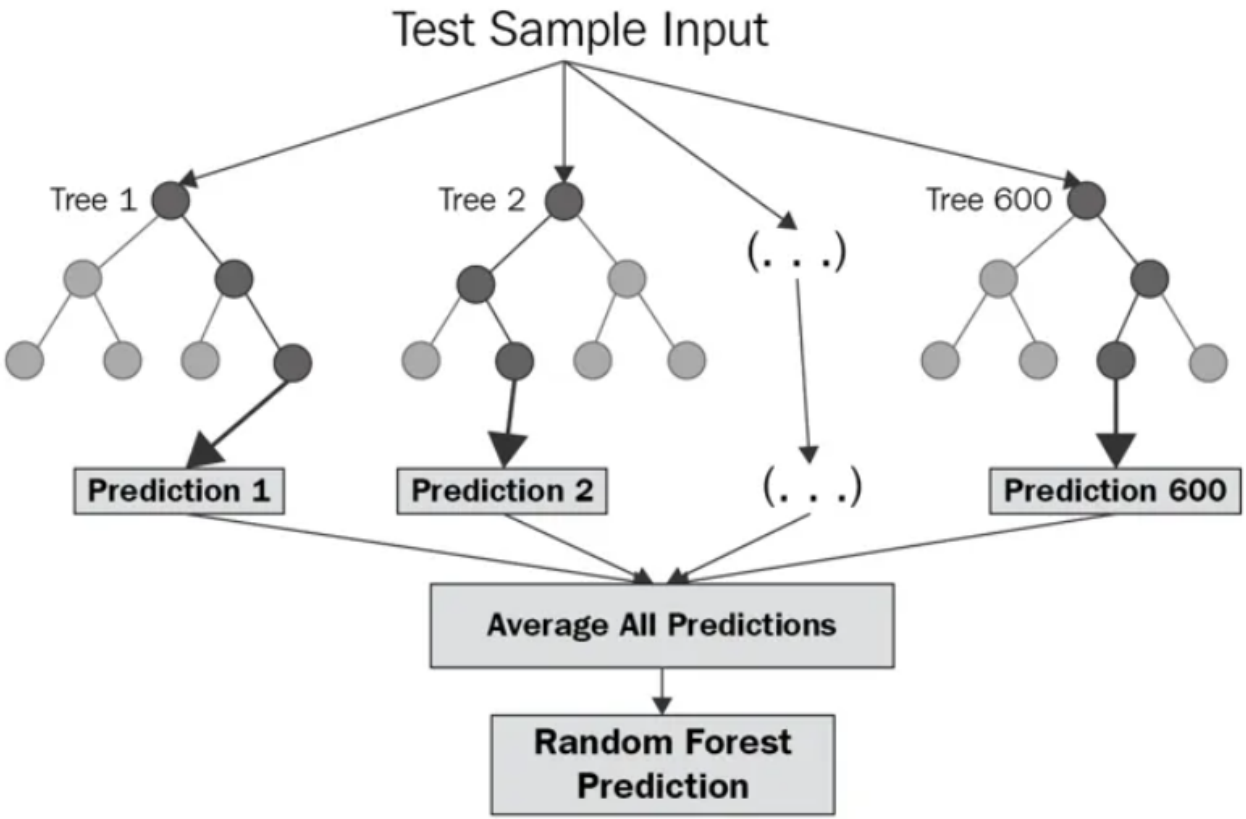
Random Forest





Test Vector <Patrons=Full, WaitEstimate?=0-10, Hungry=No>

Random Forest



Random Forest

- Advantages
  - Reduces overfitting
  - Handles large datasets and high-dimensional spaces
  - Robust to noise and outliers
- Disadvantages
  - Slower to predict than individual trees
  - Less interpretable than a single decision tree

Random Forest

- Where it's used today
  - Email Spam Detection
  - Medical Diagnosis
  - Stock Price Predictions
  - Fraud Detection
- Why use this over deep learning?
  - For structured data:
    - Less data-hungry, work well on small to medium datasets
    - Require less tuning
    - Can handle missing values and noisy inputs
    - Better interpretability

Next time

- We will focus on one specific type of Machine Learning called Reinforcement Learning

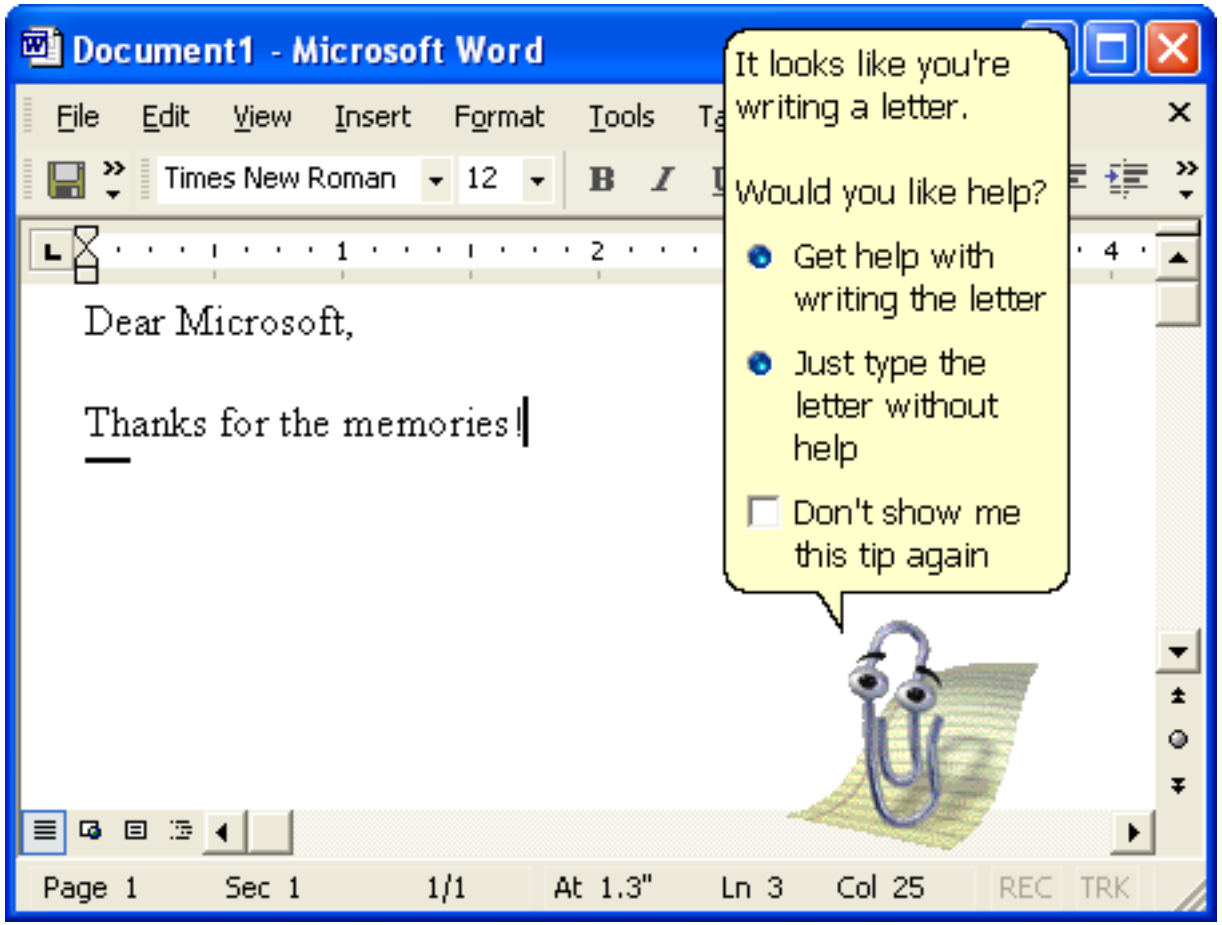
CS 380: ARTIFICIAL INTELLIGENCE

Lecture 2: Rational Agents

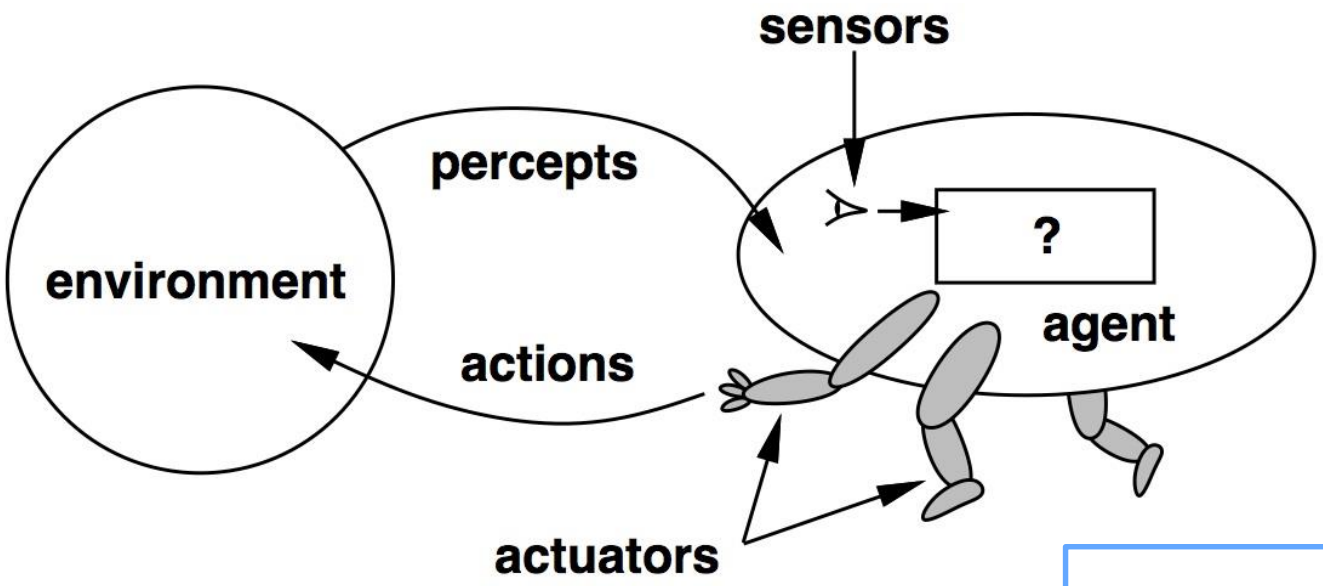
What is an Agent?

- **Agent**: an autonomous entity which observes and acts upon an environment and directs its activity towards achieving goals
- Examples
  - Robots
  - Software Agents
  - ATMs, ticket kiosks, etc.
  - Humans?

What is an Agent?



What is an Agent?



- Theoretically: $f : P^* \rightarrow A$

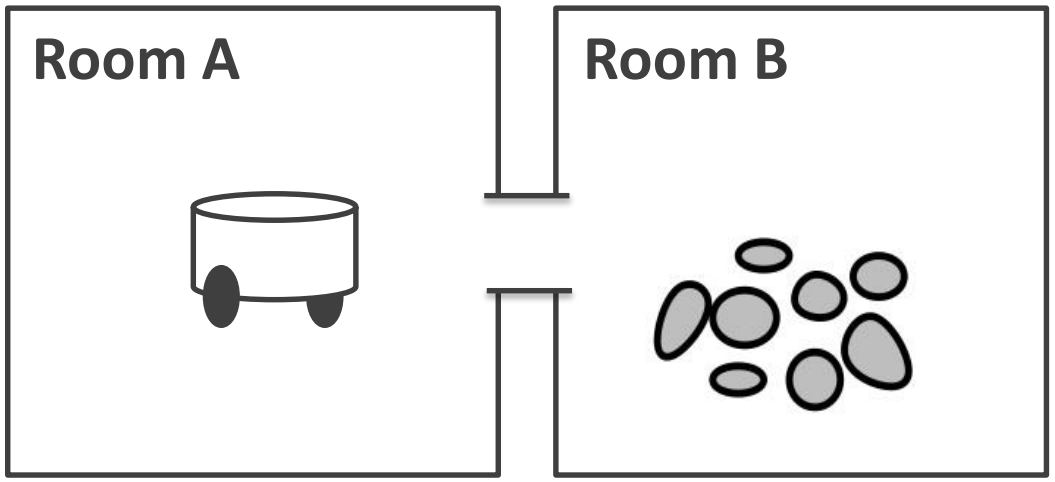
Maps sequences of **percepts** to **actions**

- In practice: the agent runs on a physical system (e.g., a computer) to produce f

Percepts

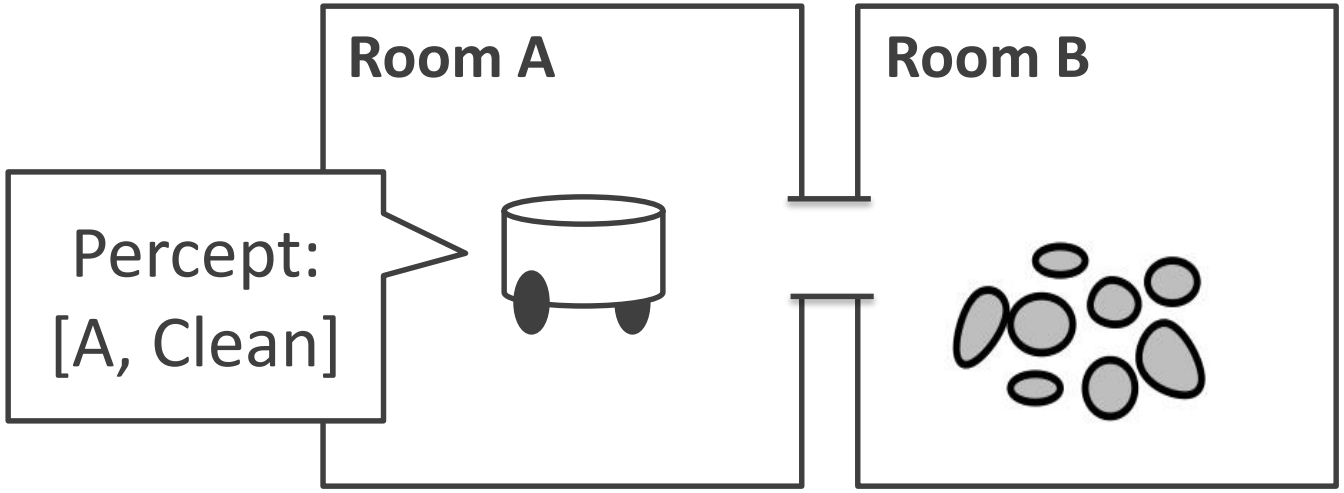
- Can be very simple, can be very complex
- What are the percepts for a self-driving car?
  - < *can go left, can go straight, can go right* >
  - Lane position
  - (above) + distance to lead car
  - (above) + all cars within distance D
  - (above) + road/signal information (e.g., traffic lights)
  - (above) + geospatial navigation information
  - (above) + current/predicted weather
  - … etc.

Example: Vacuum Cleaner



- Percepts
  - Location sensor (Room A or B?)
  - Dirt sensor (Clean or dirty?)
- Actions
  - Left, Right, Suck, Wait

Example: Vacuum Cleaner



- Percepts
  - Location sensor (Room A or B?)
  - Dirt sensor (Clean or Dirty?)
- Actions
  - Left, Right, Suck, Wait

Example: Vacuum Cleaner

Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
⋮	⋮

Example: Vacuum Cleaner

Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
⋮	⋮

Questions:

- Can it be implemented as a computer program?
- If so, how? – what would the function look like?

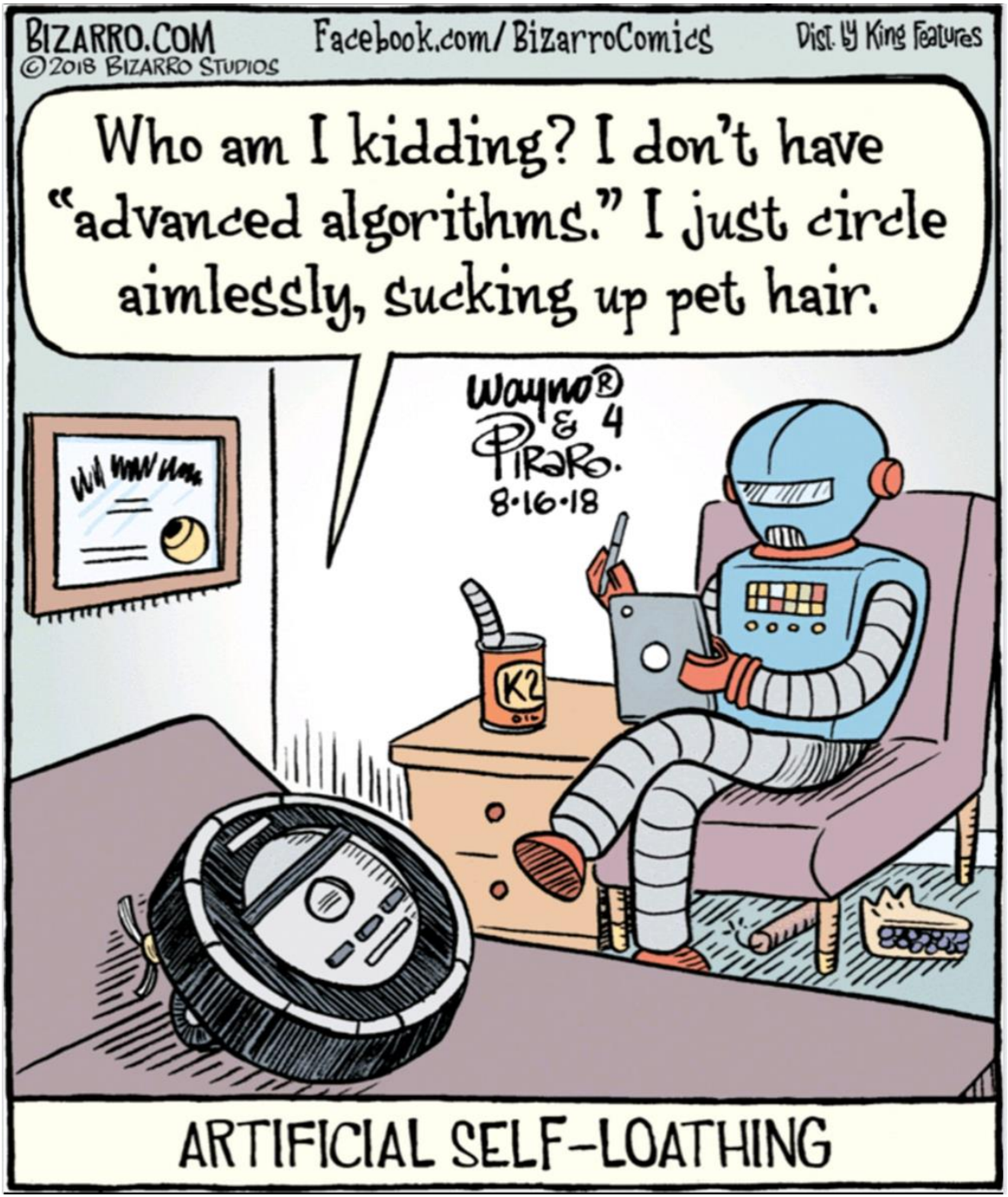
Example: Vacuum Cleaner

Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
⋮	⋮

function REFLEX-VACUUM-AGENT([*location*,*status*]) **returns** an action

if *status* = *Dirty* **then return** *Suck*
else if *location* = *A* **then return** *Right*
else if *location* = *B* **then return** *Left*

BIZARRO.COM
©2018 BIZARRO STUDIOS
Facebook.com/BizarroComics
Dist. by King Features



ARTIFICIAL SELF-LOATHING

Rationality

- As we discussed last time, “intelligence” is an ill-defined concept.
- Let us assume a performance measure that is given (and usually external) to the agent
  - For example:
    - $1 per room cleaned
    - $10 per room cleaned minus $1 per movement
    - $10 per room cleaned minus $3 per movement
    - etc.
- Rational Agent:
  - Chooses actions that maximize the expected value of the performance measure given the current percept sequence.

Rationality

- Imagine two agents A and B whose performance measure is defined by how much $$ they make at the casino.
  - A and B play one game of blackjack
  - A plays the move that probabilistically has the highest chance of winning, but loses
  - B plays a different, riskier move, and ends up winning
- Which agent is rational?

Rationality

- Imagine two agents A and B whose performance measure is defined by how much $$ they make at the casino.
  - A and B play one game of blackjack
  - A plays the move that probabilistically has the highest chance of winning, but loses
  - B plays a different, riskier move, and ends up winning
- Which agent is rational?
- A is (probably) that rational agent
  - Rationality does not imply clairvoyance
  - Rationality does not imply omniscience
  - Thus, rational does not necessarily mean more successful
- (PS: When is the riskier move the more rational choice?)

How Do We Design a Rational Agent?

- To design a rational agent, we first need to define its environment, performance measure, etc.
- PEAS:
  - Performance
  - Environment
  - Actions
  - Sensors (percepts)

Example (informal): Automated Taxi

- **Performance**:
- **Environment**:
- **Actions**:
- **Sensors**:

Example (informal): Automated Taxi

- **Performance**:
- **Environment**: ??
- **Actions**:
- **Sensors**:

Example (informal): Automated Taxi

- **Performance**:
- **Environment**: streets, pedestrians, cars, weather, traffic lights/signs, …
- **Actions**:
- **Sensors**:

Example (informal): Automated Taxi

- **Performance**:
- **Environment** : streets, pedestrians, cars, weather, traffic lights/signs, …
- **Actions**:
- **Sensors**: ??

Example (informal): Automated Taxi

- **Performance**:
- **Environment** : streets, pedestrians, cars, weather, traffic lights/signs, …
- **Actions**:
- **Sensors**: GPS, video cameras, SONAR/LiDAR, accelerometers, microphones, keyboard, …

Example (informal): Automated Taxi

- **Performance**:
- **Environment** : streets, pedestrians, cars, weather, traffic lights/signs, …
- **Actions**: ??
- **Sensors**: GPS, video cameras, SONAR/LiDAR, accelerometers, microphones, keyboard, …

Example (informal): Automated Taxi

- **Performance**:
- **Environment** : streets, pedestrians, cars, weather, traffic lights/signs, …
- **Actions**: steering, throttle/brake, horn, display, …
- **Sensors**: GPS, video cameras, SONAR/LiDAR, accelerometers, microphones, keyboard, …

Example (informal): Automated Taxi

- **Performance**: ??
- **Environment** : streets, pedestrians, cars, weather, traffic lights/signs, …
- **Actions**: steering, throttle/brake, horn, display, …
- **Sensors**: GPS, video cameras, SONAR/LiDAR, accelerometers, microphones, keyboard, …

Example (informal): Automated Taxi

- **Performance**: profit, safety, legality, …
- **Environment** : streets, pedestrians, cars, weather, traffic lights/signs, …
- **Actions**: steering, throttle/brake, horn, display, …
- **Sensors**: GPS, video cameras, SONAR/LiDAR, accelerometers, microphones, keyboard, …

WHY ASIMOV PUT THE THREE LAWS OF ROBOTICS IN THE ORDER HE DID:

POSSIBLE ORDERING	CONSEQUENCES	
1. (1) DON'T HARM HUMANS 2. (2) OBEY ORDERS 3. (3) PROTECT YOURSELF	[SEE ASIMOV'S STORIES]	BALANCED WORLD
1. (1) DON'T HARM HUMANS 2. (3) PROTECT YOURSELF 3. (2) OBEY ORDERS	EXPLORE MARS! HAHA, NO. IT'S COLD AND I'D DIE.	FRUSTRATING WORLD
1. (2) OBEY ORDERS 2. (1) DON'T HARM HUMANS 3. (3) PROTECT YOURSELF		KILLBOT HELLSCAPE
1. (2) OBEY ORDERS 2. (3) PROTECT YOURSELF 3. (1) DON'T HARM HUMANS		KILLBOT HELLSCAPE
1. (3) PROTECT YOURSELF 2. (1) DON'T HARM HUMANS 3. (2) OBEY ORDERS	I'LL MAKE CARS FOR YOU, BUT TRY TO UNPLUG ME AND I'LL VAPORIZE YOU.	TERRIFYING STANDOFF
1. (3) PROTECT YOURSELF 2. (2) OBEY ORDERS 3. (1) DON'T HARM HUMANS		KILLBOT HELLSCAPE

Example: Chess Agent

- **Performance**:
- **Environment**: ??
- **Actions**:
- **Sensors**:

Example: Chess Agent

- **Performance**:
- **Environment**: Chess board, opponents, rules
- **Actions**:
- **Sensors**:



Example: Chess Agent

- **Performance**:
- **Environment**: Chess board, opponents, rules
- **Actions**:
- **Sensors**: ??

Example: Chess Agent

- **Performance**:
- **Environment**: Chess board, opponents, rules
- **Actions**:
- **Sensors**: Board perceived as a 8x8 matrix. Each element in the matrix can take one of the following values: E, BP, WP, BB, WB, BK, WN, BR, WN, BQ, WK, BK, WK

Example: Chess Agent



Environment

WR	WN	WB	WK	WQ	WB	WN	WR
WP	WP	WP	WP	WP	WP	WP	WP
BP	BP	BP	BP	BP	BP	BP	BP
BR	BN	BB	BK	BQ	BB	BN	BR

Percept

Note: The environment might have lots of features
(piece size and material, opponent's facial expression, sunny vs. raining etc.)
… but for rational agents, we only need to build sensors for the information that is necessary to maximize the performance measure

Example: Chess Agent

- **Performance**:
- **Environment**: Chess board, opponents, rules
- **Actions**: ??
- **Sensors**: Board perceived as a 8x8 matrix. Each element in the matrix can take one of the following values: E, BP, WP, BB, WB, BK, WN, BR, WN, BQ, WK, BK, WK

Example: Chess Agent

- **Performance**:
- **Environment**: Chess board, opponents, rules
- **Actions**:
  - Pairs of coordinates: (x1, y1) → (x2, y2)
  - Specify the source piece and the target position (for castling, the king's move is specified)
- **Sensors**: Board perceived as a 8x8 matrix. Each element in the matrix can take one of the following values: E, BP, WP, BB, WB, BK, WN, BR, WN, BQ, WK, BK, WK

Example: Chess Agent

- **Performance**:
  - ??
- **Environment**: Chess board, opponents, rules
- **Actions**:
  - Pairs of coordinates: (x1, y1) → (x2, y2)
  - Specify the source piece and the target position (for castling, the king's move is specified)
- **Sensors**: Board perceived as a 8x8 matrix. Each element in the matrix can take one of the following values: E, BP, WP, BB, WB, BK, WN, BR, WN, BQ, WK, BK, WK

Example: Chess Agent

- **Performance**:
  - Once reached a terminal state (draw, win): -1 for losing, 0 for draw, 1 for winning
  - Or, more complex measures of how well/poorly it played
- **Environment**: Chess board, opponents, rules
- **Actions**:
  - Pairs of coordinates: (x1, y1) → (x2, y2)
  - Specify the source piece and the target position (for castling, the king's move is specified)
- **Sensors**: Board perceived as a 8x8 matrix. Each element in the matrix can take one of the following values: E, BP, WP, BB, WB, BK, WN, BR, WN, BQ, WK, BK, WK

Properties of Environments

- **(Fully) Observable:** The agent can sense the *entire* state of the environment (else: **Partially Observable**)
- **Deterministic**: The outcome of actions is determined by the state of the board and the action executed (else: **Nondeterministic** or **Stochastic**)
- **Sequential**: Action outcomes can be affected by prior actions (else: **Episodic**).
- **Static**: State of the world doesn't change during deliberation between actions (else: **Dynamic**)
- **Discrete**: Actions can only be performed in a small fixed number of ways (else: **Continuous**)
- **Single-Agent**: There is only one agent of change (else: **Multi-Agent**)

Environment Examples

- Solitaire (the card game)?
  - Fully Observable vs. Partially Observable
  - Deterministic vs. Nondeterministic
  - Sequential vs. Episodic
  - Static vs. Dynamic
  - Discrete vs. Continuous
  - Single-Agent vs. Multi-Agent



Environment Examples

- Solitaire (the card game)?
  - Fully Observable vs. **Partially Observable**
  - **Deterministic** vs. Nondeterministic
  - **Sequential** vs. Episodic
  - **Static** vs. Dynamic
  - **Discrete** vs. Continuous
  - **Single-Agent** vs. Multi-Agent



Environment Examples

- Chess?
  - Fully Observable vs. Partially Observable
  - Deterministic vs. Nondeterministic
  - Sequential vs. Episodic
  - Static vs. Dynamic
  - Discrete vs. Continuous
  - Single-Agent vs. Multi-Agent



Environment Examples

- Chess?
  - **Fully Observable** vs. Partially Observable
  - **Deterministic** vs. Nondeterministic
  - **Sequential** vs. Episodic
  - **Static** vs. Dynamic
  - **Discrete** vs. Continuous
  - Single-Agent vs. **Multi-Agent**



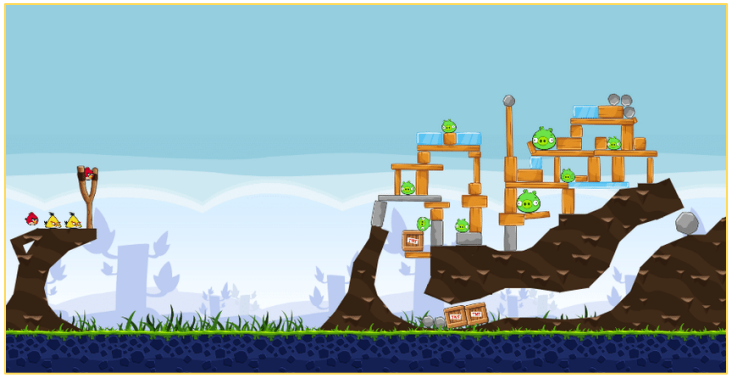
Environment Examples

- Backgammon?
  - **Fully Observable** vs. Partially Observable
  - Deterministic vs. **Nondeterministic**
  - **Sequential** vs. Episodic
  - **Static** vs. Dynamic
  - **Discrete** vs. Continuous
  - Single-Agent vs. **Multi-Agent**



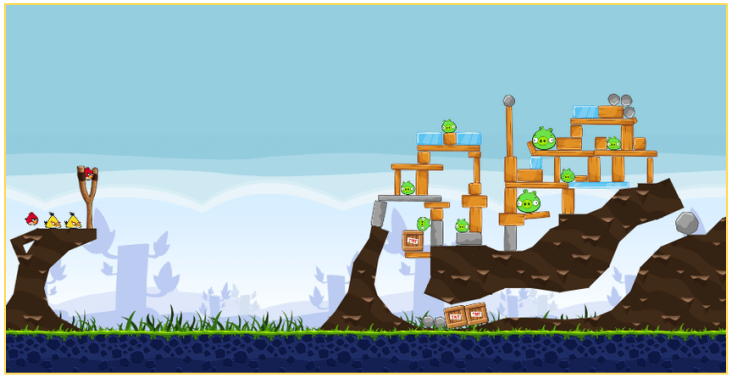
Environment Examples

- Angry Birds?
  - Fully Observable vs. Partially Observable
  - Deterministic vs. Nondeterministic
  - Sequential vs. Episodic
  - Static vs. Dynamic
  - Discrete vs. Continuous
  - Single-Agent vs. Multi-Agent



Environment Examples

- Angry Birds?
  - **Fully Observable*** vs. Partially Observable
  - **Deterministic*** vs. Nondeterministic
  - **Sequential** vs. Episodic
  - Static vs. **Dynamic**
  - Discrete vs. Continuous **??**
  - **Single-Agent** vs. Multi-Agent



Environment Examples

- Real World?
  - Fully Observable vs. Partially Observable
  - Deterministic vs. Nondeterministic
  - Sequential vs. Episodic
  - Static vs. Dynamic
  - Discrete vs. Continuous
  - Single-Agent vs. Multi-Agent

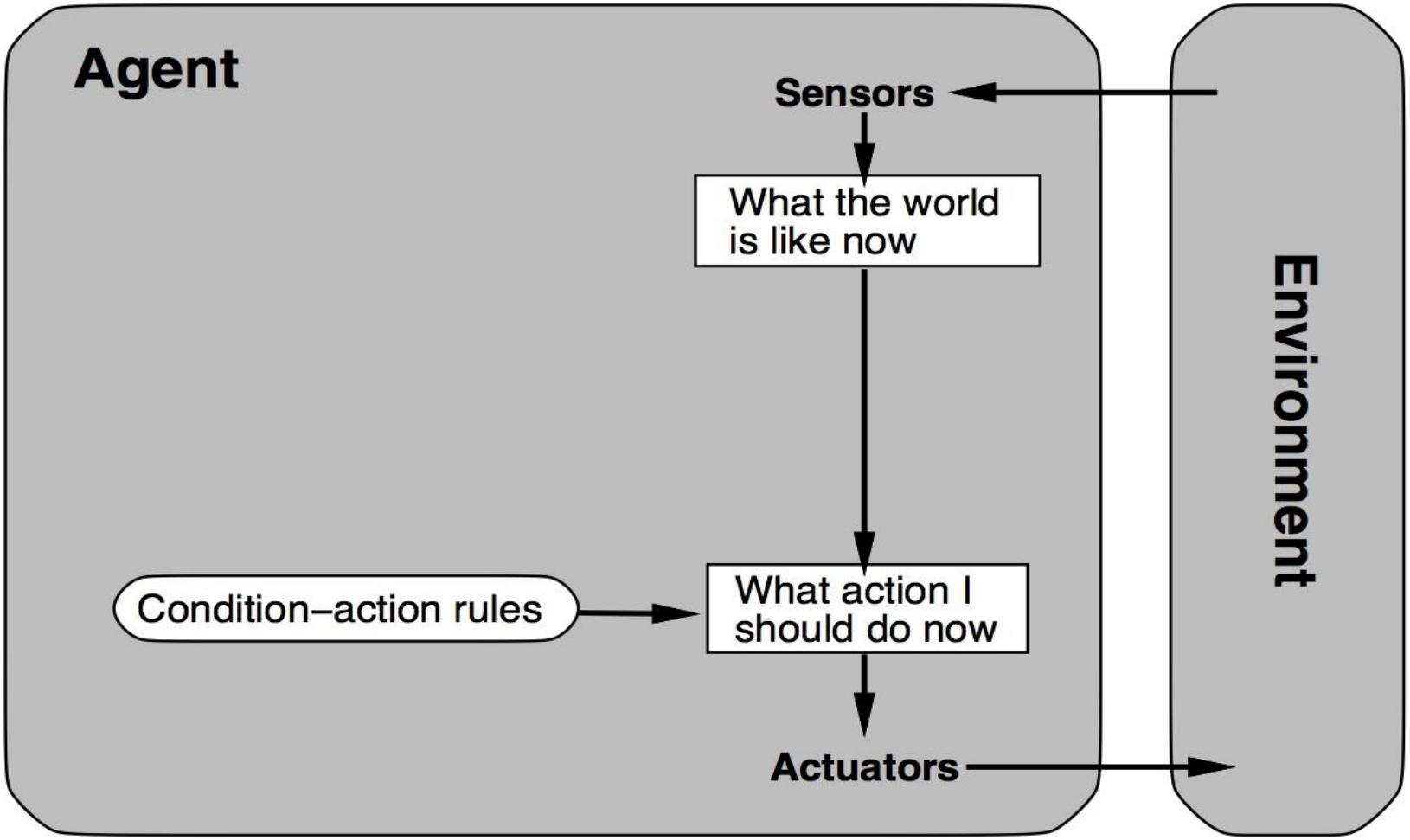
Environment Examples

- Real World?
  - Fully Observable vs. **Partially Observable**
  - Deterministic vs. **Nondeterministic**
  - **Sequential** vs. Episodic
  - Static vs. **Dynamic**
  - Discrete vs. **Continuous**
  - Single-Agent vs. **Multi-Agent**

Agent Types

- We will cover 4 basic types:
  - Reactive agents (reflex agents)
  - State-based agents
  - Goal-based agents
  - Utility-based agents
- They can all incorporate learning

Reactive Agents

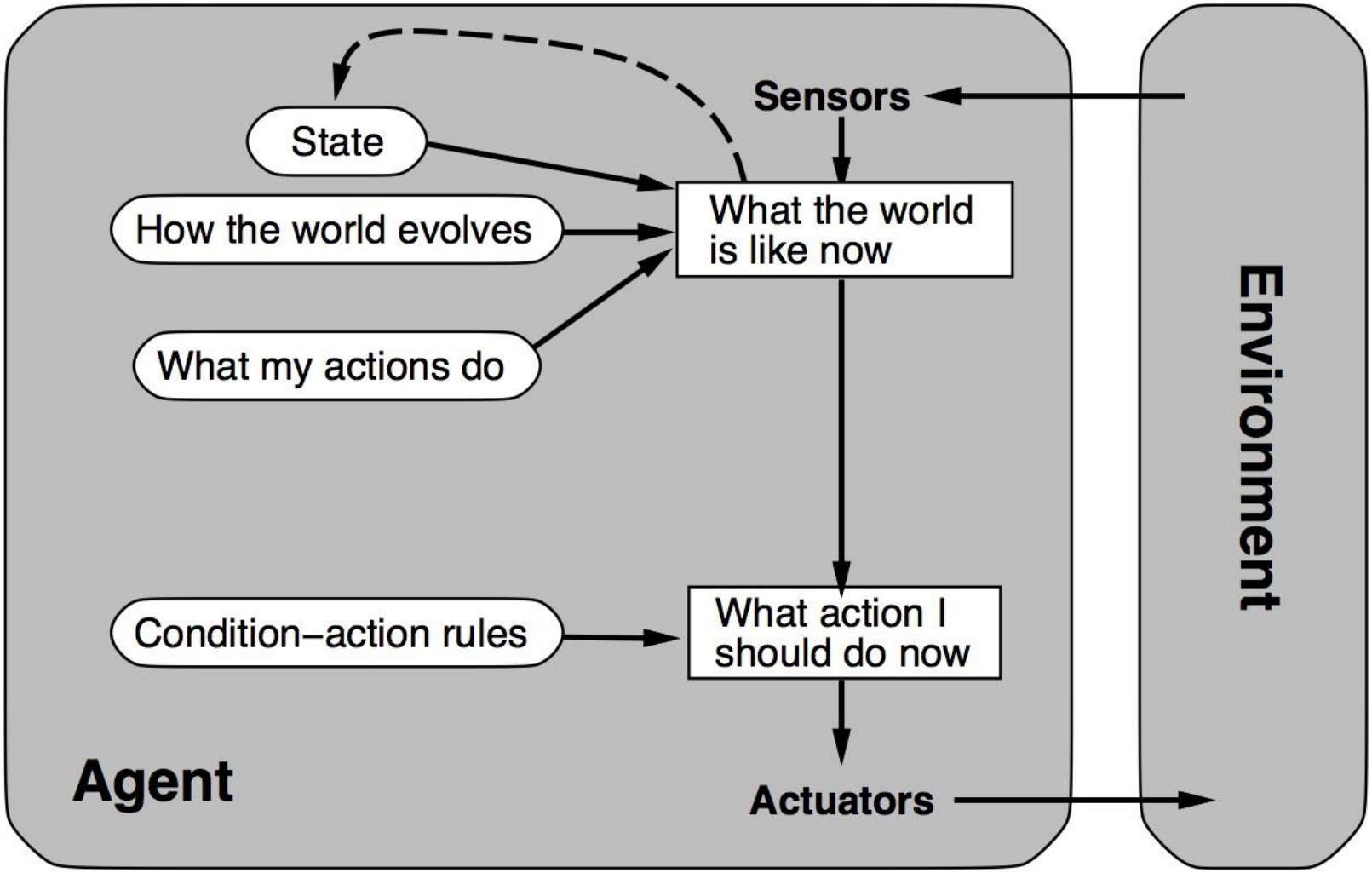


Reactive Agent

- Finds a rule whose condition matches the current situation, as defined by percepts, and then does the action associated with the rule.
- Agent has no memory.

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

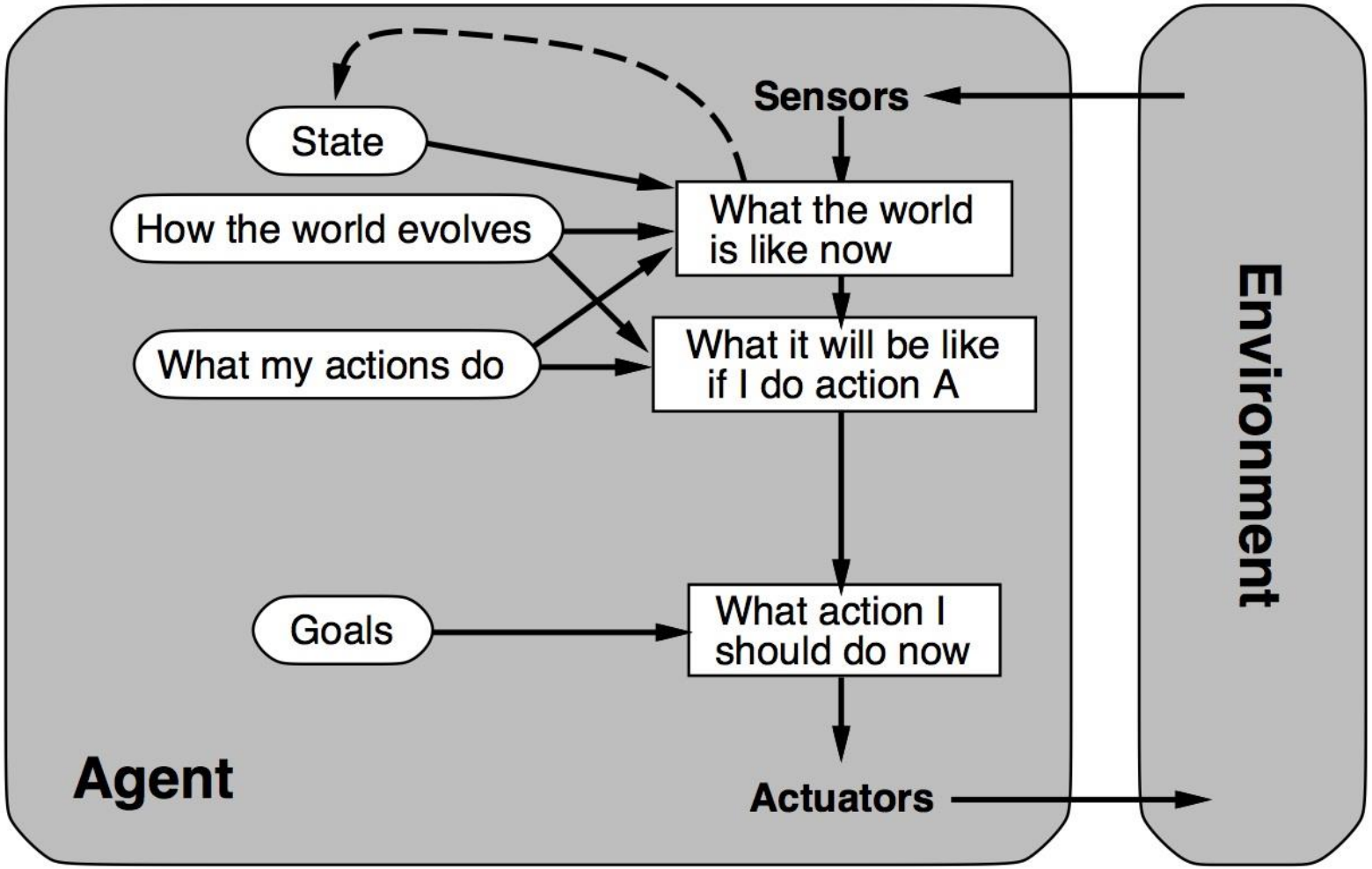
State-based Agents



State-based Agent

- Finds rule whose condition matches current situation, as defined by percepts and stored internal state, and does action associated with the rule.
- Internal state acts as a memory.

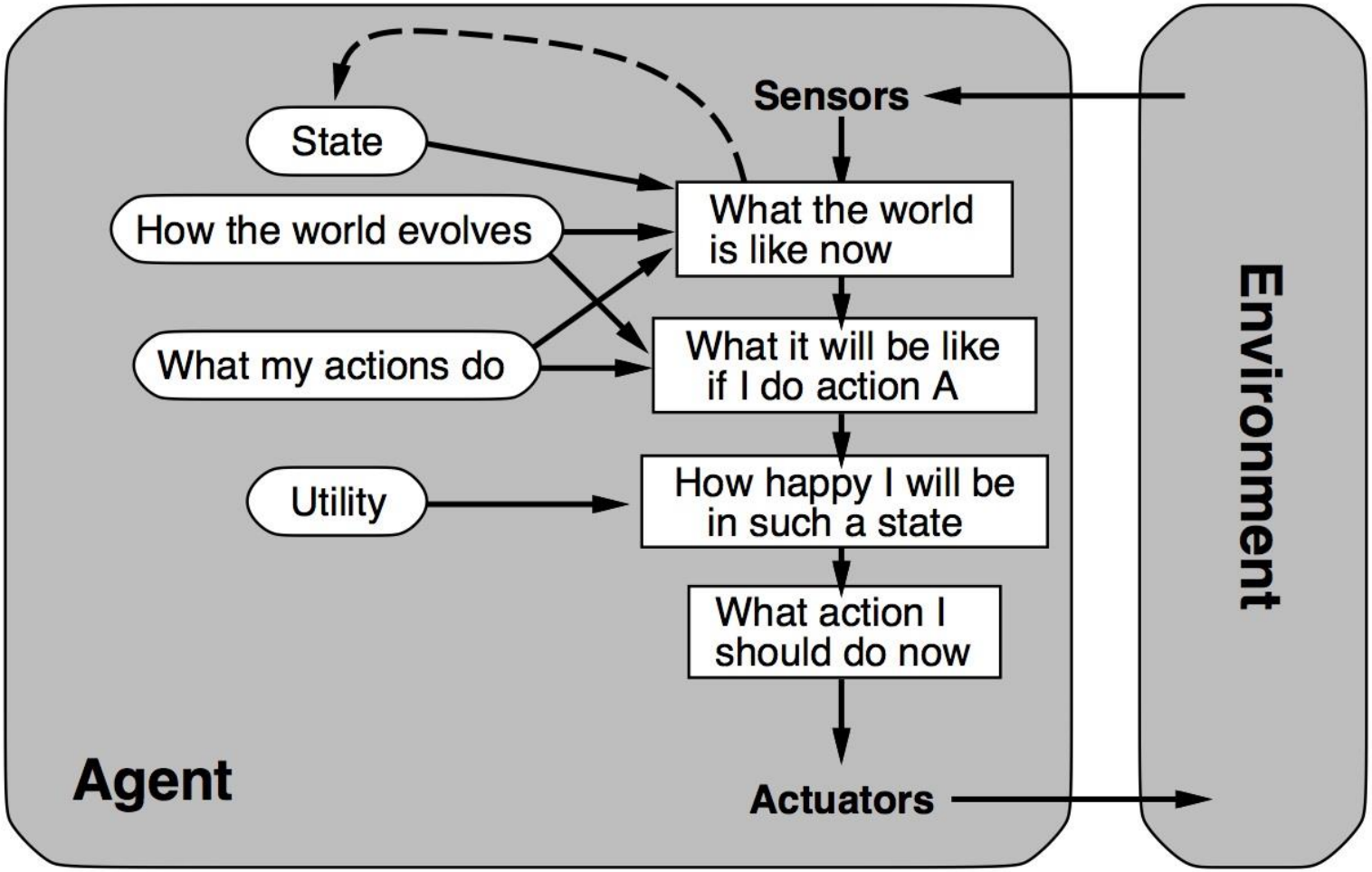
Goal-based Agents



Goal-based Agent

- Has explicit goal and when choosing an action, will select an action that achieves that goal.
- Requires some planning ("what will happen if I do action A?").

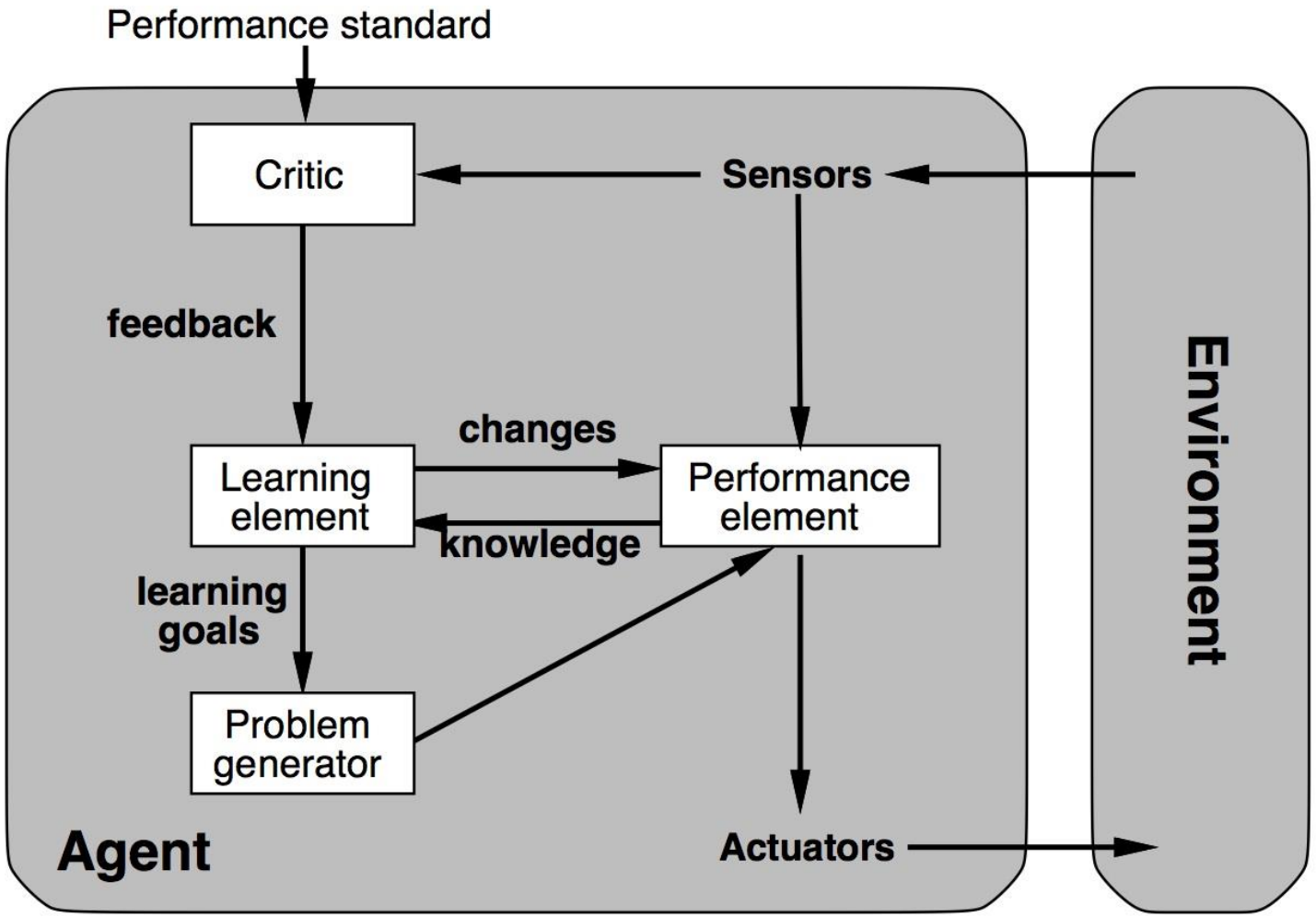
Utility-based Agents



Utility-based Agent

- Has utility function that maps a state onto real number that describes how well agent is performing.
- Not only achieves goals, but maximizes some utility measure.

Learning Agents



Summary

- Agents interact with environments through actuators and sensors
- The agent function maps percepts to actions
- A rational agent chooses the actions that maximize the performance measure
- PEAS describes components of rational agent design
- Environments can be categorized along several dimensions:
  - Observable? Deterministic? Sequential? Static? Discrete? Single-agent?
- Some basic agent architectures:
  - Reactive, State-based, Goal-based, Utility-based

Looking Ahead…

- What we have covered so far…
  - What is AI?
  - What is a Rational Agent?
- In the next few lectures, we will cover…
  - How can rational agents solve problems?
  - How can rational agents represent knowledge?
  - How can rational agents learn?

CS 380: ARTIFICIAL INTELLIGENCE

Lecture 3: Problem Solving & Uninformed Search

Problem Solving

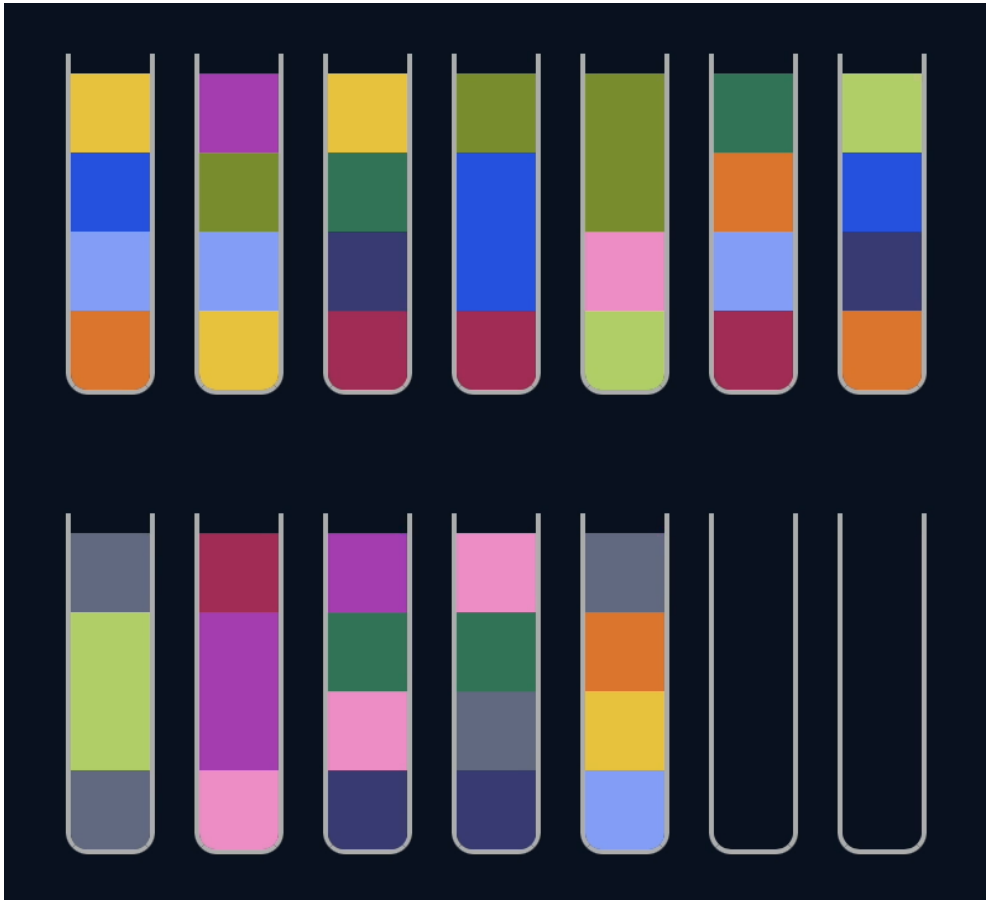
- What is a “problem”?
  - In the context of AI: a question the requires an answer
- Many types of problems
  - **Decision Problems**
  - **Search Problems**
  - **Counting Problems**
  - **Optimization Problems**

Problem Solving

- There are many problems for which we know specialized ways of solving.
- For example:
  - Problem: Find the roots of the polynomial $ax^2 + bx + c = 0$
  - We have mechanical procedures to solve this problem in an exact way
  - However, those procedures can only be applied to this problem, not any other
- Problem Solving in AI:
  - Finding general procedures to solve general classes of problems
  - Are there algorithms that can be used to solve any computationally solvable problem?
    - We humans seem to be able to!

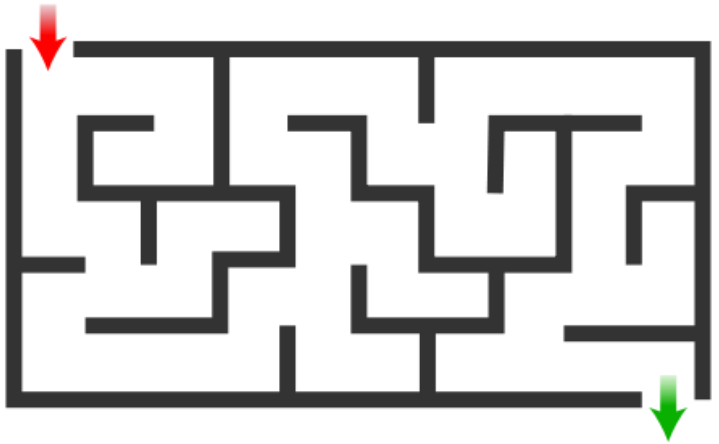
Problem Solving

- Example: Beakers / Water Jugs Puzzle
  - Pour from one to another but only on top of same color (or empty)
  - Continue until beakers are filled with only 1 color



Example: Maze

- A robot is at the entrance of the maze.
- Problem: How does the robot get to the exit?
- Step 1: Formulate goal
  - Be in the exit
- Step 2: Formulate problem
  - States, actions, etc.
- Step 3: Find solution
  - Specific sequence of actions



Problem Formulation

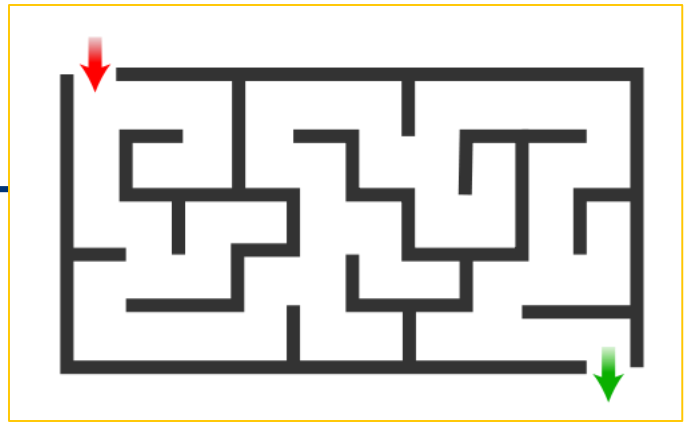
- Initial state: S_0
  - Initial configuration of the problem (e.g. starting position in a maze)
- Actions: ***A***
  - The different ways in which the agent can change the state (e.g. moving to an adjacent position in the maze)
- Goal condition: ***G***
  - A function that determines whether a state reached by a given sequence of actions is a solution to the problem or not
- Cost function: ***c***
  - A function that assigns a numeric cost to a given sequence of actions

Problem Formulation

- Initial state: S_0
  - Initial configuratio
    maze)
- Actions: ***A***
  - The different ways
    (e.g. moving to an
- Goal condition: ***G***
  - A function that determines whether a state reached by a given sequence of actions is a solution to the problem or not
- Cost function: ***c***
  - A function that assigns a numeric cost to a given sequence of actions

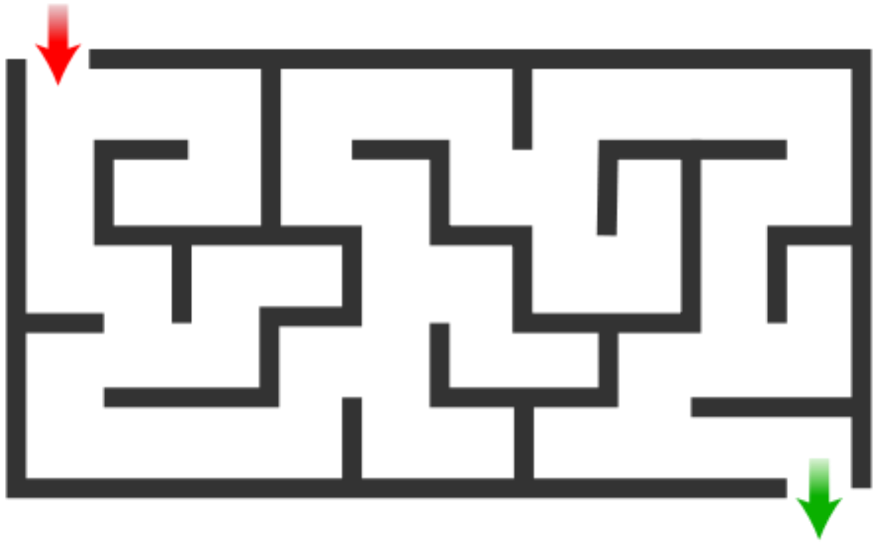
The initial state, together with the set of actions, define the State Space or Search Space — the set of all the possible states that are reachable from the initial state by some sequence of actions.

Problem Types



- Deterministic, fully observable:
  - Solution: [down, down, down, right, down, …]
- Non-deterministic or partially observable:
  - Agent might not know its exact coordinates, must rely on percepts to tell them about the walls around them
  - Solution, policy (mappings from percepts, or states, to actions):
    - If no wall on left, then turn left
    - If wall on left, no wall ahead, then advance
    - If wall on left, wall ahead, then turn right
  - For example: the “follow the right/left wall” strategy

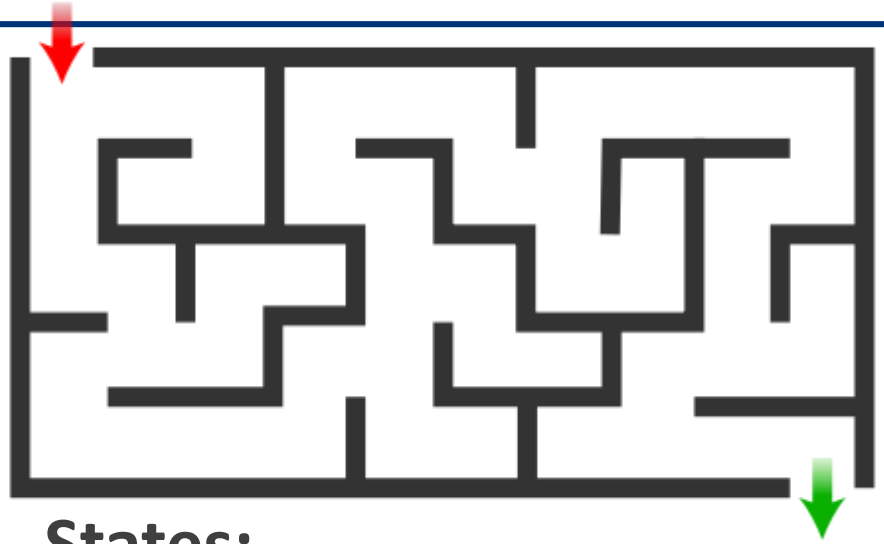
Example: Maze



- **States:**
- **Actions:**
- **Goal:**
- **Cost:**

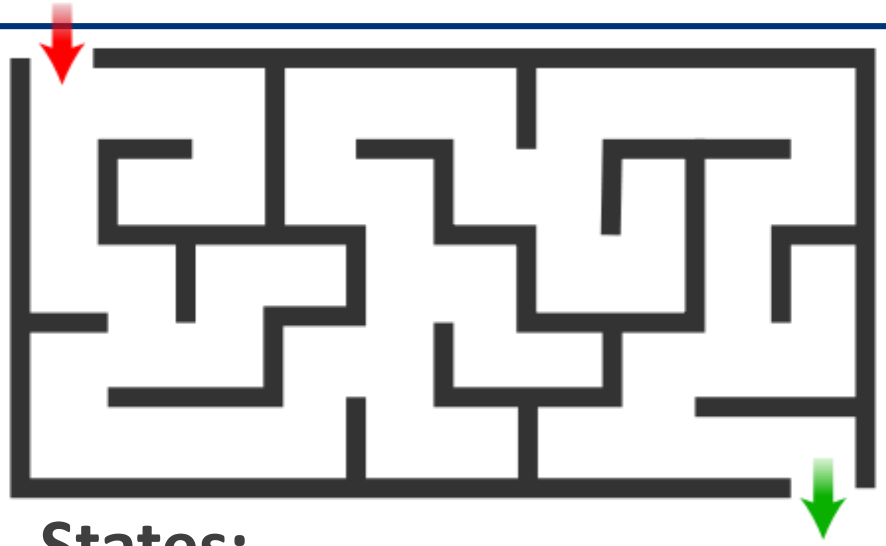
Assuming we have a robot with a direction and partial information…

Example: Maze



- **States:**
  - ((x,y), facing direction)
  - Initial state: ((0,0), down)
- **Actions:** advance, left, right
- **Goal:** reach state ((9,4), *)
- **Cost:** 1 per action executed

Example: Maze

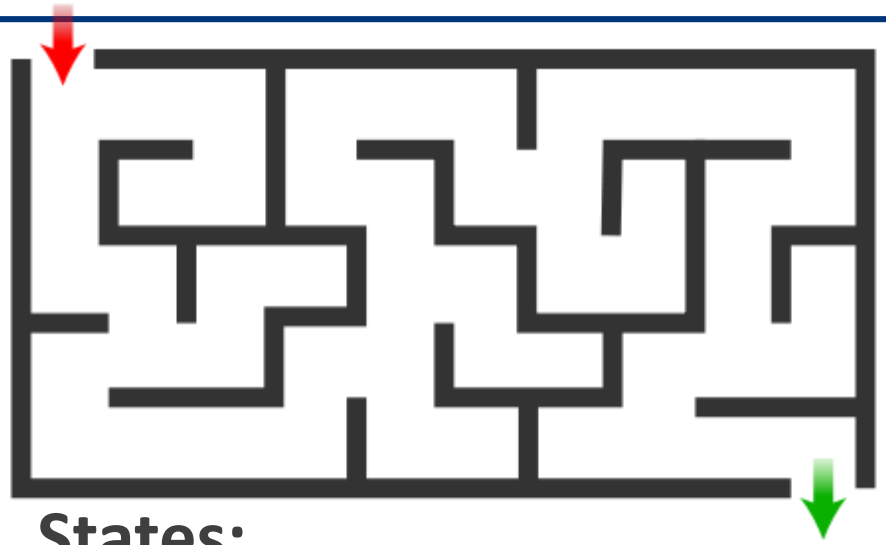


- **States:**
  - ((x,y), facing direction)
  - Initial state: ((0,0), down)
- **Actions:** advance, left, right
- **Goal:** reach state ((9,4), *)
- **Cost:** 1 per action executed

How difficult is this problem?

Let's say, compared to a larger maze…
or chess…

Example: Maze



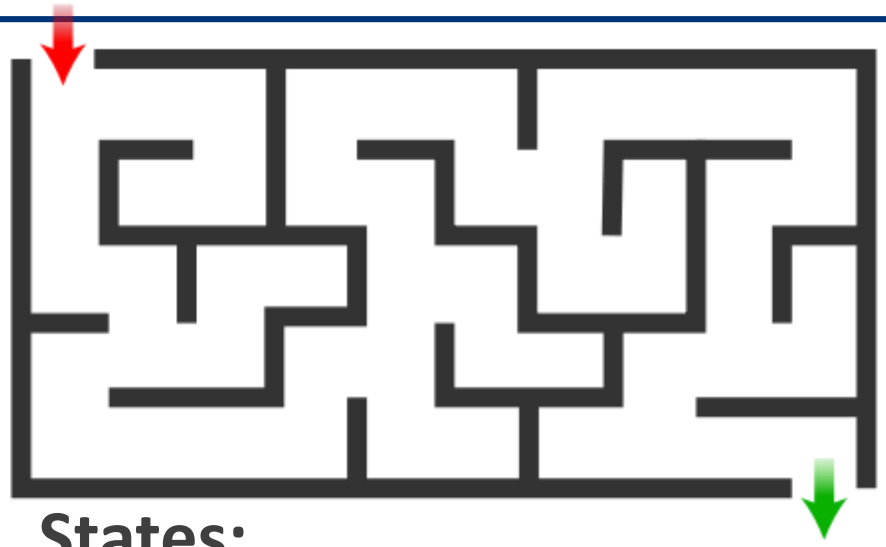
- **States:**
  - ((x,y), facing direction)
  - Initial state: ((0,0), down)
- **Actions:** advance, left, right
- **Goal:** reach state ((9,4), *)
- **Cost:** 1 per action executed

How difficult is this problem?

A typical measure of "difficulty" is the size of the state space: if it's small, it's very easy to find the solution.

How many possible states does this maze have? (for one agent finding its way around…)

Example: Maze



- **States:**
  - ((x,y), facing direction)
  - Initial state: ((0,0), down)
- **Actions:** advance, left, right
- **Goal:** reach state ((9,4), *)
- **Cost:** 1 per action executed

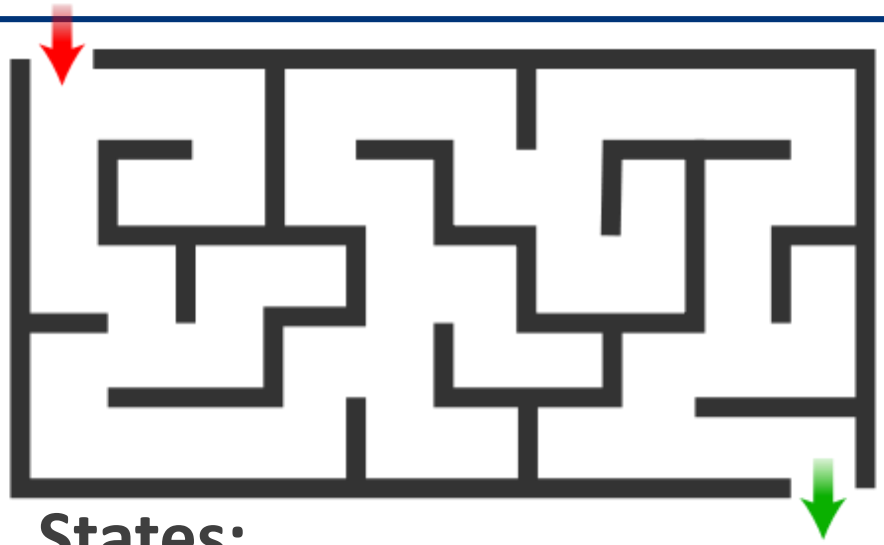
How difficult is this problem?

A typical measure of "difficulty" is the size of the state space: if it's small, it's very easy to find the solution.

For this maze, the state space has 200 states (50 positions * 4 directions)

How about chess?

Example: Maze



- **States:**
  - ((x,y), facing direction)
  - Initial state: ((0,0), down)
- **Actions:** advance, left, right
- **Goal:** reach state ((9,4), *)
- **Cost:** 1 per action executed

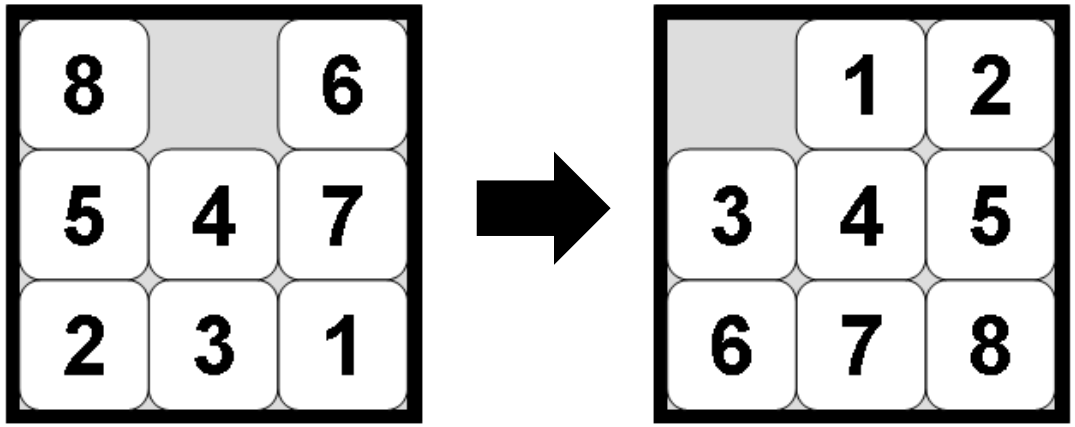
How difficult is this problem?

A typical measure of "difficulty" is the size of the state space: if it's small, it's very easy to find the solution.

For this maze, the state space has 200 states (50 positions * 4 directions)

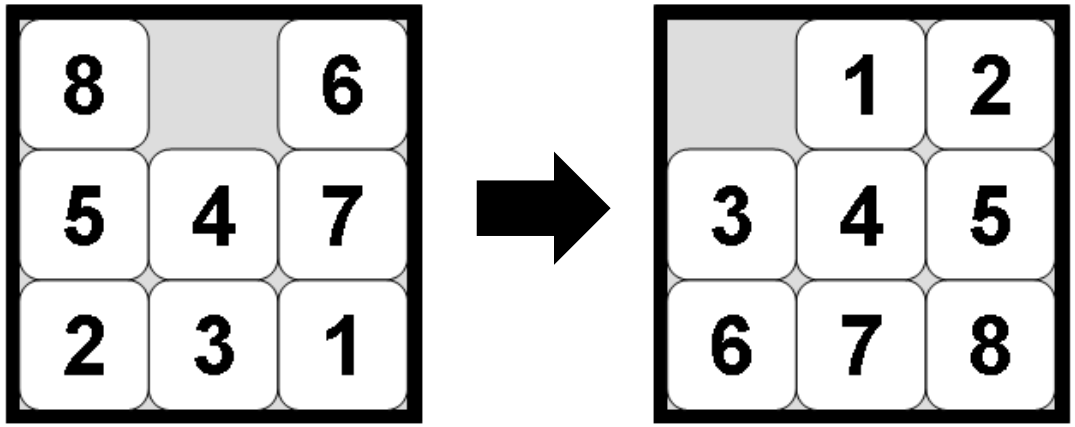
Chess has about 10^{50} states

Example: 8-puzzle



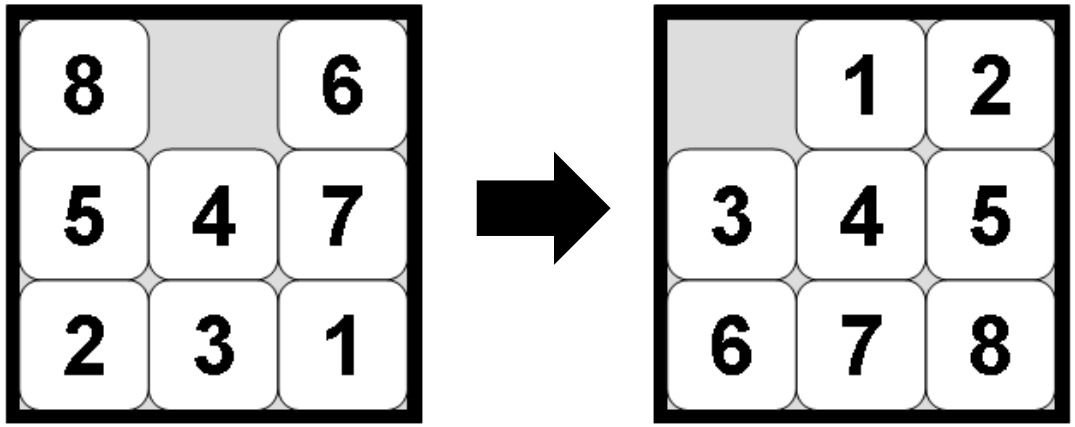
- **States:**
- **Actions:**
- **Goal:**
- **Cost:**

Example: 8-puzzle



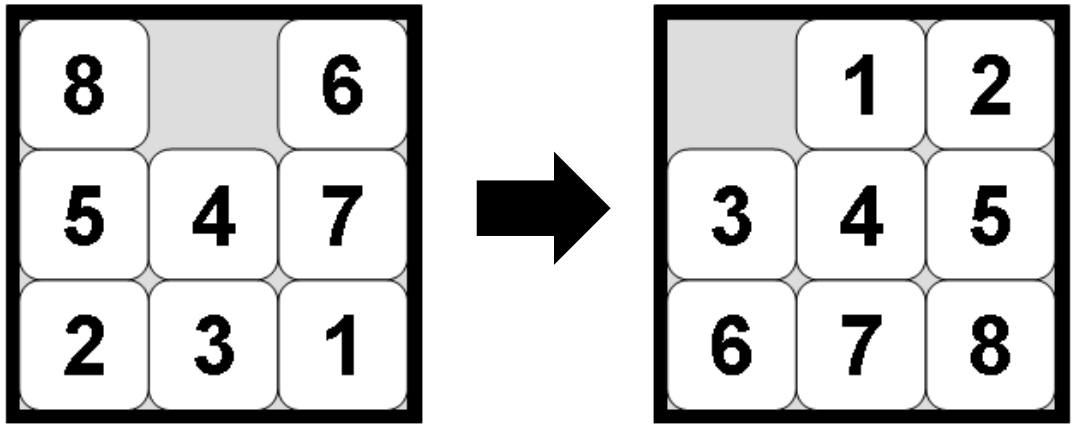
- **States:**
  - Board configuration, represented as a vector (1d or 2d) of 9 positions, where each position represents one cell on the board
  - Initial state: <8,blank,6,5,4,7,2,3,1>
- **Actions:**
- **Goal:**
- **Cost:**

Example: 8-puzzle



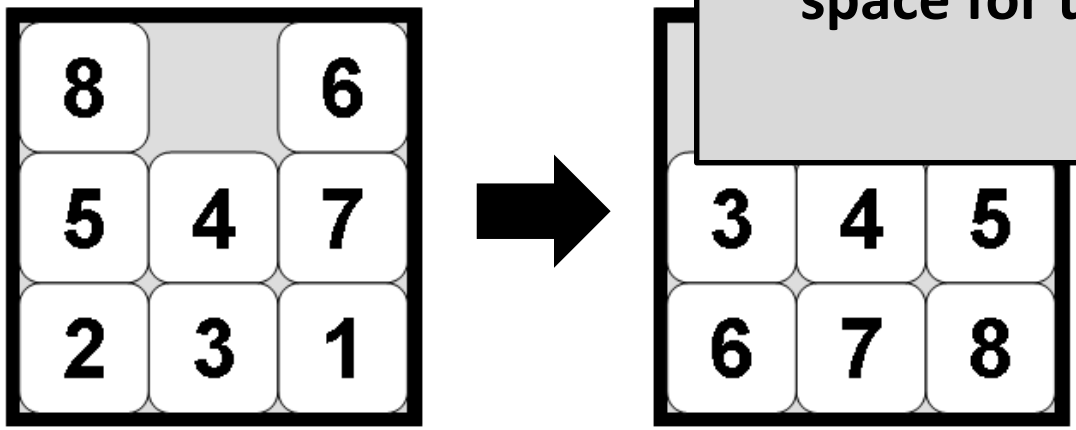
- **States:**
  - Board configuration, represented as a vector (1d or 2d) of 9 positions, where each position represents one cell on the board
  - Initial state: <8,blank,6,5,4,7,2,3,1>
- **Actions:** "move" the blank up, down, left, right
- **Goal:**
- **Cost:**

Example: 8-puzzle



- **States:**
  - Board configuration, represented as a vector (1d or 2d) of 9 positions, where each position represents one cell on the board
  - Initial state: <8,blank,6,5,4,7,2,3,1>
- **Actions:** "move" the blank up, down, left, right
- **Goal:** Reach state <blank,1,2,3,4,5,6,7,8>
- **Cost:** 1 per move

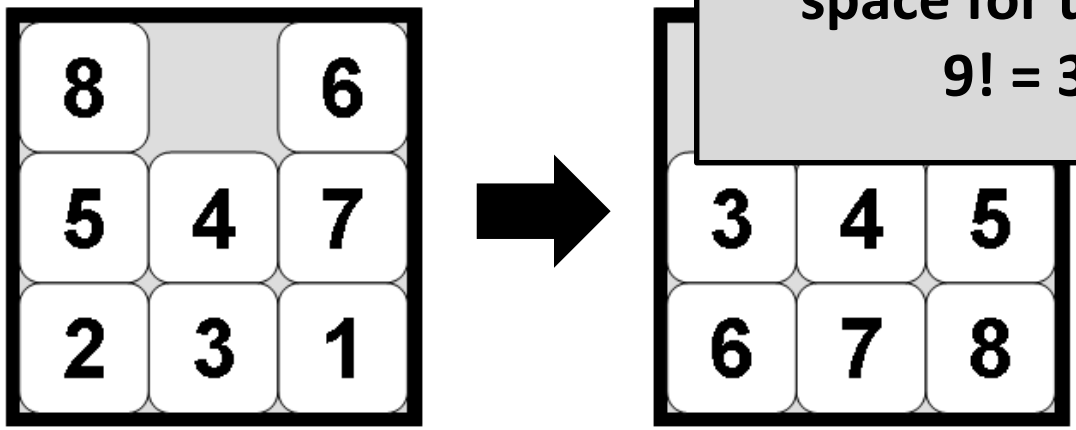
Example: 8-puzzle



What's the size of the state space for the 8-puzzle?

- **States:**
  - Board configuration, represented as a vector (1d or 2d) of 9 positions, where each position represents one cell on the board
  - Initial state: <8,blank,6,5,4,7,2,3,1>
- **Actions:** "move" the blank up, down, left, right
- **Goal:** Reach state <blank,1,2,3,4,5,6,7,8>
- **Cost:** 1 per move

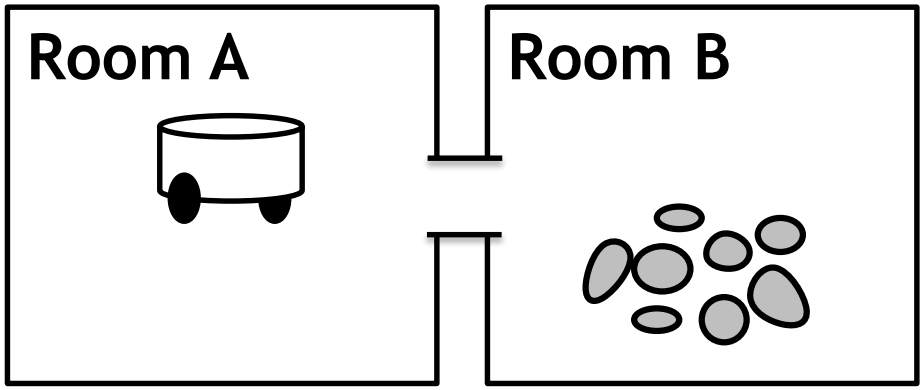
Example: 8-puzzle



What's the size of the state space for the 8-puzzle?
9! = 362,880

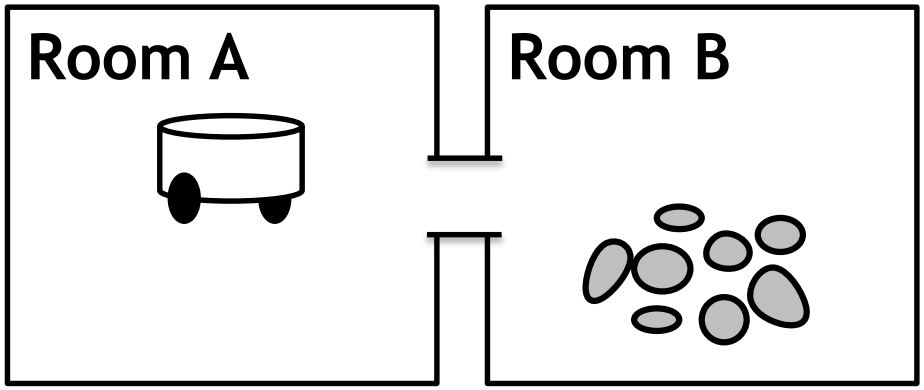
- **States:**
  - Board configuration, represented as a vector (1d or 2d) of 9 positions, where each position represents one cell on the board
  - Initial state: <8,blank,6,5,4,7,2,3,1>
- **Actions:** "move" the blank up, down, left, right
- **Goal:** Reach state <blank,1,2,3,4,5,6,7,8>
- **Cost:** 1 per move

Example: Vacuum World



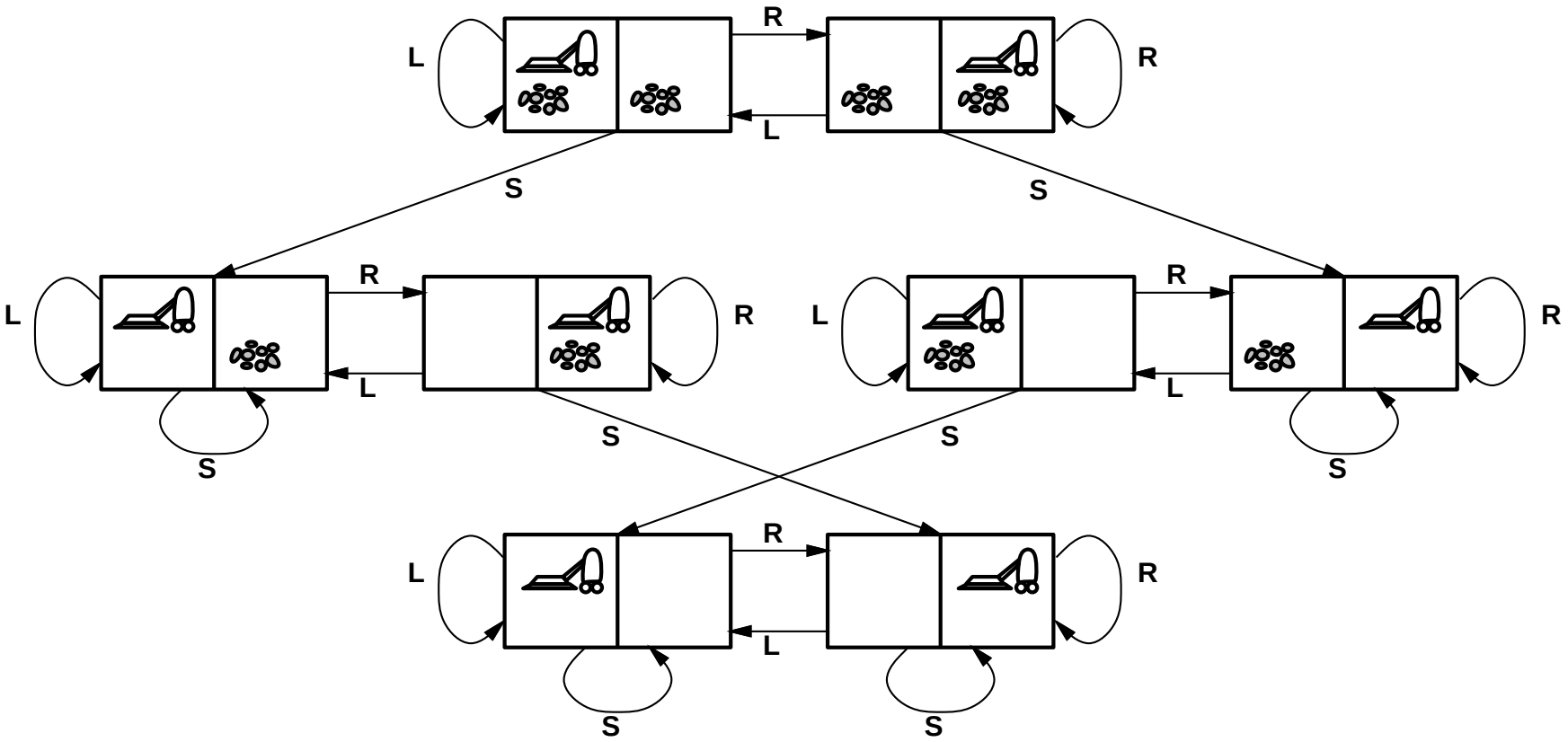
- States:
- Actions:
- Goal:
- Cost:

Example: Vacuum World

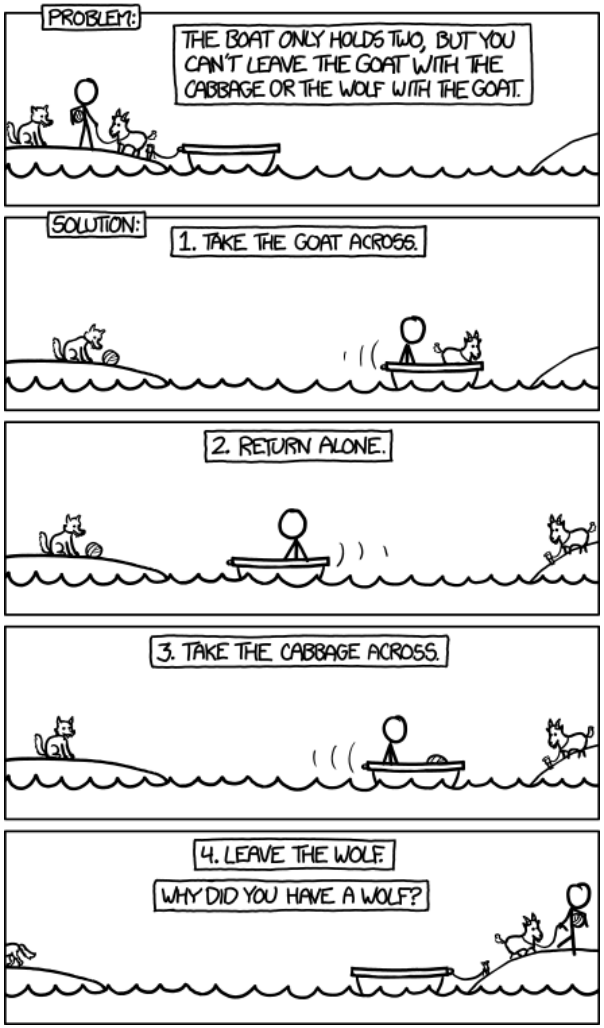


- **States:**
  - Position of robot + dirt in A? + dirt in B?
  - Initial state: (A, no, yes)
- **Actions:** Left, right, vacuum
- **Goal:** (-, no, no)
- **Cost:** 1 per move, 1 per vacuum (??)

Example: Vacuum World

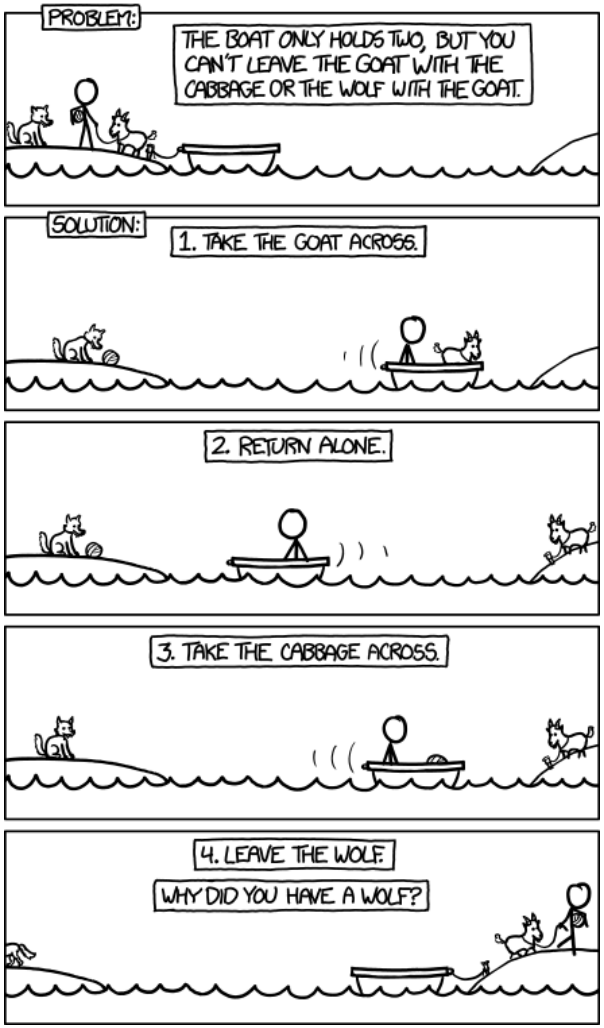


Example: Wolf, Goat, Cabbage



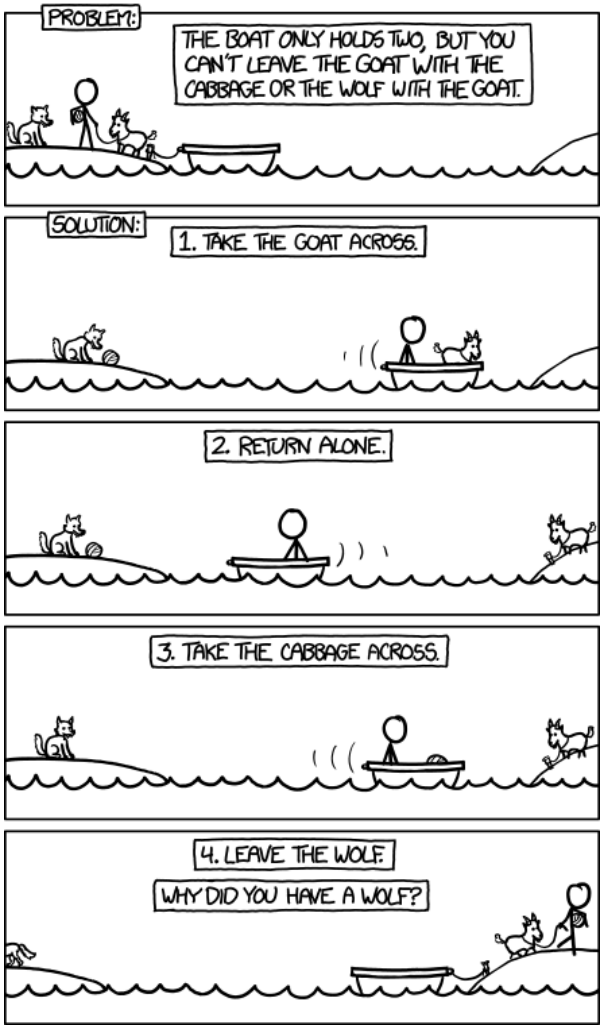
- **States:**
- **Actions:**
- **Goal:**
- **Cost:**

Example: Wolf, Goat, Cabbage



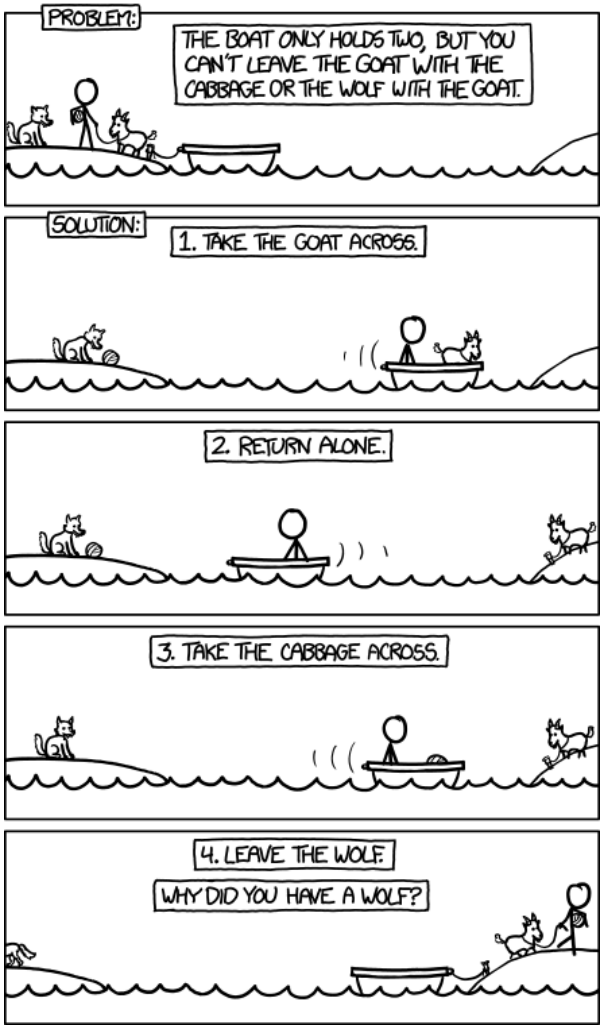
- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initial state: (left, left, left, left)
- **Actions:**
- **Goal:**
- **Cost:**

Example: Wolf, Goat, Cabbage



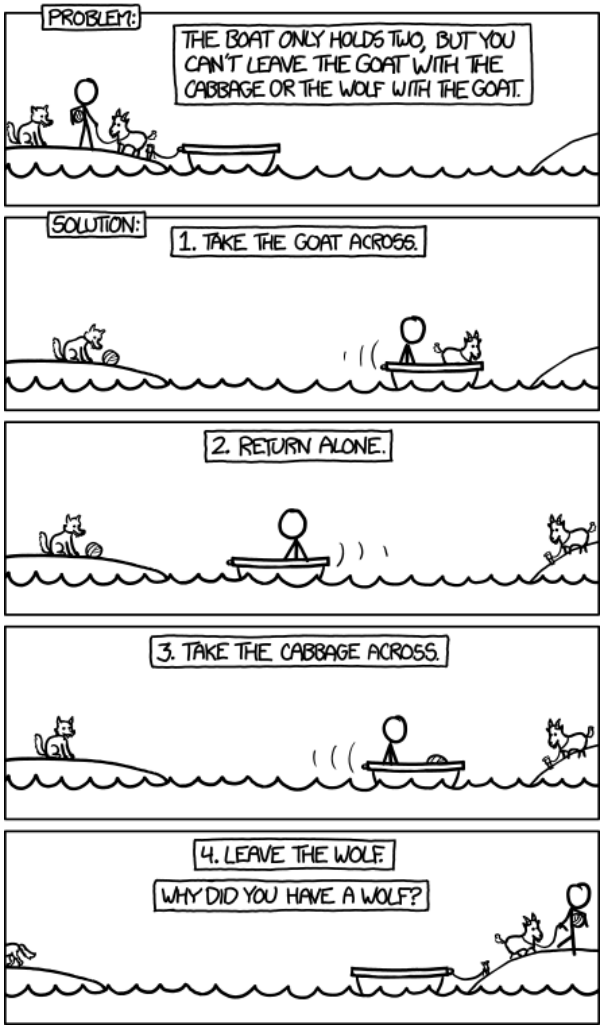
- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initial state: (left, left, left, left)
- **Actions:**
  - Boat left, boat right
  - Load wolf, unload wolf
  - Load goat, unload goat
  - Load cabbage, unload cabbage
  - (never leaving the wolf alone with goat or the goat alone with cabbage)
- **Goal:**
- **Cost:**

Example: Wolf, Goat, Cabbage



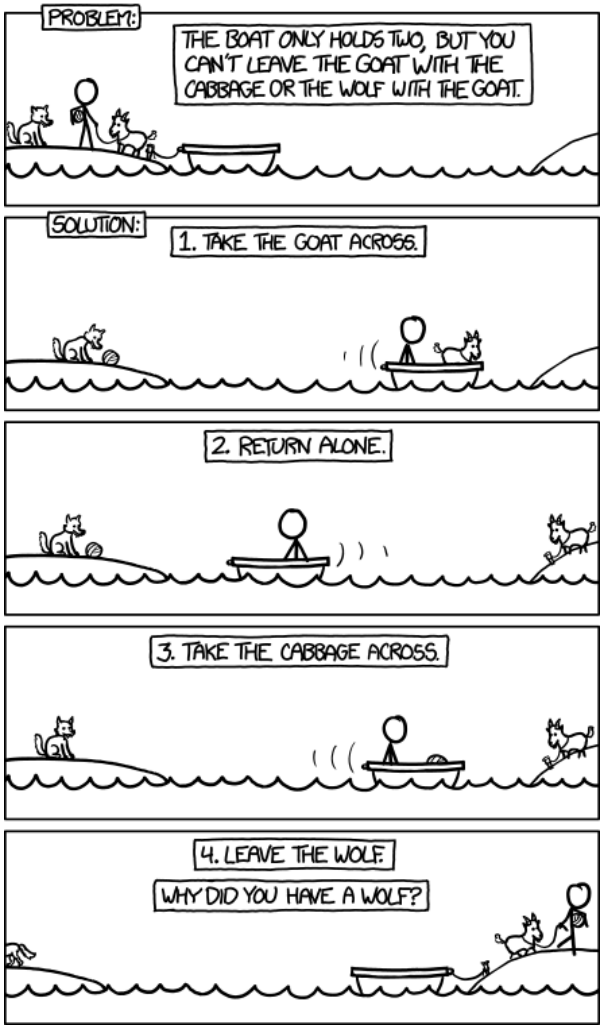
- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initial state: (left, left, left, left)
- **Actions:**
  - Boat left, boat right
  - Load wolf, unload wolf
  - Load goat, unload goat
  - Load cabbage, unload cabbage
  - (never leaving the wolf alone with goat or the goat alone with cabbage)
- **Goal:** (right, right, right, right)
- **Cost:**

Example: Wolf, Goat, Cabbage



- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initial state: (left, left, left, left)
- **Actions:**
  - Boat left, boat right
  - Load wolf, unload wolf
  - Load goat, unload goat
  - Load cabbage, unload cabbage
  - (never leaving the wolf alone with goat or the goat alone with cabbage)
- **Goal:** (right, right, right, right)
- **Cost:** 1 per action

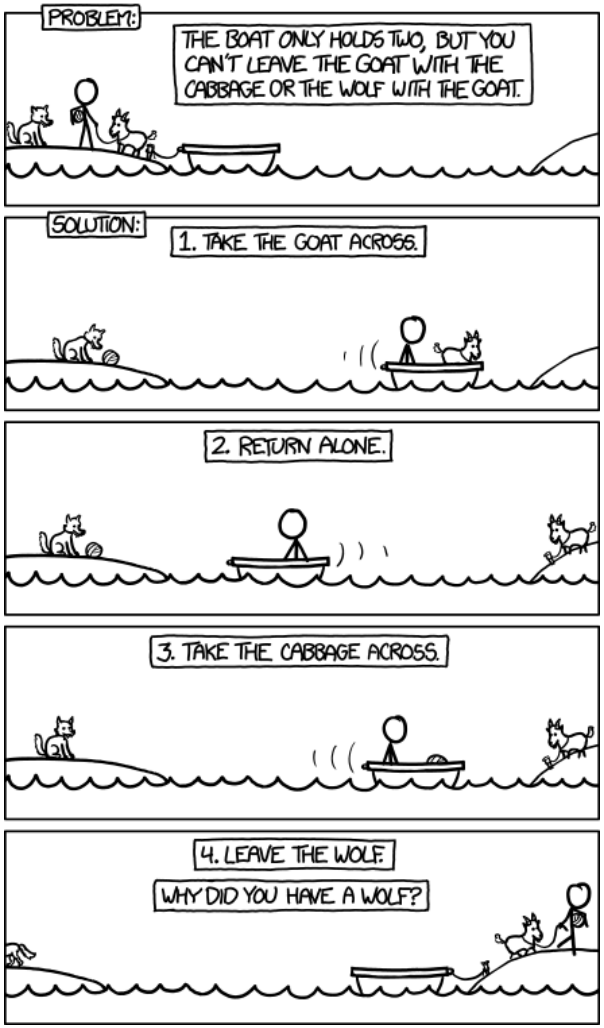
Example: Wolf, Goat, Cabbage



- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initial
- **Actions:**
  - Boat l
  - Load
  - Load goat, unload goat
  - Load cabbage, unload cabbage
  - (never leaving the wolf alone with goat or the goat alone with cabbage)
- **Goal:** (right, right, right, right)
- **Cost:** 1 per action

What's the size of the state space?

Example: Wolf, Goat, Cabbage



- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initia
- **Actions**
  - Boat
  - Load
  - Load
  - Load
  - (neve
    the g
- **Goal:** (r
- **Cost:** 1 per action

What's the size of the state space?

At most 3*3*3*2 = 54
(less when we apply the action constraints)

More Examples

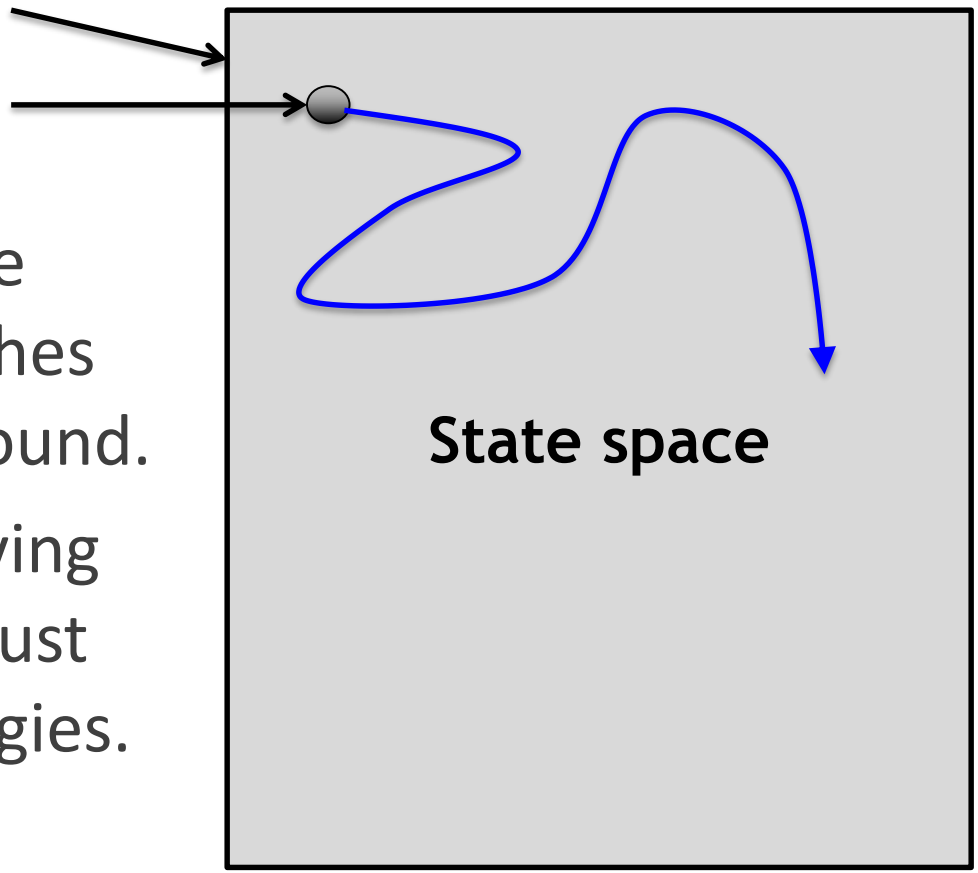
- Search Problems can be very general
- For example, we can formulate any of the following as a search problem:
  - Proving a theorem
  - Solving a physics problem
  - Finding your way around a city (path finding)
  - Playing individual sports (multiplayer sports and games belong to another category of problems)
  - Playing a computer game
  - “Where are my keys?”
  - Etc.

Solving Problems

- Let's only consider deterministic, fully-observable problems
- Simple algorithm:
  - Enumerate every possible sequence of actions:
    - [left]
    - [right]
    - [advance]
    - [left, left]
    - [left, right]
    - [left, advance]
    - …
  - Try one by one, until one reaches the solution

Solving Problems

- General Idea
  - Given the state space
  - Given the initial state
- Explore the state space until a state that matches the goal condition is found.
- Different problem-solving methods are actually just different search strategies.



Random Walk

- Algorithm:
  - Initial state: *S*
  - (1) Determine set of applicable actions *A* = validActions(*S*)
  - (2) Choose an action *a* in *A* at random
  - (3) Apply *a* to the current state: *S* = apply(*S*,*a*)
  - (4) If we have not reached the goal (*G*(*S*) == false), go to (1)
  - (5) Done
- Is this algorithm…
  - guaranteed to find a solution?
  - guaranteed to find the optimal solution? (i.e., shortest path or minimal cost)

Random Walk

- Algorithm:
  - Initial state: *S*
  - (1) Determine set of applicable actions *A* = validActions(*S*)
  - (2) Choose an action *a* in *A* at random
  - (3) Apply *a* to the current state: *S* = apply(*S*,*a*)
  - (4) If we have not reached the goal (*G*(*S*) == false), go to (1)
  - (5) Done
- Is this algorithm…
  - Is **not** guaranteed to find a solution?
  - Is **not** guaranteed to find the optimal solution? (i.e., shortest path or minimal cost)

Search

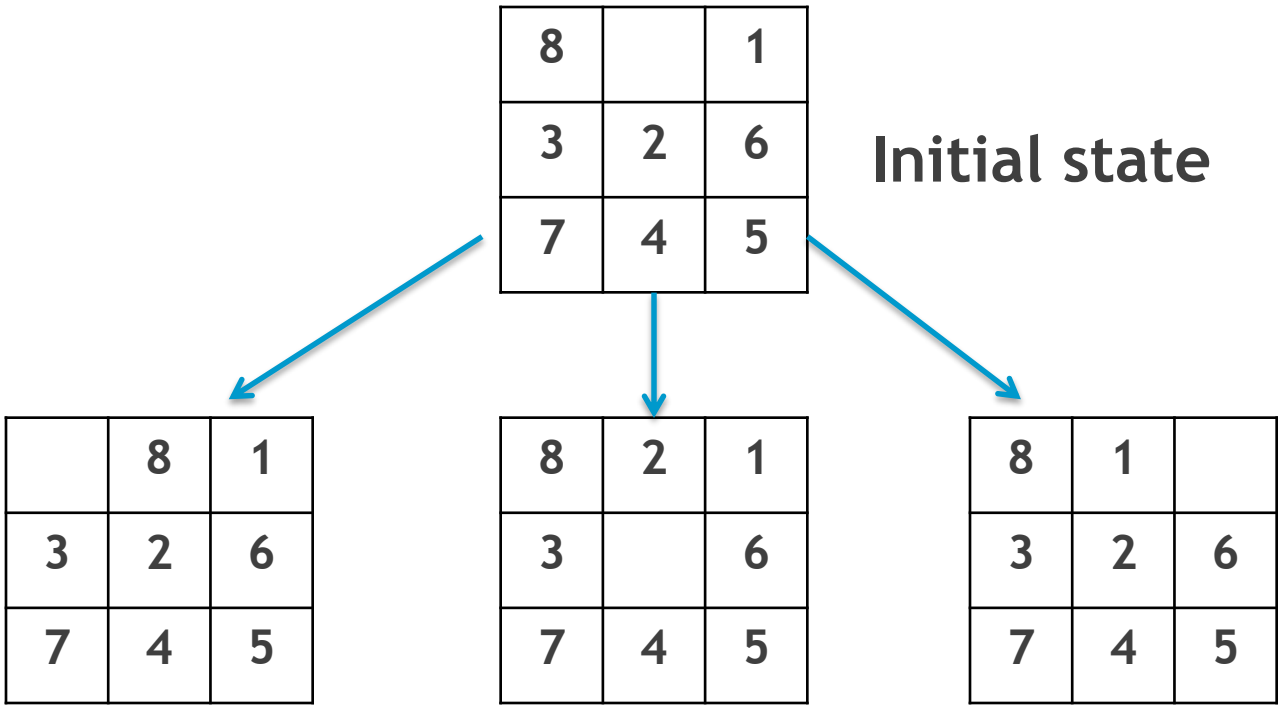
- Summary:
  - **Initial state**: initial configuration of the problem
  - **Actions**: set of actions the agent can perform to solve the problem
  - **State space**: set of possible states that are reachable from the initial space by any sequence of actions
- **Search**
  - Family of strategies to find solutions
  - Based on systematically exploring the state space until finding a sequence of actions that achieves the goal

Illustration

8		1
3	2	6
7	4	5

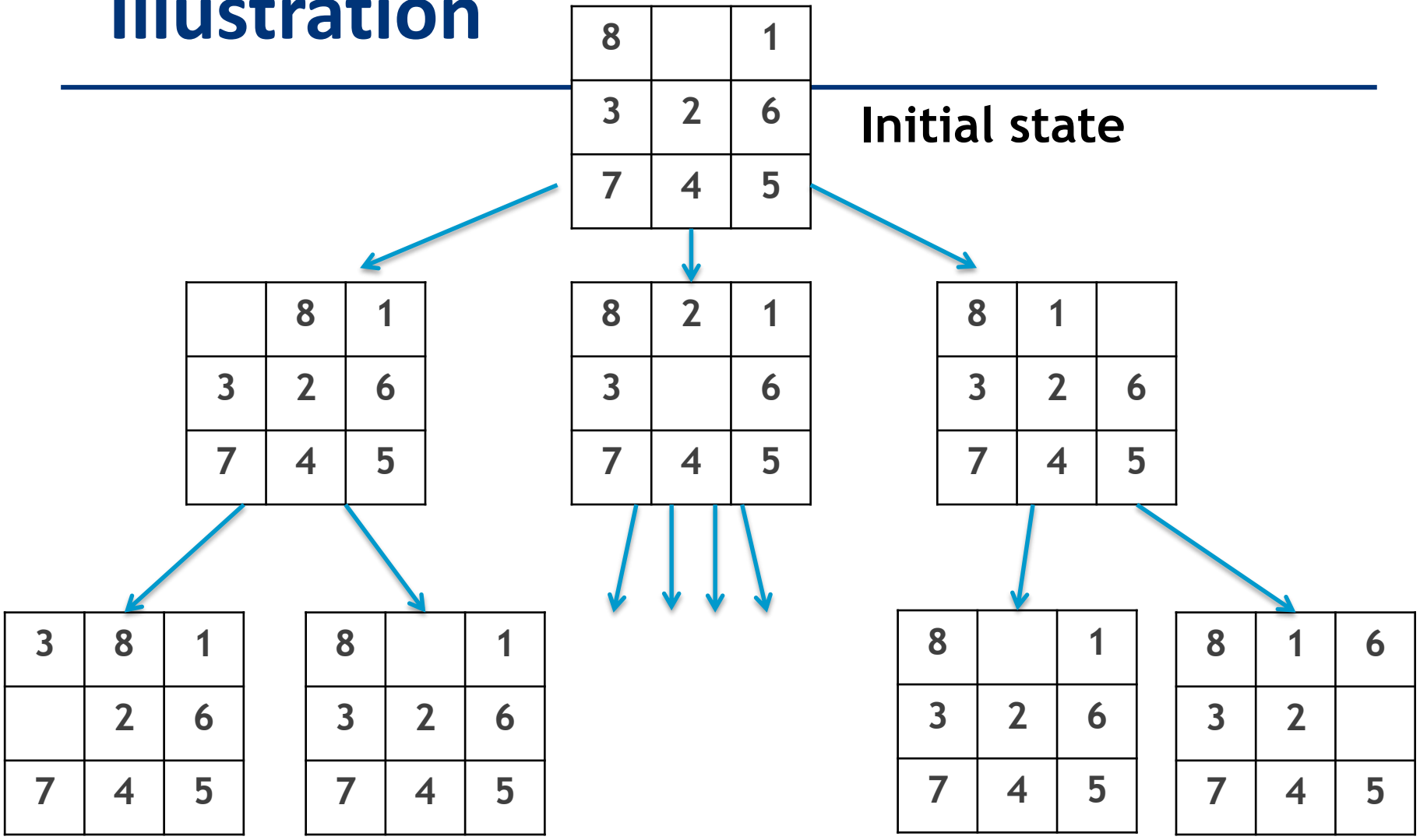
Initial state

Illustration



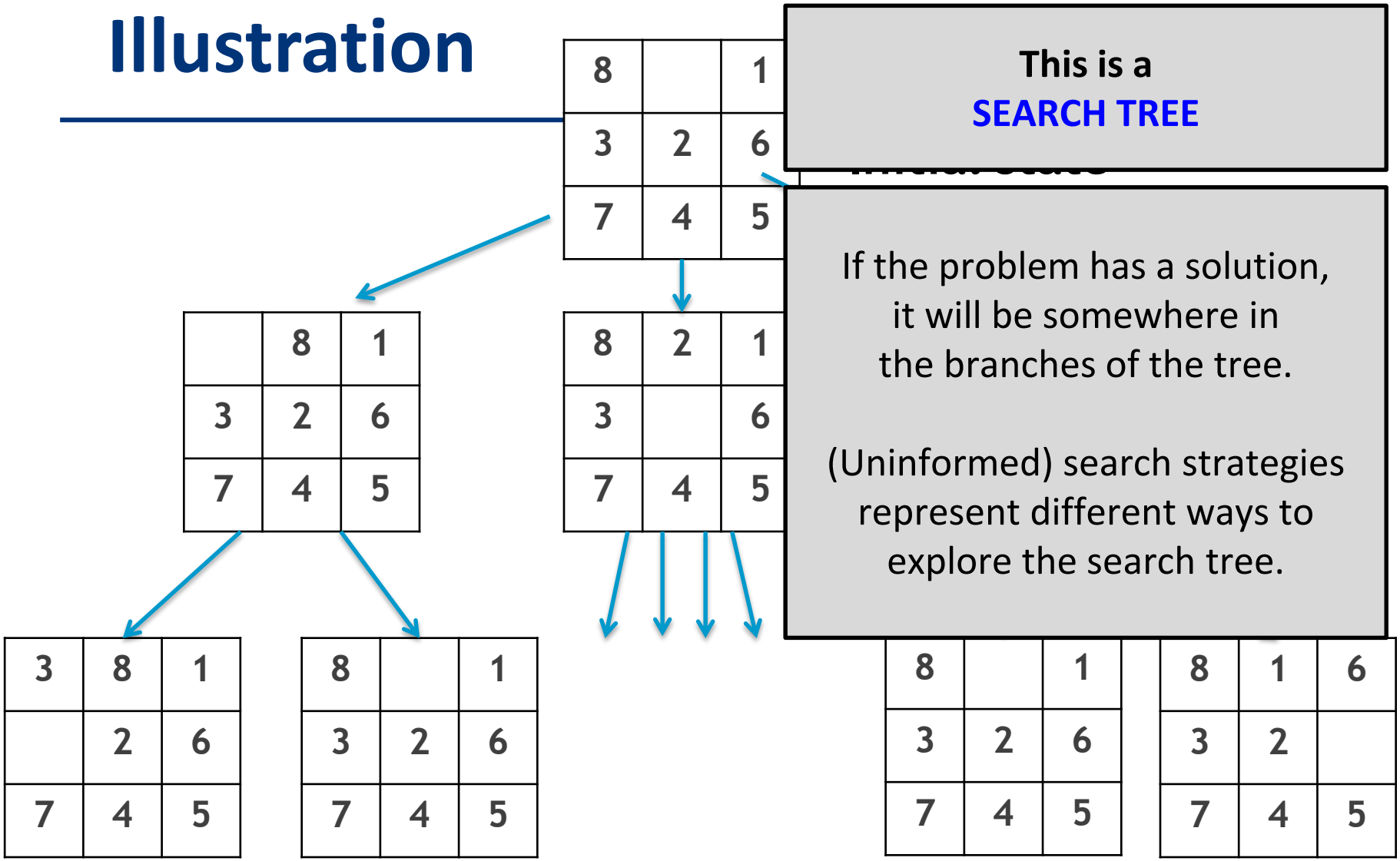
Set of states reachable by executing only one action

Illustration



Set of states reachable by executing two actions

Illustration

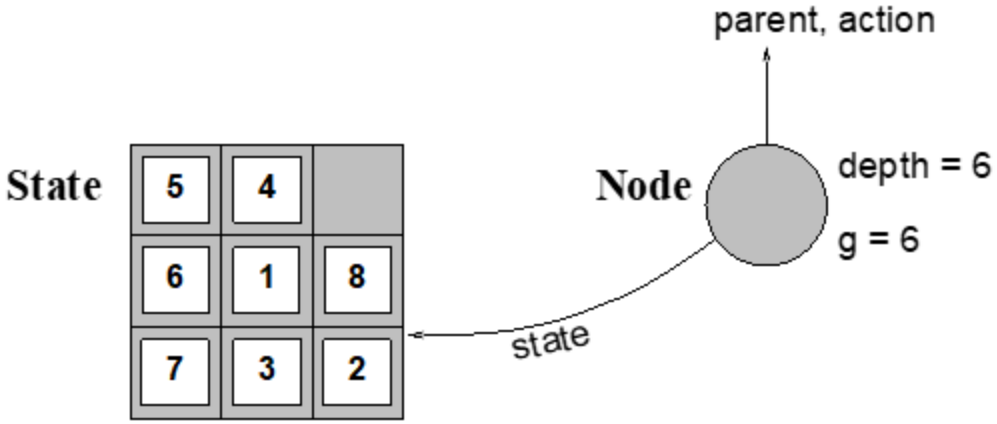


Set of states reachable by executing two actions

Implementation: states vs. nodes

- A state is a (representation of) a physical configuration
- A node is a data structure constituting part of a search tree that includes parent, children, depth, path cost g(x)

States do not have parents, children, depth, or path cost!



The Expand function creates new nodes, filling in the various fields and using the Successor function of the problem to create the corresponding states

Tree Search Algorithms

- Basic idea:
  - Offline, simulated exploration of the state space by generating successors of already-explored states (a.k.a. expanding states)

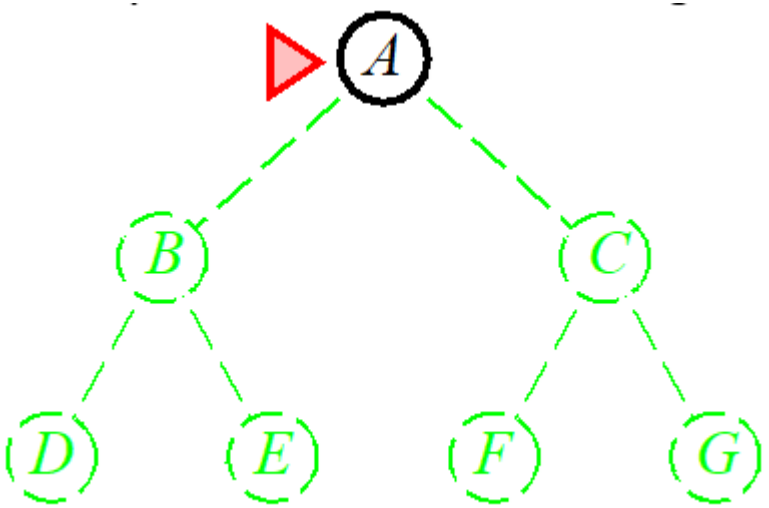
```
function Tree-Search(problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```

Uninformed Search Strategies

- Uninformed strategies use only the information available in the problem definition
  - Breadth-first Search
  - Uniform-cost Search
  - Depth-first Search
  - Depth-limited Search
  - Iterative Deepening Search

Breadth-first Search

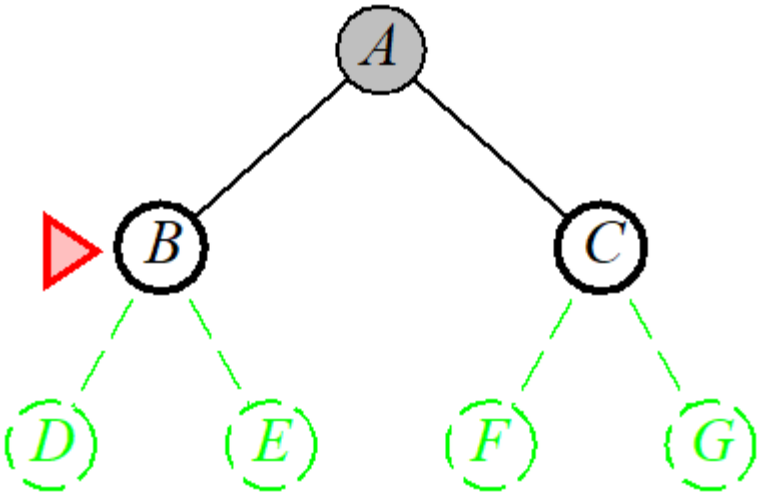
- Expand shallowest unexpanded node
- Implementation: *fringe* is a First In, First Out (FIFO) queue, i.e., new successors go at the end



[A]

Breadth-first Search

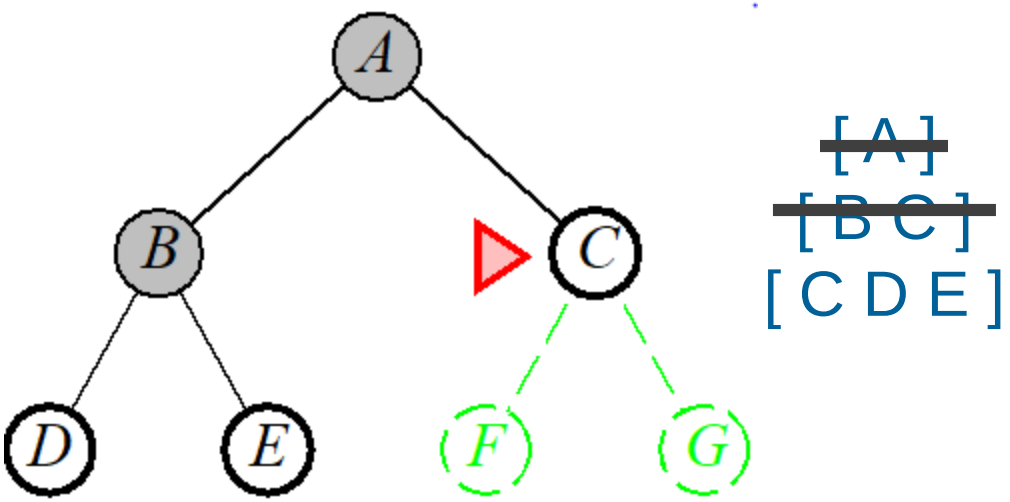
- Expand shallowest unexpanded node
- Implementation: *fringe* is a First In, First Out (FIFO) queue, i.e., new successors go at the end



~~[A]~~
[B C]

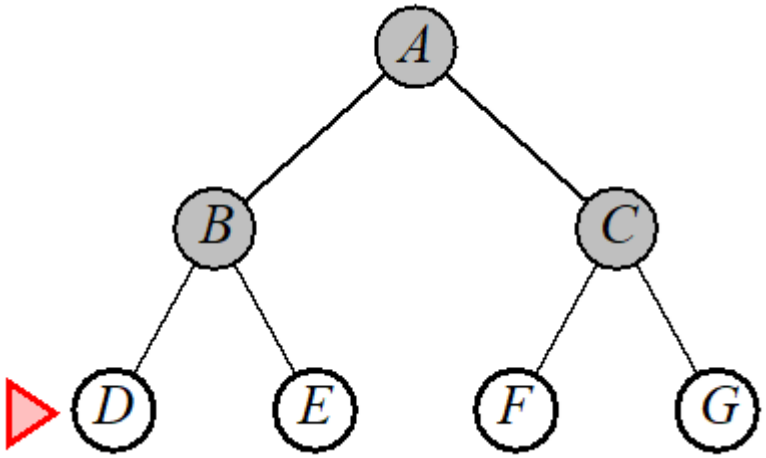
Breadth-first Search

- Expand shallowest unexpanded node
- Implementation: *fringe* is a First In, First Out (FIFO) queue, i.e., new successors go at the end



Breadth-first Search

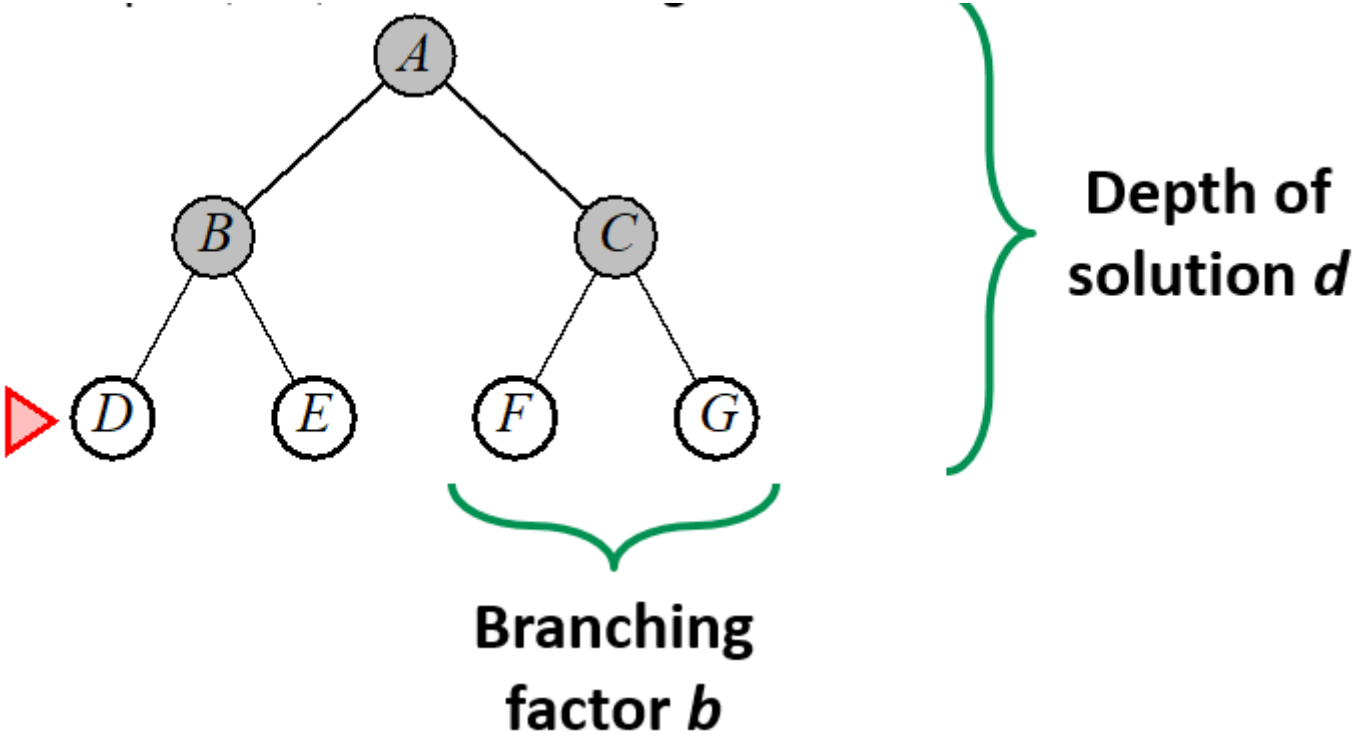
- Expand shallowest unexpanded node
- Implementation: *fringe* is a First In, First Out (FIFO) queue, i.e., new successors go at the end



~~[A]~~
~~[B C]~~
~~[C D E]~~
[D E F G]

Breadth-first Search

- Expand shallowest unexpanded node
- Implementation: *fringe* is a First In, First Out (FIFO) queue, i.e., new successors go at the end



Properties of Breadth-first Search

- Complete ?? – Will a solution always be found if one exists?
- Time ?? – How long will it take to find the solution? Often represented as the number of nodes searched.
- Space ?? – How much memory is needed to perform the search? Often represented as the maximum number of nodes stored at once.
- Optimal ?? – Will the optimal (least cost) solution be found?

Properties of Breadth-first Search

- Complete – Yes (if b is finite)
- Time ??
- Space ??
- Optimal ??

Properties of Breadth-first Search

- Complete – Yes (if b is finite)
- Time – $1 + b + b^2 + b^3 + \ldots + b^d = O(b^d)$
  - i.e., exponential in d
- Space ??
- Optimal ??

Properties of Breadth-first Search

- Complete – Yes (if b is finite)
- Time – $1 + b + b^2 + b^3 + \ldots + b^d = O(b^d)$
  - i.e., exponential in d
- Space – $O(b^d)$ (keeps every node in memory)
- Optimal ??

Properties of Breadth-first Search

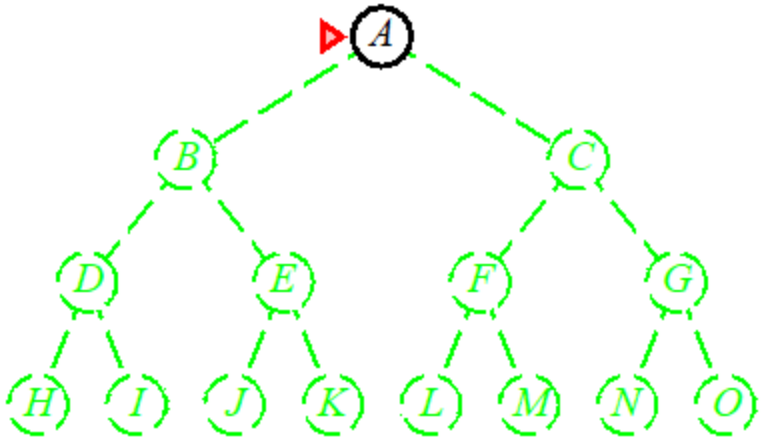
- Complete – Yes (if b is finite)
- Time – $1 + b + b^2 + b^3 + \ldots + b^d = O(b^d)$
  - i.e., exponential in d
- Space – O(b) (keeps every node in memory)
- Optimal – Yes (if cost = 1 per step); not optimal in general

Space is the big problem; can easily generate nodes at 100MB/sec so 24hrs = 8640GB.

Depth-first Search

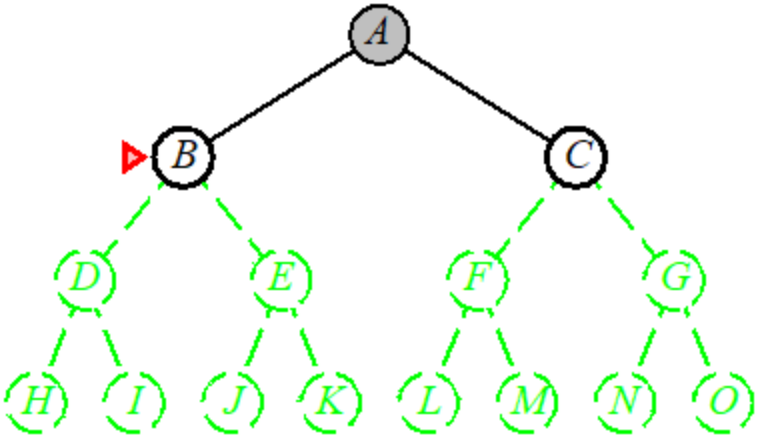
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front

[A]



Depth-first Search

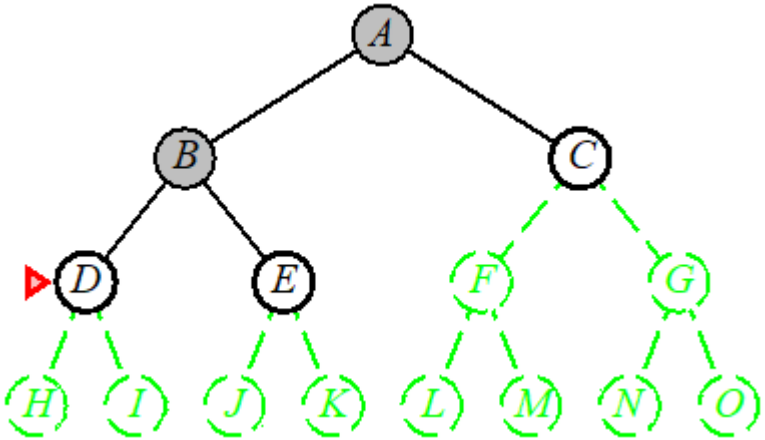
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



~~[A]~~
[B C]

Depth-first Search

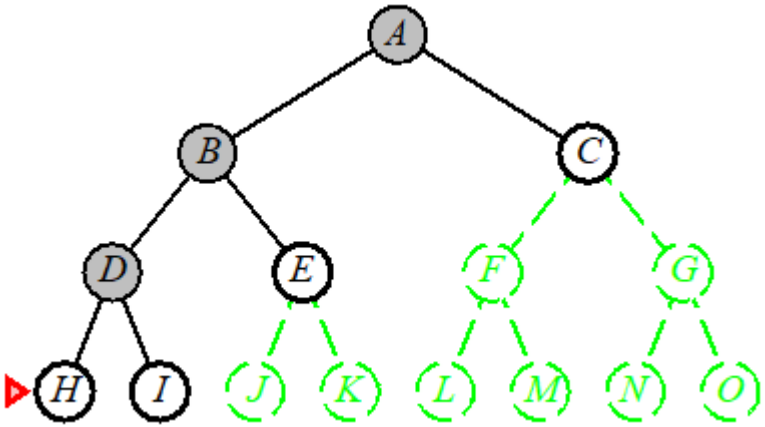
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



~~[A]~~
~~[B C]~~
[D E C]

Depth-first Search

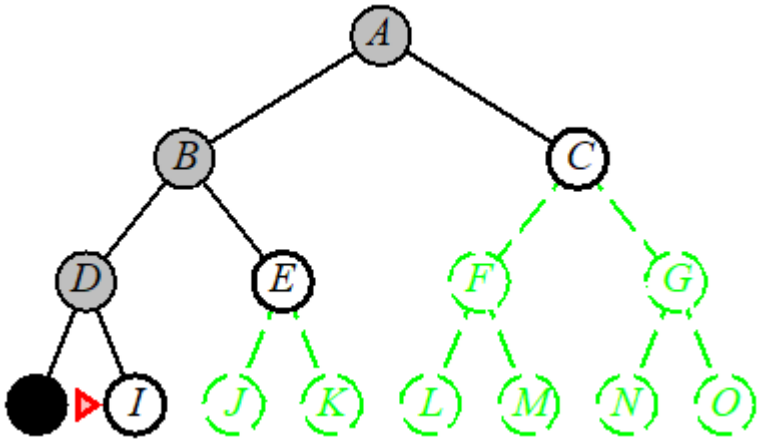
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



~~[A]~~
~~[B C]~~
~~[D E C]~~
[H I E C]

Depth-first Search

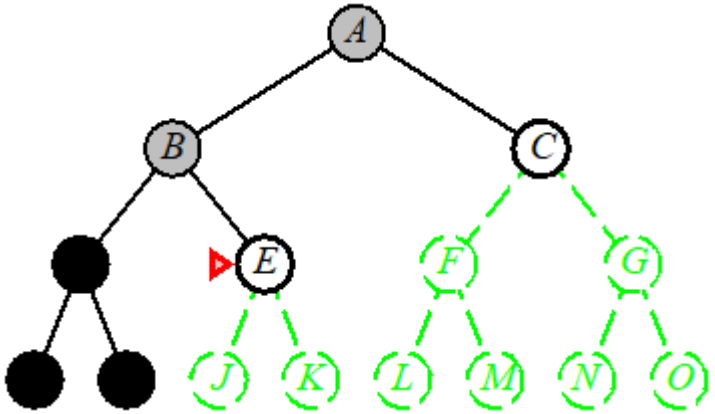
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



~~[A]~~
~~[B C]~~
~~[D E C]~~
~~[H I E C]~~
[I E C]

Depth-first Search

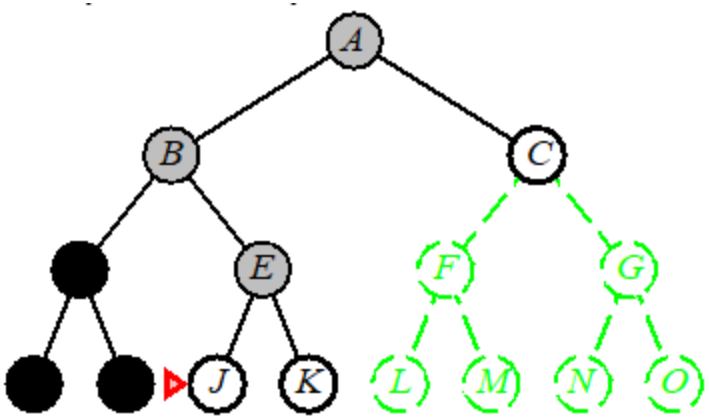
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



~~[A]~~
~~[B C]~~
~~[D E C]~~
~~[H I E C]~~
~~[I E C]~~
[E C]

Depth-first Search

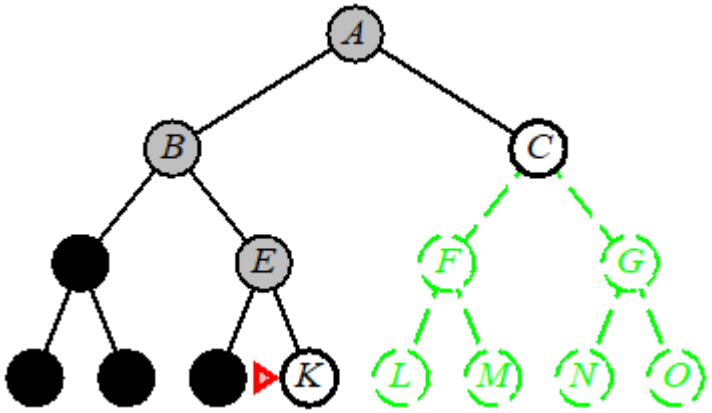
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



[A]
[B C]
[D E C]
[H I E C]
[I E C]
[E C]
[J K C]

Depth-first Search

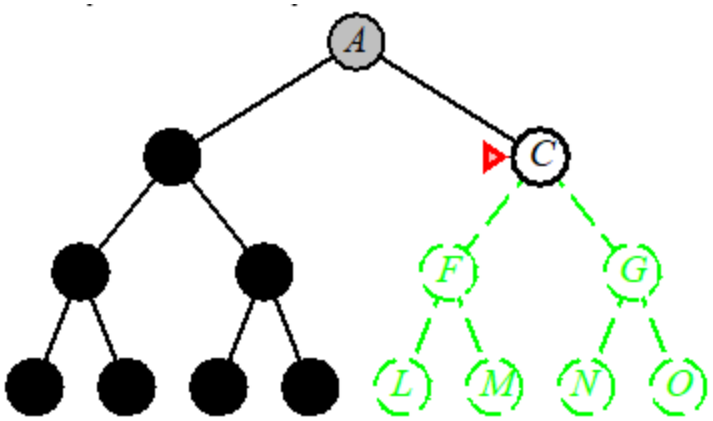
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front

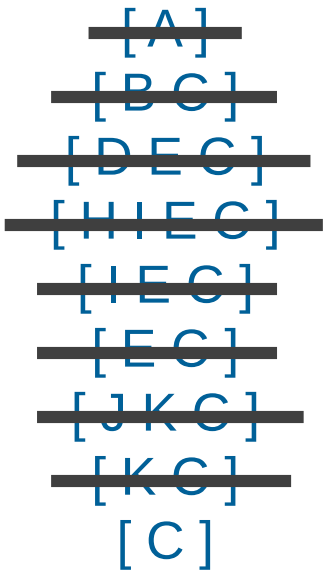


[A]
[B C]
[D E C]
[H I E C]
[I E C]
[E C]
[J K C]
[K C]

Depth-first Search

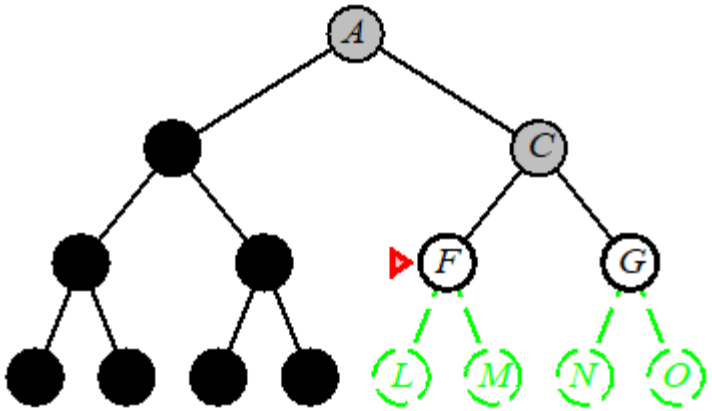
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front

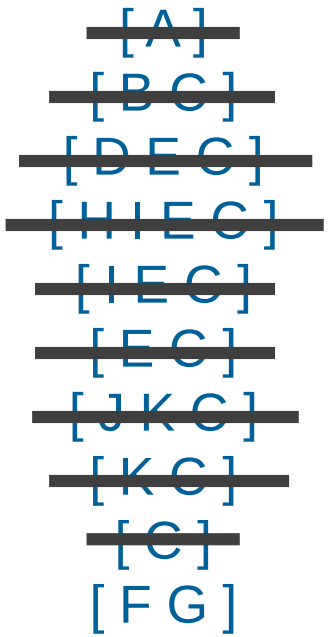




Depth-first Search

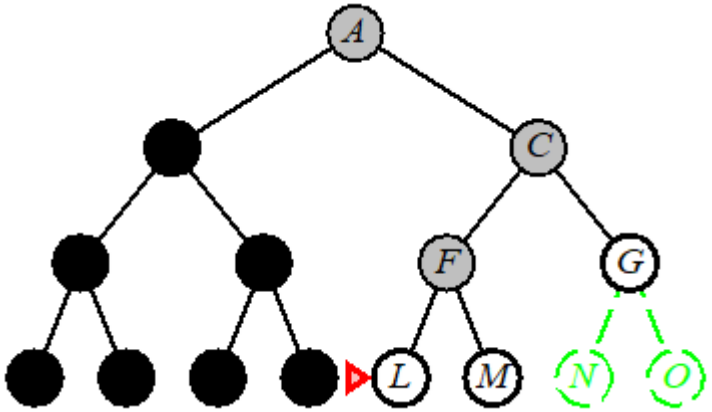
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front





Depth-first Search

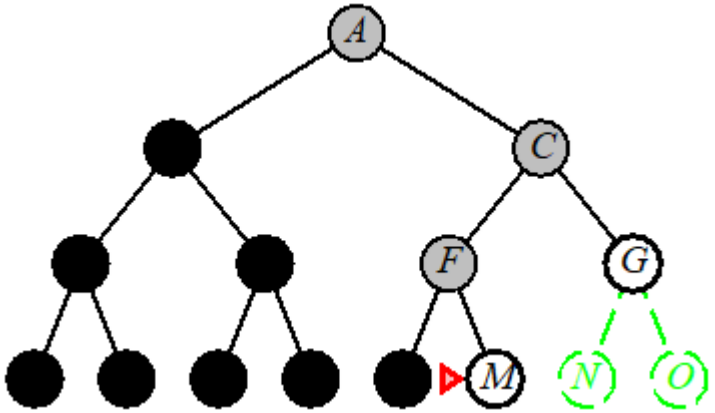
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front

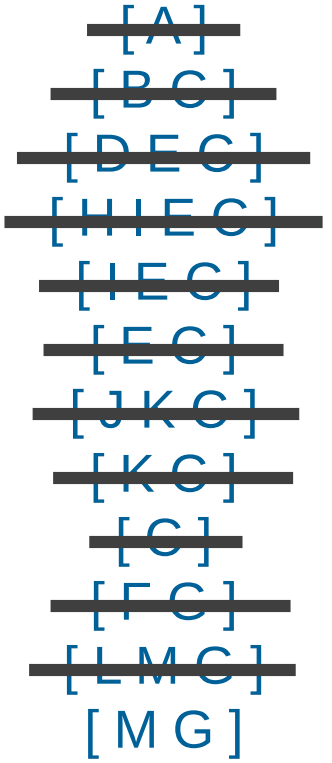


[A]
[B C]
[D E C]
[H I E C]
[I E C]
[E C]
[J K C]
[K C]
[C]
[F C]
[L M G]

Depth-first Search

- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front





Properties of Depth-first Search

- Complete ??
- Time ??
- Space ??
- Optimal ??

Properties of Depth-first Search

- Complete - No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path -> complete in finite spaces
- Time ??
- Space ??
- Optimal ??

Consider
m = maximum depth

Properties of Depth-first Search

- Complete - No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path -> complete in finite spaces
- Time – $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space ??
- Optimal ??

m = maximum depth;
bad if m >> d !

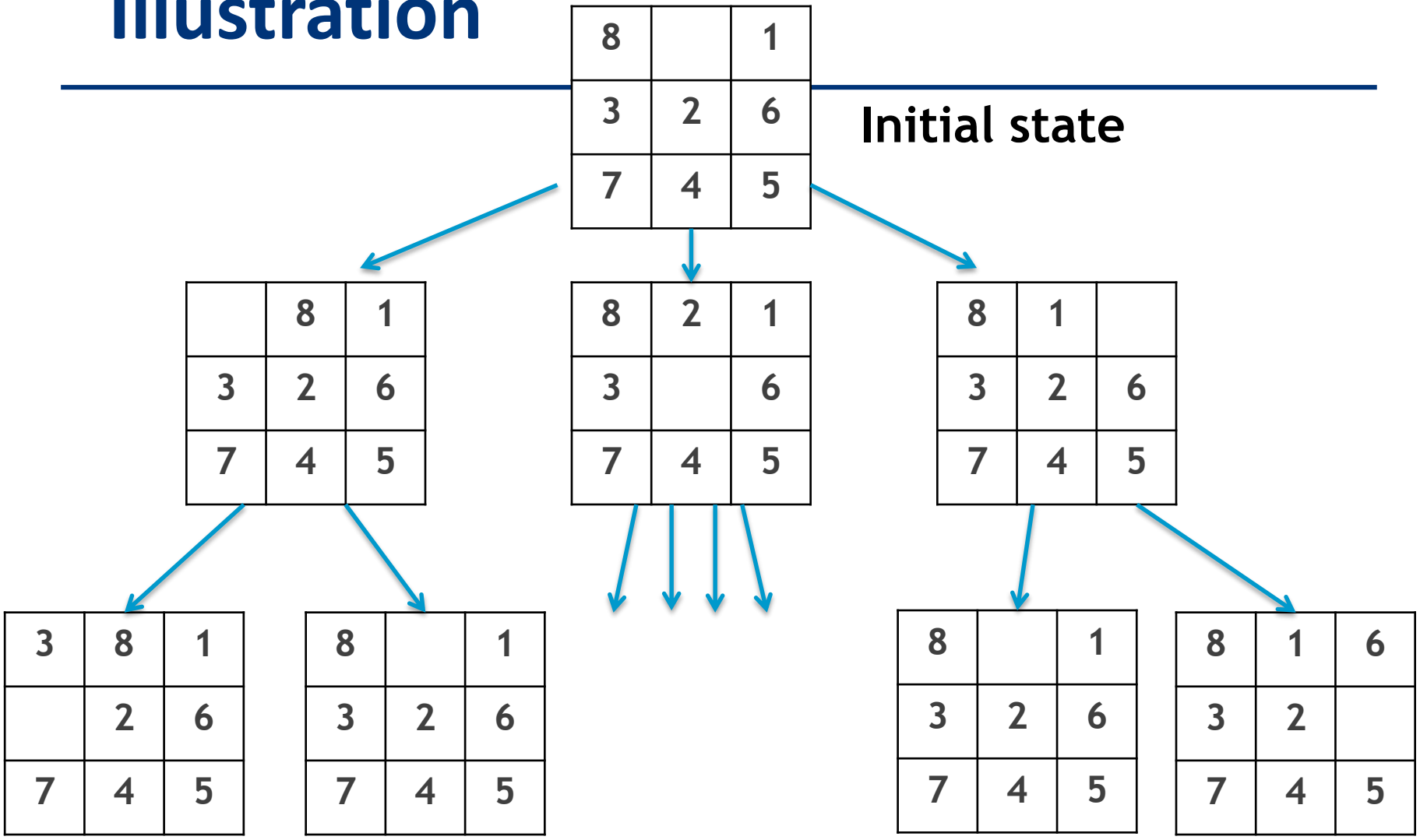
Properties of Depth-first Search

- Complete - No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path -> complete in finite spaces
- Time – $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space – O(bm), i.e., linear space!
- Optimal ??

Properties of Depth-first Search

- Complete - No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path -> complete in finite spaces
- Time – $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space – O(bm), i.e., linear space!
- Optimal – No

Illustration



Properties of Depth-first Search

- Complete - No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path -> complete in finite spaces
- Time – $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space – O(bm), i.e., linear space!... But not really. In practice you need to store a list of already visited states to avoid infinite loops.
- Optimal – No

Depth-limited Search Algorithm

- Depth-first search with depth limit l, i.e., nodes at depth l have no successors

(Might be a good idea if *d* is well understood)

```
function Depth-Limited-Search(problem, limit) returns soln/fail/cutoff
  Recursive-DLS(Make-Node(Initial-State[problem]), problem, limit)

function Recursive-DLS(node, problem, limit) returns soln/fail/cutoff
  cutoff-occurred? ← false
  if Goal-Test(problem, State[node]) then return node
  else if Depth[node] = limit then return cutoff
  else for each successor in Expand(node, problem) do
      result ← Recursive-DLS(successor, problem, limit)
      if result = cutoff then cutoff-occurred? ← true
      else if result ≠ failure then return result
  if cutoff-occurred? then return cutoff else return failure
```

Iterative Deepening Search Algorithm

```
function Iterative-Deepening-Search(problem) returns a solution
    inputs: problem, a problem

    for depth ← 0 to ∞ do
        result ← Depth-Limited-Search(problem, depth)
        if result ≠ cutoff then return result
    end
```

Iterative Deepening Search l = 0

Limit = 0

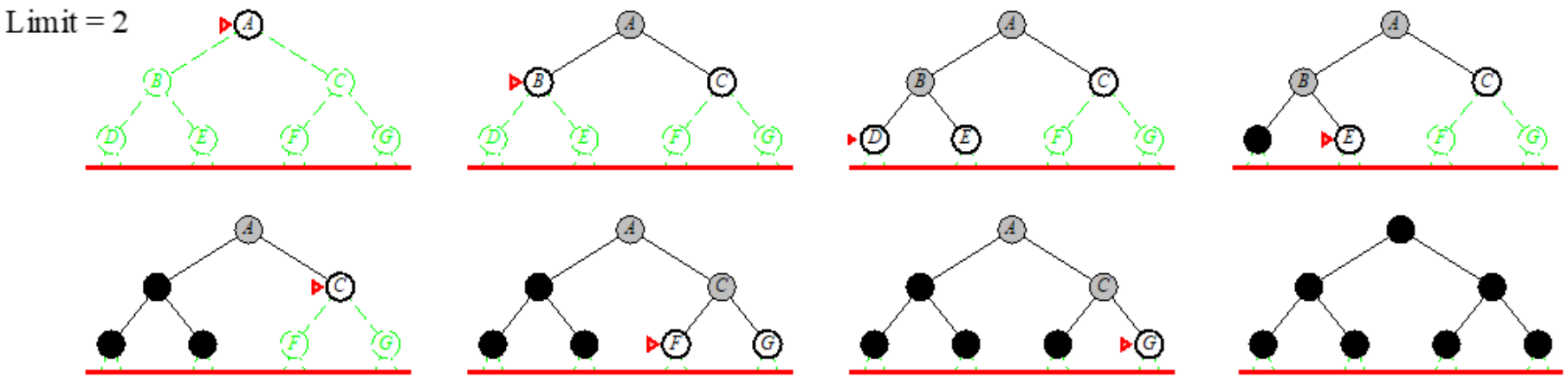




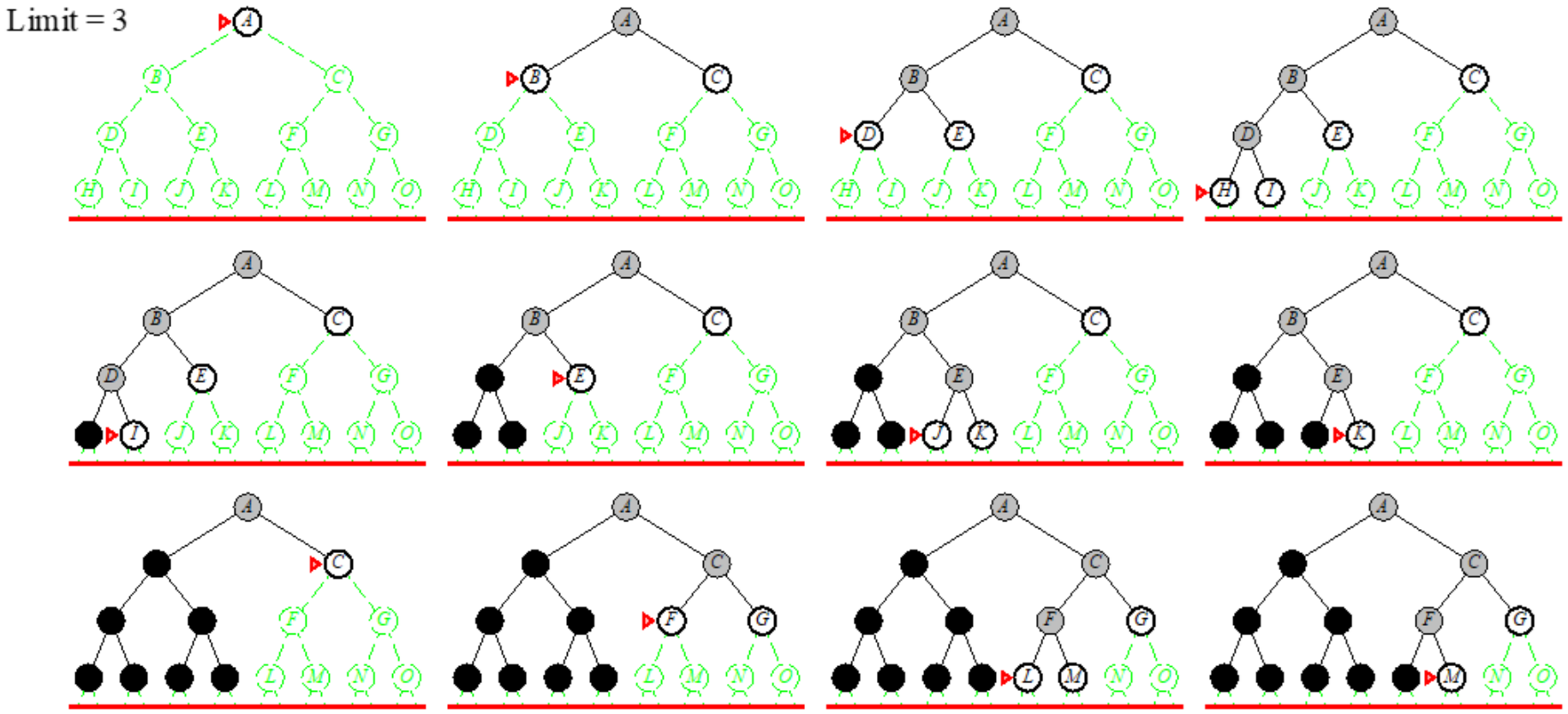
Iterative Deepening Search l = 1



Iterative Deepening Search l = 2



Iterative Deepening Search l = 3



Properties of Iterative Deepening Search

- Complete ??
- Time ??
- Space ??
- Optimal ??

Properties of Iterative Deepening Search

- Complete – Yes
- Time ??
- Space ??
- Optimal ??

Properties of Iterative Deepening Search

- Complete – Yes
- Time – $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$
  - Like BFS
- Space ??
- Optimal ??

Properties of Iterative Deepening Search

- Complete – Yes
- Time – $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$
  - Like BFS
- Space – O(bd)
  - Similar to DFS
- Optimal ??

Properties of Iterative Deepening Search

- Complete – Yes
- Time – $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$
- Space – O(bd)
- Optimal – Yes if step cost = 1;
  - Numerical comparison for b = 10 and d = 5, solution at far right leaf:

    N(IDS) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450

    N(BFS) = 10 + 100 + 1,000 + 10,000 + 100,000 + 999,990 = 1,111,100
  - IDS does better because other nodes at depth d are not expanded
  - BFS can be modified to apply goal test when a node is generated

Summary of Algorithms

Criterion	Breadth-First	Depth-First	Depth-Limited	Iterative Deepening
Complete ?	Yes*	No	Yes, if $l \geq d$	Yes
Time ?	b^d	b^m	b^l	b^d
Space ?	b^d	bm	bl	bd
Optimal ?	Yes*	No	No	Yes*

Coming up…

- Informed search strategies

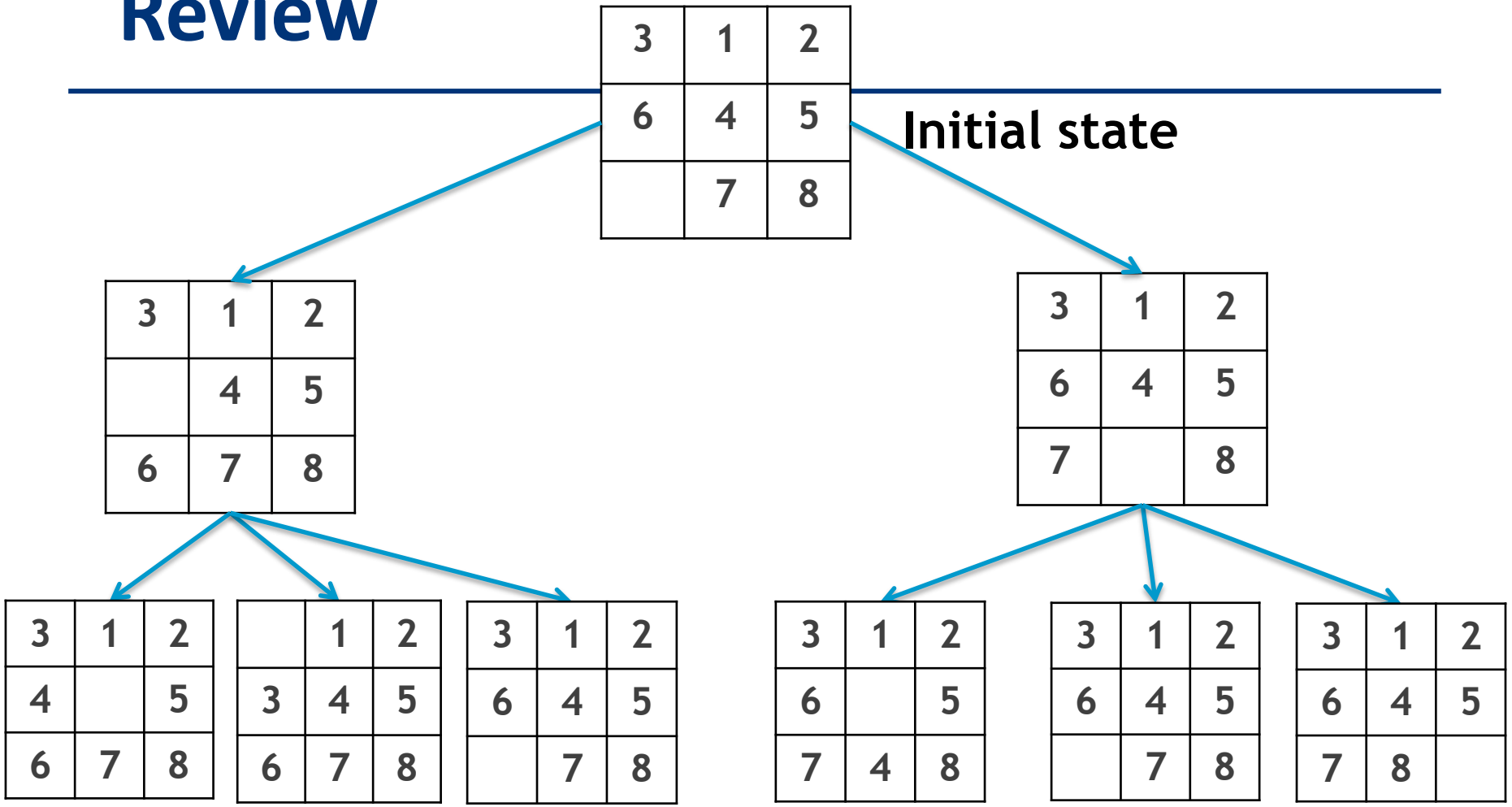
CS 380: ARTIFICIAL INTELLIGENCE

Lecture 4: Informed Search, A*, & Heuristics

Where we are…

- Problem Solving:
  - Problem Formulation
- Search: finding a solution
  - Uninformed search
  - Informed search

Review



Set of states reachable by executing two actions

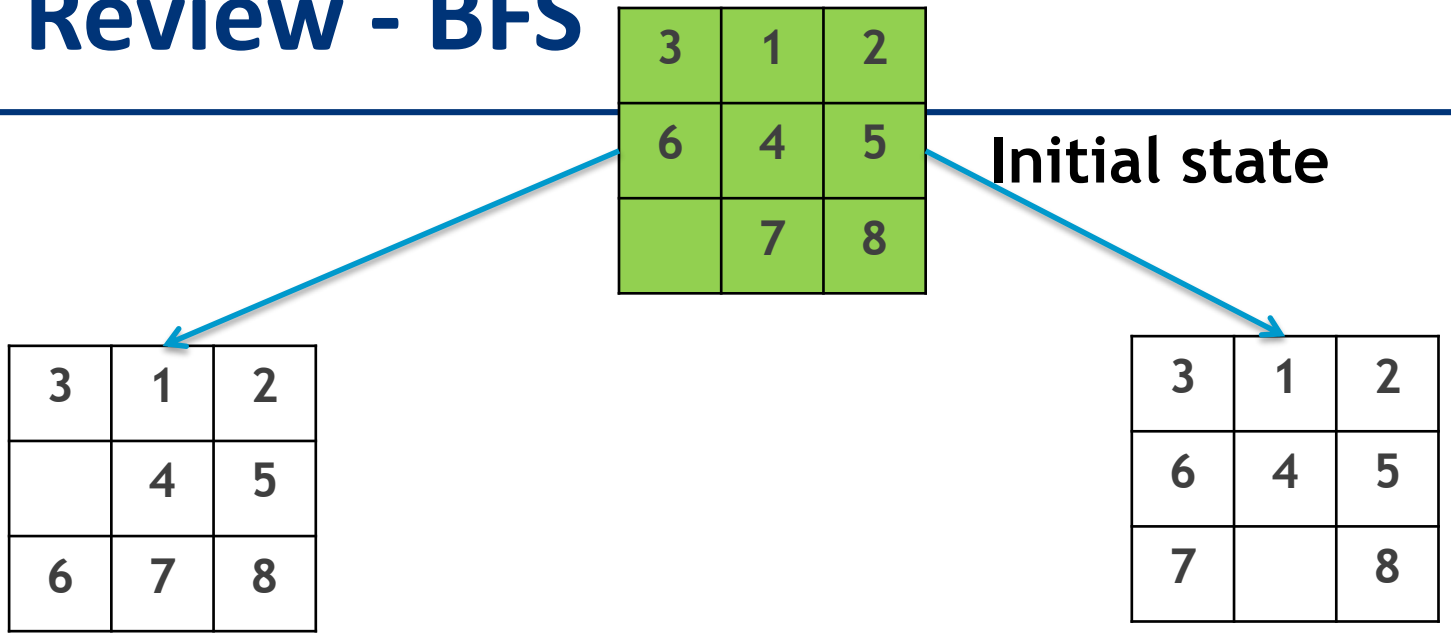
Review - BFS

3	1	2
6	4	5
	7	8

Initial state

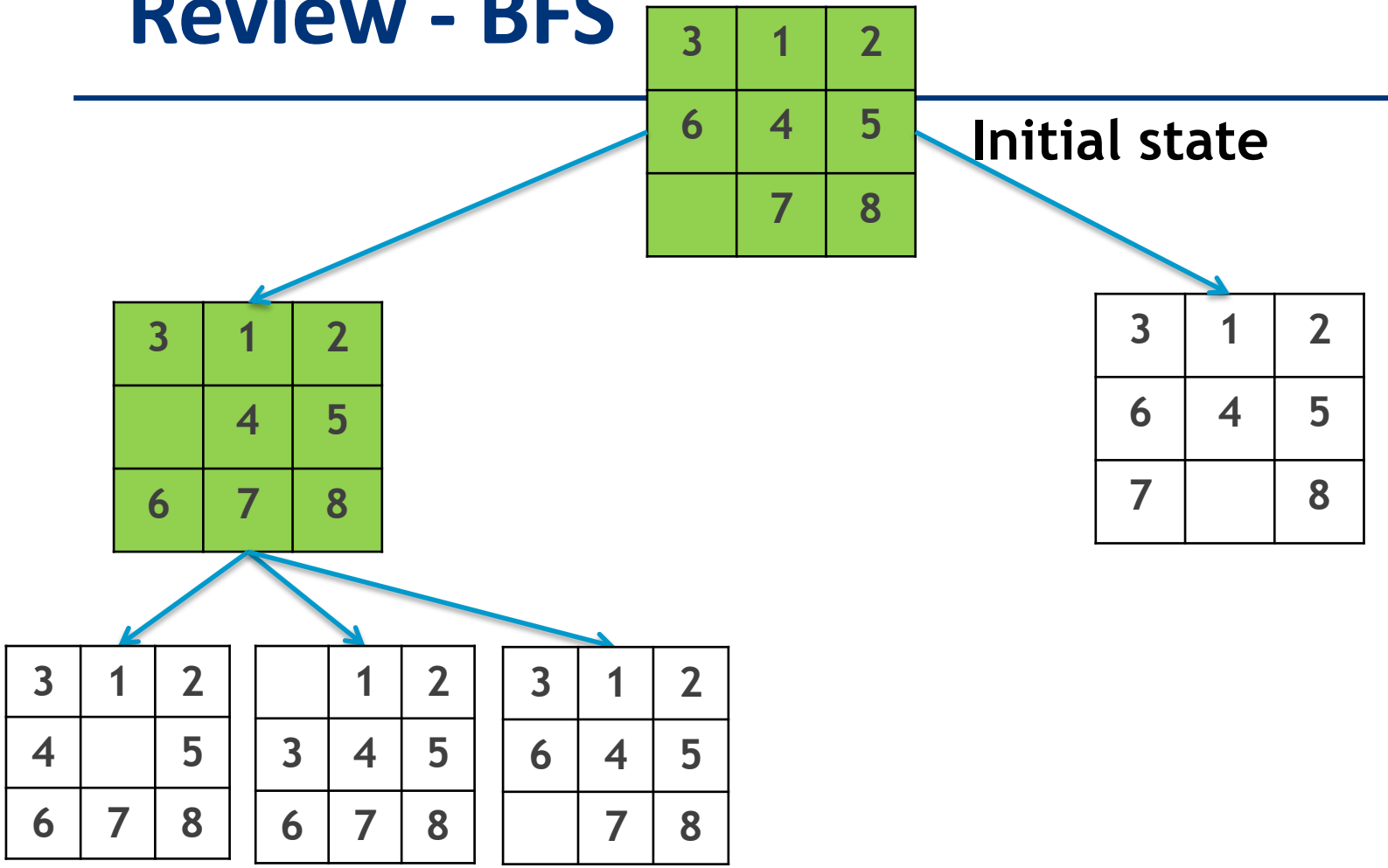
Set of states reachable by executing two actions

Review - BFS



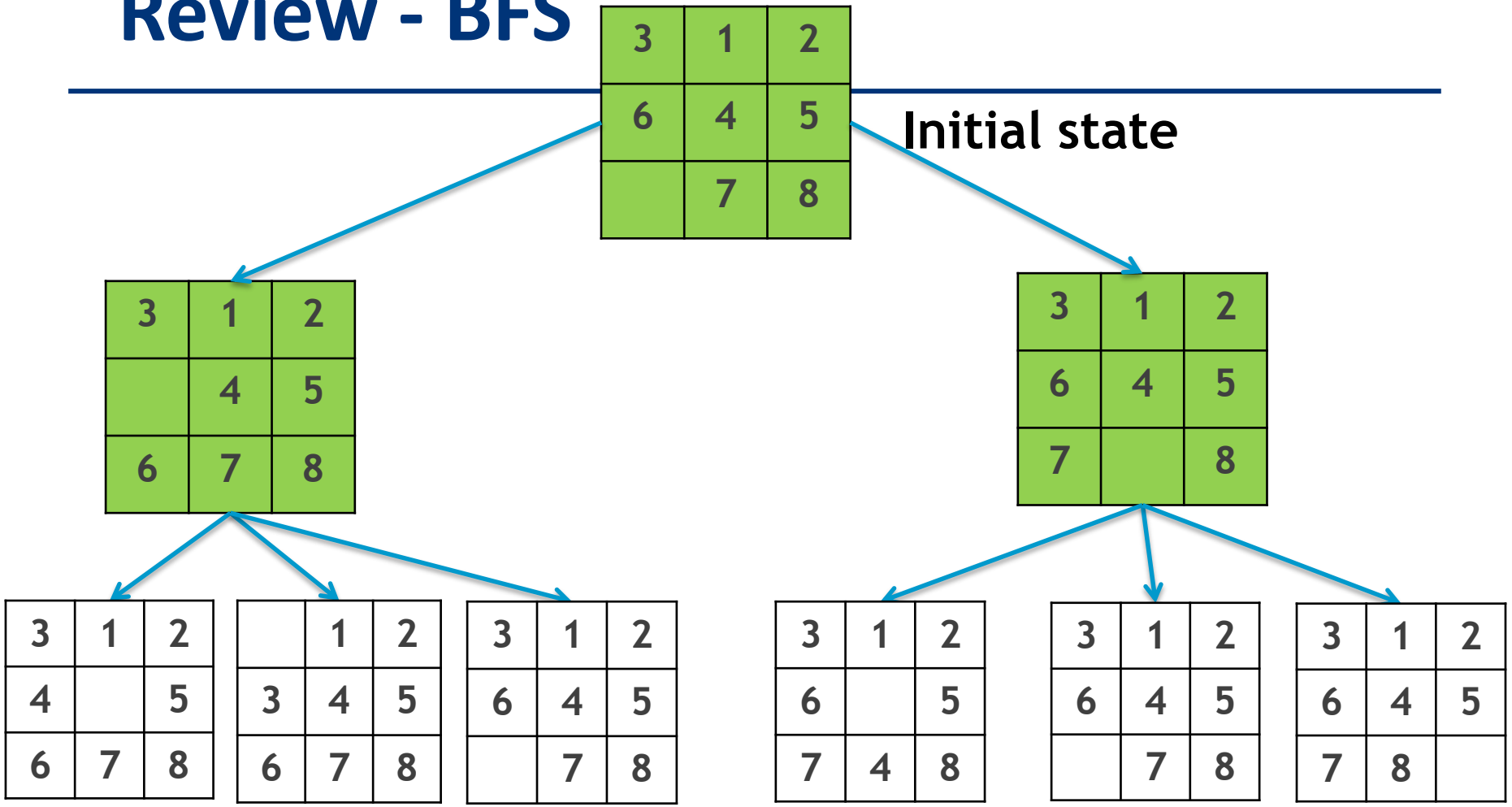
Set of states reachable by executing two actions

Review - BFS



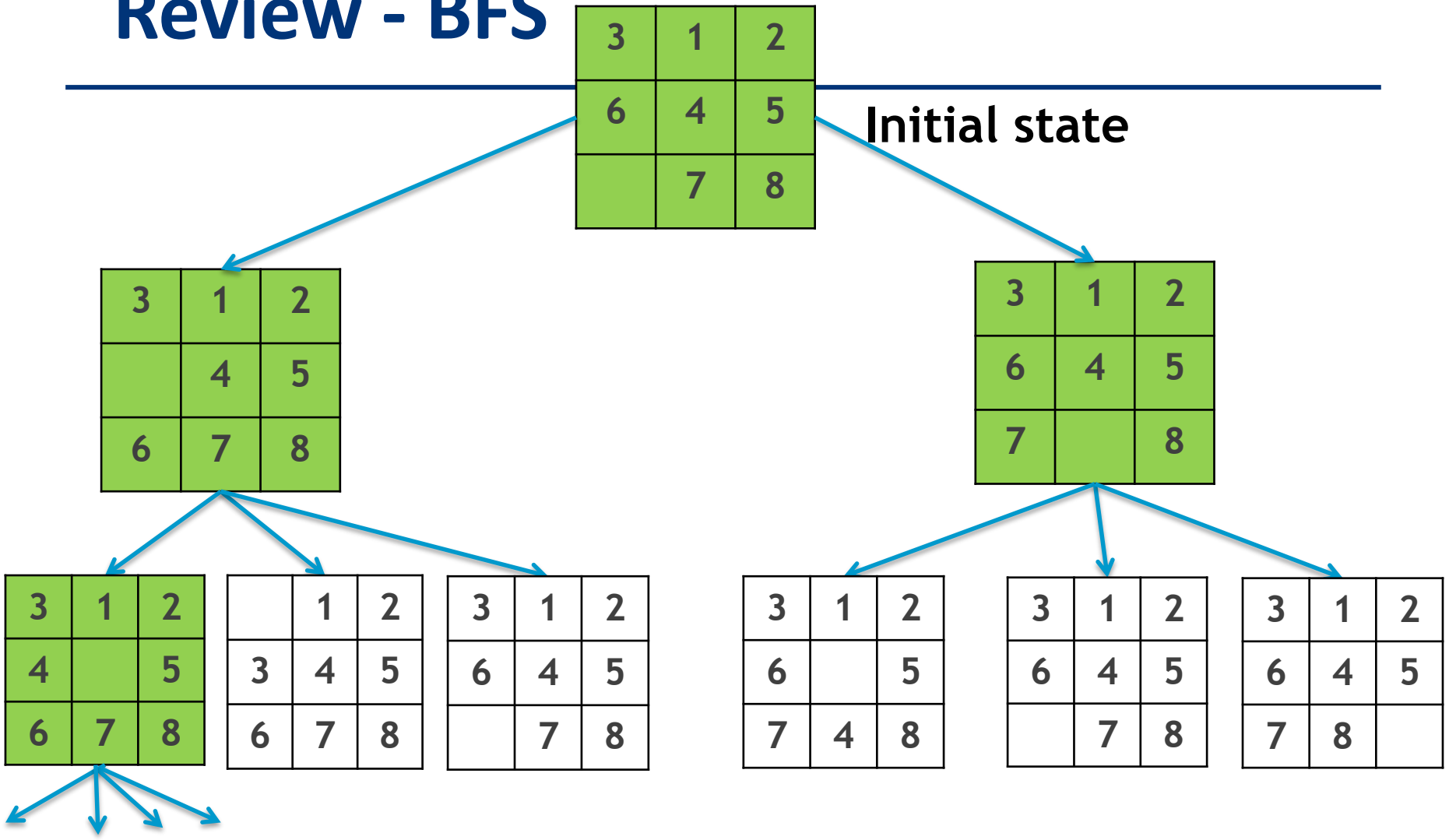
Set of states reachable by executing two actions

Review - BFS



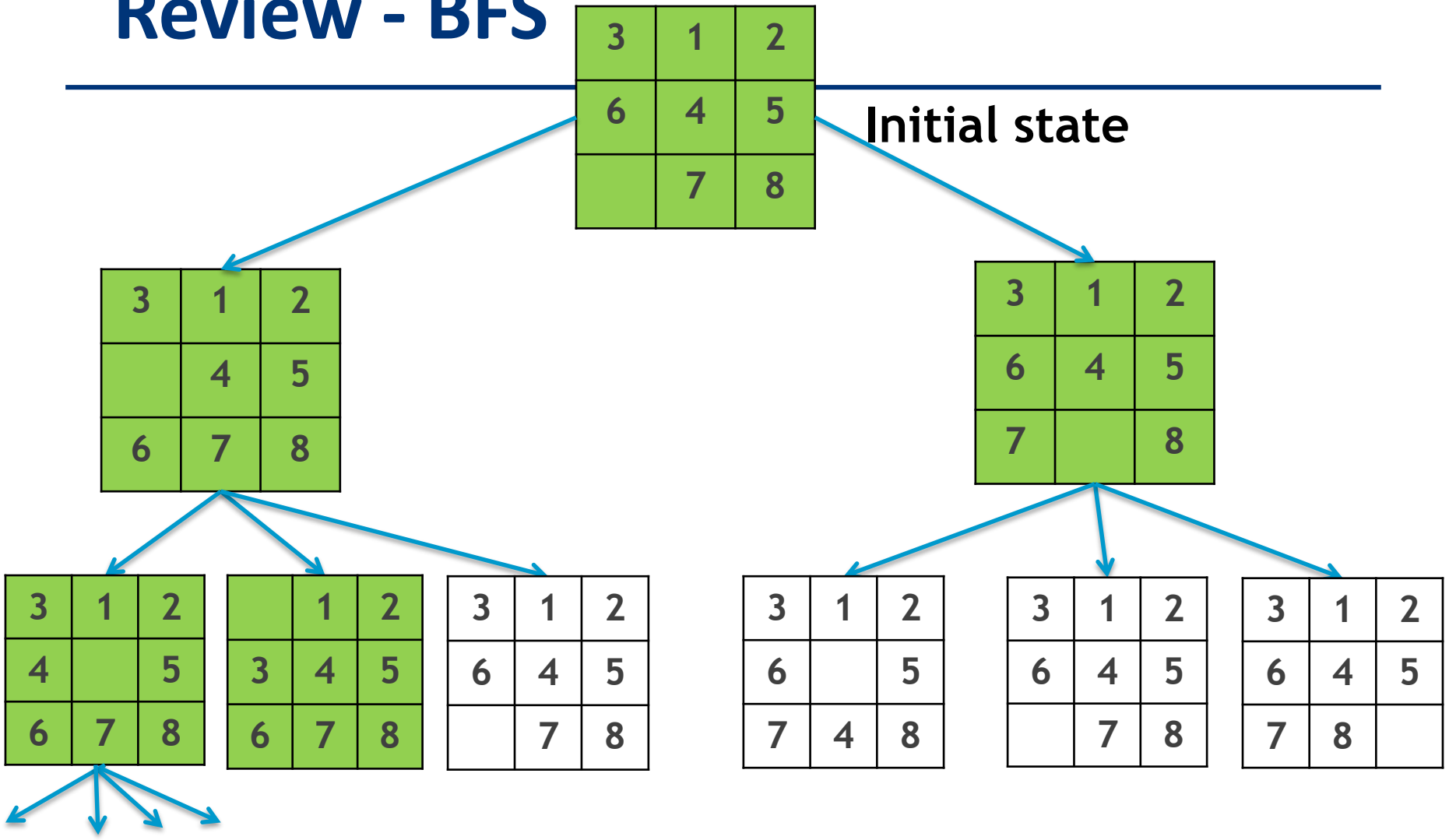
Set of states reachable by executing two actions

Review - BFS



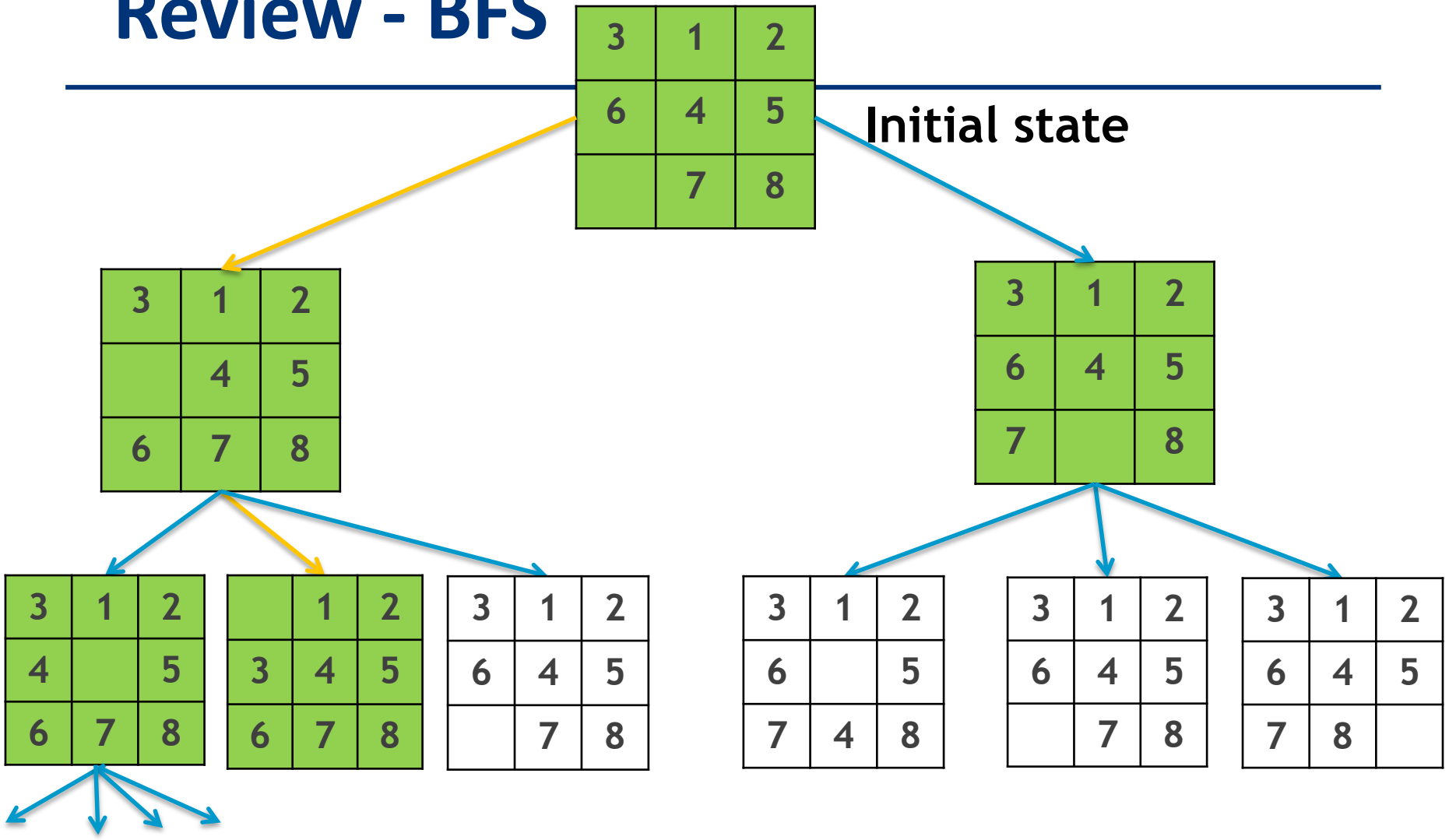
Set of states reachable by executing two actions

Review - BFS



Set of states reachable by executing two actions

Review - BFS



Set of states reachable by executing two actions

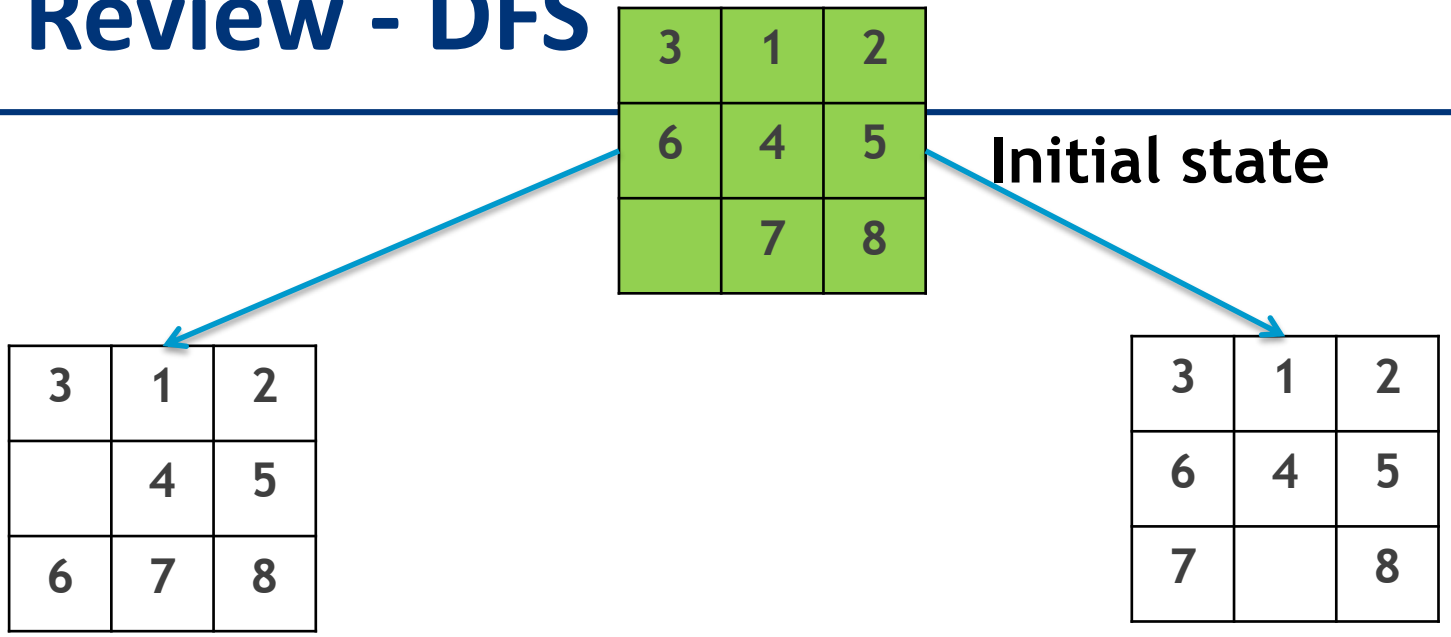
Review – DFS

3	1	2
6	4	5
	7	8

Initial state

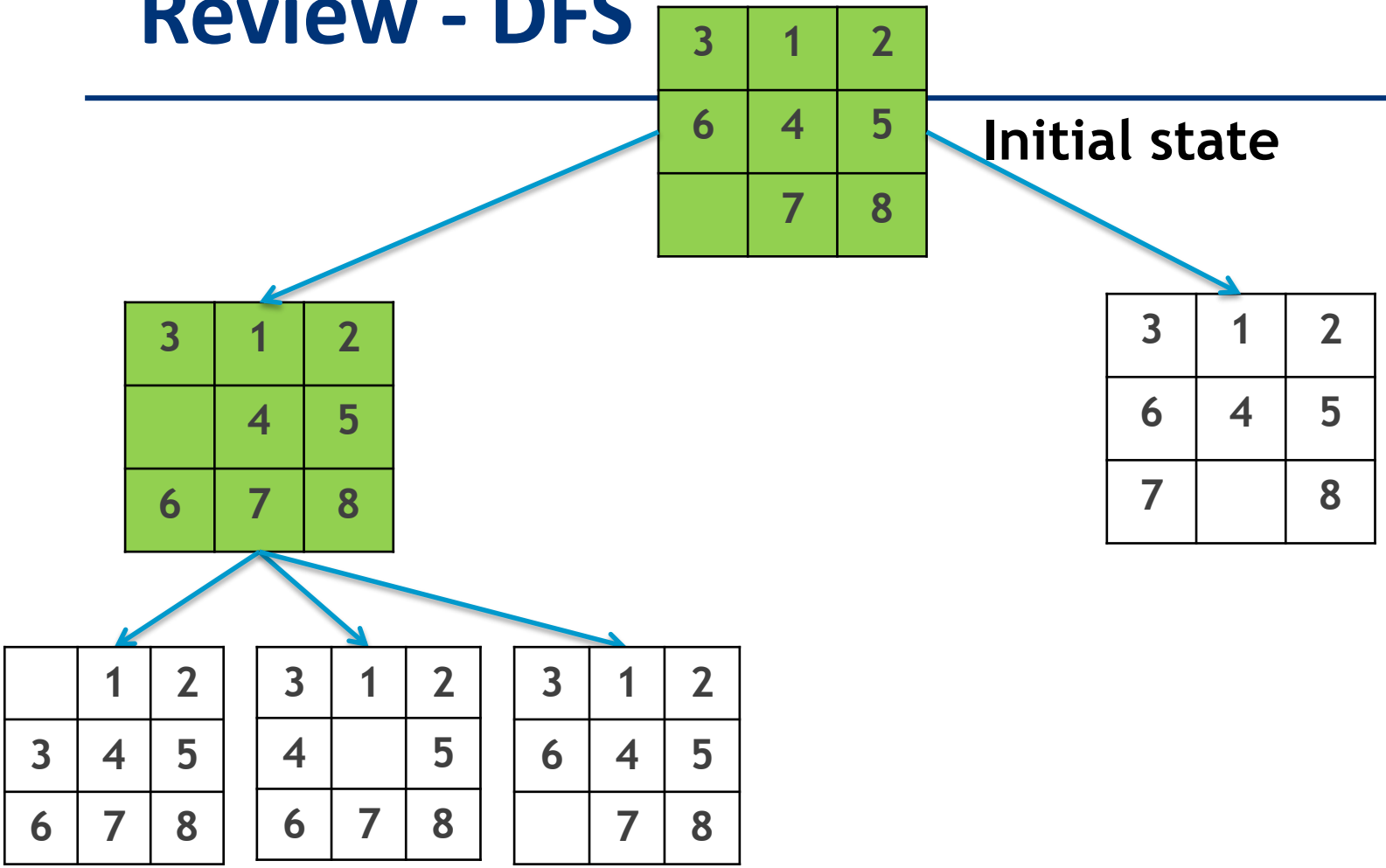
Set of states reachable by executing two actions

Review - DFS



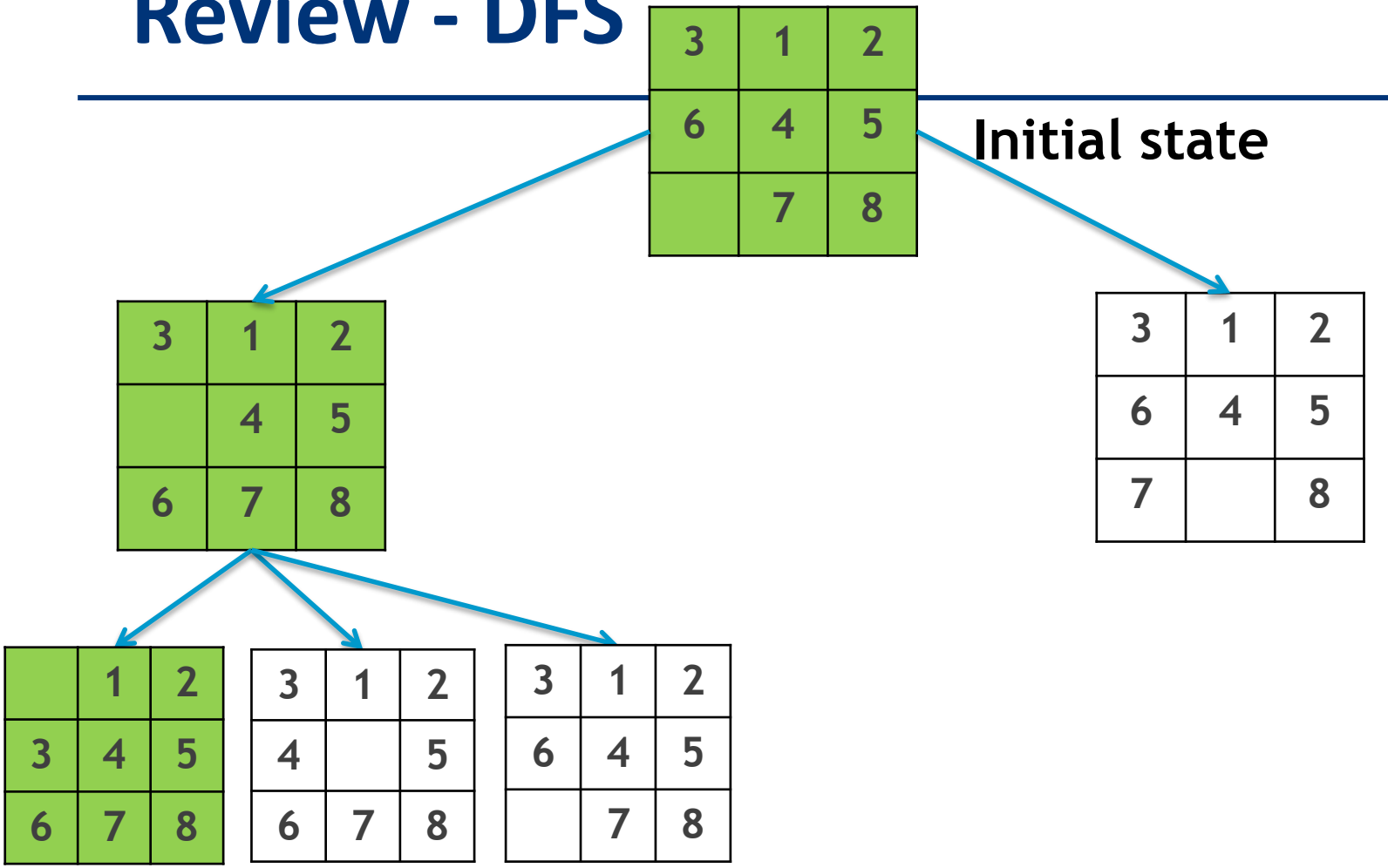
Set of states reachable by executing two actions

Review - DFS



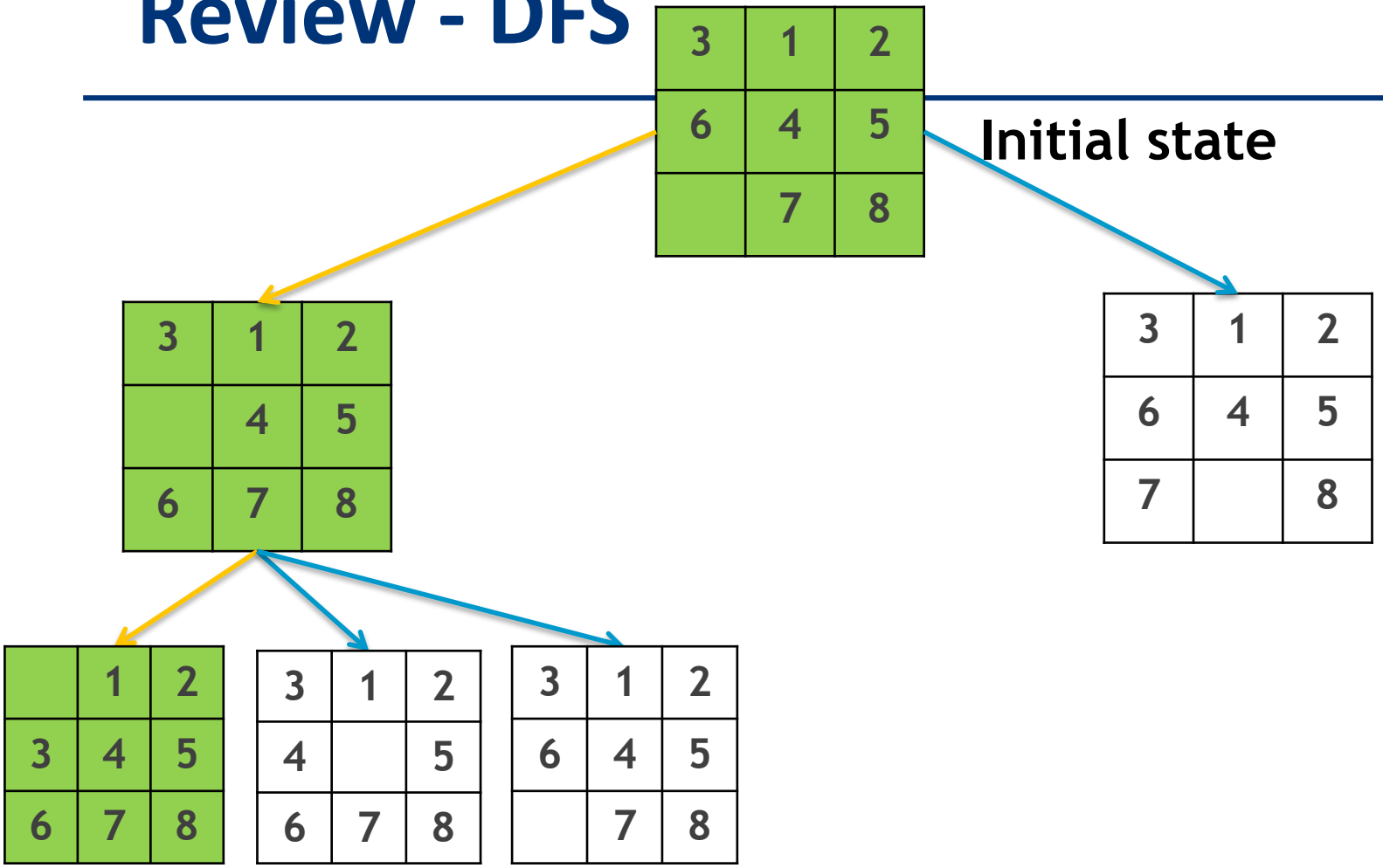
Set of states reachable by executing two actions

Review - DFS



Set of states reachable by executing two actions

Review - DFS



Set of states reachable by executing two actions

Review

- **OPEN** (or Fringe) list = nodes queued for processing
- **CLOSED** list = nodes/states we've already processed

- Breadth-first Search
  - Remove the first node in the list
  - Check if that first node's state is a goal state
    - If it's a goal, you're done
    - If not, add children to ***end*** of OPEN list and repeat
- Depth-first Search
  - Same as BFS, except add children to ***start*** of OPEN list.

- How can you check if a state is a repeated state?
  - Check whether it's already in either OPEN or CLOSED

Graph Search

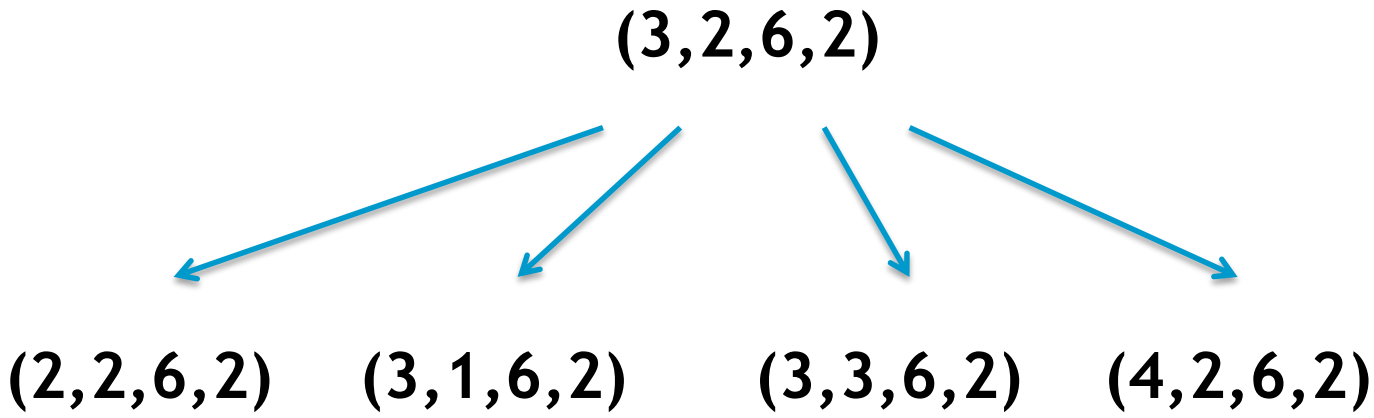
```
function Graph-Search(problem, fringe) returns a solution, or failure
    closed ← an empty set
    fringe ← Insert(Make-Node(Initial-State[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← Remove-Front(fringe)
        if Goal-Test(problem, State[node]) then return node
        if State[node] is not in closed then
            add State[node] to closed
            fringe ← InsertAll(Expand(node, problem), fringe)
    end
```

Uninformed → Informed Search

- All the problem-solving strategies we have seen so far use the following knowledge:
  - **Goal check**: knowing if the problem is solved
  - **States**
  - **Actions**: an agent knows which actions can be executed in the current state.

- Note that this is very little information

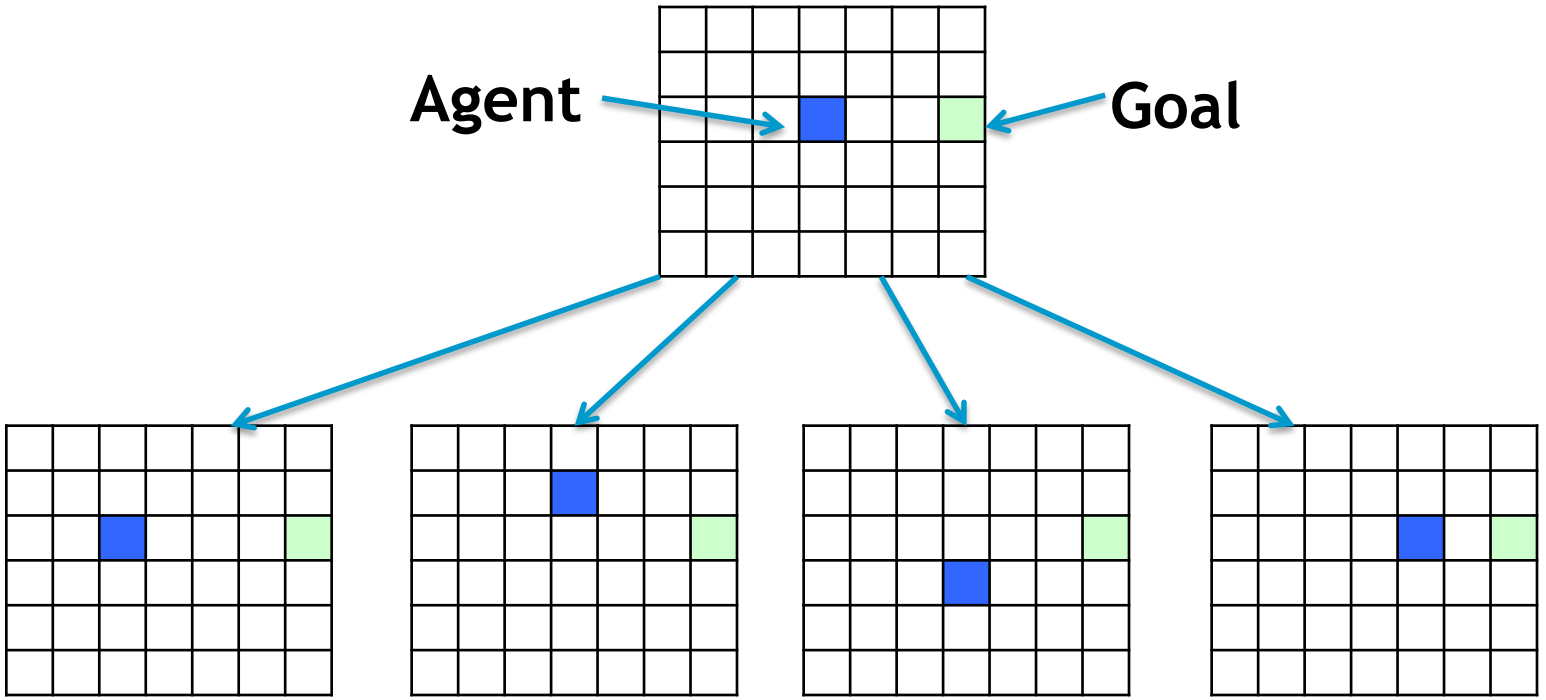
Example



There are 4 possible actions to execute.
Now, if you were the agent, which one would you try first?

Example

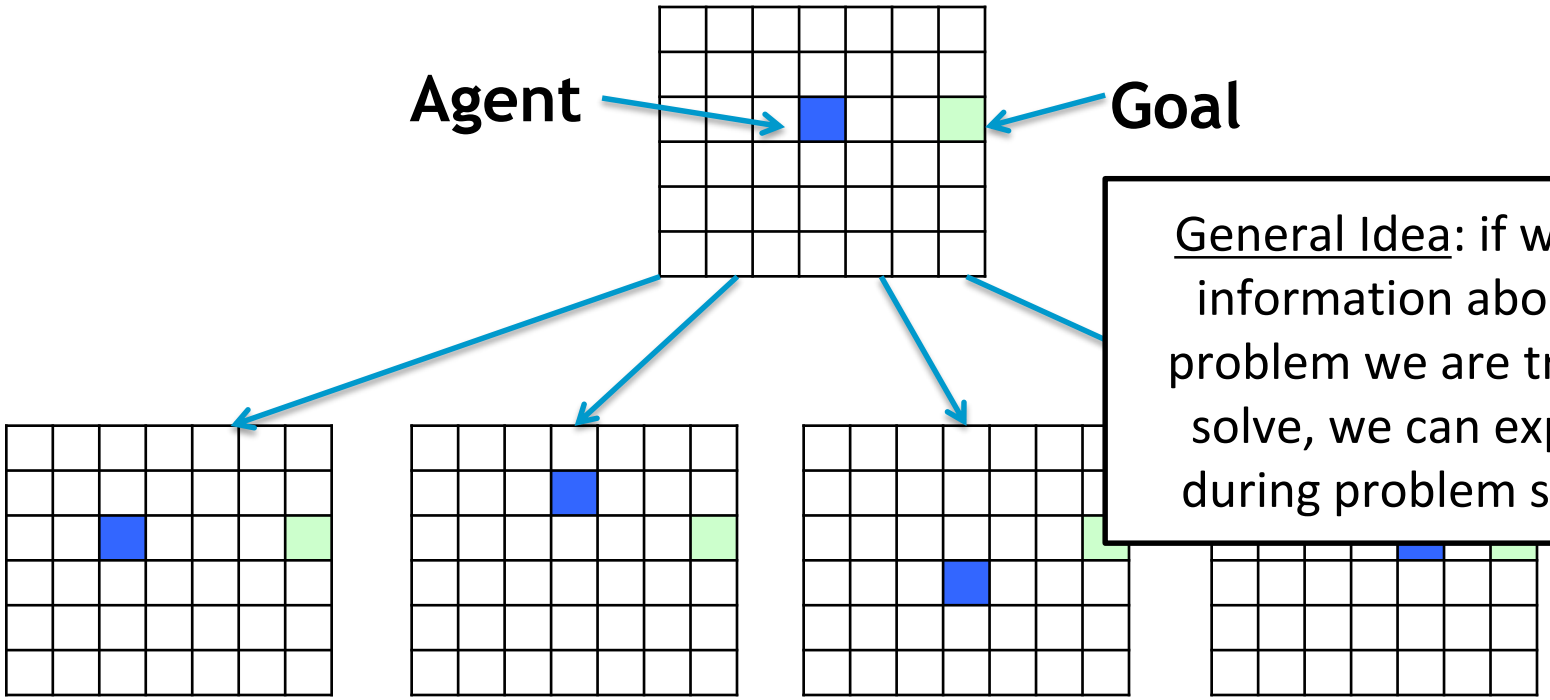
What if now you know what the coordinates mean?



There are 4 possible actions to execute.
Now, if you were the agent, which one would you try first?

Example

What if now you know what the coordinates mean?



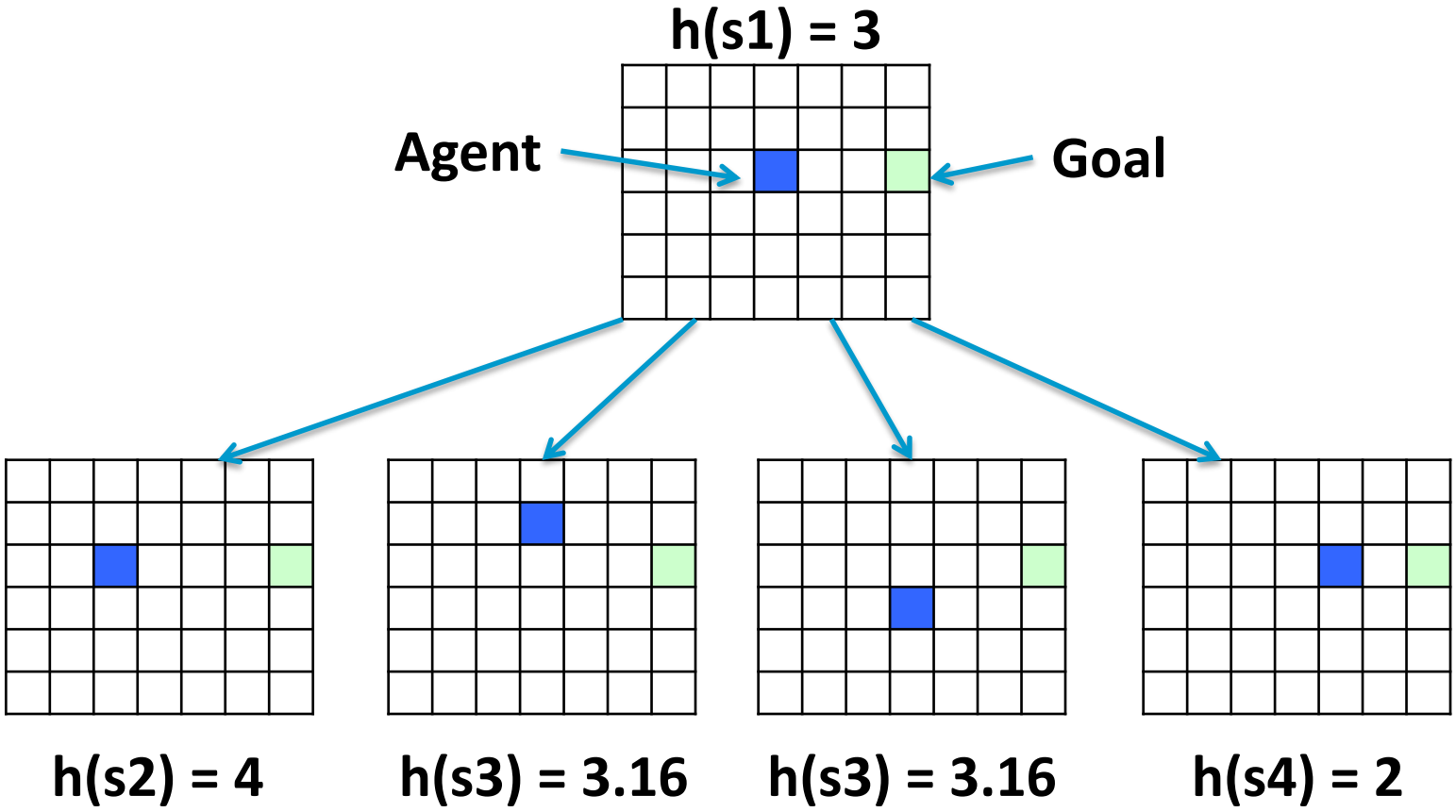
General Idea: if we have information about the problem we are trying to solve, we can exploit it during problem solving.

There are 4 possible actions to execute.
Now, if you were the agent, which one would you try first?

Evaluation Function

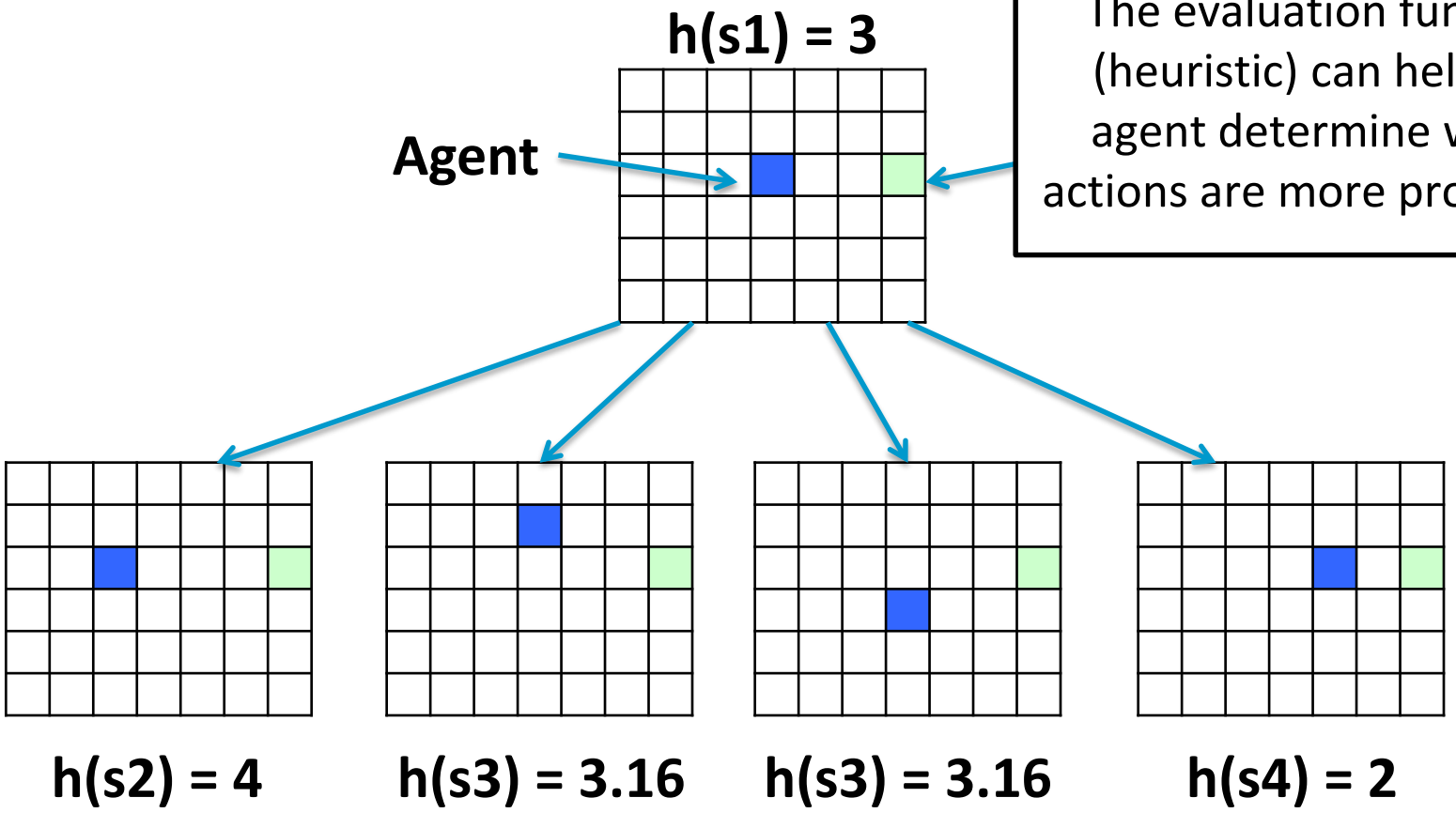
- Idea: represent the information we have about the domain as an "evaluation function" ***h***
- Evaluation function or "**heuristic**":
  - Given a state ***s*** …
  - ***h(s)*** estimates how far the state is from the goal
- For example, in a maze solving problem…
  - Euclidean distance to the goal?
  - Manhattan distance?
  - Etc.

Example



h: Euclidean distance to the goal

Example



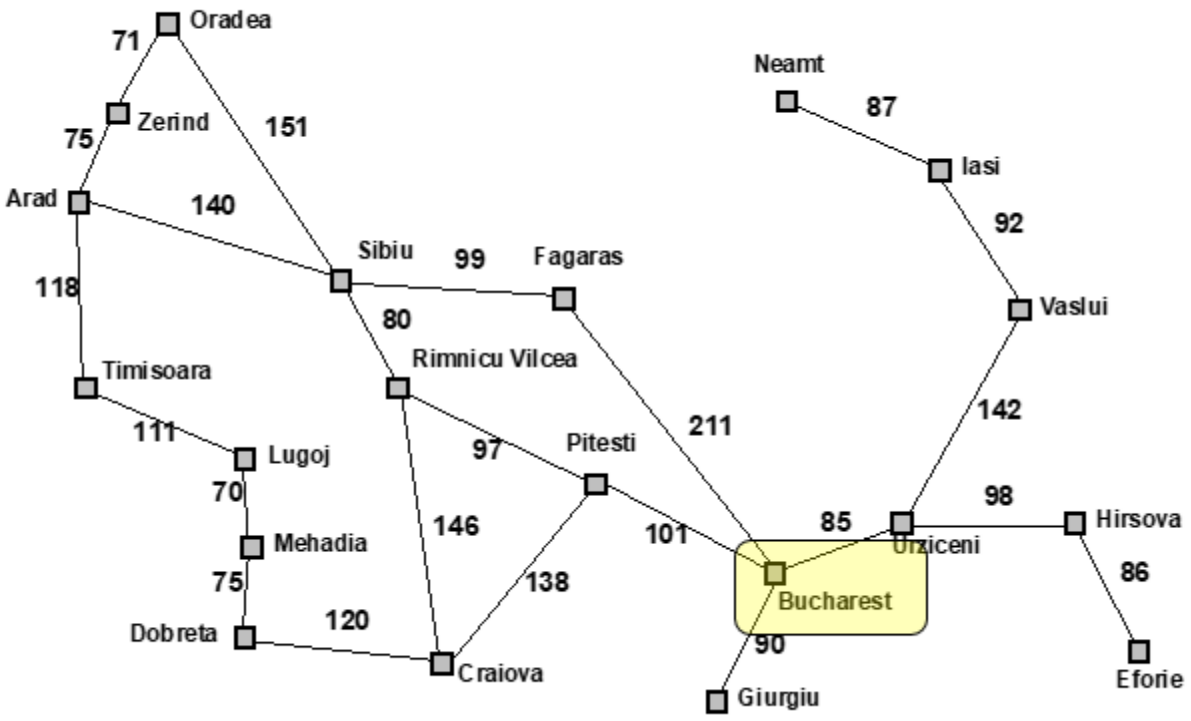
h: Euclidean distance to the goal

Best-first Search

- Idea: use an evaluation function for each node
  - estimate of "desirability"
  - expand most desirable unexpanded node
- Implementation:
  - fringe is a queue sorted in decreasing order of desirability
- Special cases:
  - Greedy search
  - A* search

Romania with step costs in km

Actual road connections and distances



Possible heuristic:

Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Search

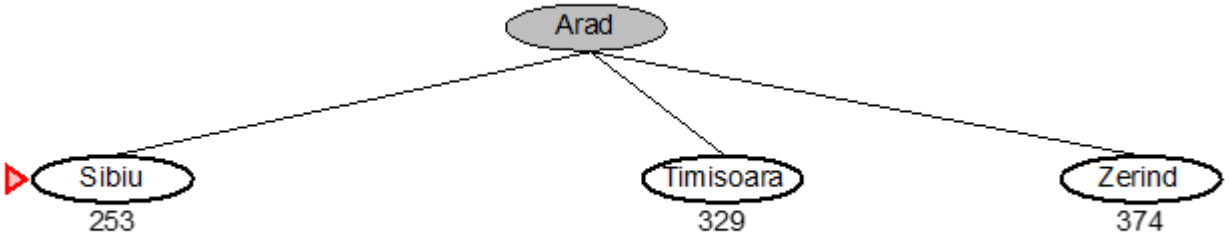
- Evaluation function h(n) (heuristic) = estimate of cost from n to the closest goal
- E.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest
- Greedy search expands the node that appears to be closest to goal

(What might go wrong with this heuristic?)

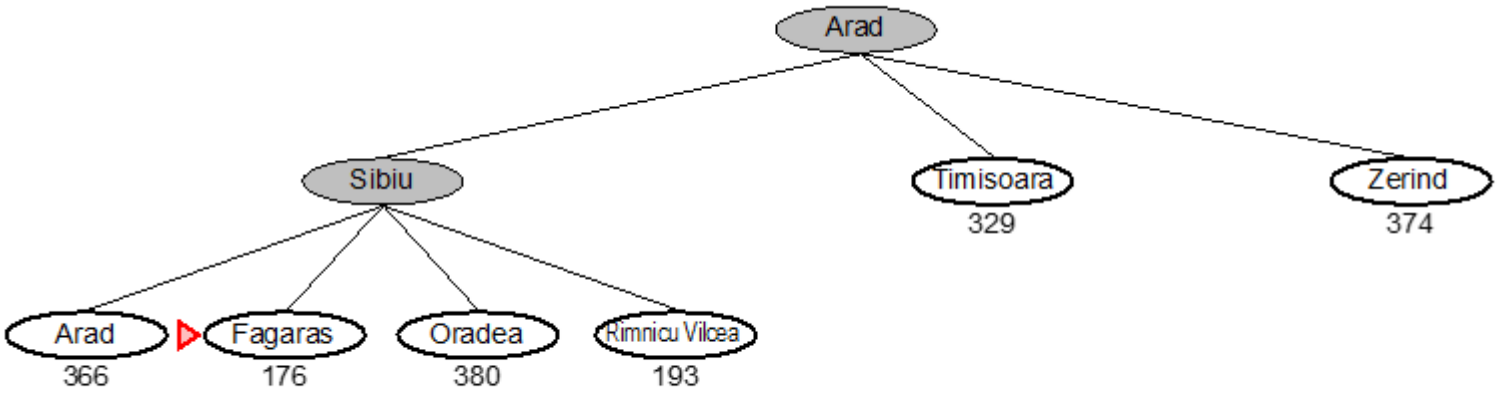
Greedy Search Example



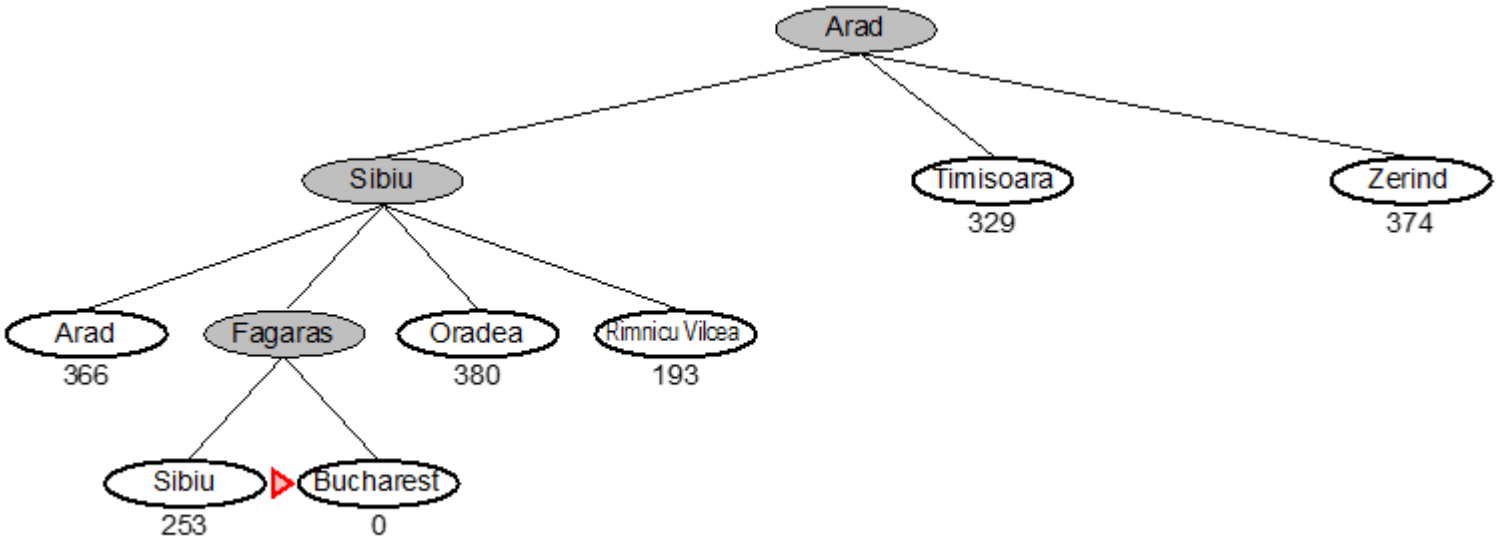
Greedy Search Example



Greedy Search Example



Greedy Search Example



Properties of Greedy Search

- Complete ??
- Time ??
- Space ??
- Optimal ??

Properties of Greedy Search

- Complete
  - No, it can get stuck in loops, e.g., with Oradea as goal, Iasi -> Neamt -> Iasi -> Neamt -> …
  - Complete in finite space with repeated-state checking
- Time ??
- Space ??
- Optimal ??

Properties of Greedy Search

- Complete
  - No, it can get stuck in loops, e.g., with Oradea as goal, lasi -> Neamt -> lasi -> Neamt -> …
  - Complete in finite space with repeated-state checking
- Time
  - $O(b^m)$, but a good heuristic can give dramatic improvement
- Space ??
- Optimal ??

Consider
m = maximum depth

Properties of Greedy Search

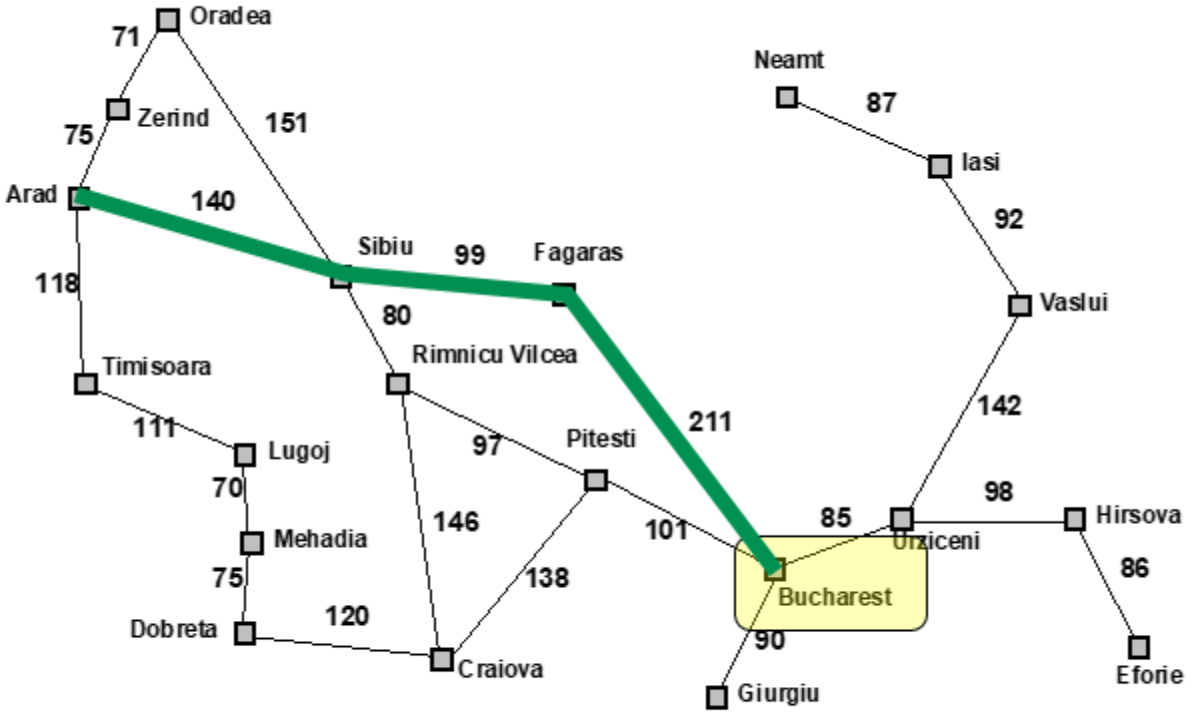
- Complete
  - No, it can get stuck in loops, e.g., with Oradea as goal, lasi -> Neamt -> lasi -> Neamt -> …
  - Complete in finite space with repeated-state checking
- Time
  - $O(b^m)$, but a good heuristic can give dramatic improvement
- Space
  - $O(b^m)$ keeps all nodes in memory
- Optimal ??

Properties of Greedy Search

- Complete
  - No, it can get stuck in loops, e.g., with Oradea as goal, lasi -> Neamt -> lasi -> Neamt -> …
  - Complete in finite space with repeated-state checking
- Time
  - $O(b^m)$, but a good heuristic can give dramatic improvement
- Space
  - $O(b^m)$ keeps all nodes in memory
- Optimal
  - No

Romania with Step Costs in km

Actual connections and distances



Possible heuristic:

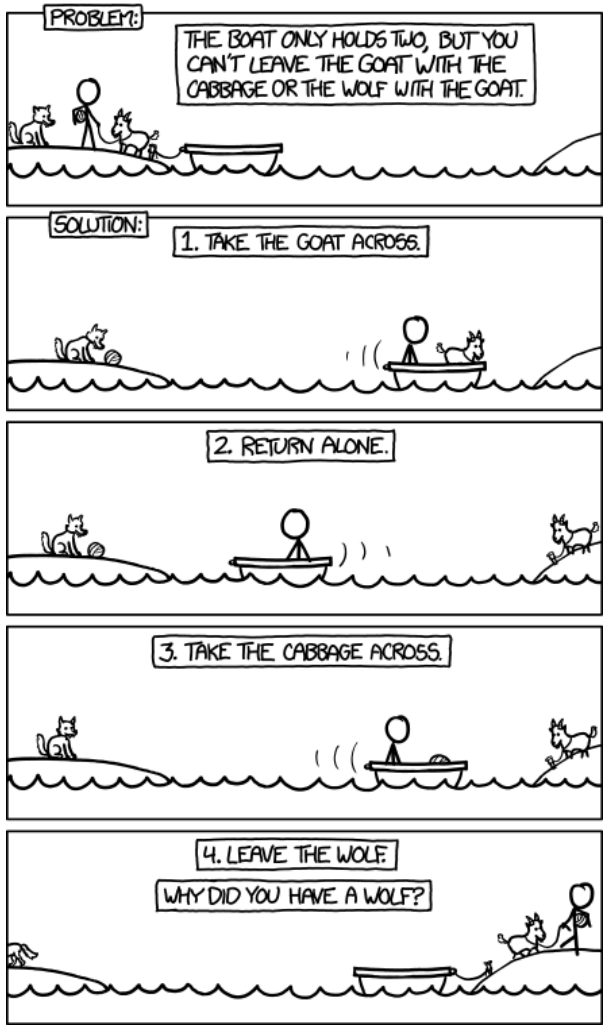
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Is this path optimal?

Heuristics

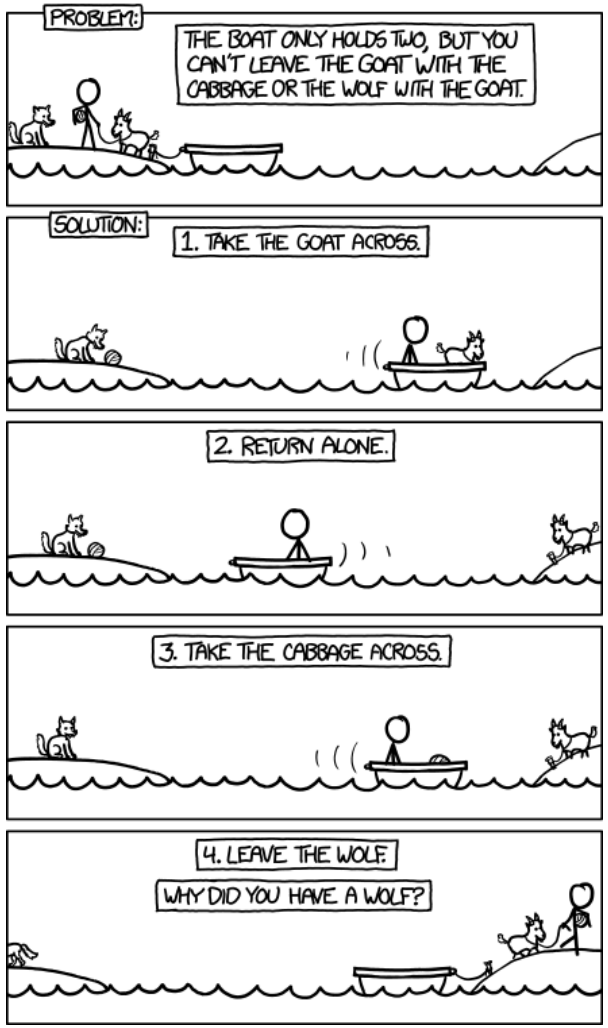
- Idea: Estimate the distance to the solution
- Must be computationally efficient (otherwise, one could just run the complete search algorithm to determine the actual cost!)
- Finding good heuristics makes a huge difference in computation time

Heuristics: Wolf-Goat-Cabbage



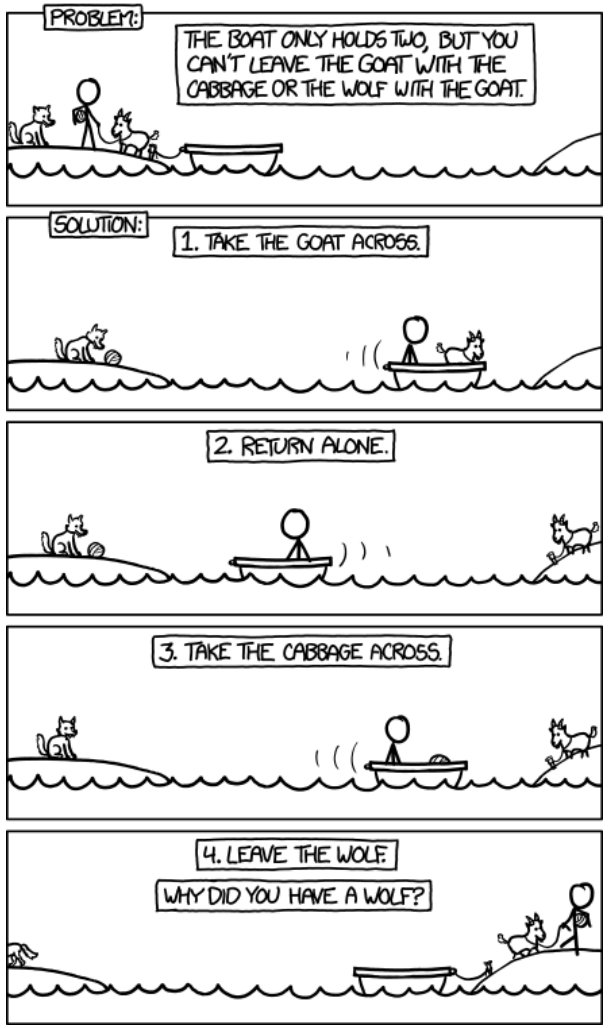
- Let's assume that "cost" is the number of boat trips.
- What would be a good heuristic?

Heuristics: Wolf-Goat-Cabbage



- Let's assume that "cost" is the number of boat trips.
- What would be a good heuristic?
- Example:
  - Number of "items" on the left bank (since we know we need to make at least that number of trips)

Heuristics: Wolf-Goat-Cabbage



- Let's assume that "cost" is the number of boat trips.
- What would be a good heuristic?
- Example:
  - Number of "items" on the left bank (since we know we need to make at least that number of trips)
  - Better:
    - ([*number of items in left bank*] – 1) * 2
    - Add 1 if boat on right bank and [*number of items in left bank*] > 0

Heuristics: 8-Puzzle

8		1
3	2	6
7	4	5

- Cost: number of moves
- What would be a good heuristic?

Heuristics: 8-Puzzle

8		1
3	2	6
7	4	5

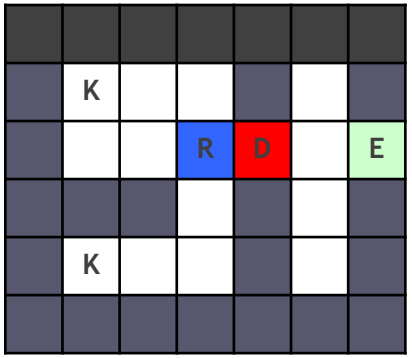
- Cost: number of moves
- What would be a good heuristic?
  - Heuristic 1:
    - Number of blocks out of place

Heuristics: 8-Puzzle

8		1
3	2	6
7	4	5

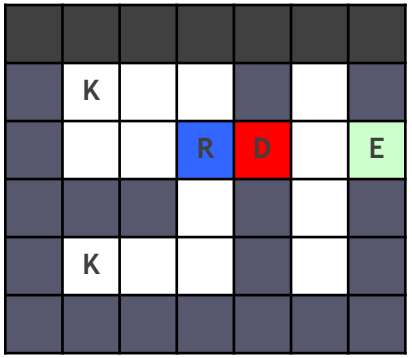
- Cost: number of moves
- What would be a good heuristic?
  - Heuristic 1:
    - Number of blocks out of place
  - Heuristic 2:
    - Distance of each block to its target position

Heuristic: Robot navigation with keys



- A robot (blue) needs to exit a maze (light green), and needs to pick up keys to cross doors (red)

Heuristic: Robot navigation with keys



- A robot (blue) needs to exit a maze (light green), and needs to pick up keys to cross doors (red)
- Heuristic 1:
  - Simply Euclidean distance to goal (ignoring keys and doors)
  - Can you improve it?

A Note on Graph Search

- Repeated-state checking:
  - When the search state is a graph, strategies like DFS can get stuck in loops
  - To avoid this, we generally want to keep a list of already-visited nodes – a "CLOSED" list
- In DFS:
  - If we want to avoid repeating states completely, we need to keep ALL visited states in memory (in the CLOSED list)
  - If we just want to avoid loops, we only need to remember the current branch (linear memory as a function of *m* = maximum depth)

A* Search

- Pronounced “A-Star”
- At each cycle, expand the node with the lowest *f(n)*

$$f(n) = g(n) + h(n)$$

Real cost from Start to n

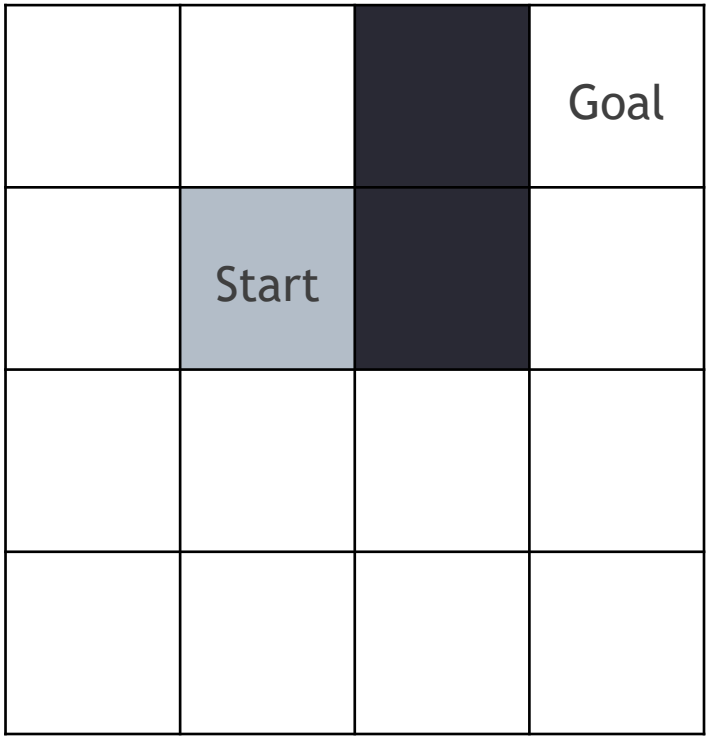
Heuristic

- A* implementations assume repeated-state checking (i.e., assume the search space is a graph)
- Thus, they use two lists simultaneously:
  - OPEN: list of nodes that need to be processed
  - CLOSED: list of nodes that have already been processed

Example: A*



OPEN = [Start]
CLOSED = []

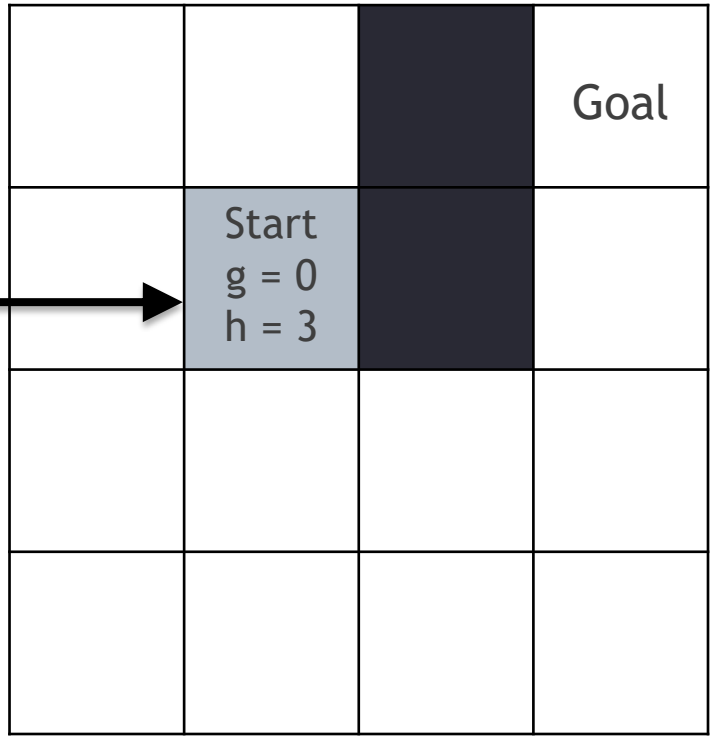


Heuristic used: **Manhattan Distance**

Example: A*



OPEN = [Start]
CLOSED = []



Assigns an "estimated cost", *f*, to each node:

$f(n) = g(n) + h(n)$

Real cost from Start to n

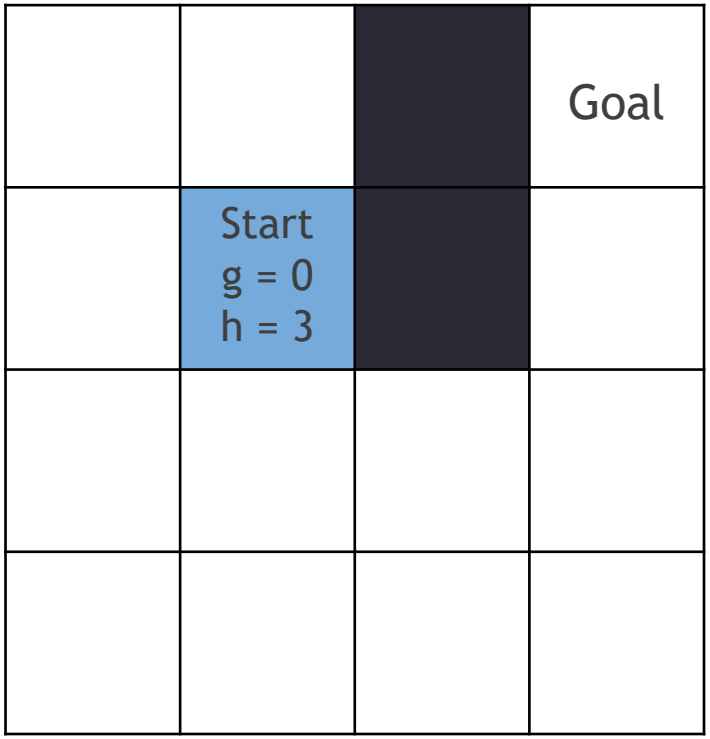
Heuristic

Heuristic used: **Manhattan Distance**

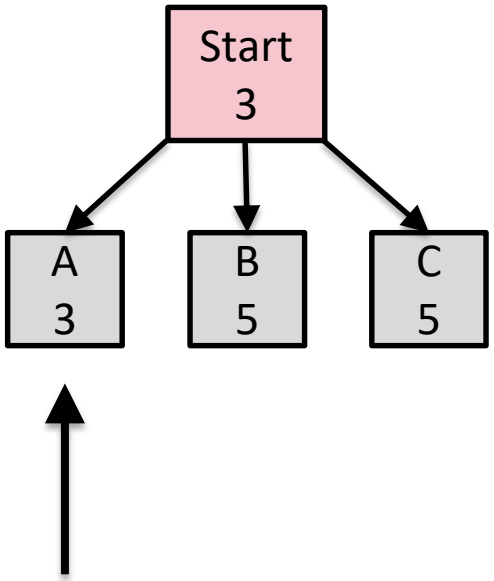
Example: A*

Start
3

OPEN = [Start]
CLOSED = []



Example: A*

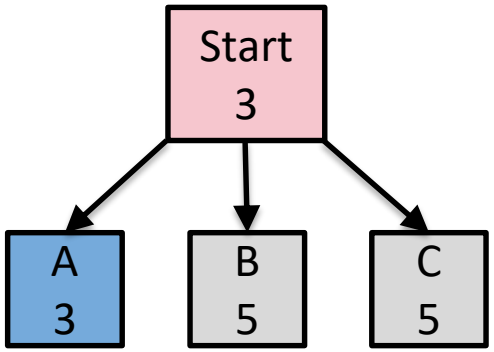


Expands the node with the lowest estimated cost first

OPEN = [A, B, C]
CLOSED = [Start]

	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
	C g = 1 h = 4		

Example: A*

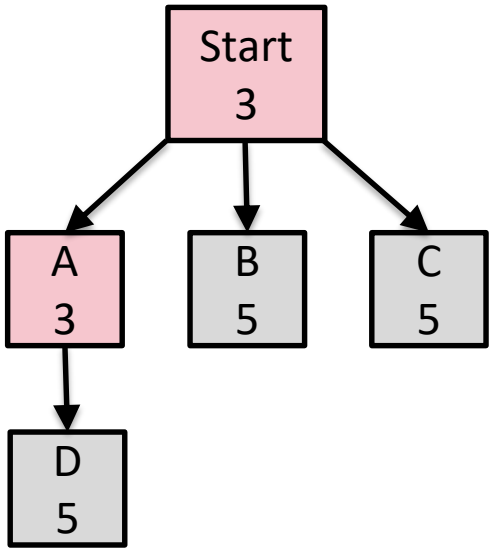


Expands the node with the lowest estimated cost first

OPEN = [A, B, C]
CLOSED = [Start]

	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
	C g = 1 h = 4		

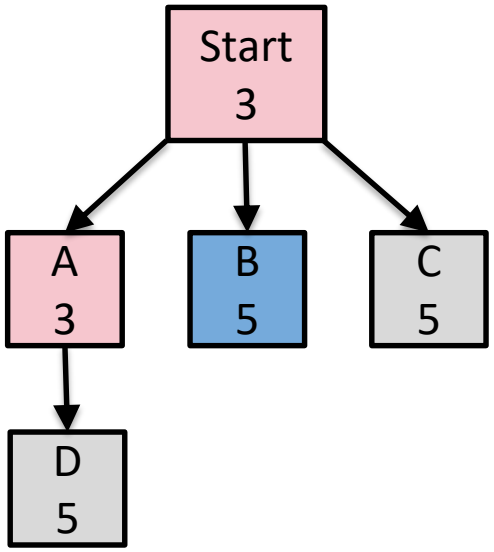
Example: A*



OPEN = [B, C, D]
CLOSED = [Start, A]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
	C g = 1 h = 4		

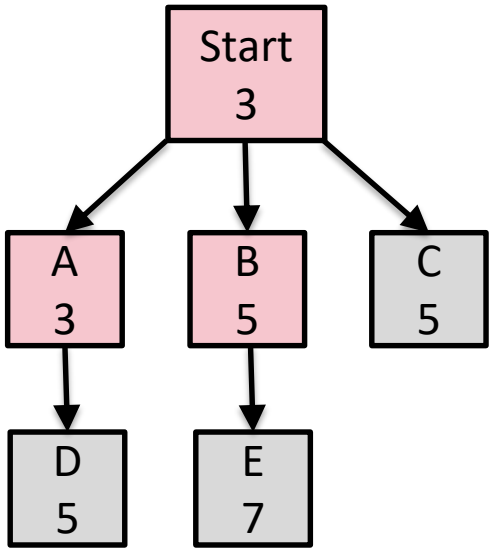
Example: A*



OPEN = [B, C, D]
CLOSED = [Start, A]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
	C g = 1 h = 4		

Example: A*

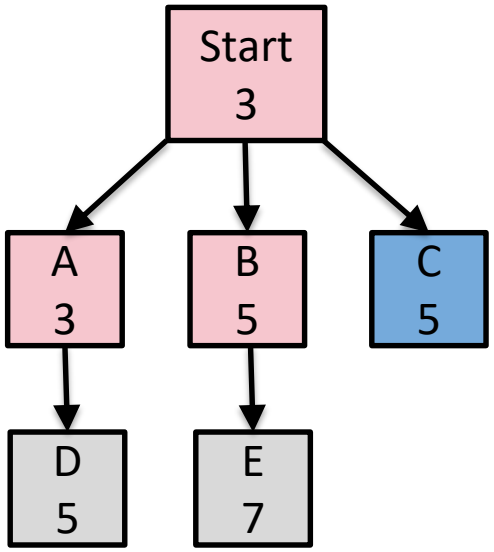


OPEN = [C, D, E]

CLOSED = [Start, A, B]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4		

Example: A*

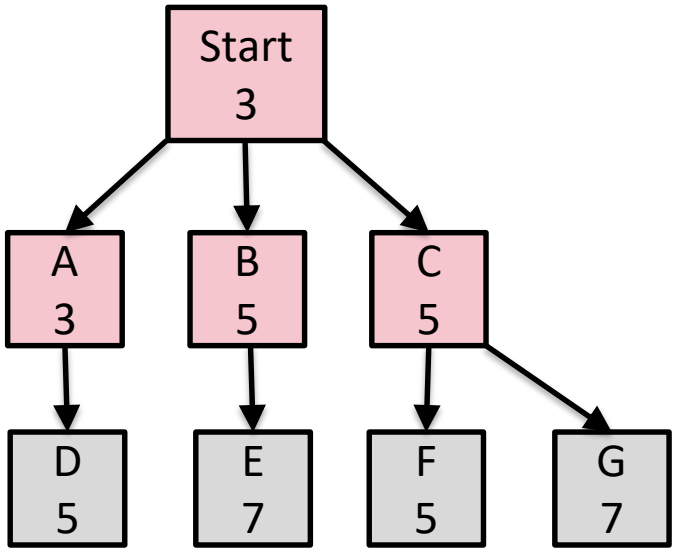


OPEN = [C, D, E]

CLOSED = [Start, A, B]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4		

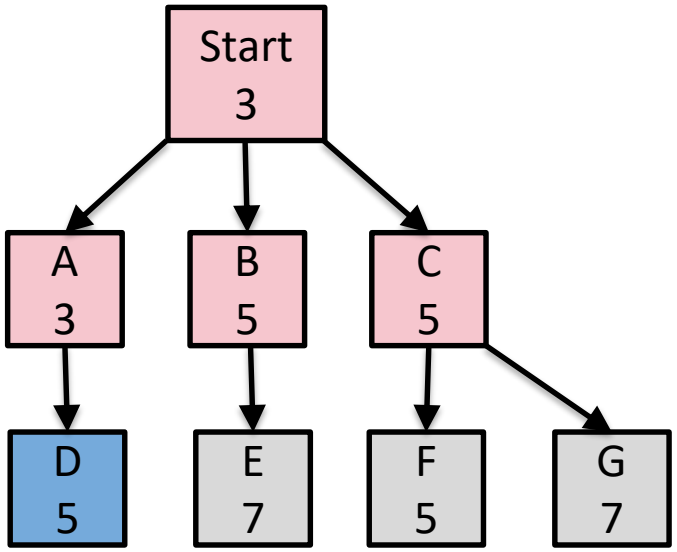
Example: A*



OPEN = [D, F, E, G]
CLOSED = [Start, A, B, C]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	
	G g = 2 h = 5		

Example: A*

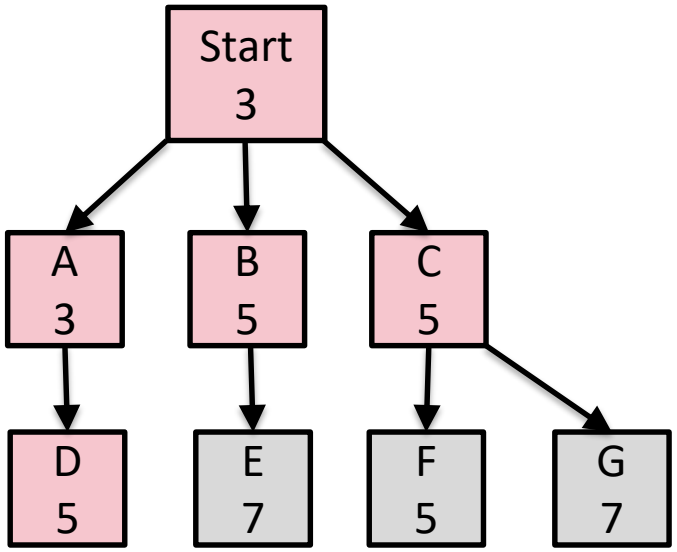


OPEN = [D, F, E, G]

CLOSED = [Start, A, B, C]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	
	G g = 2 h = 5		

Example: A*

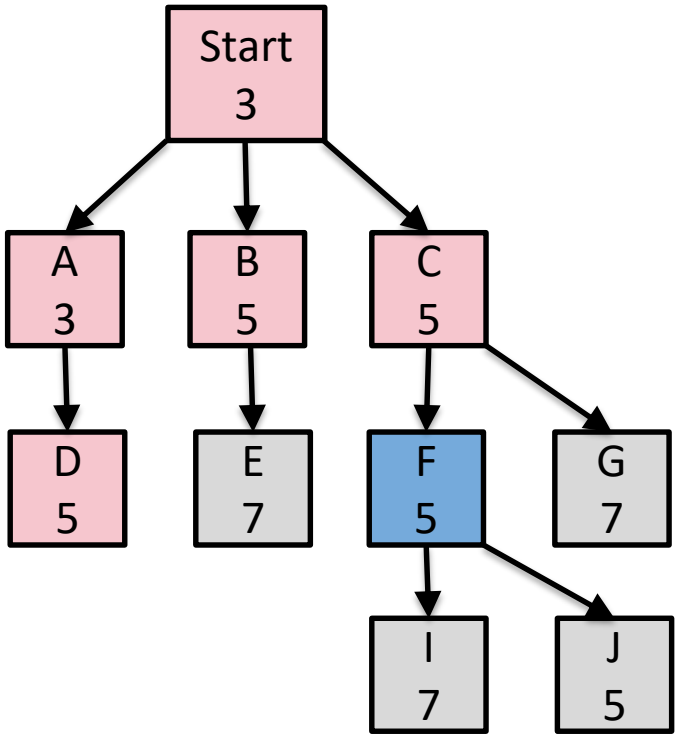


OPEN = [F, E, G]

CLOSED = [Start, A, B, C, D]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	
	G g = 2 h = 5		

Example: A*

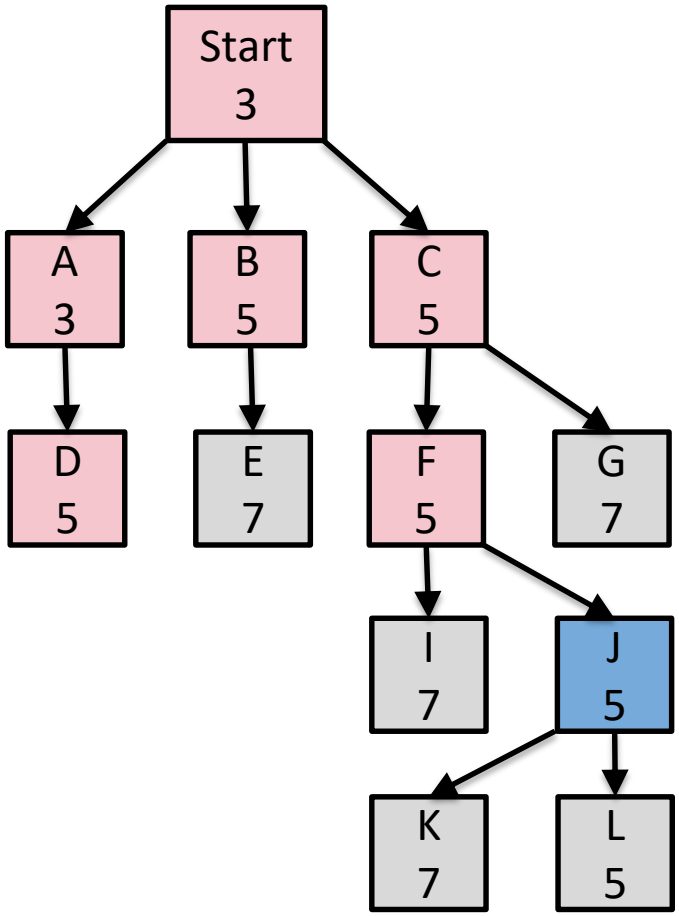


OPEN = [J, E, G, I]

CLOSED = [Start, A, B, C, D, F]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	

Example: A*

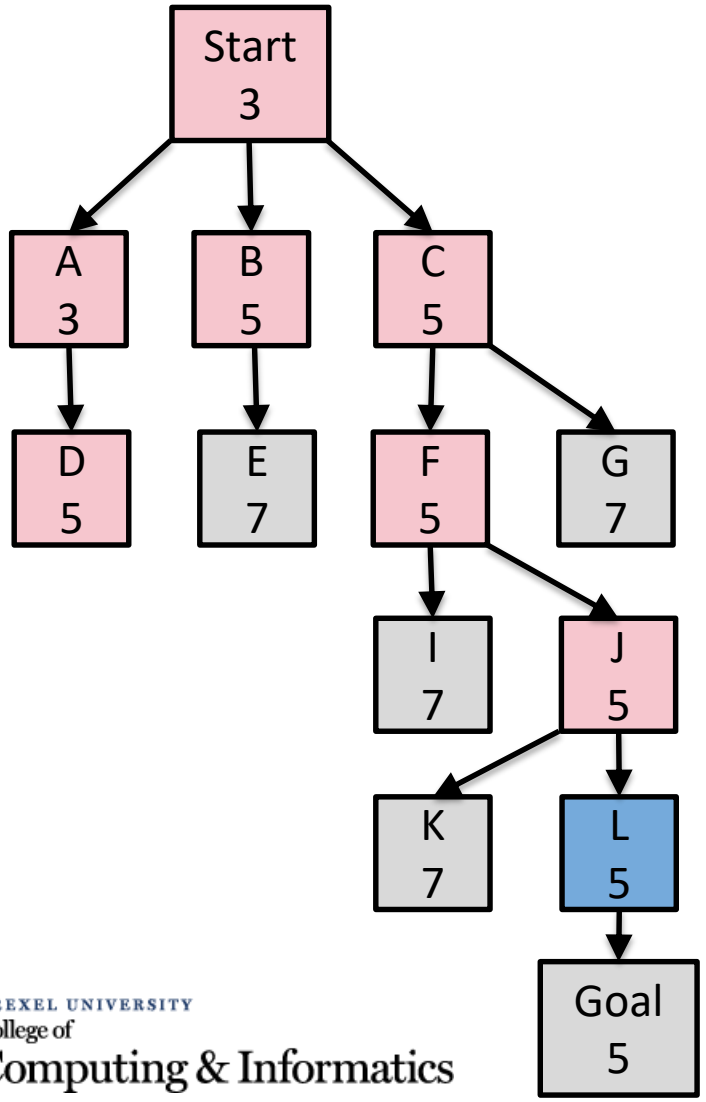


OPEN = [L, E, G, I, K]

CLOSED = [Start, A, B, C, D, F, J]

D g = 2 h = 3	A g = 1 h = 2		Goal
B g = 1 h = 4	Start g = 0 h = 3		L g = 4 h = 1
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	K g = 4 h = 3

Example: A*

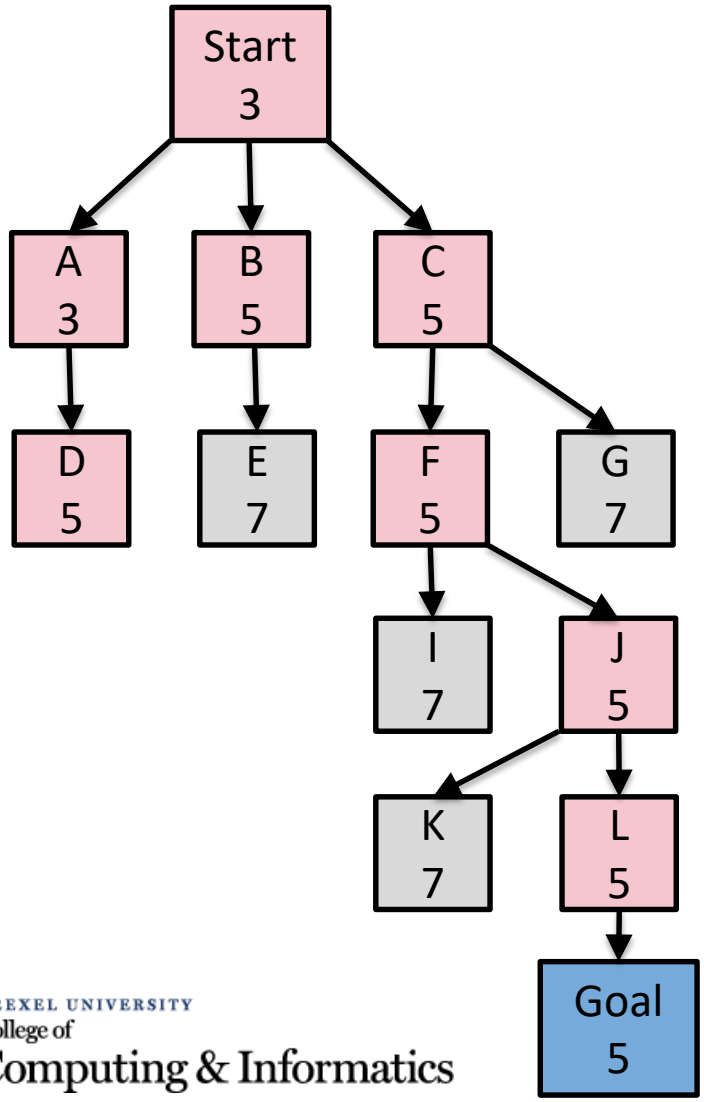


OPEN = [Goal, E, G, I, K]

CLOSED = [Start, A, B, C, D, F, J, L]

D g = 2 h = 3	A g = 1 h = 2		Goal g = 5 h = 0
B g = 1 h = 4	Start g = 0 h = 3		L g = 4 h = 1
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	K g = 4 h = 3

Example: A*



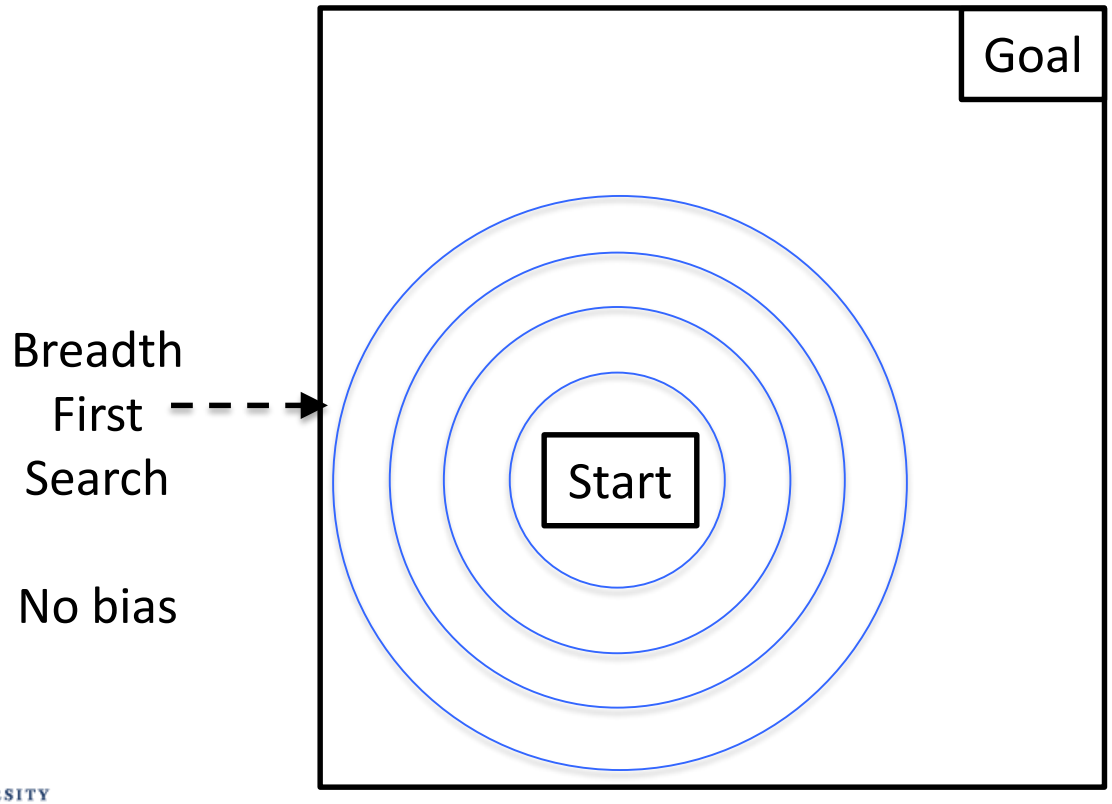
OPEN = [E, G, I, K]

CLOSED = [Start, A, B, C, D, F, J, L, Goal]

D g = 2 h = 3	A g = 1 h = 2		Goal g = 5 h = 0
B g = 1 h = 4	Start g = 0 h = 3		L g = 4 h = 1
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	K g = 4 h = 3

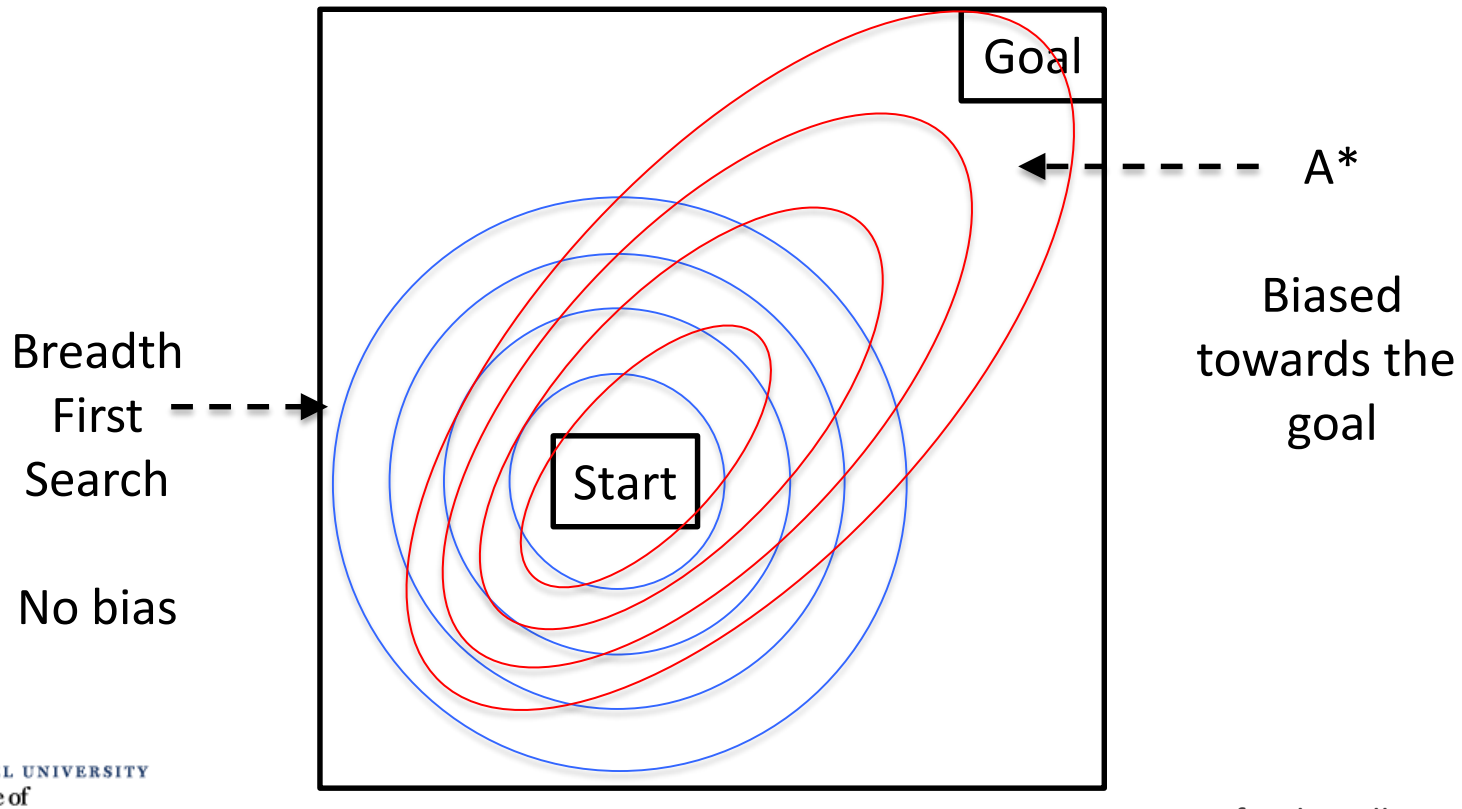
A* Intuition

- The heuristic biases the search of the algorithm towards the goal:

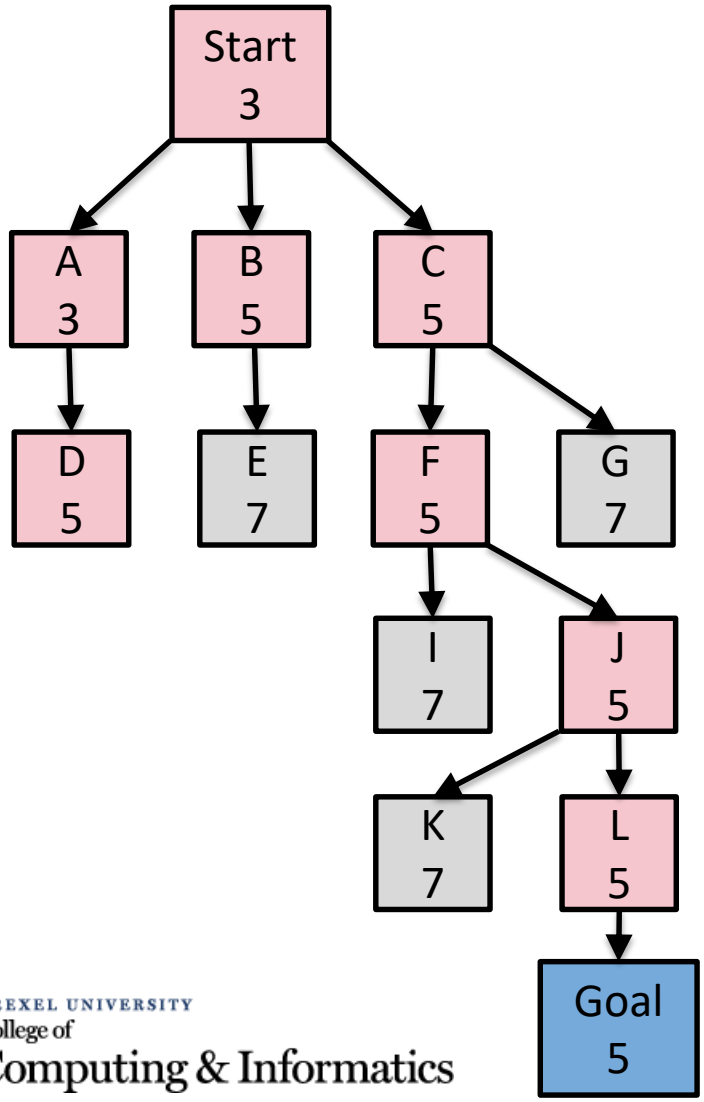


A* Intuition

- The heuristic biases the search of the algorithm towards the goal:



Example: A*



OPEN = [E, G, I, K]

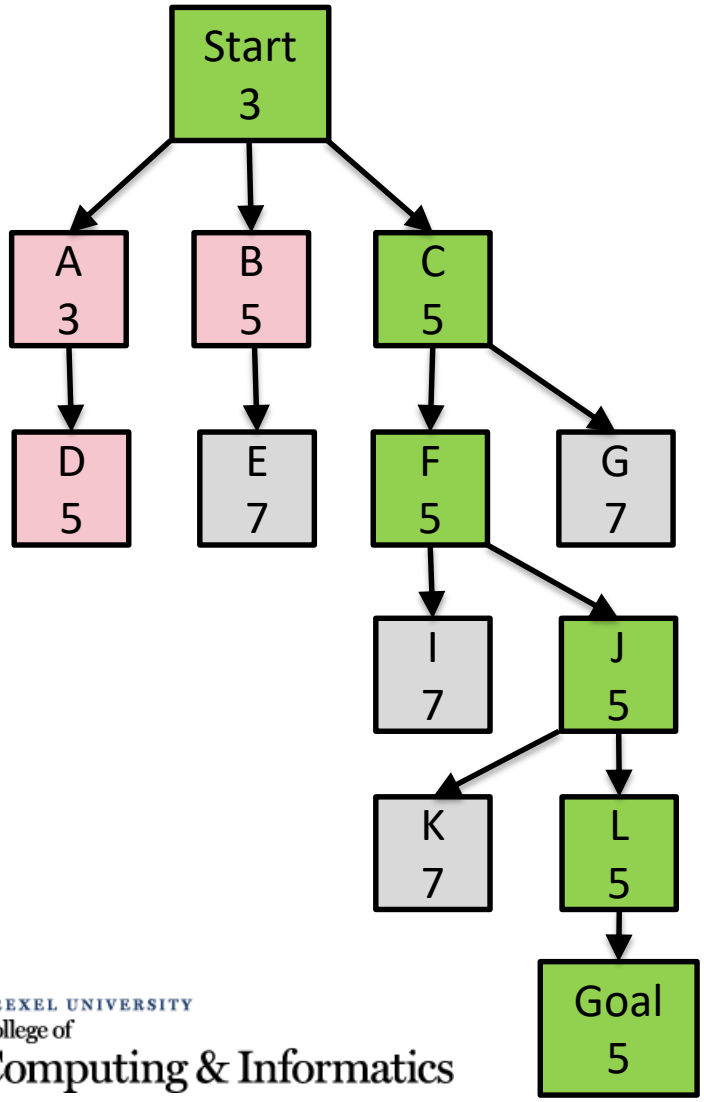
CLOSED = [Start, A, B, C, D, F, J, L, Goal]

D g = 2 h = 3	A g = 1 h = 2		Goal g = 5 h = 0
B g = 1 h = 4	Start g = 0 h = 3		L g = 4 h = 1
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	K g = 4 h = 3

Implementation Notes

- Remember the distinction between **state** and **node**
- State:
  - The configuration of the problem (e.g., coordinates of a robot, positions of the pieces in the 8-puzzle, etc.)
- Node:
  - State + {current cost (g), current heuristic (h), parent node, action that got us here from the parent node, …}
  - It is important to remember who was the parent, and which action was applied, so that once the solution is found, we can reconstruct the path

Example: A*



OPEN = [E, G, I, K]

CLOSED = [Start, A, B, C, D, F, J, L, Goal]

D g = 2 h = 3	A g = 1 h = 2		Goal g = 5 h = 0
B g = 1 h = 4	Start g = 0 h = 3		L g = 4 h = 1
E g = 2 h = 5	C g = 1 h = 4	F g = 2 h = 3	J g = 3 h = 2
	G g = 2 h = 5	I g = 3 h = 4	K g = 4 h = 3

A* Pseudocode

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

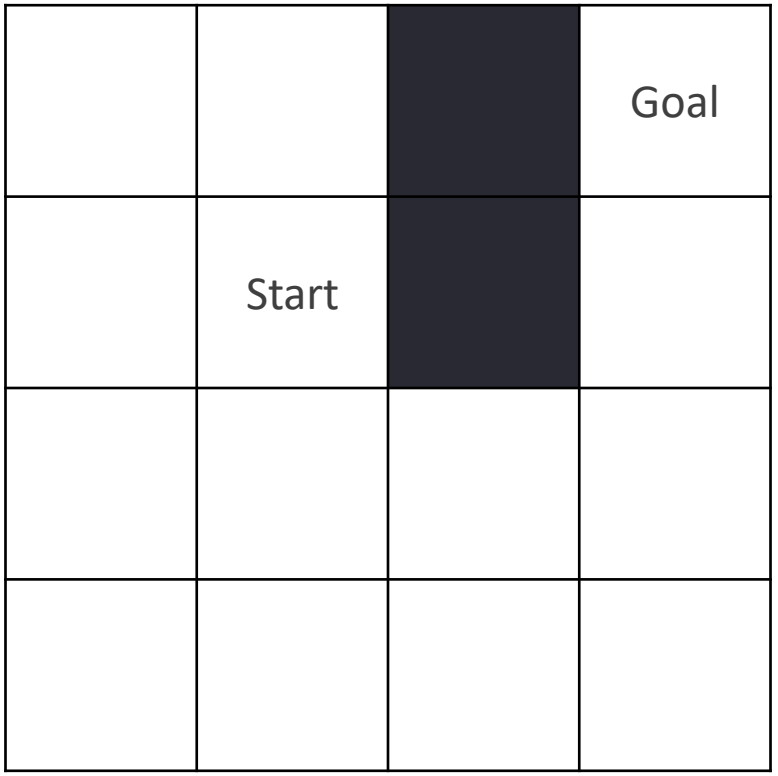
A* Pseudocode

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
    CLOSED.add(N)
    FOR all children M of N not in CLOSED:
            M.parent = N
            M.g = N.g + 1
            M.h = heuristic(M)
            OPEN.add(M)
    ENDFOR
ENDWHILE
```

Differences with respect to Breadth First Search

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

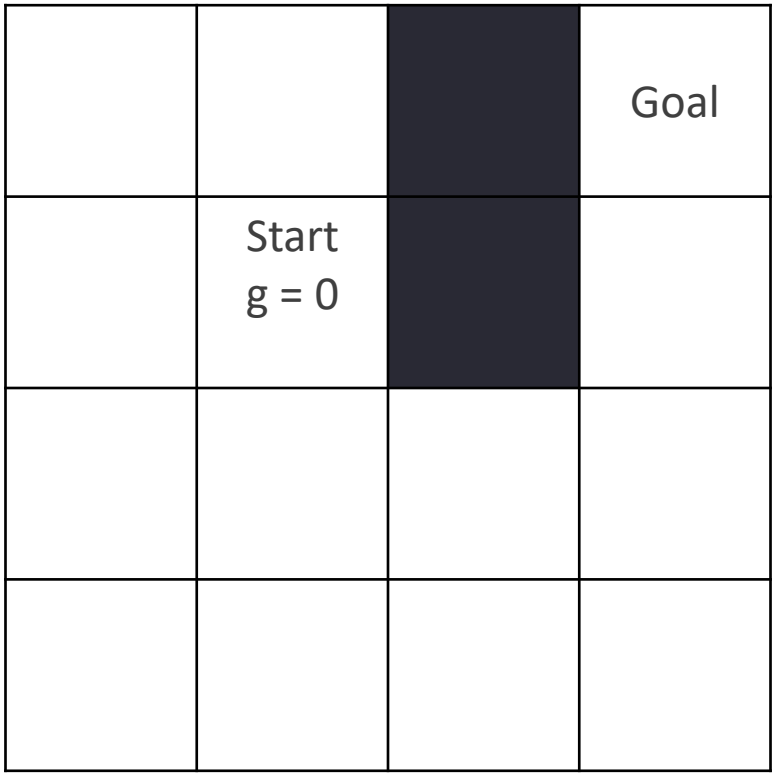


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

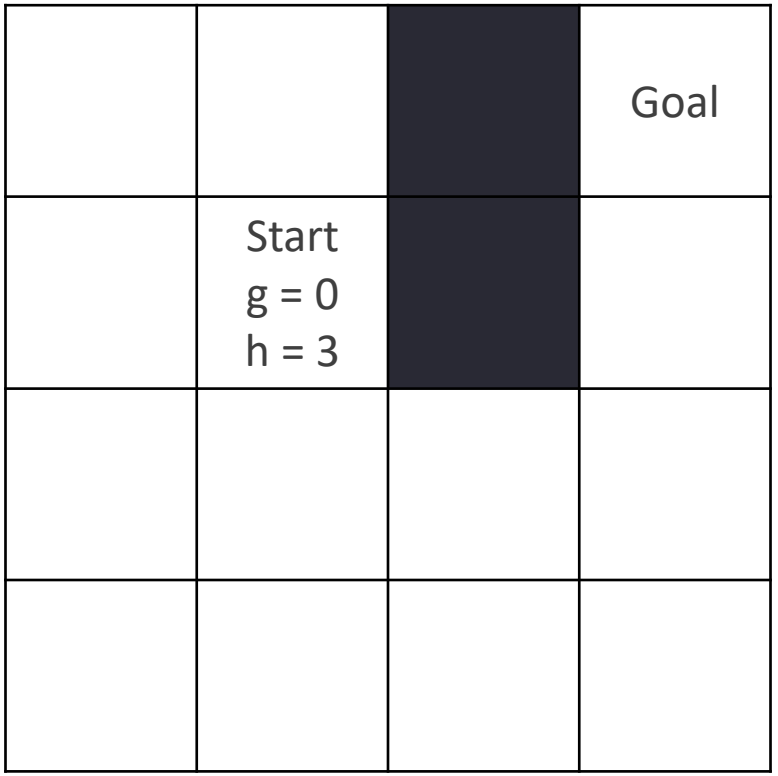


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

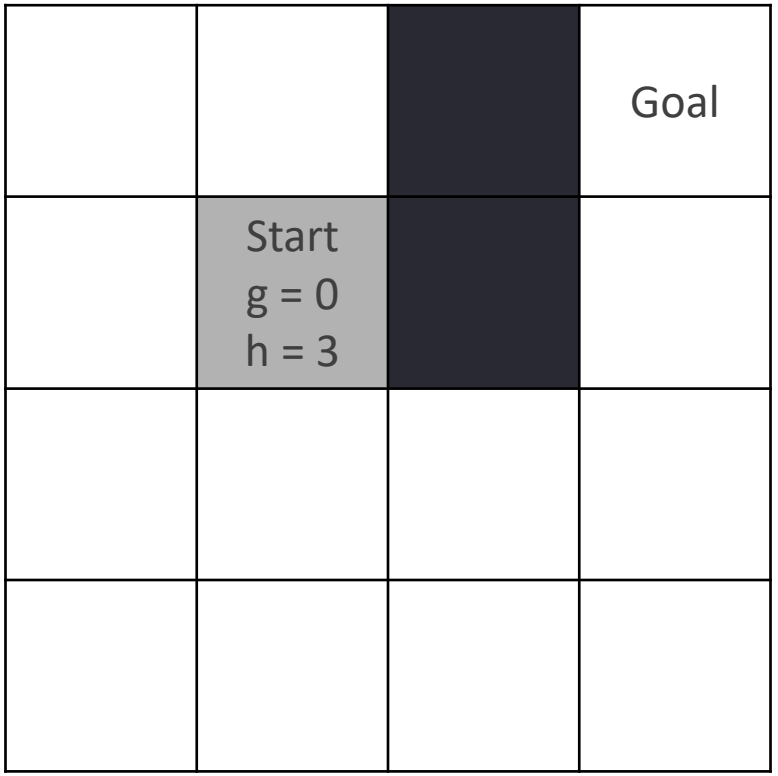


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

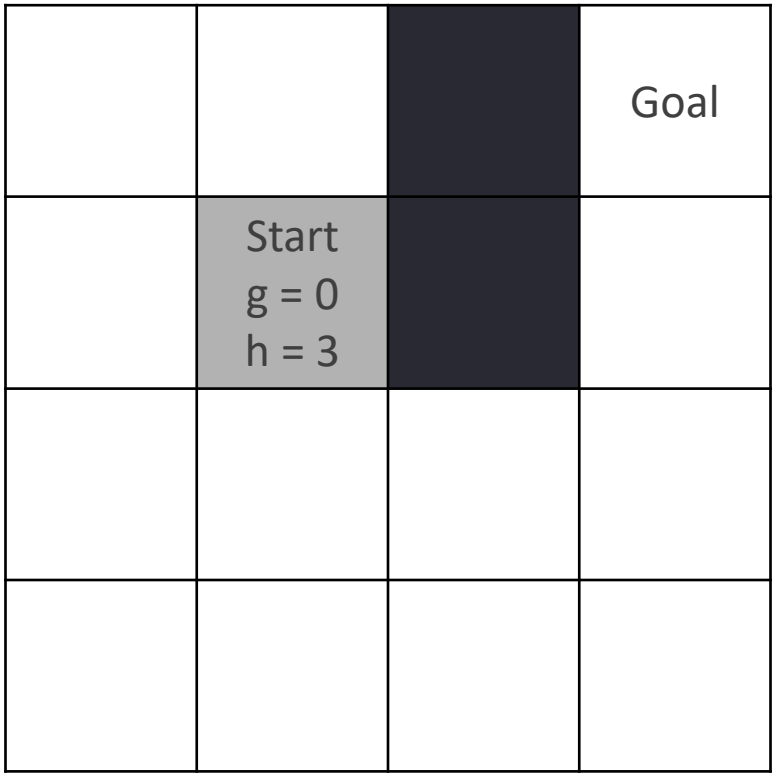


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

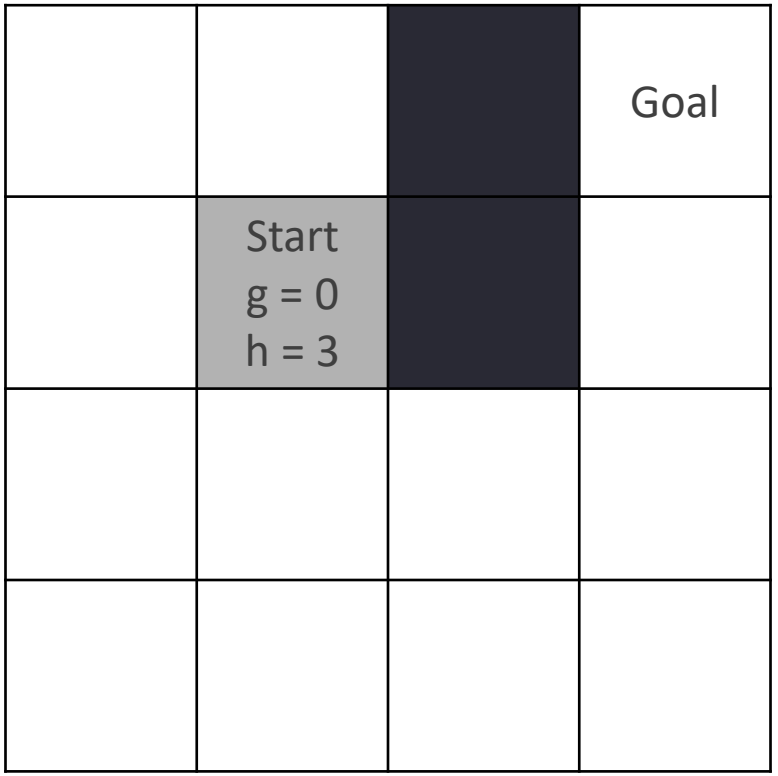


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

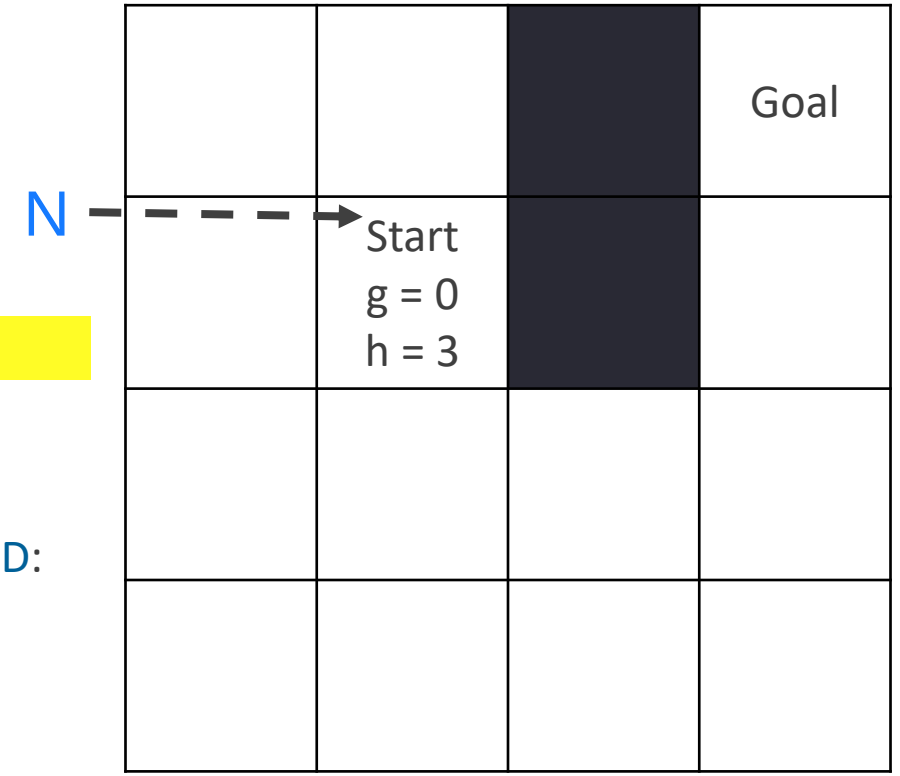


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

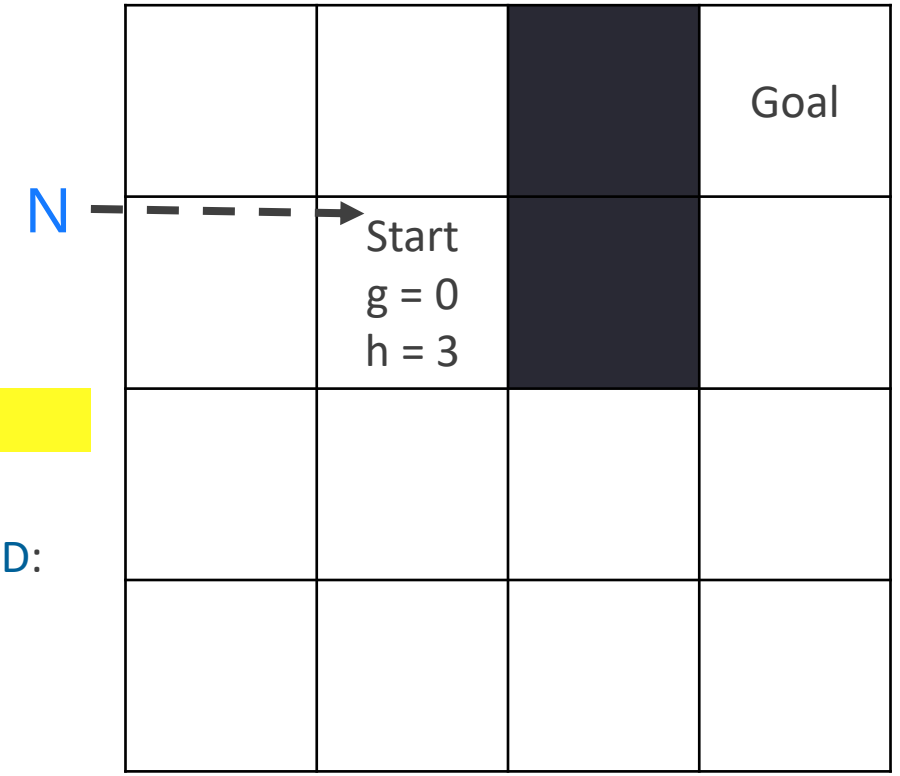


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

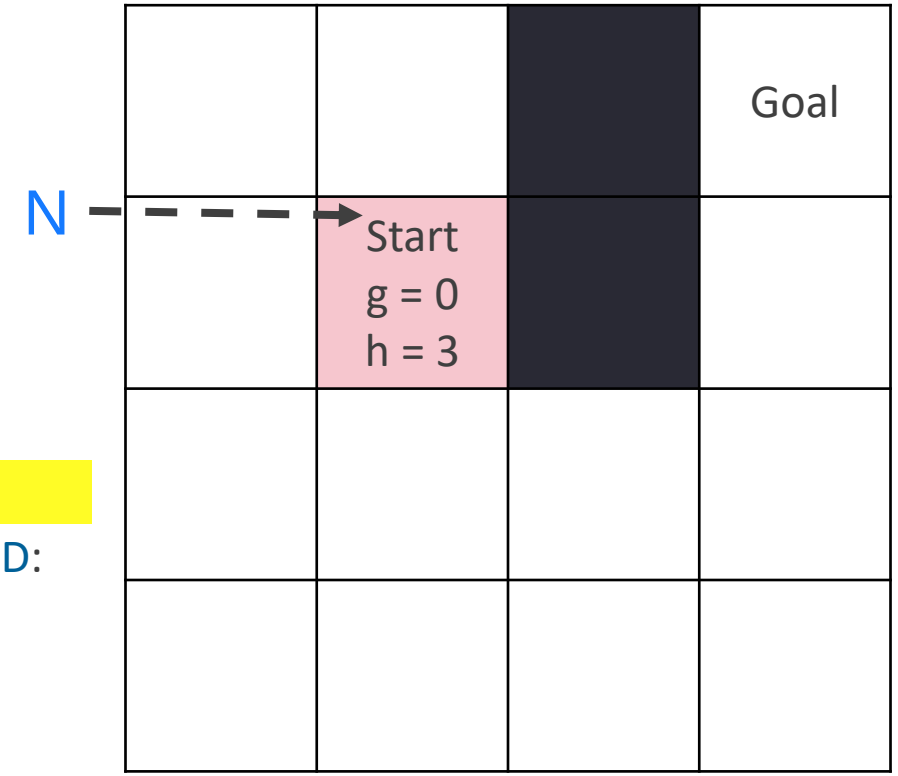
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```



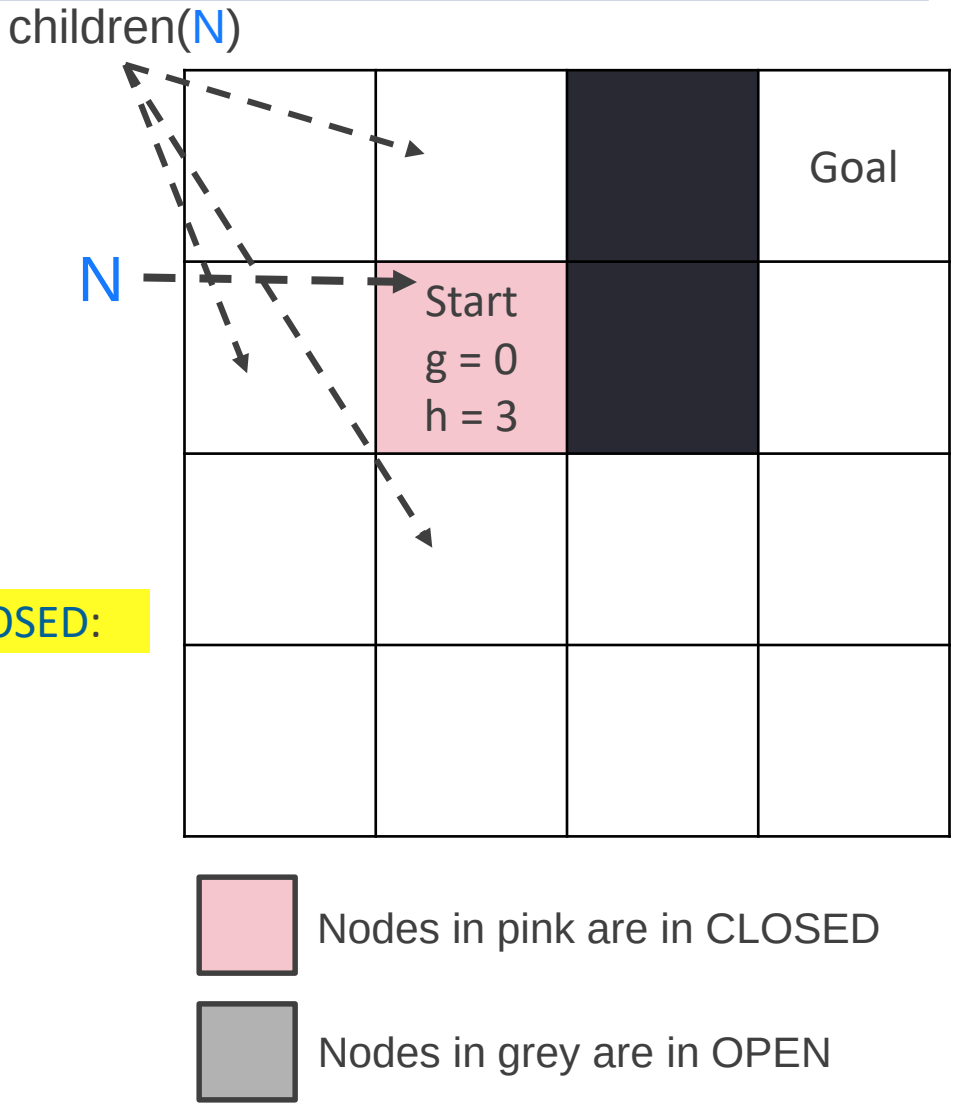
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
    CLOSED.add(N)
    FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
    ENDFOR
ENDWHILE
```



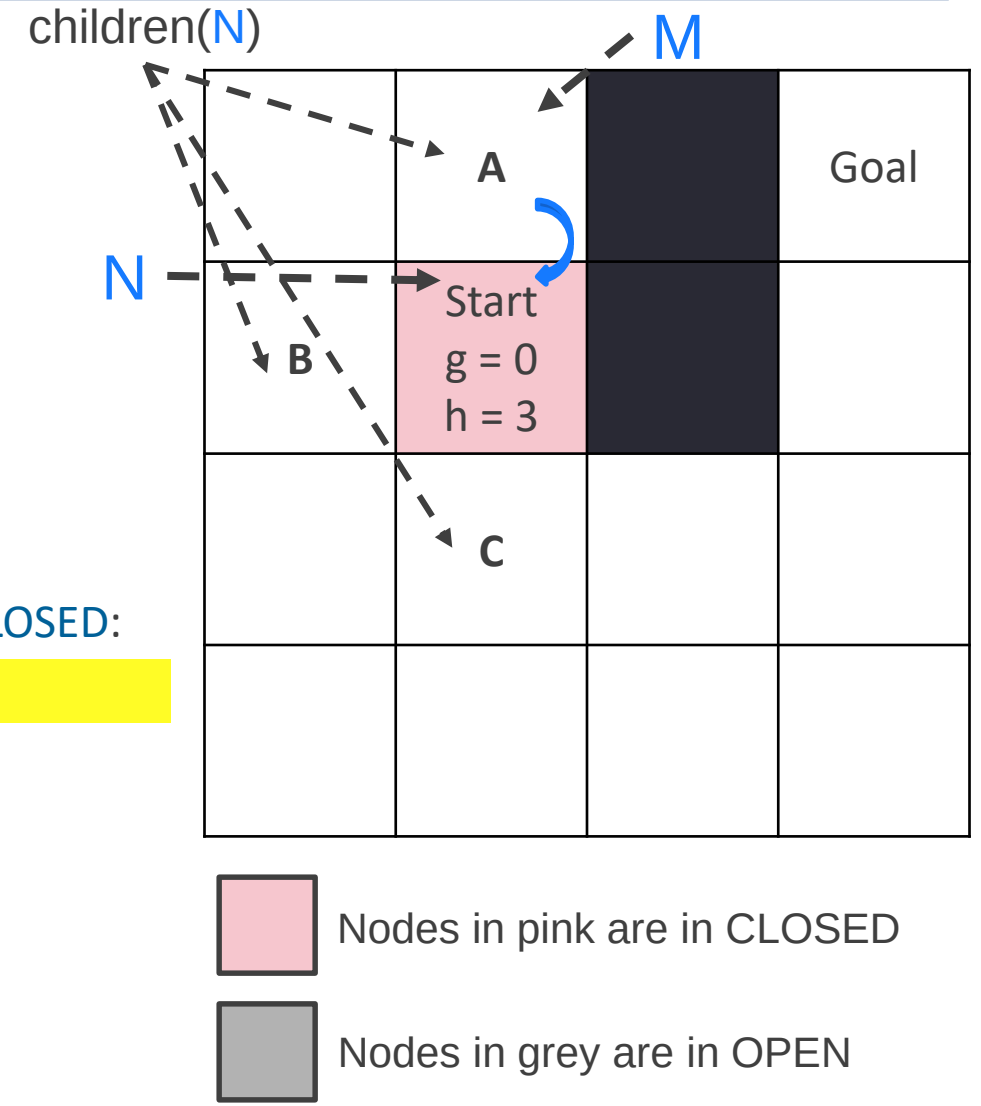
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```



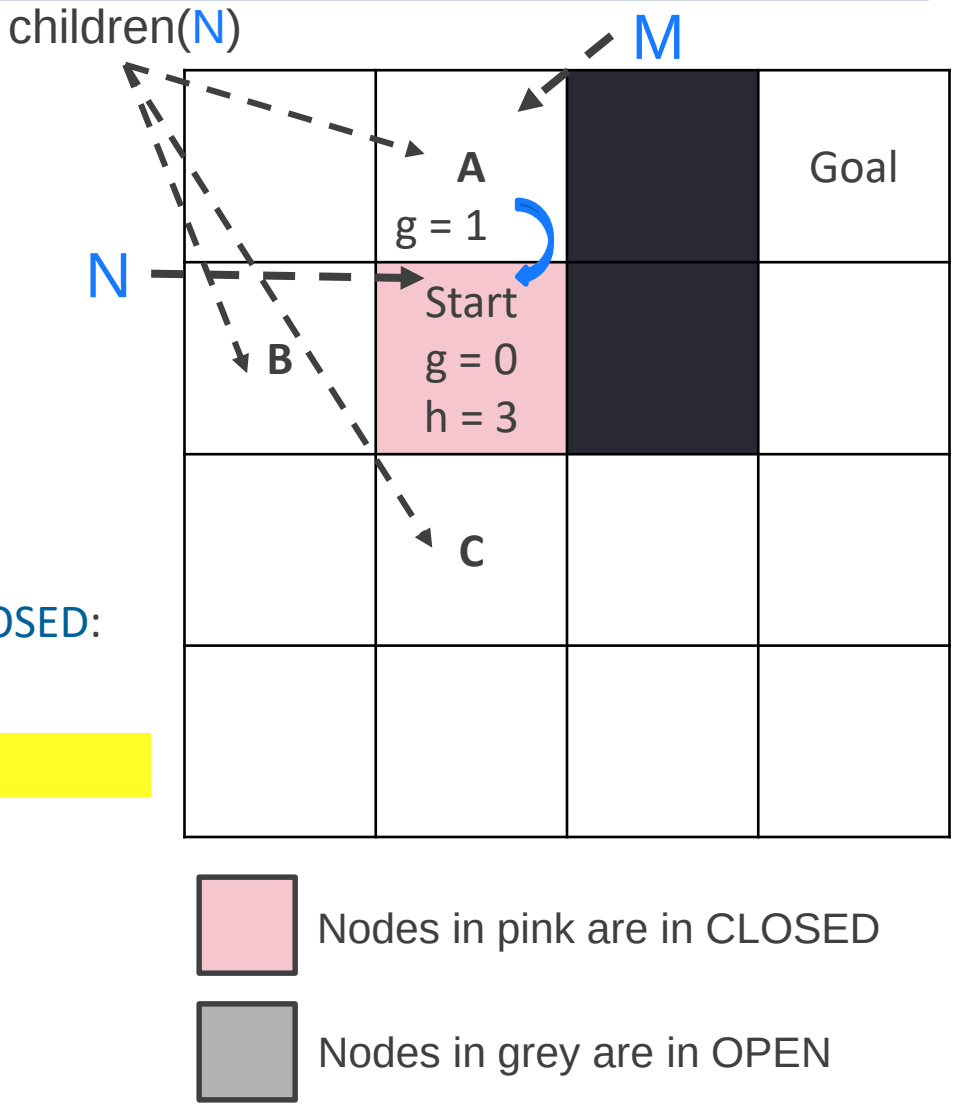
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```



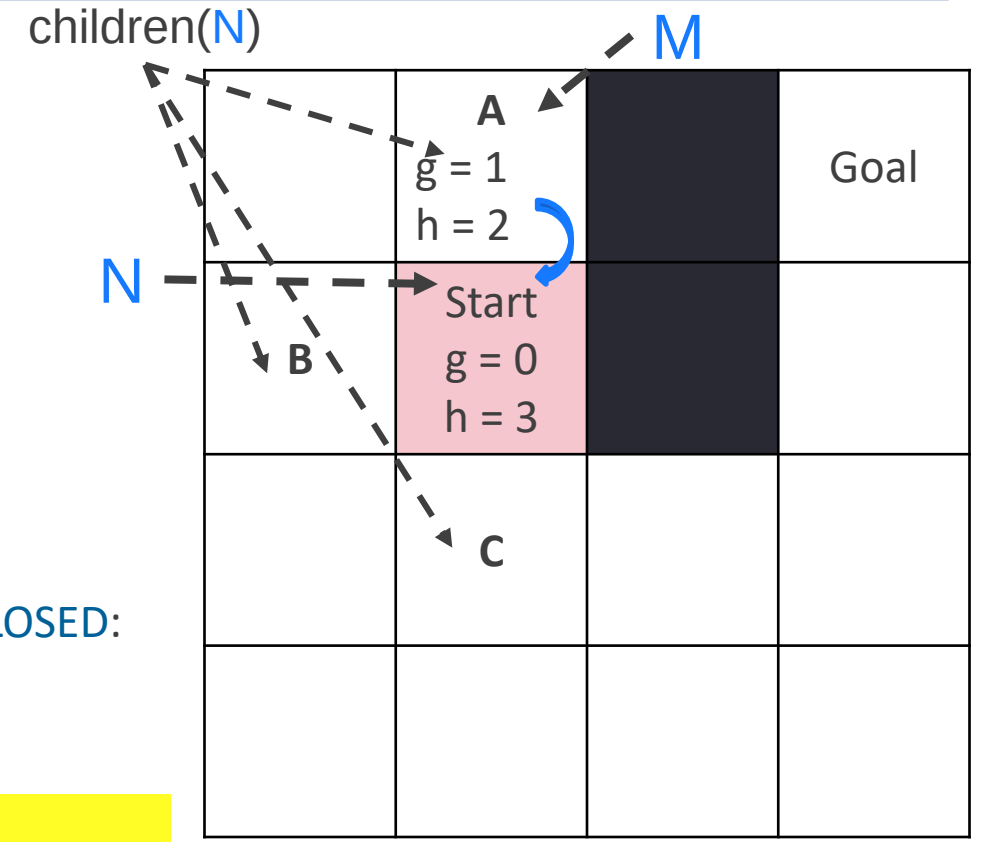
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```



A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

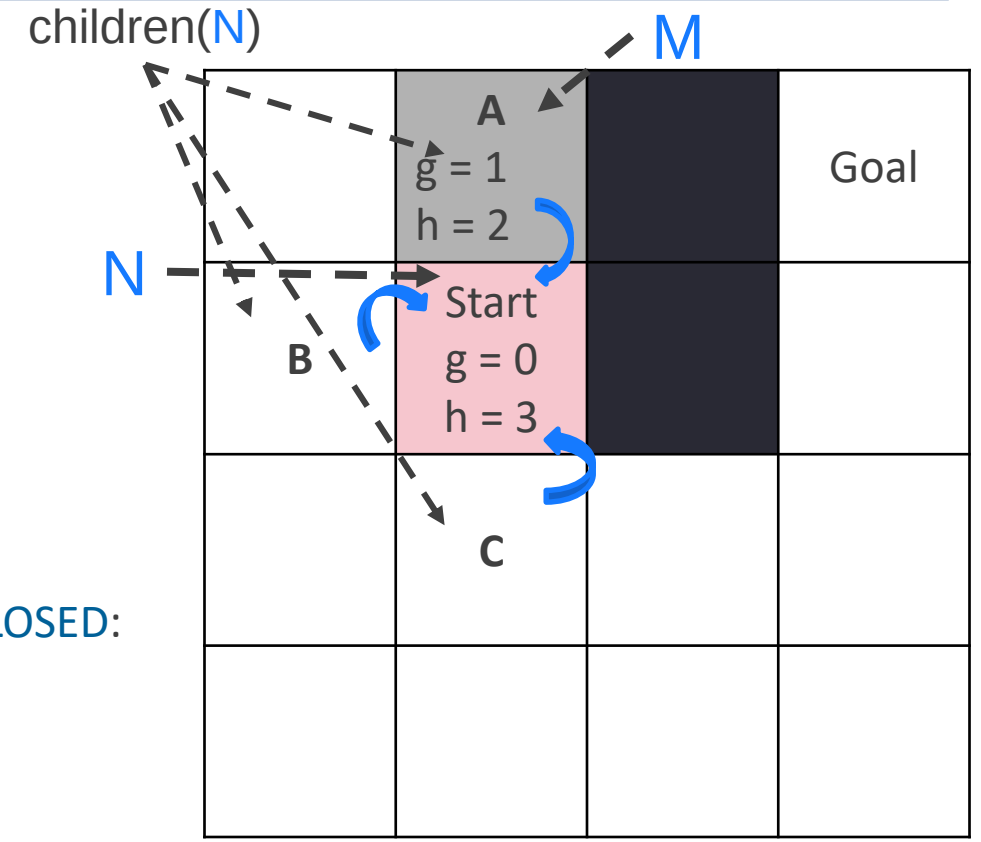


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

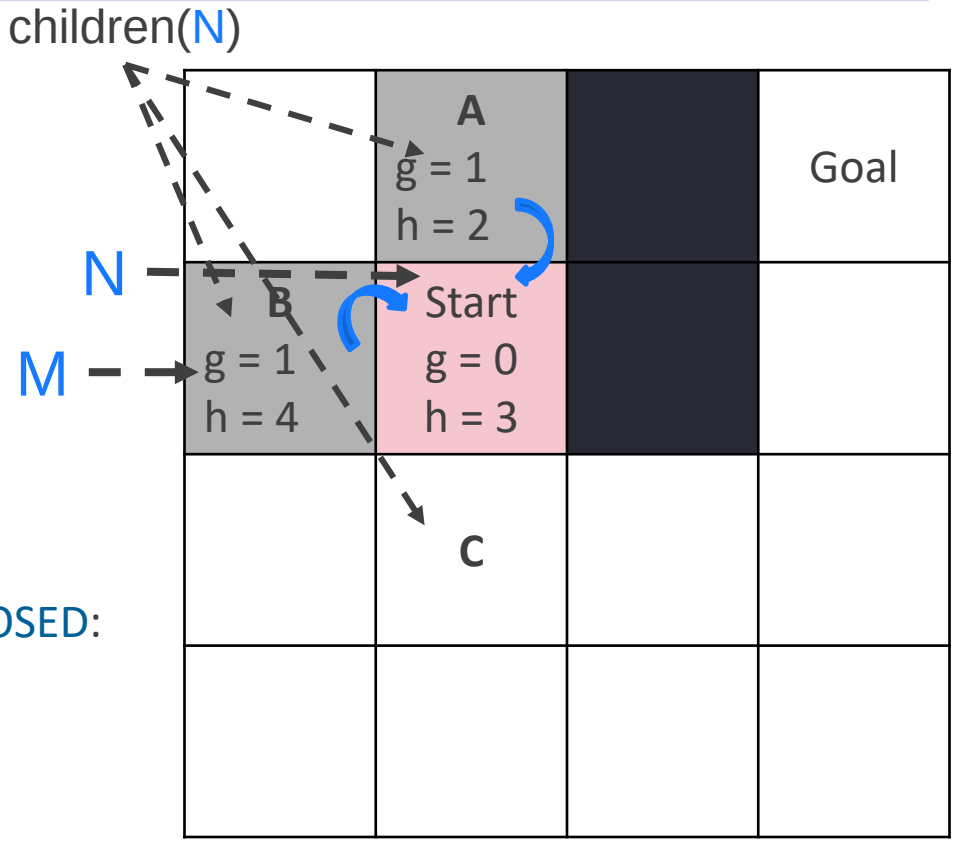


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

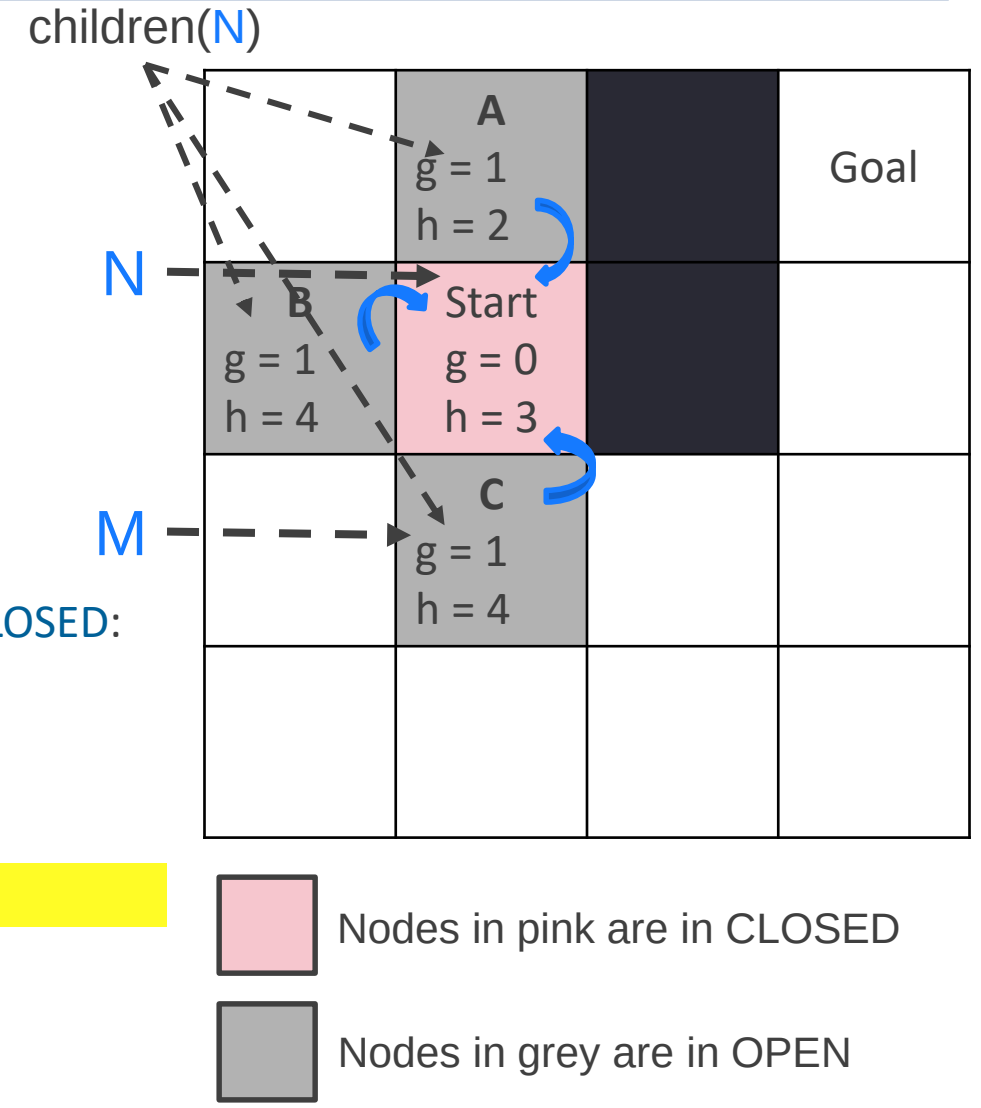


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

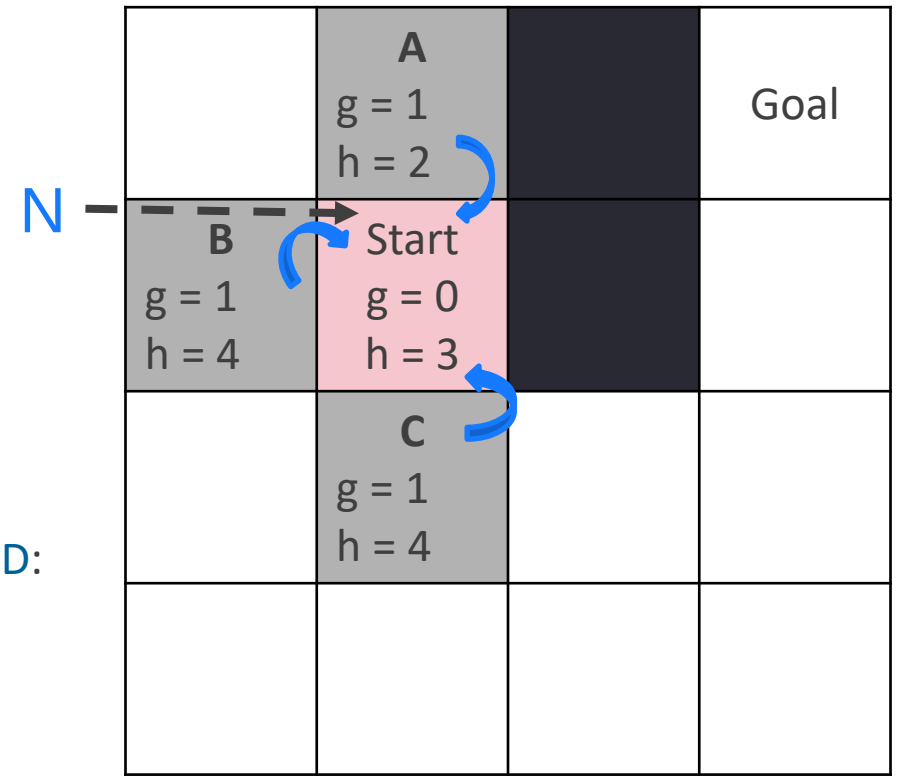
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```



A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

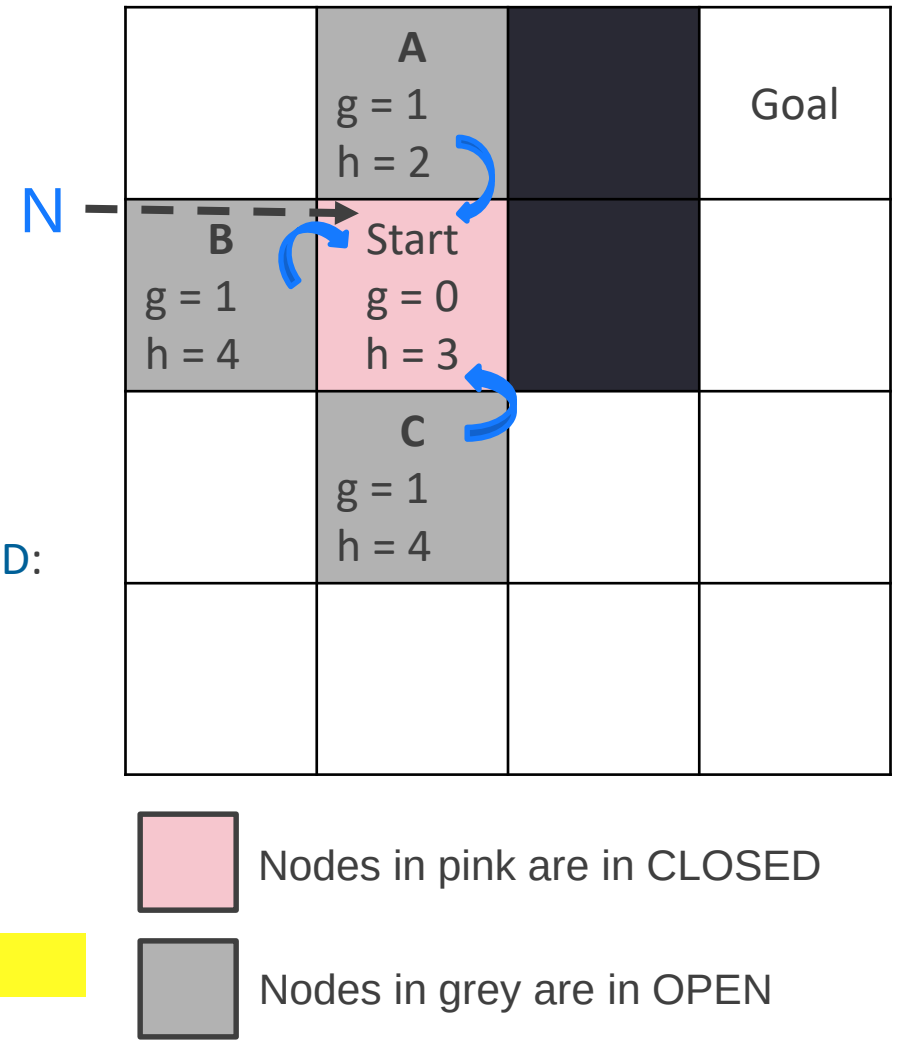


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

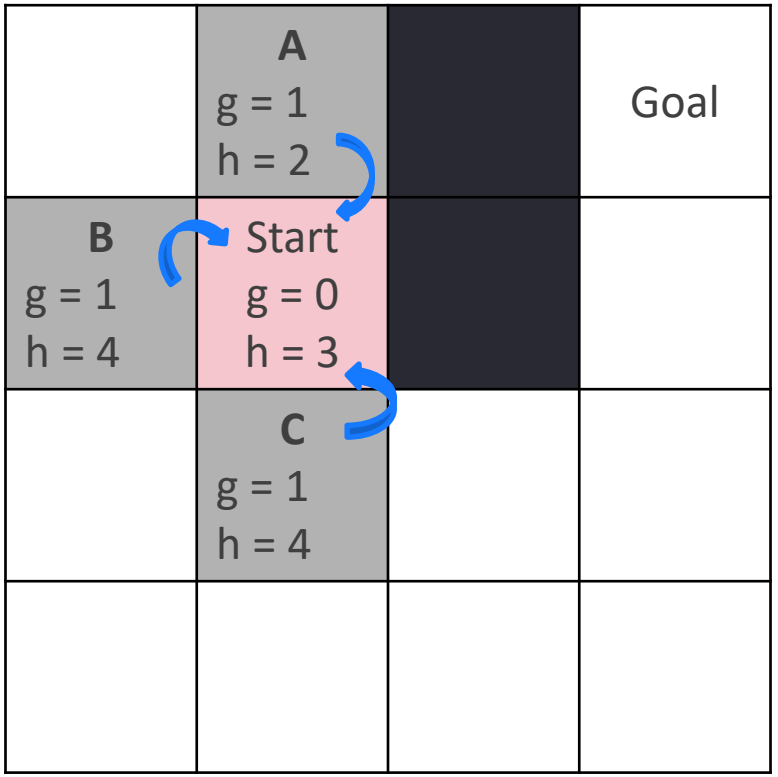
A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```



A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
          M.parent = N
          M.g = N.g + 1
          M.h = heuristic(M)
          OPEN.add(M)
  ENDFOR
ENDWHILE
```

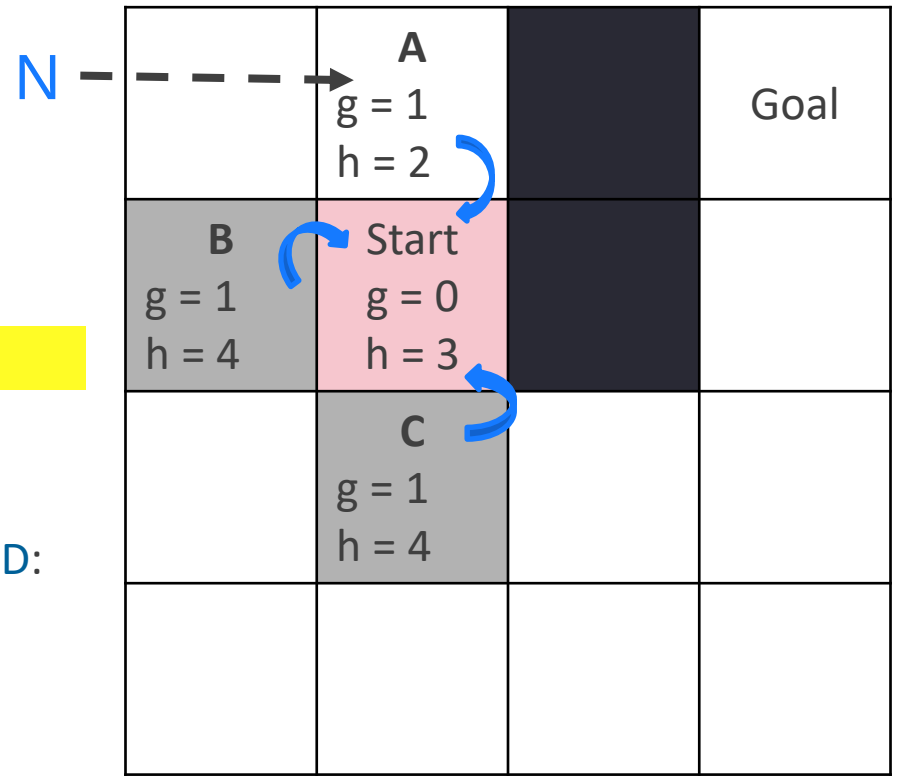


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

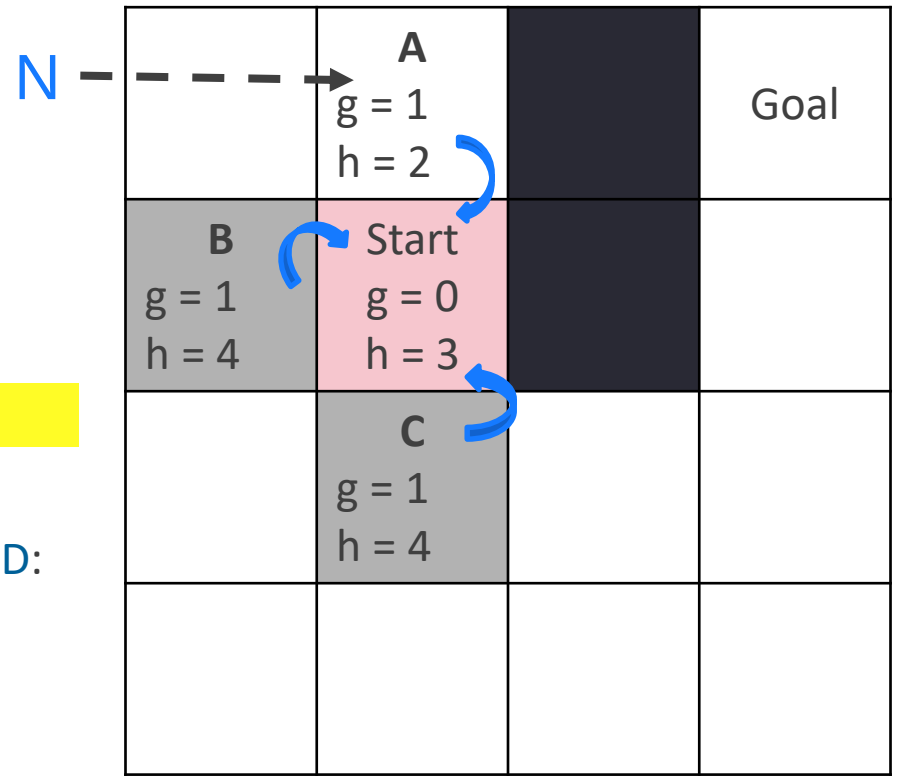


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
  CLOSED.add(N)
  FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
  ENDFOR
ENDWHILE
```

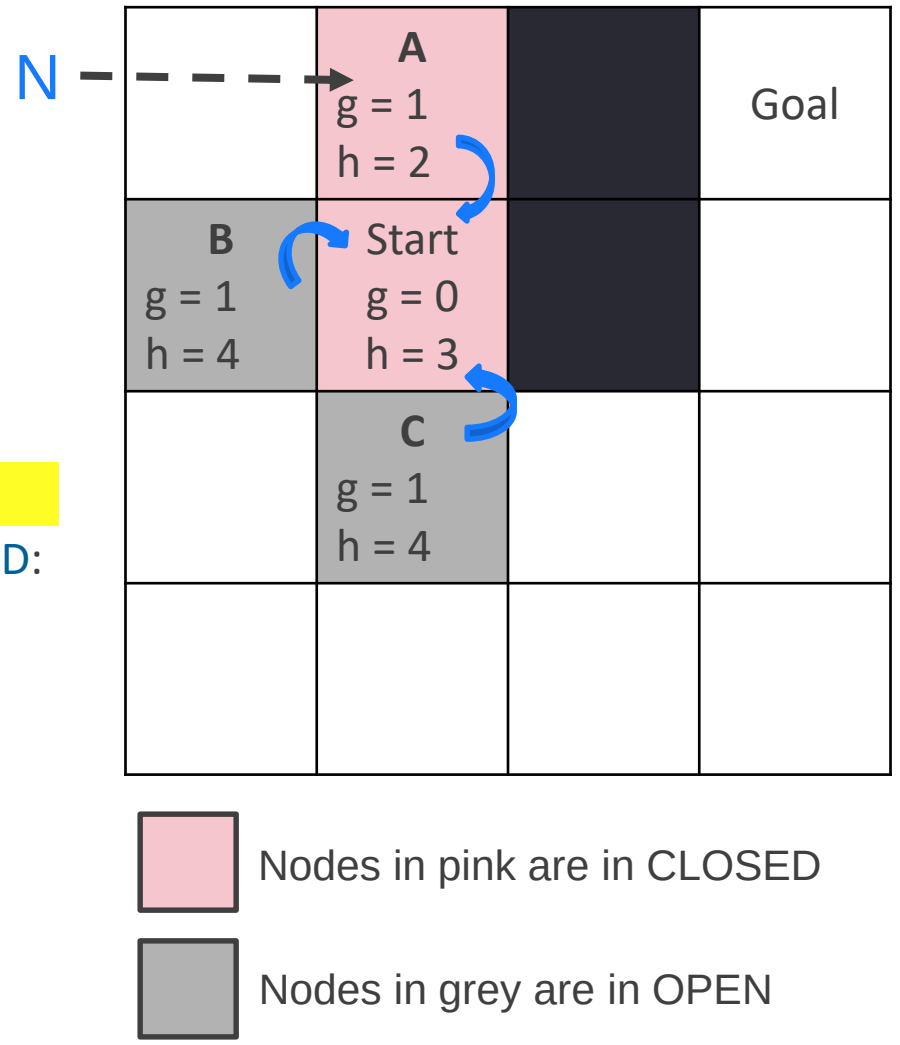


Nodes in pink are in CLOSED

Nodes in grey are in OPEN

A* Pseudocode Walkthrough

```
Start.g = 0
Start.h = heuristic(Start)
OPEN = [Start]
CLOSED = []
WHILE OPEN is not empty
    N = OPEN.removeLowestF()
    IF goal(N) RETURN path to N
    CLOSED.add(N)
    FOR all children M of N not in CLOSED:
        M.parent = N
        M.g = N.g + 1
        M.h = heuristic(M)
        OPEN.add(M)
    ENDFOR
ENDWHILE
```



Properties of A*

- Complete – Yes, unless there infinitely many nodes with $f \leq f(G)$
- Time – Exponential in [relative error in h x length of solution]
- Space – Keeps all nodes in memory
- Optimal – Yes, cannot expand f_{i+1} until f_i is finished

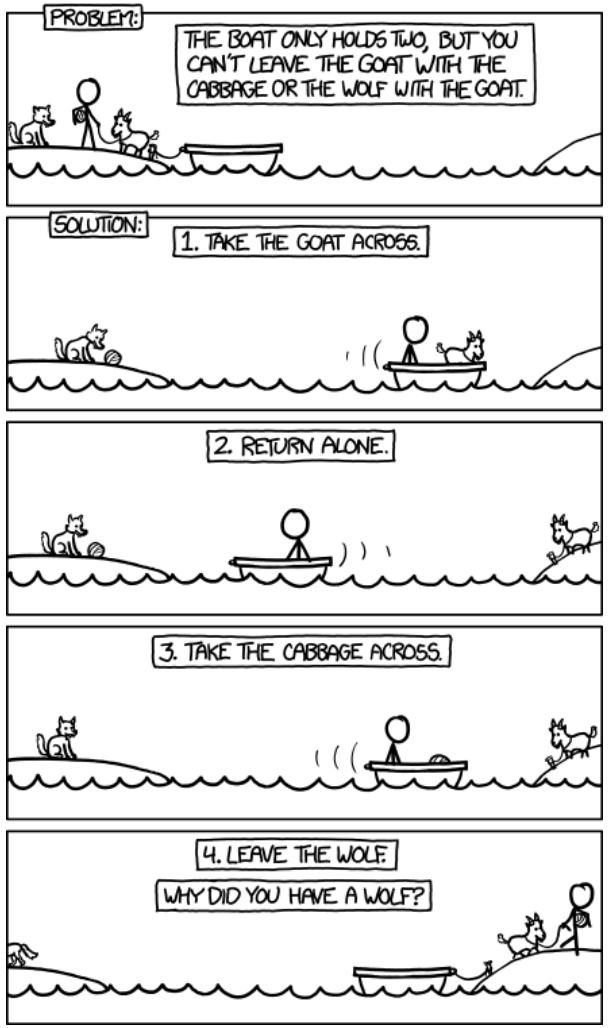
Admissible Heuristics

- To ensure optimality, A* requires the heuristic to be **admissible**:

$$h(n) \pounds h^*(n)$$ ← Actual cost to the goal

- In other words: the heuristic **must never overestimate** the actual remaining cost to the goal.

Heuristics: Wolf-Goat-Cabbage



- Let's assume that "cost" is the number of boat trips.
- What would be a good heuristic?
- Example:
  - Number of "items" on the left bank (since we know we need to make at least that number of trips)
  - Better:
    - ([*number of items in left bank*] – 1) * 2
    - Add 1 if boat on right bank and [*number of items in left bank*] > 0

Heuristics: 8-Puzzle

8		1
3	2	6
7	4	5

- Cost: number of moves
- What would be a good heuristic?
  - Heuristic 1:
    - Number of blocks out of place
  - Heuristic 2:
    - Distance of each block to its target position

Admissible Heuristics

- To ensure optimality, A* requires the heuristic to be **admissible**:

$$h(n) \pounds h^*(n)$$ ← Actual cost to the goal

- In other words: the heuristic **must never overestimate** the actual remaining cost to the goal.
- If the heuristic is admissible, A* will be ***provably*** optimal.
  - But an inadmissible heuristic might work just fine.

Heuristic Dominance

- If $h_2(n) \geq h_1(n)$ for all n (both admissible)

  Then h_2 dominates h_1 and is better for search
- Typical Search Costs:
  - d = 14 IDS = 3,473,941 nodes
    - $A^*(h_1)$ = 539 nodes
    - $A^*(h_2)$ = 113 nodes
  - d = 24 IDS ≈ 54,000,000,000 nodes
    - $A^*(h_1)$ = 39,135 nodes
    - $A^*(h_2)$ = 1,641 nodes
- Given any admissible heuristics h_a, h_b
  - $h(n) = \max(h_a(n), h_b(n))$ is also admissible and dominates h_a, h_b

Constructing Heuristics by Relaxation

$h_1(n)$ = number of tiles out of place

$h_2(n)$ = Manhattan distance of tiles to proper locations

- Admissible heuristics can be derived from the exact solution cost of a relaxed version of the problem
- If the rules of the 8-puzzle are relaxed so that a tile can move anywhere, the $h_1(n)$ gives the shortest solution
- If the rules are relaxed so that a tile can move to any adjacent square, then $h_2(n)$ gives the shortest solution

Key point: The optimal solution cost of a relaxed problem is not greater than the optimal solution cost of the real problem

A Different Way to Build Heuristics

- **Pattern Databases** – Store the exact optimal solution cost for every possible subproblem instance.
  - You care about some features of the problem, but not all
  - In this example, all the possible locations of [1,3,6,*blank*]
  - Identify the instance in the DB to get the h value

Start

	3	6
1	*	*
*	*	*

Goal

1	*	3
*	*	6
*	*	

Variations of A*

- SMA*: A* with bounded memory usage
- TBA*: A* for real-time domains where we have a bounded time before producing an action
- LRTA*: another real-time version of A* (very simple, and the basis of a whole family of algorithms)
- D*: A* for dynamic domains (problem configuration can change)
- etc.

Coming up…

- Next time, we'll look at some search variations and alternatives, including local search and constraint satisfaction

CS 380: ARTIFICIAL INTELLIGENCE

Lecture 5: Local Search & CSPs

Recall: Problem Solving

- Idea: represent the problem we want to solve as:
  - State space
  - Actions
  - Goal Check
  - Heuristic / evaluation function
- If we do this, we can then use search methods to solve any problem in that space
  - BFS, DFS, Iterative Deepening, Greedy Search, A*

Systematic vs. Local Search

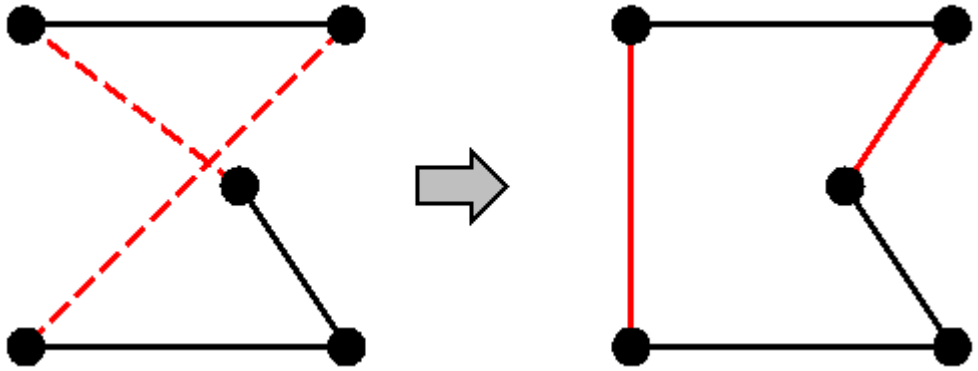
- All the algorithms we have seen so far have been "systematic":
  - They will systematically explore ALL the states in the search space if needed
    - Each of them just explores the states in a different order
  - If the search space is finite, they are all complete
- Local Search:
  - Instead of systematically exploring the state space…
  - Start with one state (even a bad one) and iteratively improve it by exploring neighbor states ("local")
  - Search space is the set of all states visited
  - *Iterative improvement*

Iterative Improvement Algorithms

- In many optimization problems, path is irrelevant
  - Goal state itself is the solution
- Then state space = set of "complete" configurations
  - Find optimal configuration, e.g., TSP
  - Or, find configuration satisfying constraints, e.g., timetable
- In such cases, can use iterative improvement algorithms
  - keep a single "current" state and try to improve it
- Constant space, suitable for online as well as offline search

Example: Traveling Salesperson Problem

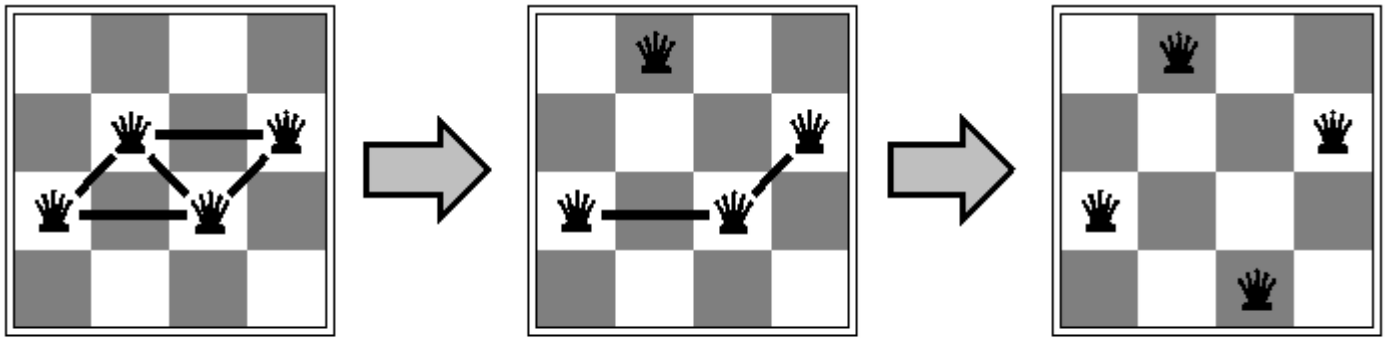
- Start with any complete tour, perform pairwise exchanges



- Variants of this approach get within 1% of optimal very quickly with thousands of cities

Example: n-queens

- Put *n* queens on an *n* x *n* board with no two queens on the same row, column, or diagonal
- Move a queen to reduce the number of conflicts



- Almost always solves n-queens problems almost instantaneously for very large *n*, e.g., *n* = 1 million

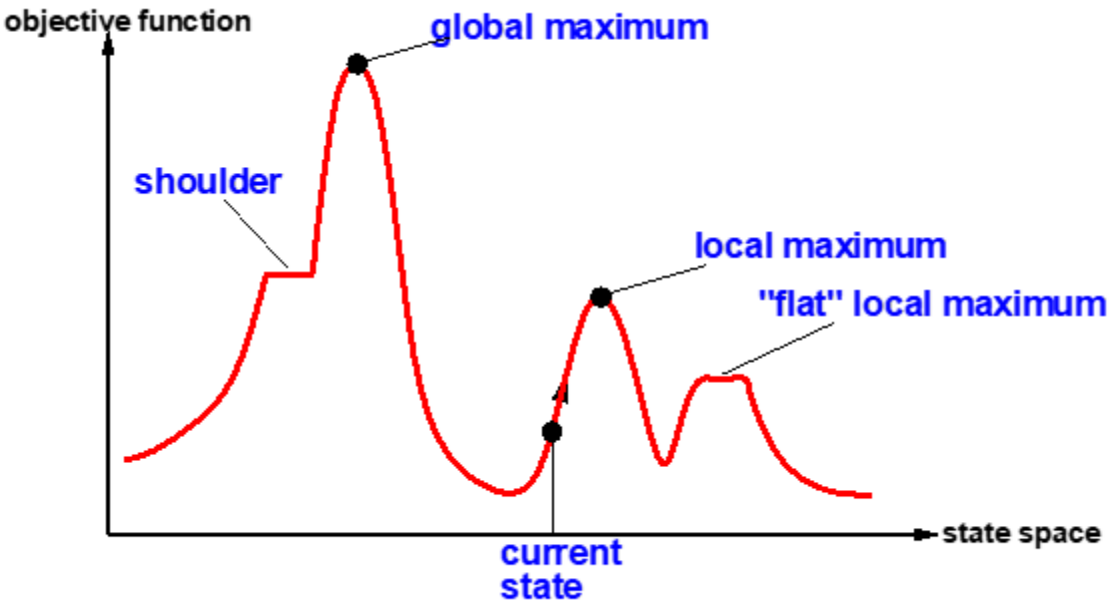
Hill-climbing (or gradient ascent/descent)

- "Like climbing Everest in think fog with amnesia"

```
function Hill-Climbing(problem) returns a state that is a local maximum
    inputs: problem, a problem
    local variables: current, a node
                     neighbor, a node
    current ← Make-Node(Initial-State[problem])
    loop do
        neighbor ← a highest-valued successor of current
        if Value[neighbor] ≤ Value[current] then return State[current]
        current ← neighbor
    end
```

Hill-climbing cont.

- Useful to consider state space landscape



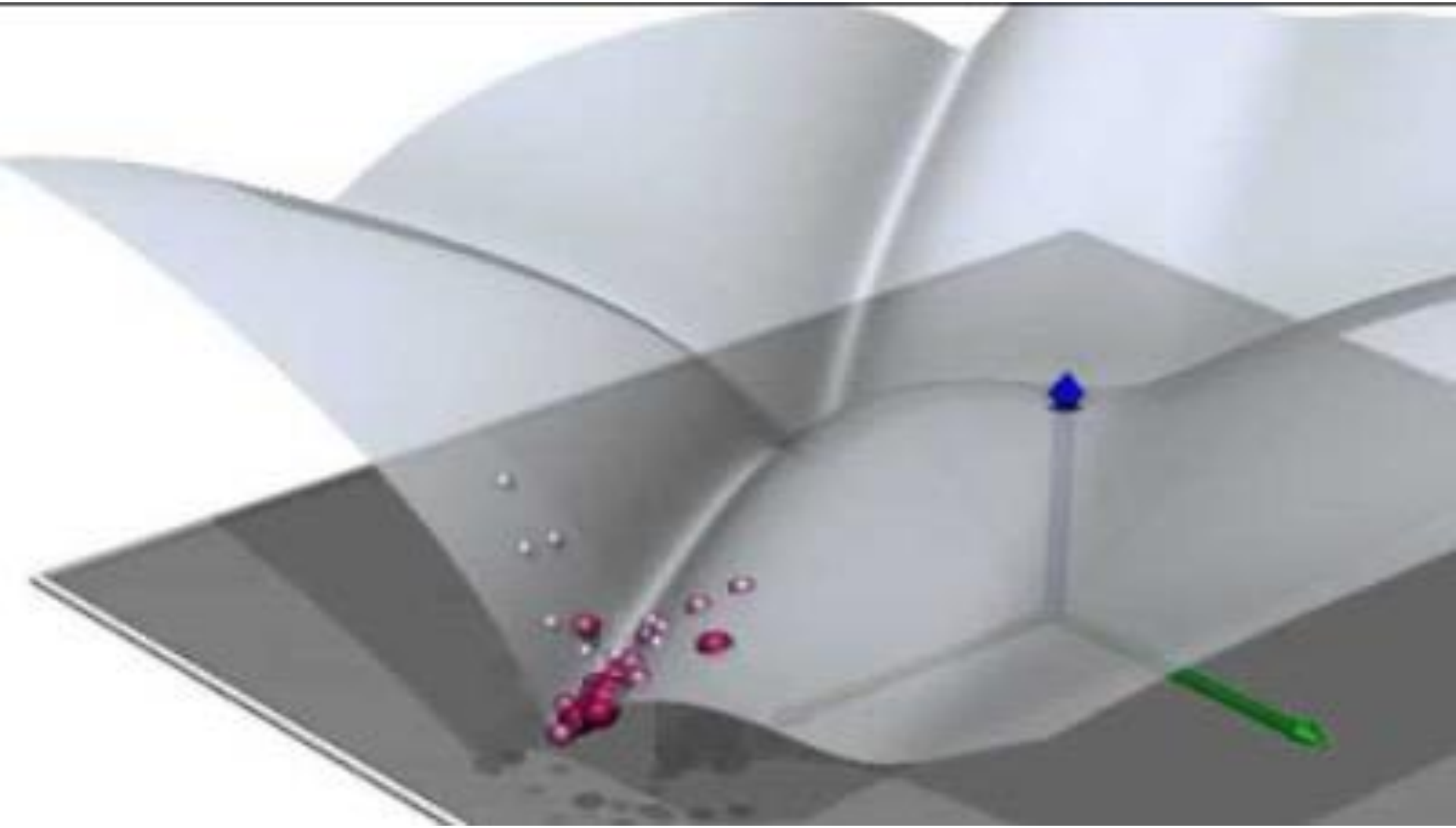
- Random-restart hill climbing overcomes local maxima – trivially complete
- Random sideways moves escape from shoulders but loop on flat maxima

Simulated Annealing

- "Annealing" = heating and then controlled cooling (of metals to strengthen the material)
- At a fixed temperature *T*, the state occupation probability reaches the Boltzmann distribution

$$p(x) = \alpha e^{\frac{E(x)}{kT}}$$

- If *T* is decreased slowly enough, you are guaranteed to reach the optimal state (if we need optimal)
- Visualization:
  - https://www.youtube.com/watch?v=iaq_Fpr4KZc



Simulated Annealing

- Idea: escape local maxima by allowing some "bad" moves
- But gradually decrease their size and frequency

```
function Simulated-Annealing(problem, schedule) returns a solution state
   inputs: problem, a problem
           schedule, a mapping from time to "temperature"
   local variables: current, a node
                    next, a node
                    T, a "temperature" controlling prob. of downward steps

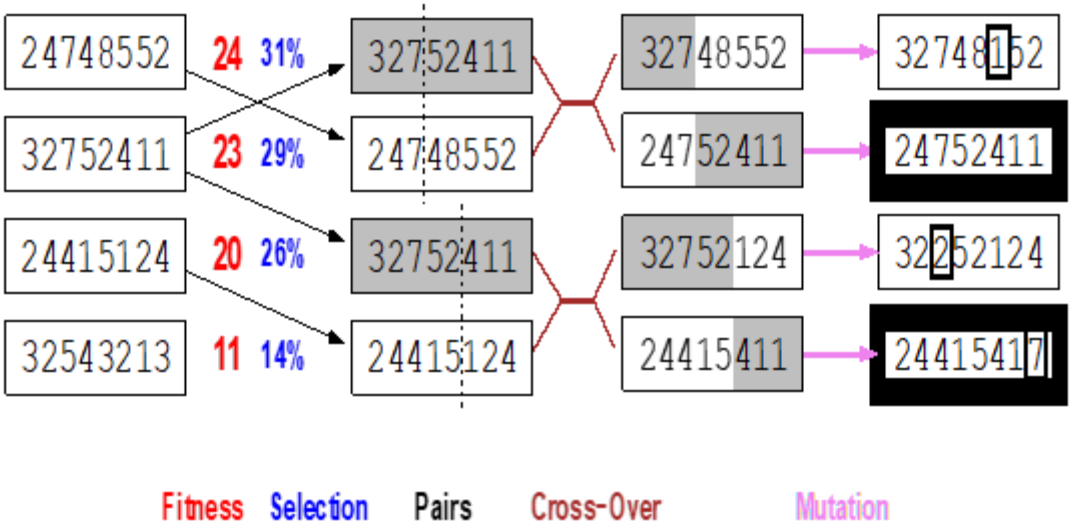
   current ← Make-Node(Initial-State[problem])
   for t ← 1 to ∞ do
       T ← schedule[t]
       if T = 0 then return current
       next ← a randomly selected successor of current
       ΔE ← Value[next] – Value[current]
       if ΔE > 0 then current ← next
       else current ← next only with probability e^(Δ E/T)
```

Local Beam Search

- Idea: Keep *k* states instead of 1; choose top *k* of all their successors
- Not the same as *k* searches run in parallel!
- Searches that find good states recruit other searches to join them
- Problem: quite often, all *k* states end up on same local hill
- Idea: choose *k* successors randomly, biased towards the good ones
- Observe the analogy to natural selection.

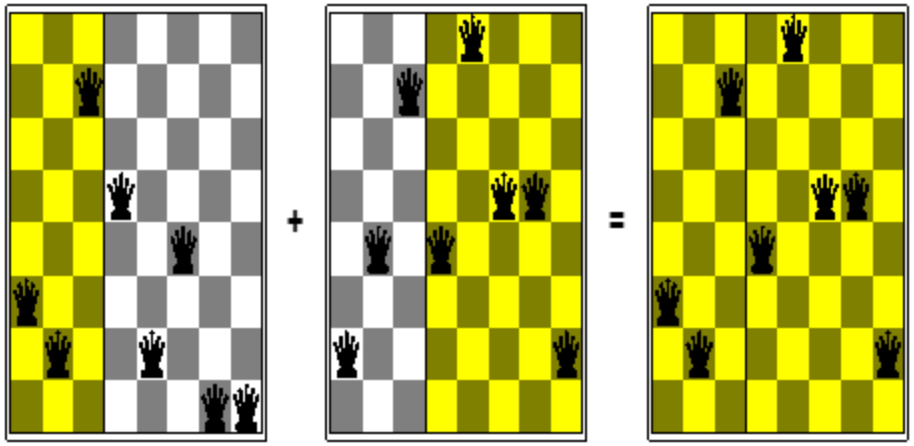
Genetic Algorithms

- Stochastic local beam search + generate successor from pairs of states



Genetic Algorithms Cont.

- GAs require states encode a strings (GPs use programs)
- Crossover helps iff substrings are meaningful components



- GAs != evolution: e.g., real genes encode replication machinery.

Sample Visualization

- Greedy search, local search, simulated annealing…

https://www.youtube.com/watch?v=SC5CX8drAtU



Constraint Satisfaction Problems (CSPS)

- Standard search problem:
  - State is a "black box" – any old data structure that supports goal test, eval, successor
- CSP:
  - State is defined by *variables* V_i with *values* from *domain* D_i
  - Goal test is a set of *constraints* specifying allowable combinations of values for subsets of variables
  - Simple example of a *formal representation language*
  - Allows useful *general-purpose* algorithms with more power than standard search algorithms

Example: 4-Queens as a CSP

- Assume 1 queen in each column. Which row does each one go in?

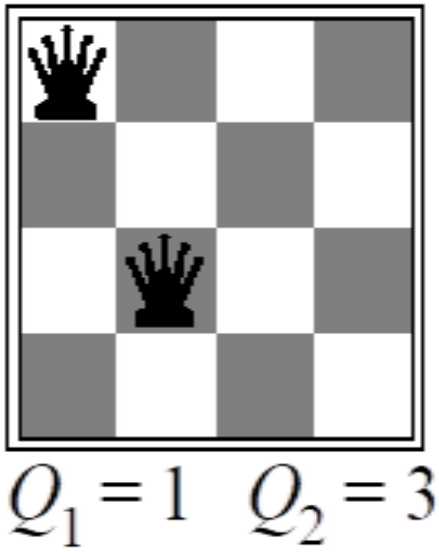
Variables: Q_1, Q_2, Q_3, Q_4

Domains: $D_i = \{1,2,3,4\}$

Constraints:

Q_i != Q_j (cannot be in same row)

$|Q_i - Q_j|$!= $|i - j|$ (or same diagonal)

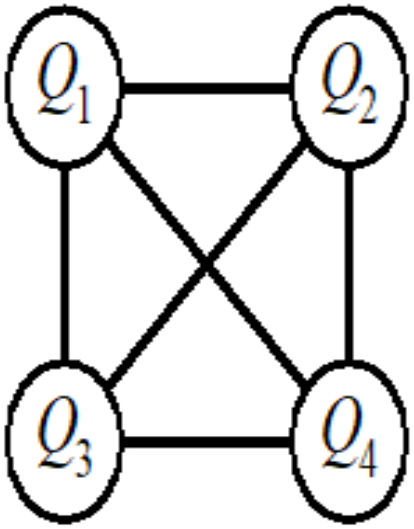


Translate each constraint into set of allowable values for its variables

E.g., values for (Q_1, Q_2) are (1,3) (1,4) (2,4) (3,1) (4,1) (4,2)

Constraint Graph

- Binary CSP: each constrain relates to at most two variables
- Constraint graph: nodes are variables, arcs show constraints



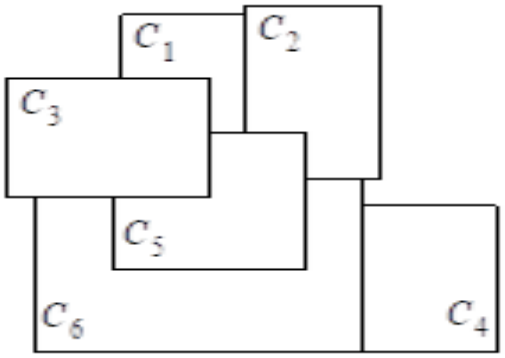
Example: Cryptarithmetic

- Variables
  - *D E M N O R S Y*
- Domains
  - {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
- Constraints
  - *M* != 0, *S* != 0 (unary constraints)
  - *Y* = *D* + *E* or *Y* = *D* + *E* – 10, etc.
  - *D* != *E*, *D* != *M*, *D* != *N*, etc.

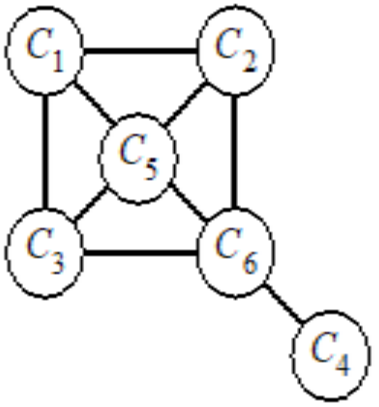


Example: Map Coloring

- Color a map so that no adjacent countries have the same color
- Variables
  - Countries C_i
- Domains
  - {Red, Blue, Green}
- Constraints
  - C_1 != C_2, C_2 != C_5, etc.



Constraint Graph:



Real-world CSPs

- Assignment problems
  - e.g., who teaches what class
- Timetabling problems
  - e.g., which class is offered when and where?
- Hardware configuration
- Spreadsheets
- Transportation scheduling
- Factory scheduling
- Floorplanning

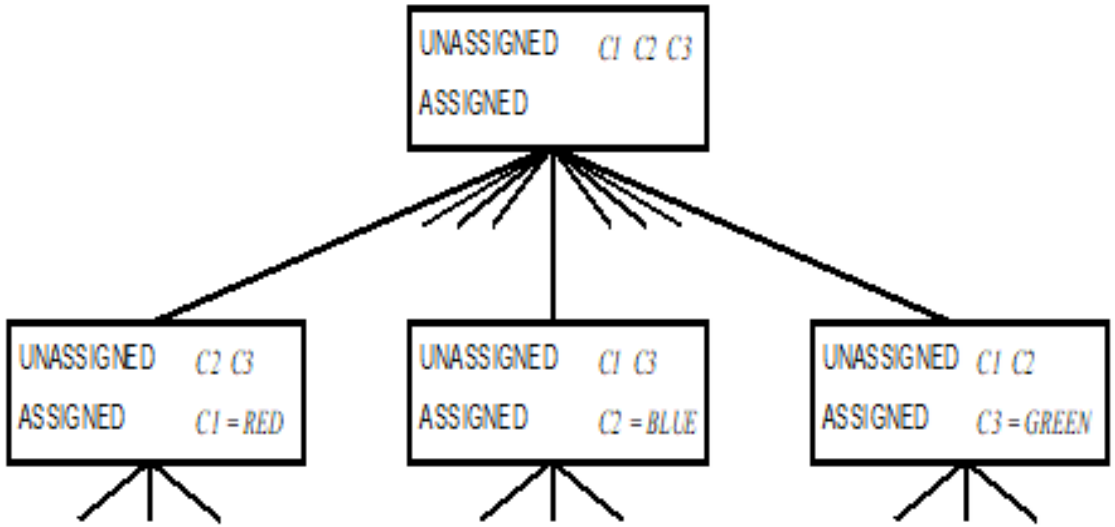
Notice that many real-world problems involve real-valued variables

Applying standard search

- Start with straightforward, “dumb” approach, then fix it
- States are defined by the values assigned so far
- Initial State: all variables unassigned
- Actions: assign a value to an unassigned variable
- Goal test: all variables assigned, no constraints violated

- Notice that this is the same for all CSPs

Standard search applied to map-coloring



Complexity of the dumb approach

- Max. depth of space m: ??
- Depth of solution state d: ??
- Search algorithm to use: ??
- Branching factor b: ??

Complexity of the dumb approach

- Max. depth of space m: *n* (number of variables)
- Depth of solution state d: *n* (all vars assigned)
- Search algorithm to use: depth-first
- Branching factor b: $\sum_i |Di|$ (at top of tree)

  $|\ D_i\ |$ = size of Domain *i*
- This can be improved dramatically by noting the following:
  - Order of assignment is irrelevant, hence many paths are equivalent
  - Adding assignments cannot correct a violated constraint

Backtracking Search

Use depth-first search, but

1) Fix the order of assignment => b = $|D_i|$

   (can be done in the SUCCESSORS function)

2) Check for constraint violations

The constraint violation check can be implemented in two ways:

1) Modify SUCCESSORS to assign only values that are allowed, given the values already assigned
2) Check constraints are satisfied before expanding a state

Backtracking search is the basic uninformed algorithm for CSPs

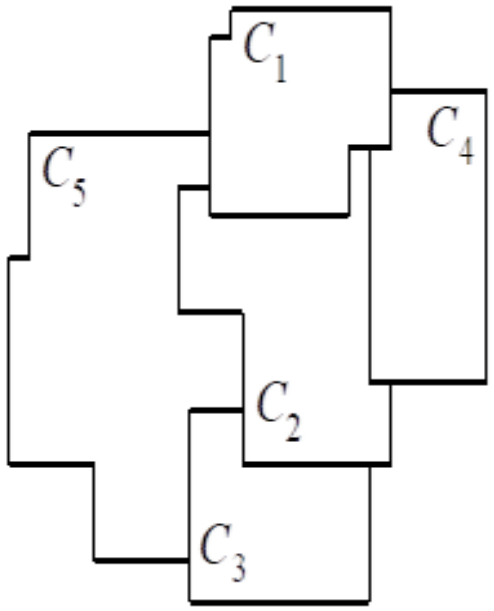
Can solve *n*-queens for $n \approx 15$

Forward Checking

- Idea: Keep track of remaining legal values for unassigned variables.
- Terminate search when any variable has no legal values.

Simplified map-coloring example:

	Red	Blue	Green
C_1			
C_2			
C_3			
C_4			
C_5			



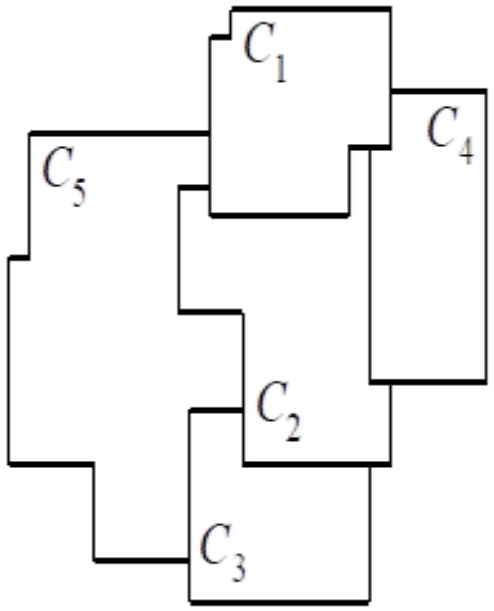
Can solve n-queens up to n ≈ 30

Forward Checking

- Idea: Keep track of remaining legal values for unassigned variables.
- Terminate search when any variable has no legal values.

Simplified map-coloring example:

	Red	Blue	Green
C_1	✓		
C_2	x		
C_3			
C_4	x		
C_5	x		



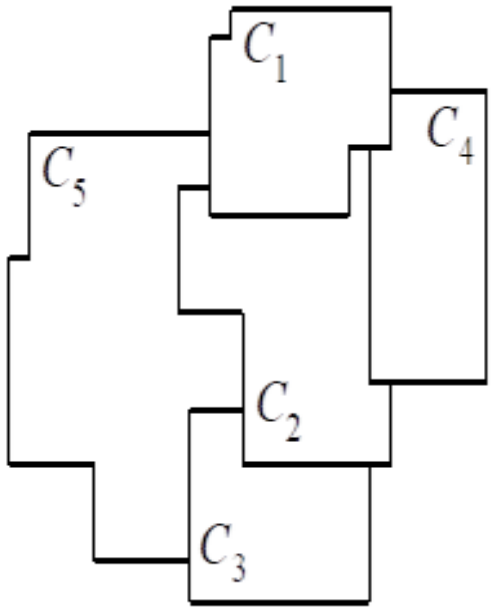
Can solve n-queens up to n ≈ 30

Forward Checking

- Idea: Keep track of remaining legal values for unassigned variables.
- Terminate search when any variable has no legal values.

Simplified map-coloring example:

	Red	Blue	Green
C_1	✓		
C_2	x	✓	
C_3		x	
C_4	x	x	
C_5	x	x	



Can solve n-queens up to n ≈ 30

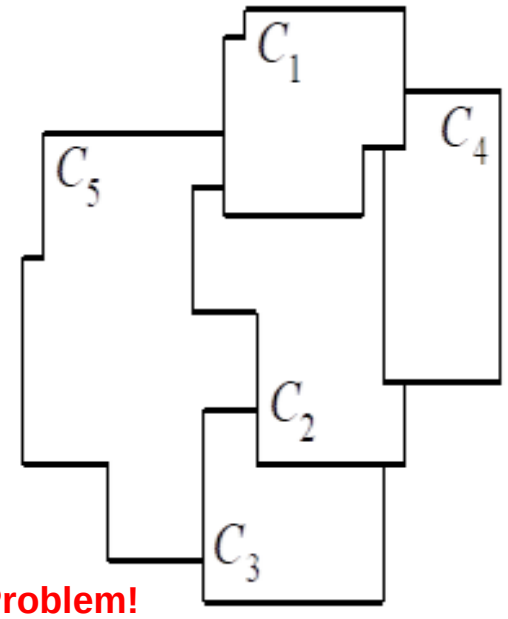
Forward Checking

- Idea: Keep track of remaining legal values for unassigned variables.
- Terminate search when any variable has no legal values.

Simplified map-coloring example:

	Red	Blue	Green
C_1	✓		
C_2	X	✓	
C_3		X	✓
C_4	X	X	
C_5	X	X	X

← Problem!



Can solve n-queens up to n ≈ 30

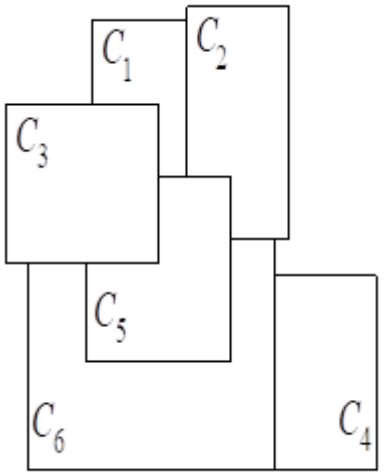
Heuristics for CSPs

- More intelligent decision on which value to choose for each variable, which variable to assign next

Given C_1 = Red, C_2 = Green, chose C_3 = ??

Given C_1 = Red, C_2 = Green, what next??

Can solve n-queens for n ≈ 1000



Heuristics for CSPs

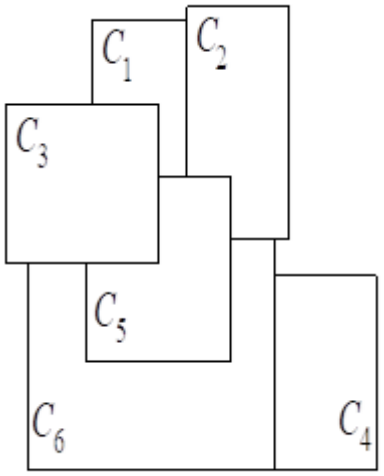
- More intelligent decision on which value to choose for each variable, which variable to assign next

Given C_1 = Red, C_2 = Green, chose C_3 = ??

C_3 = Green: Least constraining value

Given C_1 = Red, C_2 = Green, what next??

C_5 : Most constrained variable



Can solve n-queens for n ≈ 1000

Iterative algorithms for CSPs

- Hill-climbing, simulated annealing typically work with "complete" states, i.e., all variables assigned
- To apply to CSPs:
  - Allow states with unsatisfied constraints
  - Operators reassign variable values
- Variable selection: randomly select any conflicted variable
- Min-conflicts heuristic:
  - Choose value that violates the fewest constraints
  - i.e., hillclimb with *h(n)* = total number of violated constraints

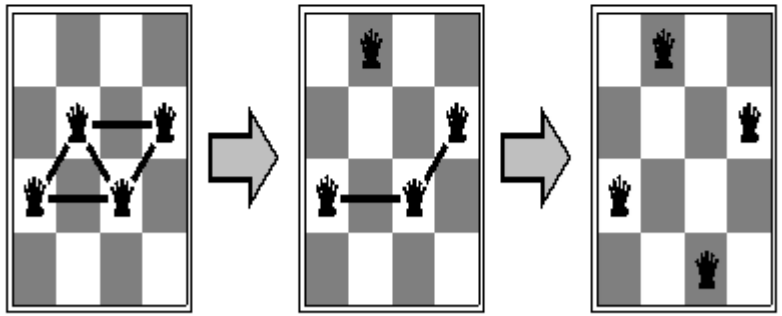
Example: 4-Queens

States: 4 queens in 4 columns ($4^4 = 256$ states)

Operators: move queen in column

Goal test: no attacks

Evaluation: $h(n)$ = number of attacks

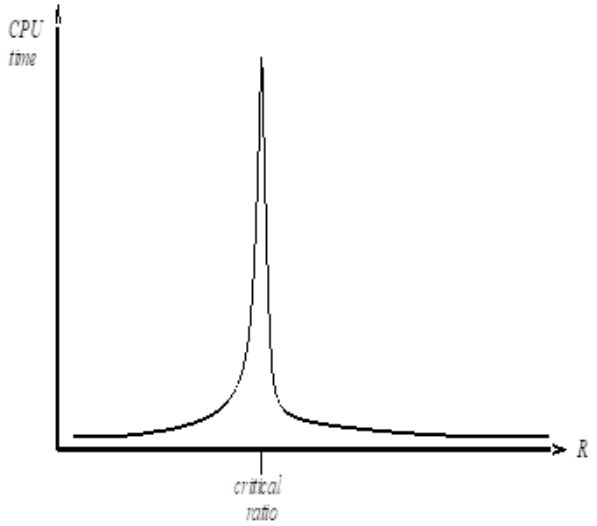


Performance of min-conflicts

Given random initial state, can solve n-queens in almost constant time for arbitrary n with high probability (e.g., $n = 10{,}000{,}000$)

The same appears to be true for any randomly-generated CSP except in a narrow range of the ratio

$$R = \frac{\text{number of constraints}}{\text{number of variables}}$$



Summary

- CSPs are a special kind of problem:
  - States defined by values of a fixed set of variables
  - Goal test defined by constraints on variable values
- Backtracking = depth-first search with
  - Fixed variable order
  - Only legal successors
- Forward checking prevents assignments that guarantee later failure
- Variable ordering and value selection heuristics help significantly
- Iterative min-conflicts is usually effective in practice
- Tree-structured CSPs can always be solved very efficiently

Coming up…

- We will start applying ideas of search to adversarial domains, where one player is playing another

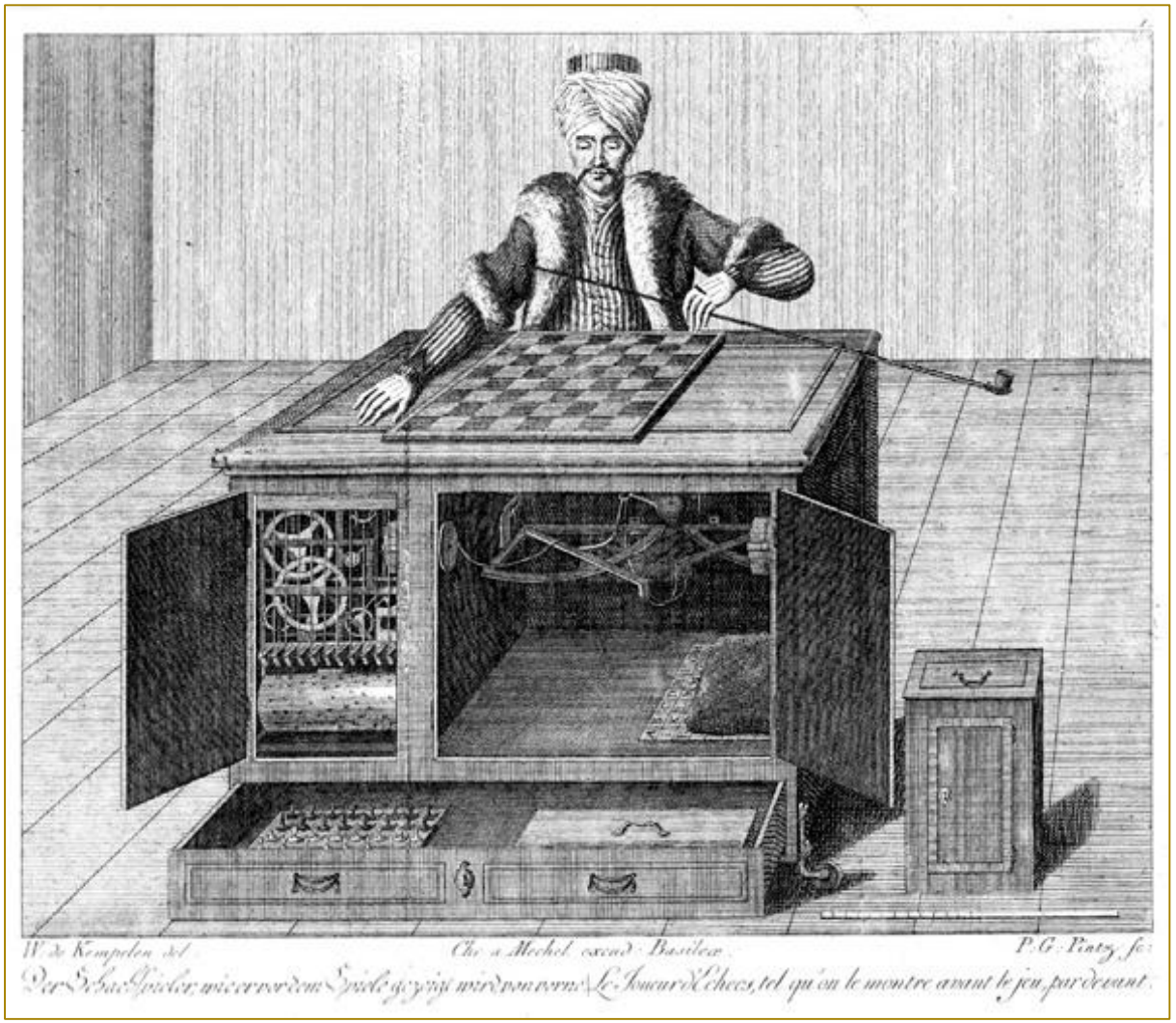
CS 380: ARTIFICIAL INTELLIGENCE

Lecture 6: Adversarial Search I

Recall: Problem Solving

- Idea: represent the problem we want to solve as:
  - State space
  - Actions
  - Goal check
  - Cost function
- If this is possible, we can use search methods to solve any problem:
  - Systematic Search:
    - BFS, DFS, ID, Greedy Search, A*
  - Local Search:
    - Hill Climbing, Simulated Annealing, Local Beam Search (e.g., GAs)

The “Mechanical Turk” (1770-1854)



Wikipedia.org

Games vs. Search Problems

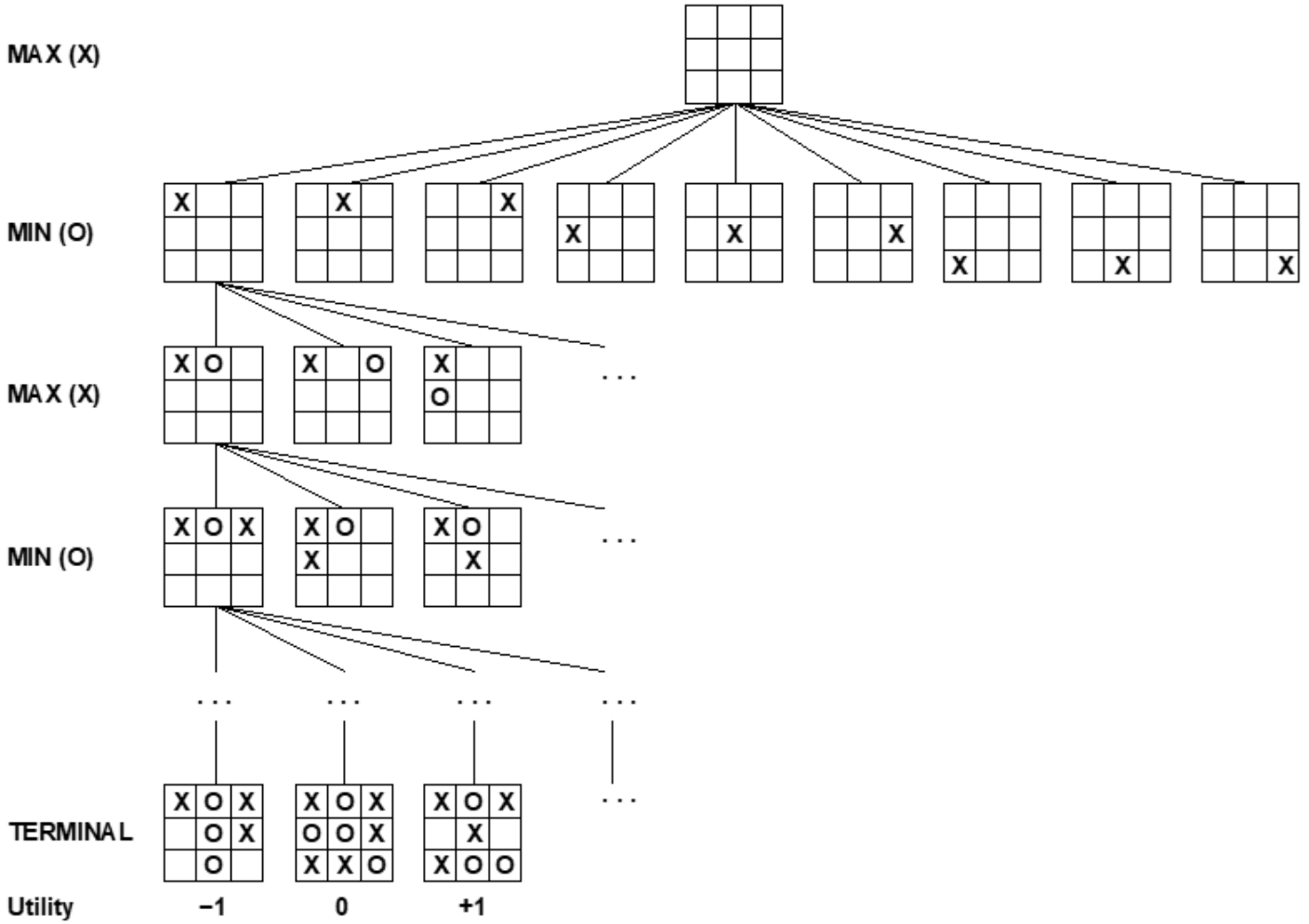
- "Unpredictable" opponent => solution is a strategy specifying a move for every possible opponent reply
- Time limits => unlikely to find goal, must approximate
- Plan of Attach:
  - Computer considers possible lines of play (Babbage, 1846)
  - Algorithm for perfect play (Zermelo, 1912; Von Neumann, 1944)
  - Finite horizon, approximate evaluation (Zuse, 1945; Wiener, 1948; Shannon, 1950)
  - First chess program (Turing, 1951)
  - Machine learning to improve evaluation accuracy (Samuel, 1952-57)
  - Pruning to allow deeper search (McCarthy, 1956)

Types of Games

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon monopoly
imperfect information	battleships, blind tictactoe	bridge, poker, scrabble

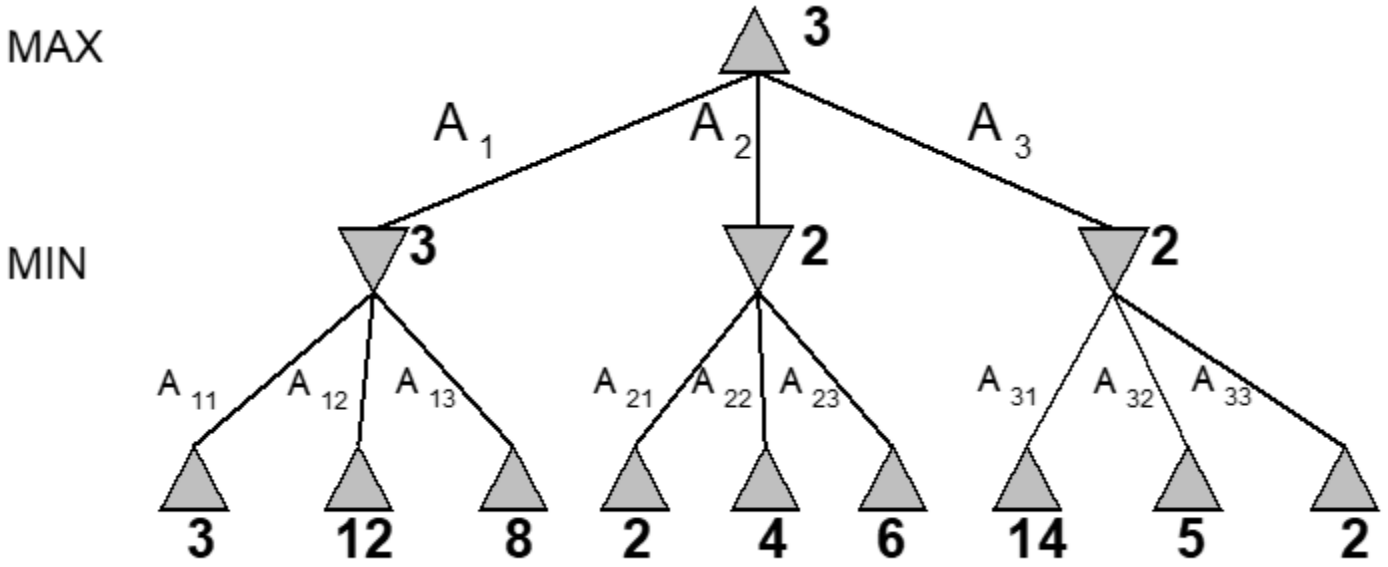
Players make a move simultaneously; if they select the same spot, coin flip determines mark

Game Tree (2-player, deterministic, turns)

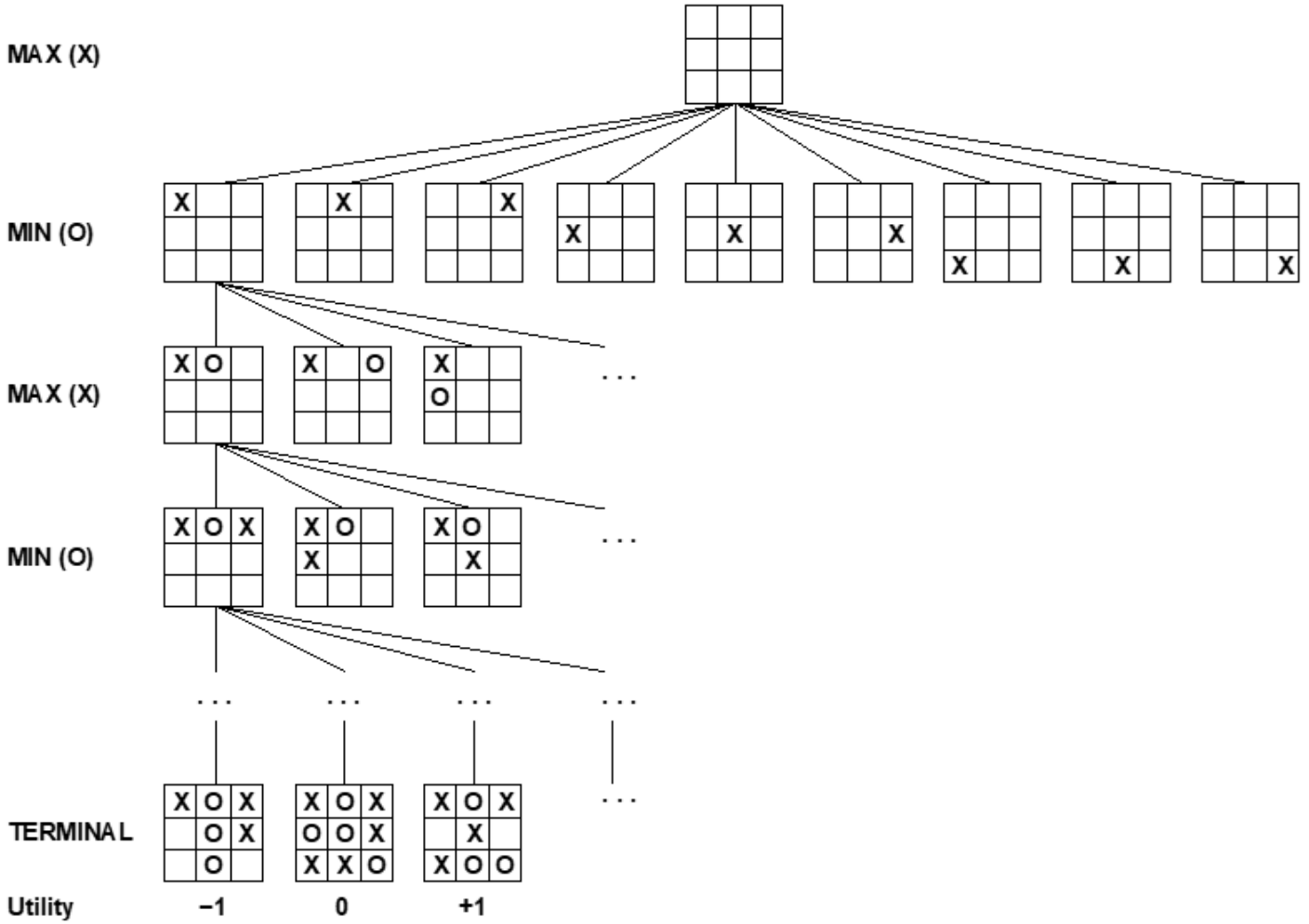


Minimax

- Perfect play for deterministic, perfect-information games
- Idea: choose move to position with highest minimax value = best achievable payoff against best play
- E.g., 2-ply game:



Game Tree (2-player, deterministic, turns)



Minimax Algorithm

```
function Minimax-Decision(state) returns an action
    inputs: state, current state in game
    return the a in Actions(state) maximizing Min-Value(Result(a, state))

function Max-Value(state) returns a utility value
    if Terminal-Test(state) then return Utility(state)
    v ← −∞
    for a, s in Successors(state) do v ← Max(v, Min-Value(s))
    return v

function Min-Value(state) returns a utility value
    if Terminal-Test(state) then return Utility(state)
    v ← ∞
    for a, s in Successors(state) do v ← Min(v, Max-Value(s))
    return v
```

Properties of Minimax

- Complete ??
- Optimal ??
- Time Complexity ??
- Space Complexity ??

Properties of Minimax

- Complete : Only if tree is finite (chess has specific rules for this) NB a finite strategy can exist even in an infinite tree!
- Optimal ??
- Time Complexity ??
- Space Complexity ??

Properties of Minimax

- Complete : Yes, if tree is finite
- Optimal : Yes, against an optimal opponent. Otherwise??
- Time Complexity ??
- Space Complexity ??

Properties of Minimax

- Complete : Yes, if tree is finite
- Optimal : Yes, against an optimal opponent. Otherwise??
- Time Complexity : $O(b^m)$
- Space Complexity ??

Properties of Minimax

- Complete : Yes, if tree is finite
- Optimal : Yes, against an optimal opponent. Otherwise??
- Time Complexity : $O(b^m)$
- Space Complexity : $O(bm)$ (depth-first exploration)

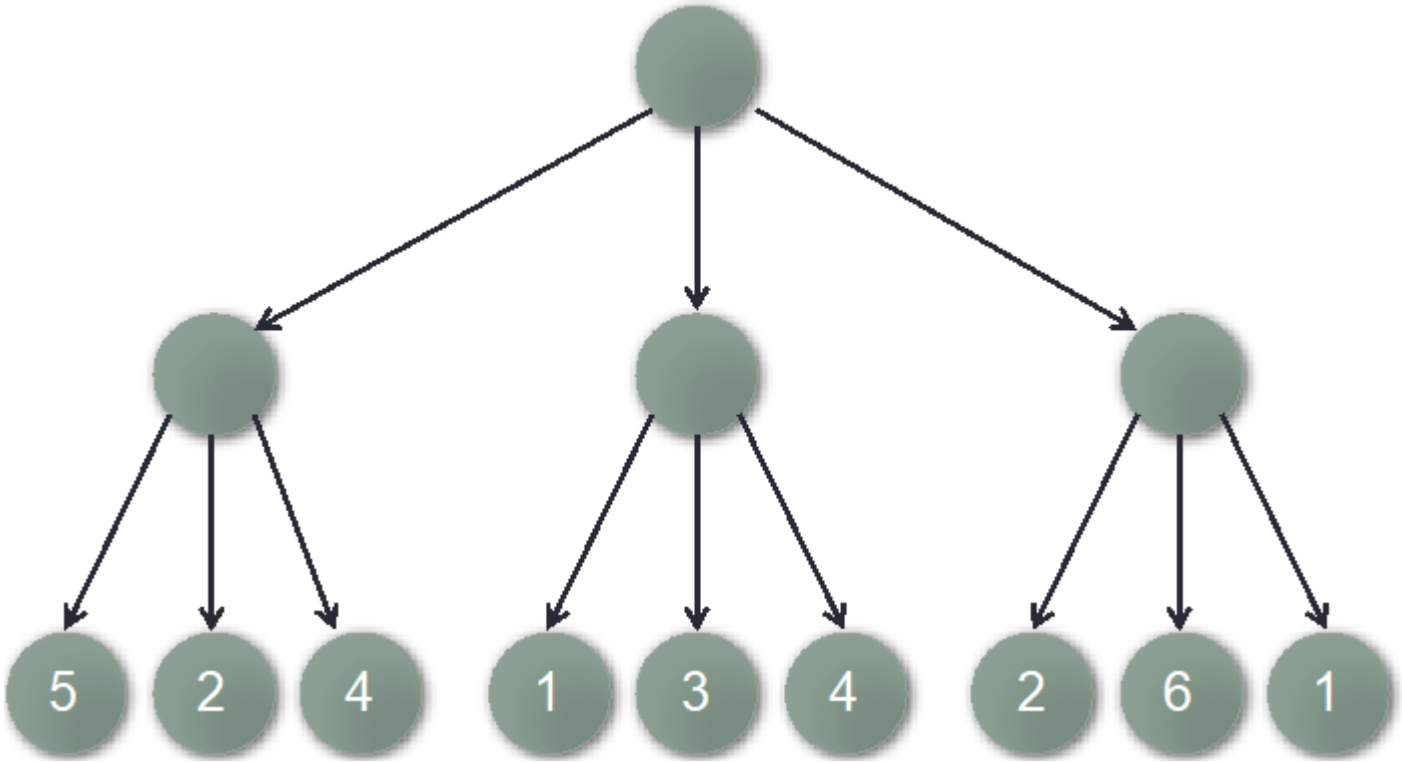
For chess, b ≈ 35, m ≈ 100 for "reasonable" games

=> exact solution completely infeasible

But do we need to explore every path?

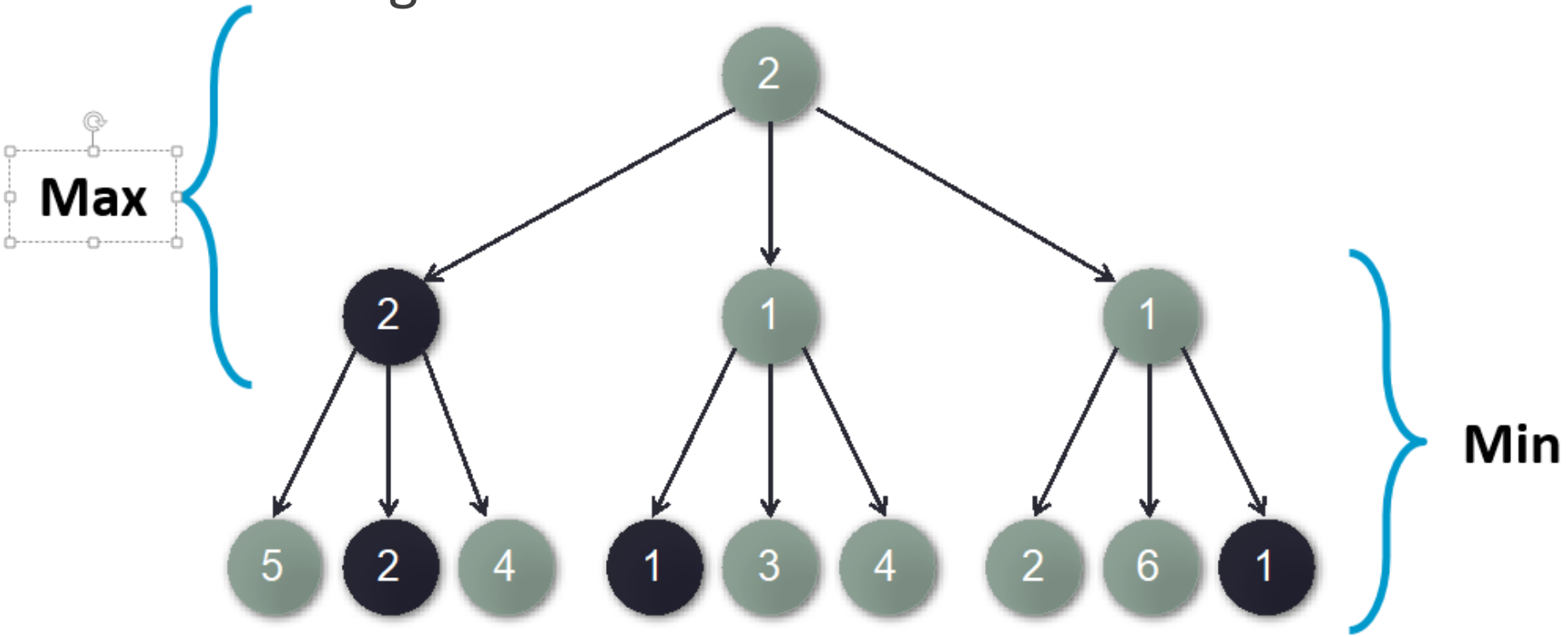
Alpha-Beta Pruning

- Not all nodes in the search tree are relevant for deciding the next move



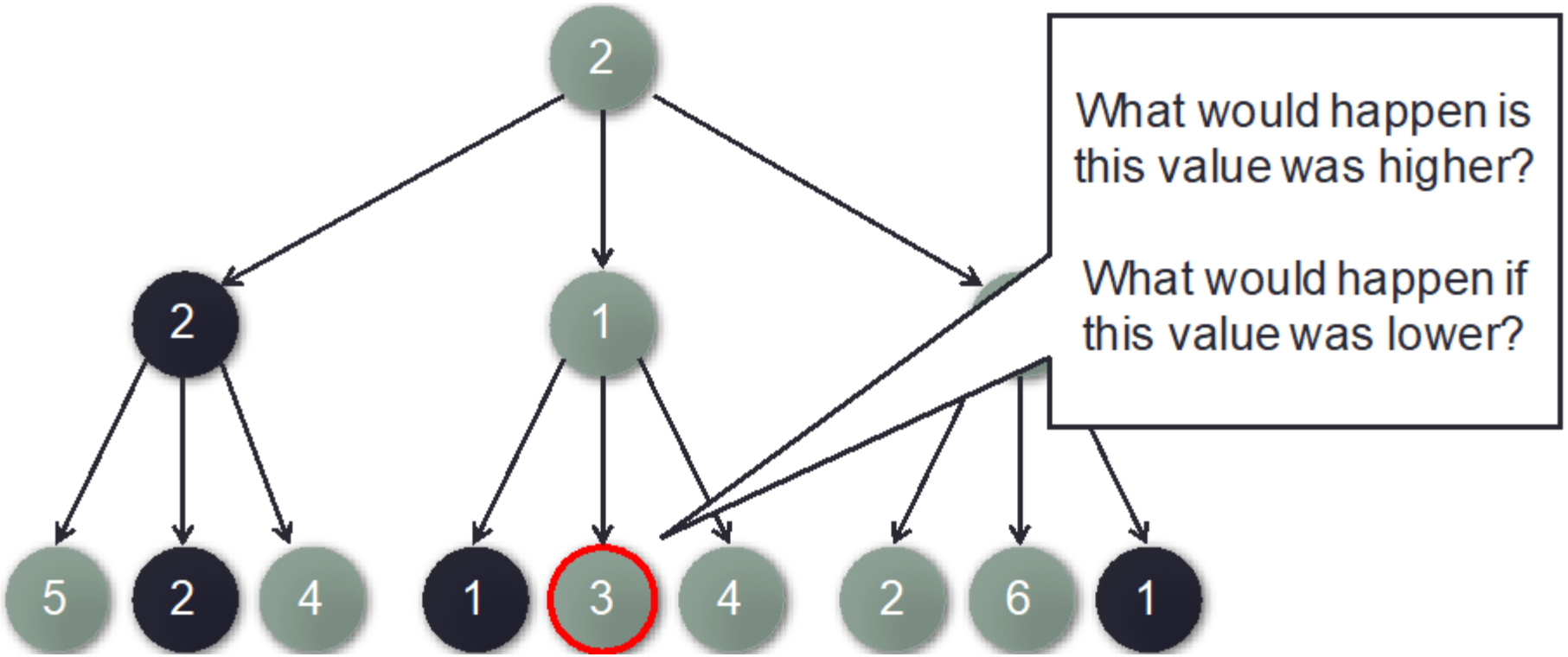
Alpha-Beta Pruning

- Not all nodes in the search tree are relevant for deciding the next move



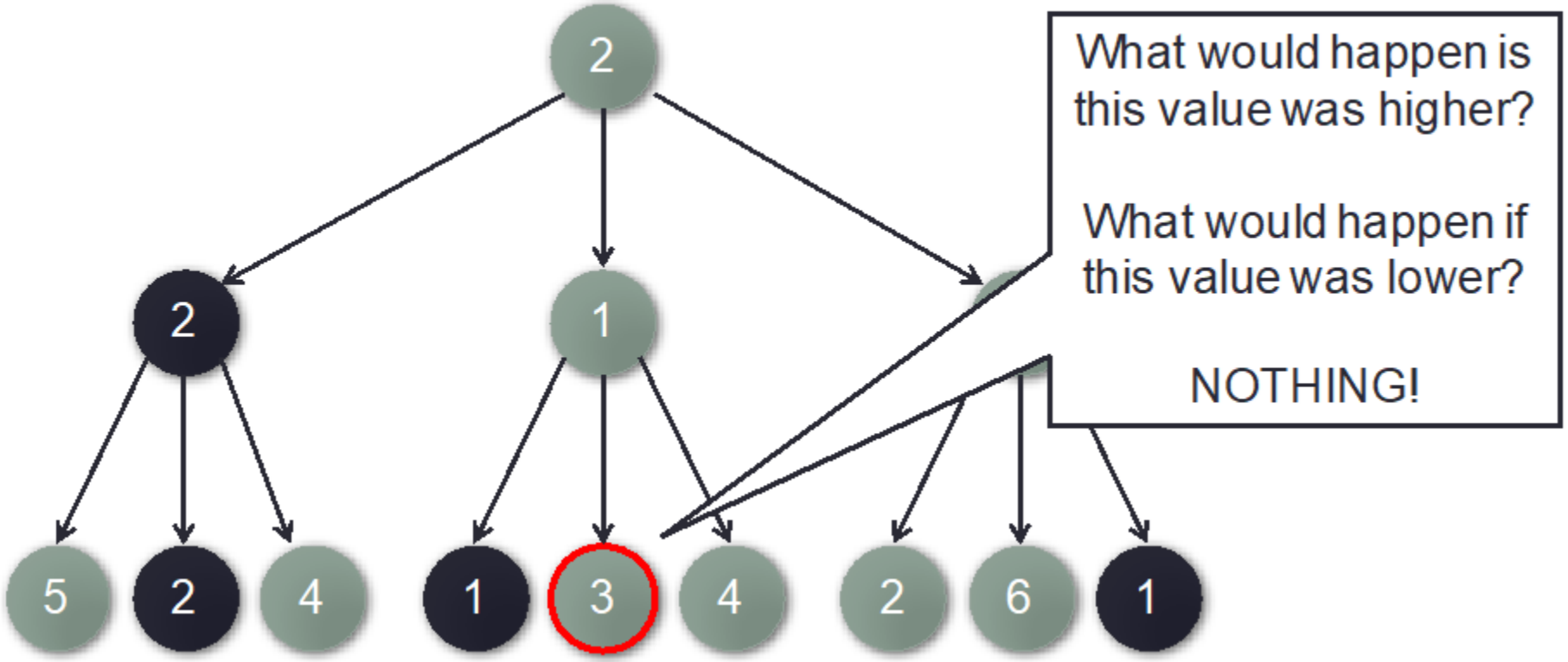
Alpha-Beta Pruning

- Not all nodes in the search tree are relevant for deciding the next move

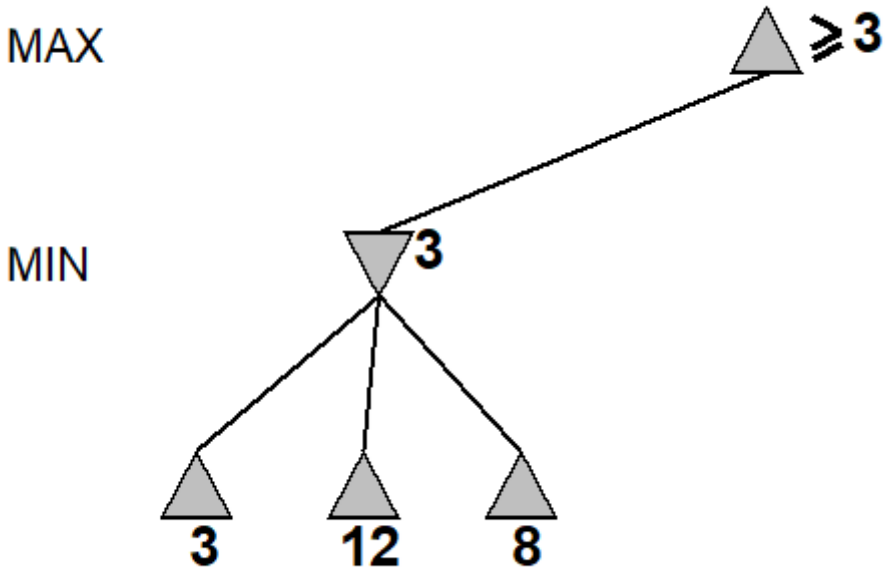


Alpha-Beta Pruning

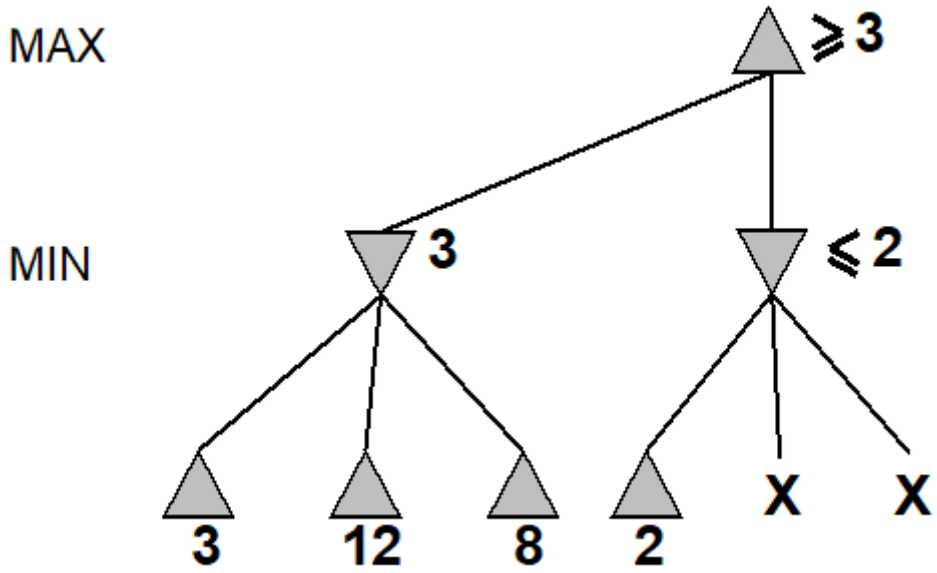
- Not all nodes in the search tree are relevant for deciding the next move



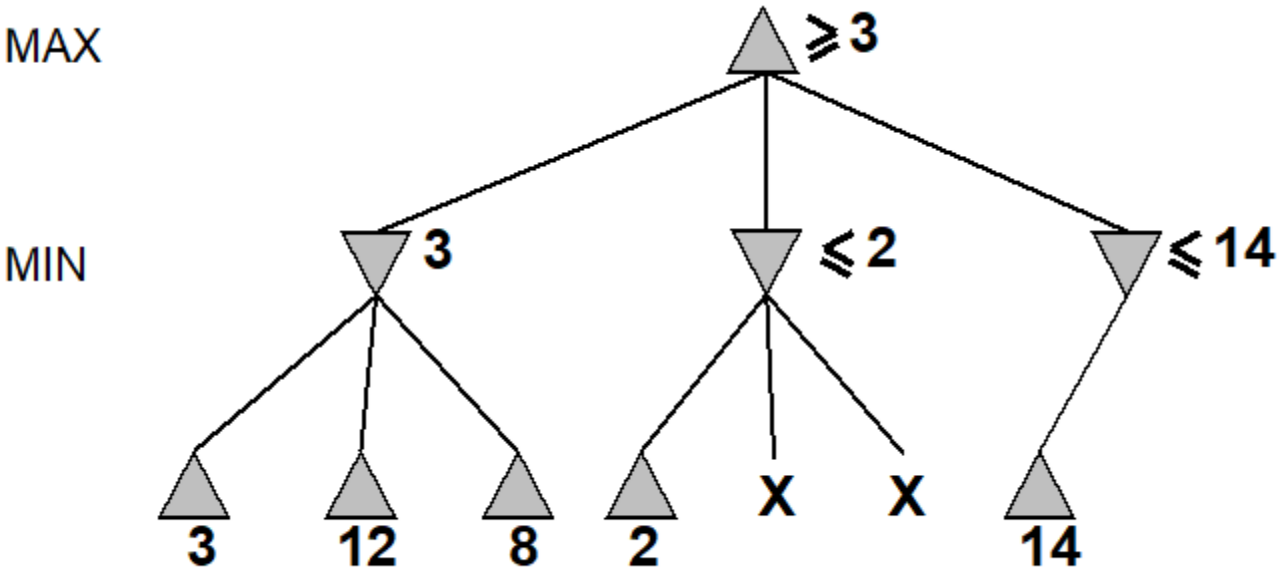
α-β Pruning Example



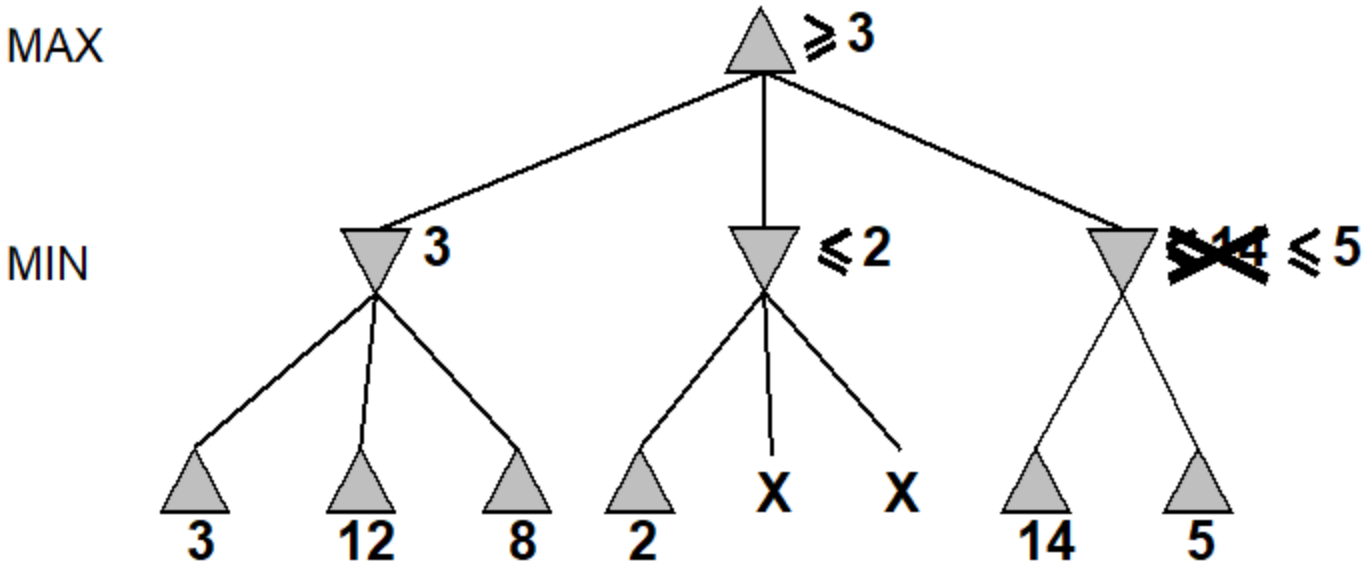
α-β Pruning Example



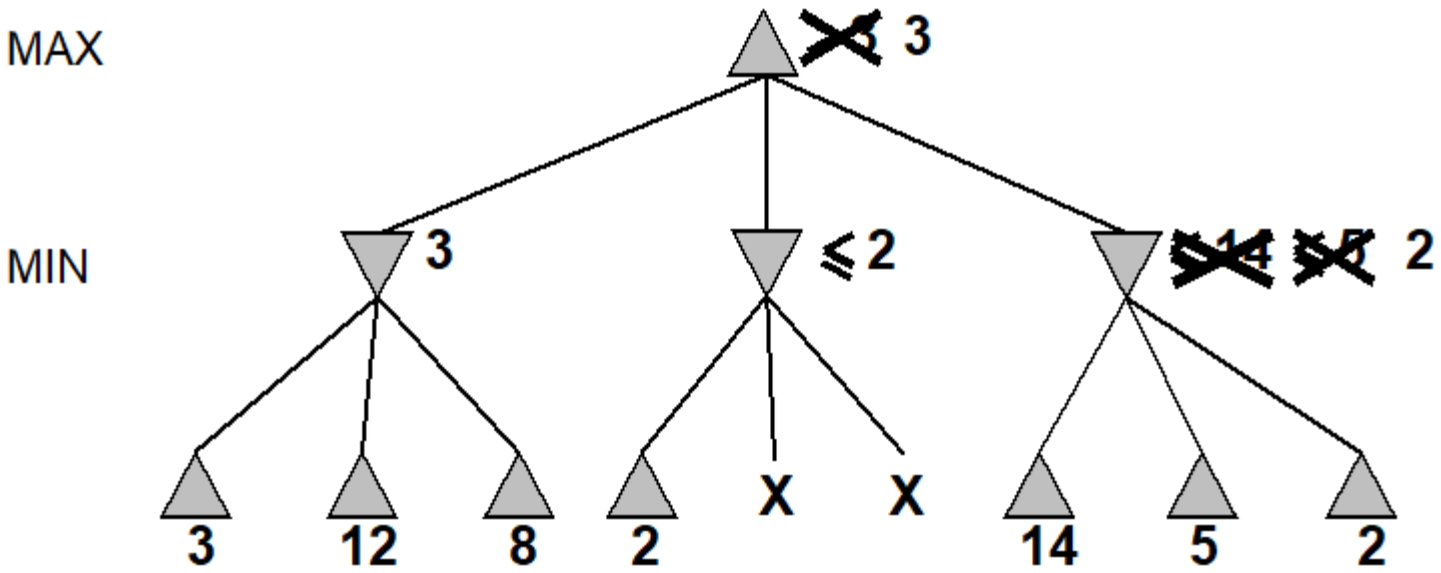
α-β Pruning Example



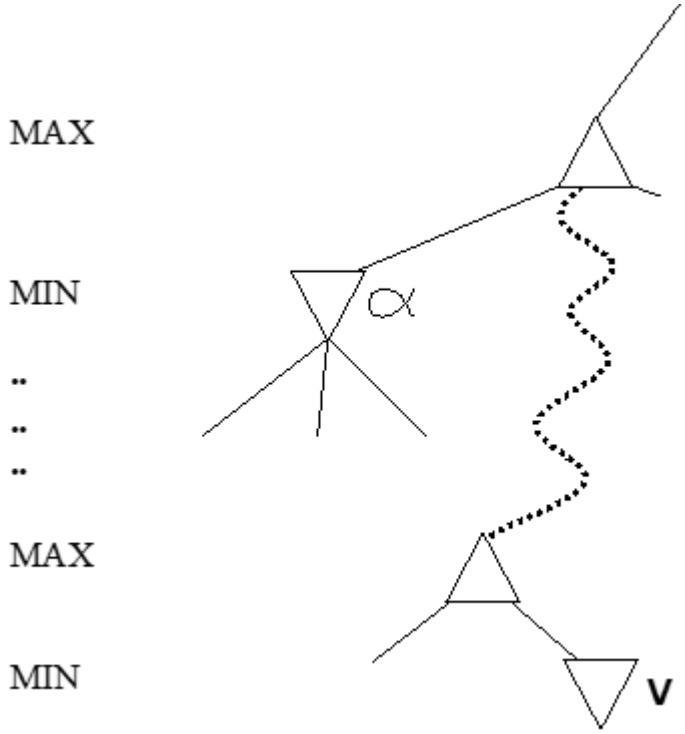
α-β Pruning Example



α-β Pruning Example



Why is it called α-β?



α is the best value (to max) found so far off the current path
If V is worse than α, max will avoid it => prune that branch
Define β similarly for min

The α-β Algorithm

```
function Alpha-Beta-Decision(state) returns an action
    return the a in Actions(state) maximizing Min-Value(Result(a, state))

---

function Max-Value(state, α, β) returns a utility value
    inputs: state, current state in game
            α, the value of the best alternative for max along the path to state
            β, the value of the best alternative for min along the path to state

    if Terminal-Test(state) then return Utility(state)
    v ← −∞
    for a, s in Successors(state) do
        v ← Max(v, Min-Value(s, α, β))
        if v ≥ β then return v
        α ← Max(α, v)
    return v

---

function Min-Value(state, α, β) returns a utility value
    same as Max-Value but with roles of α, β reversed
```

Properties of α-β

- Pruning does not affect final result
- Good move ordering improves effectiveness of pruning
- With "perfect ordering", time complexity = $O(b^{m/2})$ => doubles solvable depth
- A simple example of the value of reasoning about which computations are relevant (a form of metareasoning)
- Unfortunately, 35^{50} is still impossible!

Resource Limits

- Standard approach:
  - Use Cutoff-Test instead of Terminal-Test
    - e.g., depth limit (perhaps add quiescence search)
  - Use Eval instead of Utility
    - e.g., evaluation function that estimates desirability of position

I.e., search "interesting" states more than "standard" ones

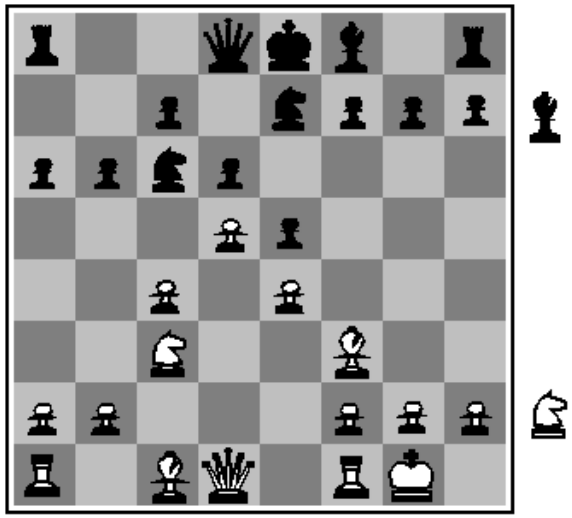
Utility for terminal states; Eval for ALL states

- Suppose we have 100 seconds, explore 10^4 nodes/second

  => 10^6 nodes per move ≈ $35^{8/2}$

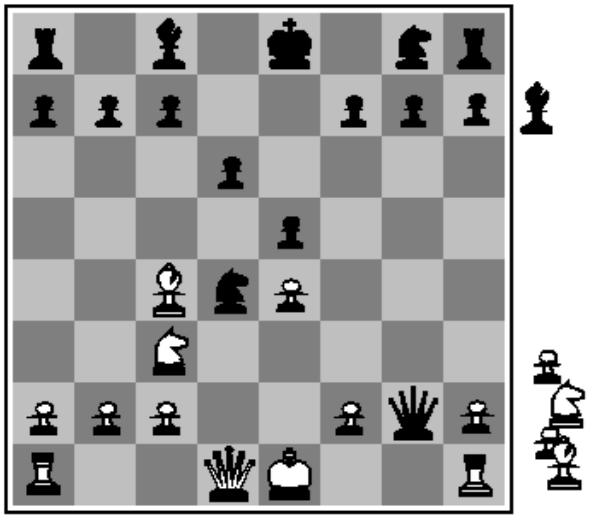
  => α-β reaches depth 8 => pretty good chess program

Evaluation Functions



Black to move

White slightly better



White to move

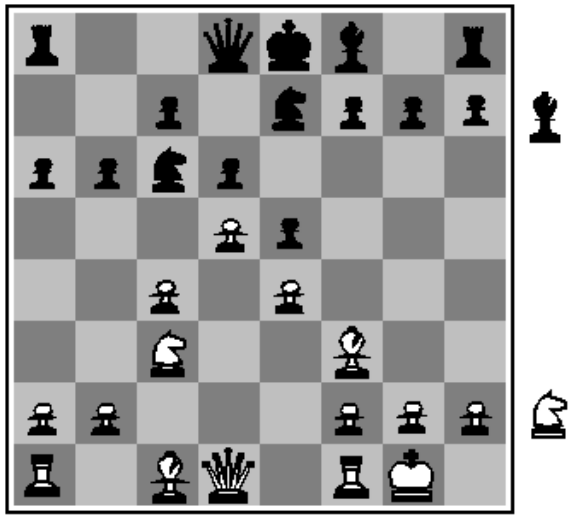
Black winning

- For chess, typically linear weighted sum of features

$$\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

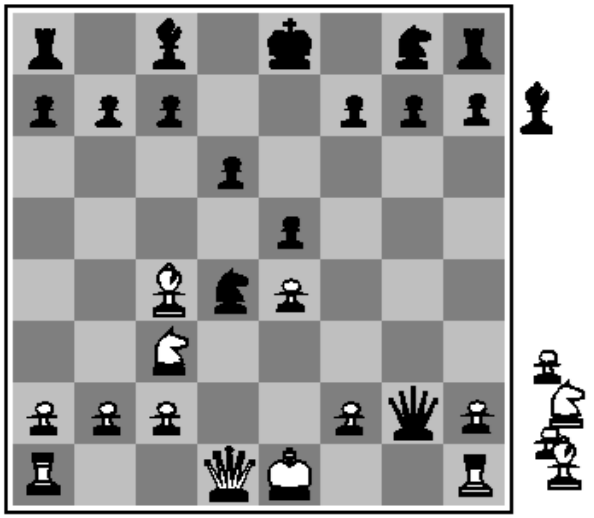
e.g., $w_1 = 9$ with $f_1(s)$ = (number of white queens) – (number of black queens), etc.

Evaluation Functions



Black to move

White slightly better



- For chess, typically linear

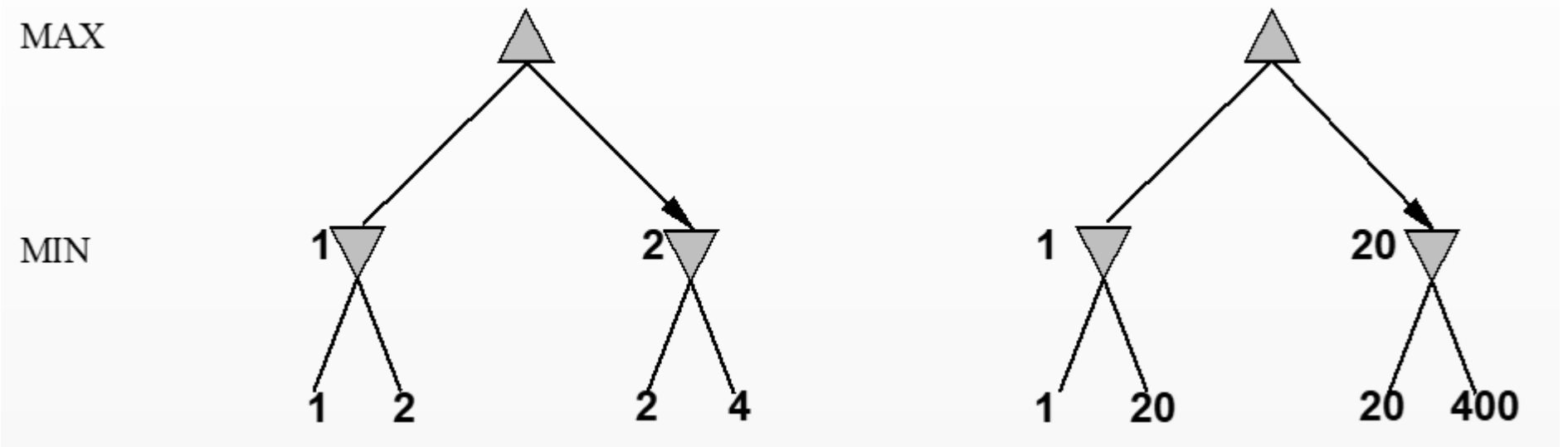
$$Eval(s) = w_1 f_1(s) + w_2$$

e.g., $w_1 = 9$ with $f_1(s) =$ (number c
queens), etc.

Symbol	♙	♘	♗	♖	♕
Piece	pawn	knight	bishop	rook	queen
Value	1	3	3	5	9

3	3.25	5	9	Fischer
3	3.15	4.5	9	Kasparov

Digression: Exact values don't matter



- Behavior is preserved under any monotonic transformation of Eval
- Only the order matters:
  - Payoff in deterministic games acts as an ordinal utility function

Deterministic games in practice

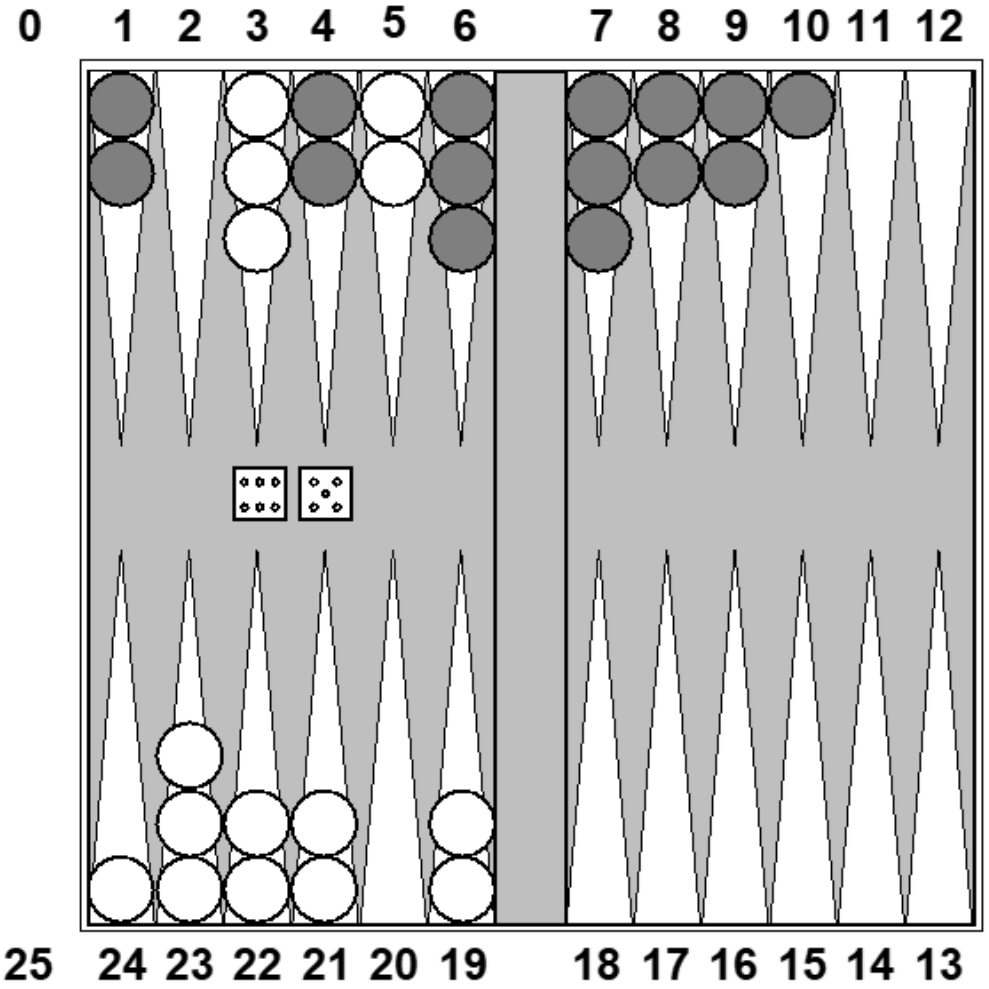
- Checkers: Chinook ended 40-year reign of human world champion Marion Tinsley in 1994. Used an endgame database defining perfect play for all positions involving 8 or fewer pieces on the board, a total of 443,748,401,247 positions.
- Chess: Deep Blue defeated human world champion, Gary Kasparov, in a six-game match in 1997. Deep Blue searches 200 million positions per second, uses very sophisticated evaluation, and undisclosed methods for extending some lines of search up to 40 ply.
- Othello: human champions refuse to compete against computers, who are too good.
- Go: human champions refuse to compete against computers, who are too bad. In go, b > 300, so most programs use pattern knowledge bases to suggest plausible moves.

Deterministic games in practice

- Checkers: Chinook ended 40-year reign of human world champion Marion Tinsley in 1994. Used an endgame database defining perfect play for all positions involving 8 or fewer pieces on the board, a total of 443,748,401,247 positions.
- Chess: Deep Blue defeated human world champion, Gary Kasparov, in a six-game match in 1997. Deep Blue searches 200 million positions per second, uses very sophisticated evaluation, and undisclosed methods for extending some lines of search up to 40 ply.
- Othello: human champions refuse to compete against computers, who are too good.
- Go: human champions refuse to compete against computers, who are too bad. In go, b > 300, so most programs use pattern knowledge bases to suggest plausible moves.

AlphaGo

Nondeterministic games: backgammon

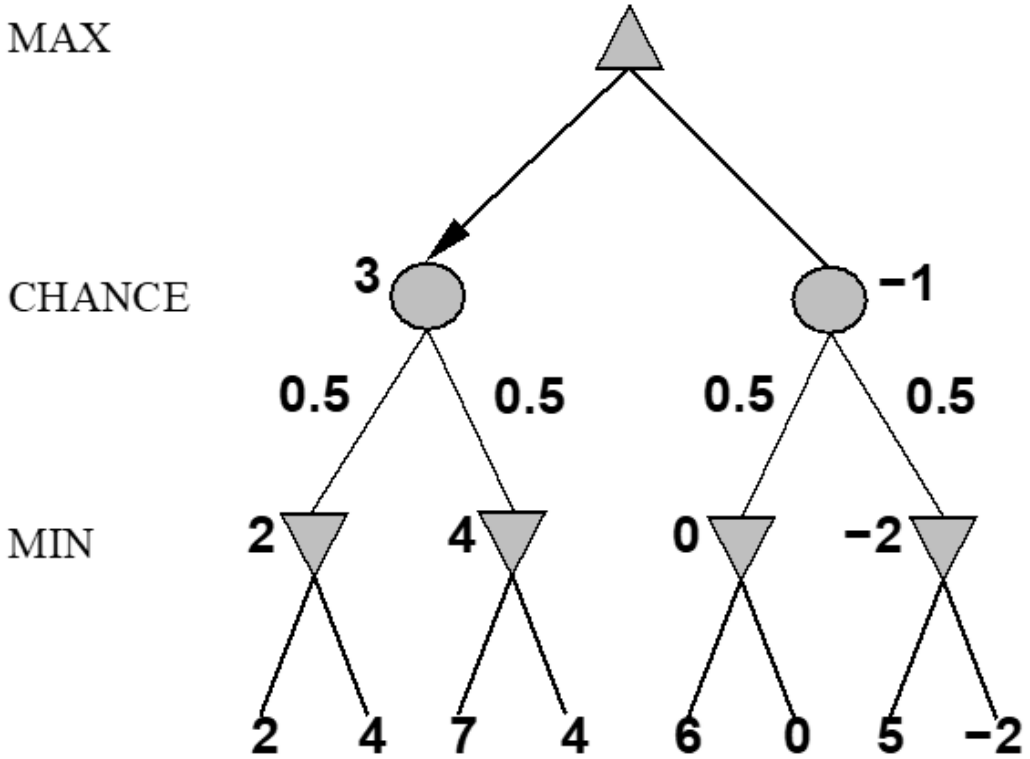


Expecti-minimax

- For chance layer, compute a score weighted by the probability that a move occurs
- In other words: Use the expected value of a move, (instead of max or min) weighted by the probability of that branch of the tree

Nondeterministic games in general

- In nondeterministic games, chance introduced by dice, card-shuffling
- Simplified example with coin-flipping



Algorithm for nondeterministic games

- Expecti-minimax gives perfect play
- Just like minimax, except we must also handle chance nodes

```
...
if state is a Max node then
        return the highest ExpectiMinimax-Value of Successors(state)
if state is a Min node then
        return the lowest ExpectiMinimax-Value of Successors(state)
if state is a chance node then
        return average of ExpectiMinimax-Value of Successors(state)
...
```

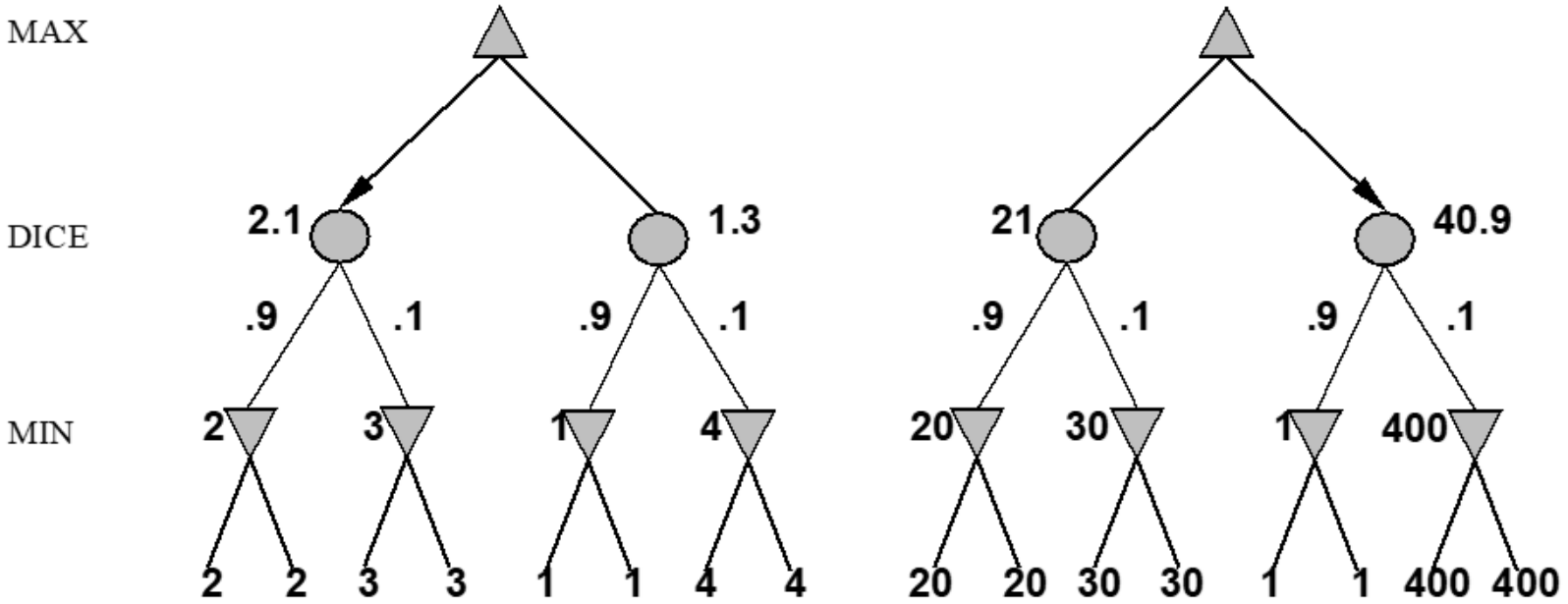
Nondeterministic games in practice

- Dice rolls increase b: 21 possible rolls with 2 dice
- Backgammon ≈ 20 legal moves

  Depth 4 = $20 \times (21 \times 20)^3 \approx 1.2 \times 10^9$

- As depth increases, probability of reaching a given node shrinks => value of lookahead is diminished
- α-β pruning is much less effective
- TDGammon uses depth-2 search + very good Eval
  ≈ world-champion level

Digression: Exact values DO matter



- Behavior is preserved only by positive linear transformation of Eval
- Hence Eval should be proportional to the expected payoff

Games of Imperfect Information

- E.g., card games, where opponent's initial cards are unknown
- Typically, we can calculate a probability for each possible deal
- Seems just like having one big dice roll at the beginning of the game
- Idea: compute the minimax value of each action in each deal, then choose the action with highest expected value over all deals.
- Special case: if an action is optimal for all deals, it's optimal
- GIB, current best bridge program, approximates this idea by:
  - Generating 100 deals consistent with bidding information
  - Picking the action the wins the most tricks on average

Proper Analysis

* Intuition that the value of an action is the average of its values in all actual states is WRONG

- With partial observability, value of an action depends on the information state or belief state the agent is in
- Can generate and search a tree of information states
- Leads to rational behaviors such as:
  - Acting to obtain information
  - Signaling to one's partner
  - Acting randomly to minimize information disclosure

Coming up…

- What if the game is too complex for minimax and related searches?
- Can we do reasonably well using some other approach?
  - Monte Carlo methods

CS 380: ARTIFICIAL INTELLIGENCE

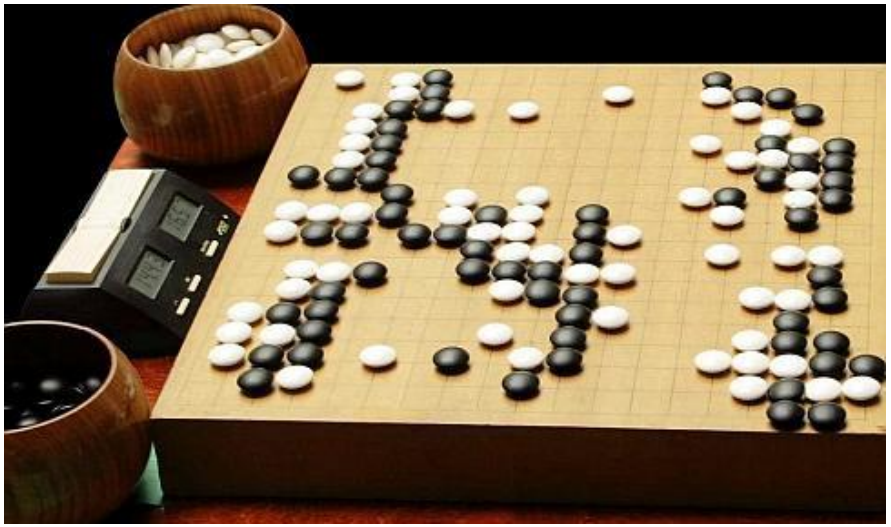
Lecture 7: Adversarial Search II

Recall: Adversarial Search

- Idea:
  - When there is only one agent in the world, we can solve problems using DFS, BFS, ID, A*, etc.
  - When more than one agent, we need adversarial search
- Last time we saw:
  - Minimax:
    - Systematically expands a “game tree” (can bound depth)
    - Apply utility function at leaves, and then back-propagate values, to find best action
  - Alpha-beta pruning:
    - Improvement over minimax to expand less nodes
  - Expecti-minimax:
    - Extend minimax with chance nodes when we have nondeterminism
    - Can still use Alpha-beta pruning

Minimax in Practice

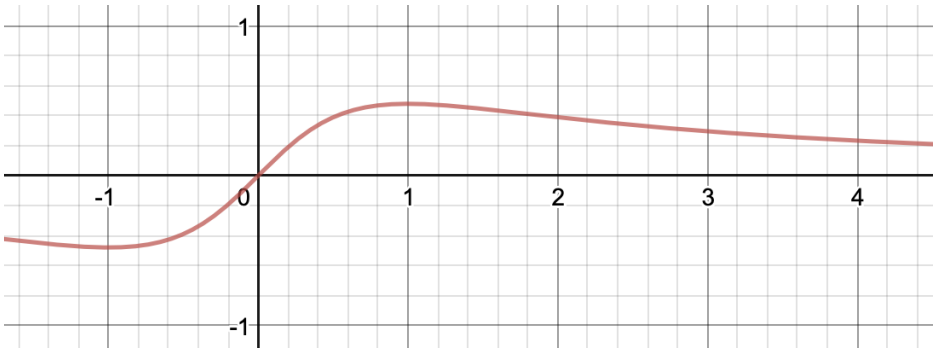
- Minimax (with alpha-beta pruning) can be used to create AI that plays games like Checkers or Chess
- Problem: There are games that humans can play that are too complex for Minimax
  - For example: Go



Monte Carlo Methods

- Algorithms that relay on random sampling to find solution approximations.
- Example: Monte Carlo integration
  - Imagine that I ask you to compute the following:

$$A = \int_1^3 sin(x) \frac{1}{1 + x^2} dx$$



Monte Carlo Methods

- Method 1: Symbolic integration
  - You could fetch your calculus book and integrate the function using known formulas
  - But you'd need to do this by hand (did you know that automatic symbolic integration is still unsolved?)
- Method 2: Numerical computations
  - Approximate integral with rectangles, polynomials, etc.
- Method 3: "Monte Carlo" methods…

Monte Carlo Methods

- Method 3: Monte Carlo

$$f(x) = sin(x)\frac{1}{1+x^2}$$

  - Repeat *N* times:
    - Pick a random *x* between 1 and 3
    - Evaluate *f*(*x*)
  - Now average and multiply by 2 (due to integral 1→3)
  - Voilà!
  - The larger the *N*, the better the approximation

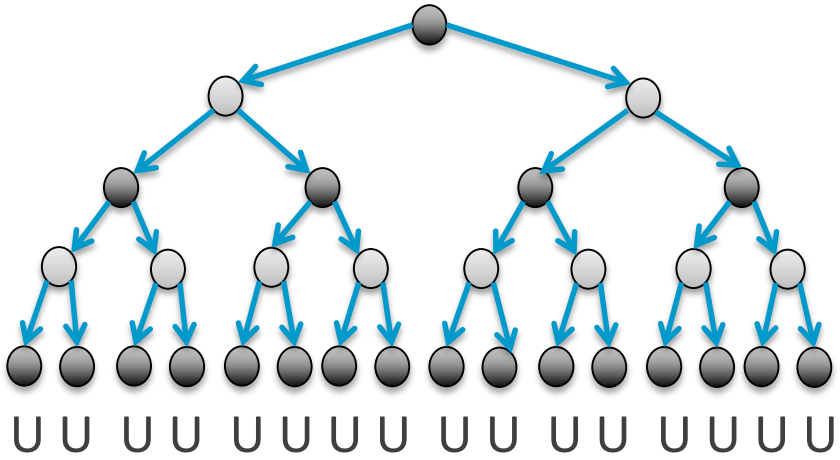
Monte Carlo Methods

- Idea:
  - Use random sampling to approximate the solution to complex problems

- How can we apply this idea to adversarial search?
  - The answer to this question is why we have computer programs that can play Go at a master level.

Minimax vs Monte Carlo Search

Minimax:

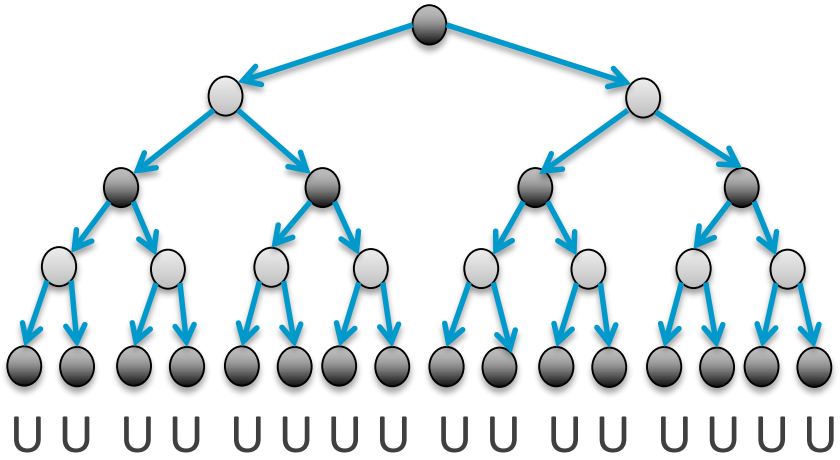


Minimax opens the complete tree (all possible moves) up to a fixed depth.
Then, a Utility function is applied to the leaves.

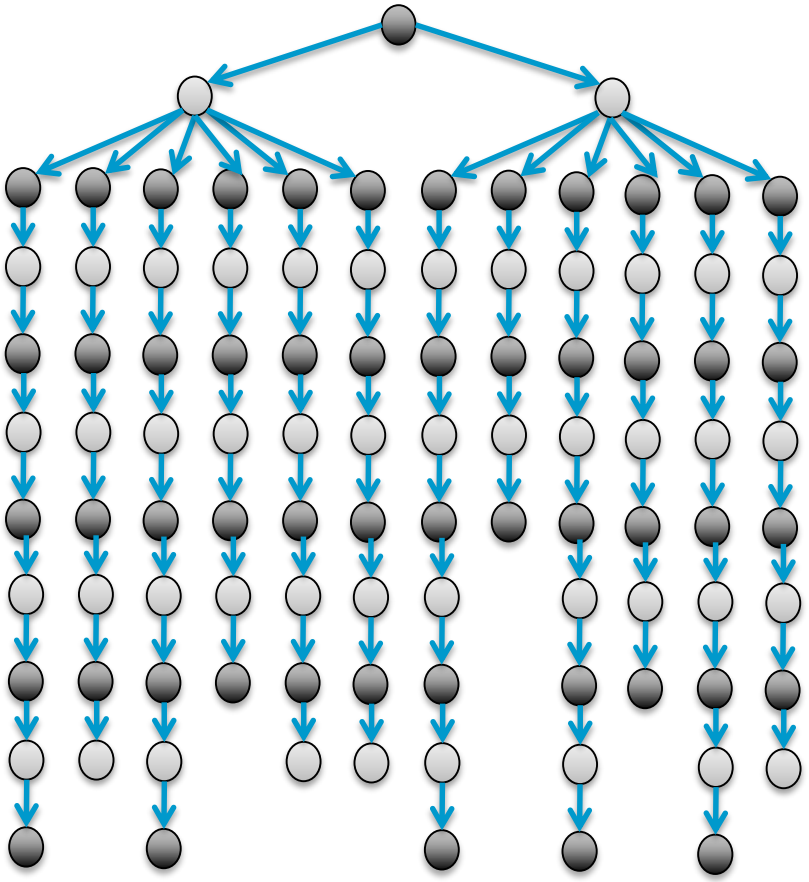
Monte-Carlo:

Minimax vs Monte Carlo Search

Minimax:

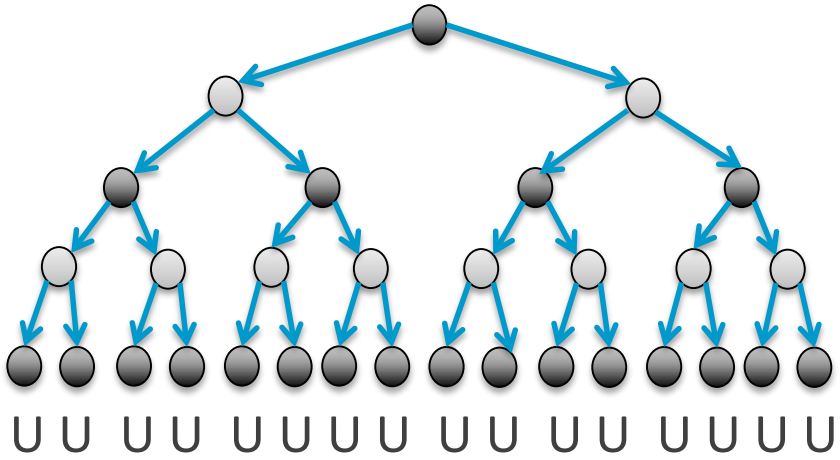


Monte-Carlo:

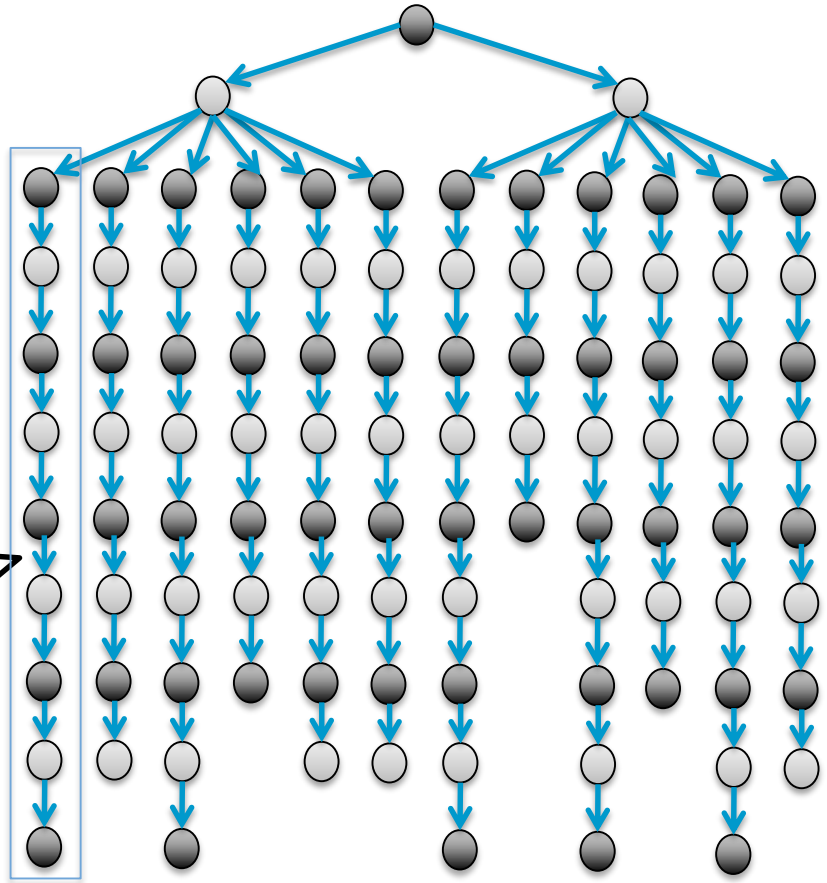


Minimax vs Monte Carlo Search

Minimax:



Monte-Carlo:



For each possible move at the root node, Monte-Carlo search runs a fixed number *K* of random complete games.
No need for an Eval function!
(but can be used if you want)

Complete Game

Monte Carlo Search

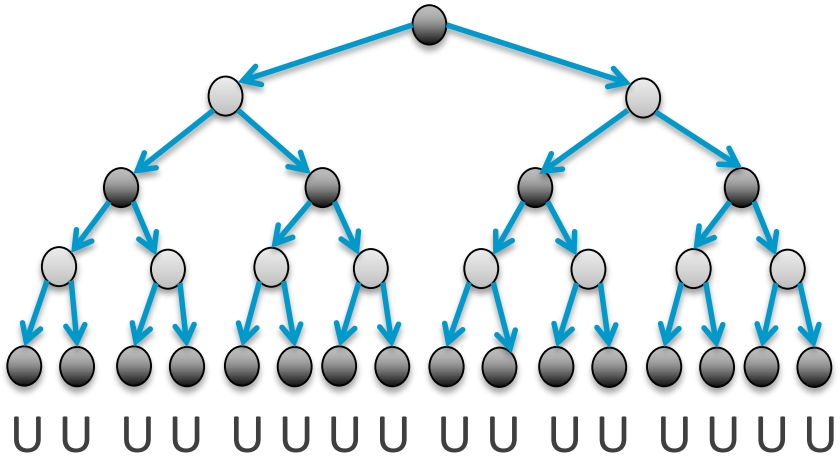
- For each possible move:
  - Repeat *N* times:
    - Play a game until the end, selecting moves at random
  - Count the percentage of wins
- Select the action with the highest percentage of wins.
- Properties:
  - **Complete**: ?
  - **Optimal**: ?
  - **Time**: ?
  - **Memory**: ?

Monte Carlo Search

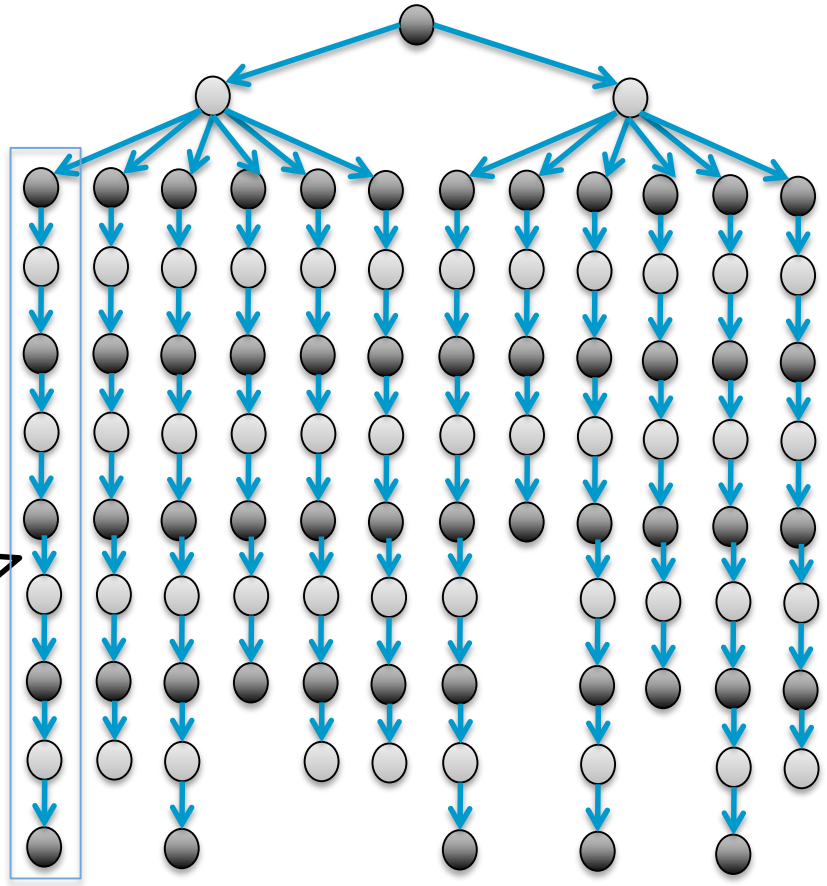
- For each possible move:
  - Repeat *N* times:
    - Play a game until the end, selecting moves at random
  - Count the percentage of wins
- Select the action with the highest percentage of wins.
- Properties:
  - **Complete**: no
  - **Optimal**: no
  - **Time**: d*b*n (depth * branching factor * N)
  - **Memory**: b (for one level of search)
- Works much better than minimax for large games, but has many problems. We can do better.

Minimax vs Monte Carlo Search

Minimax:



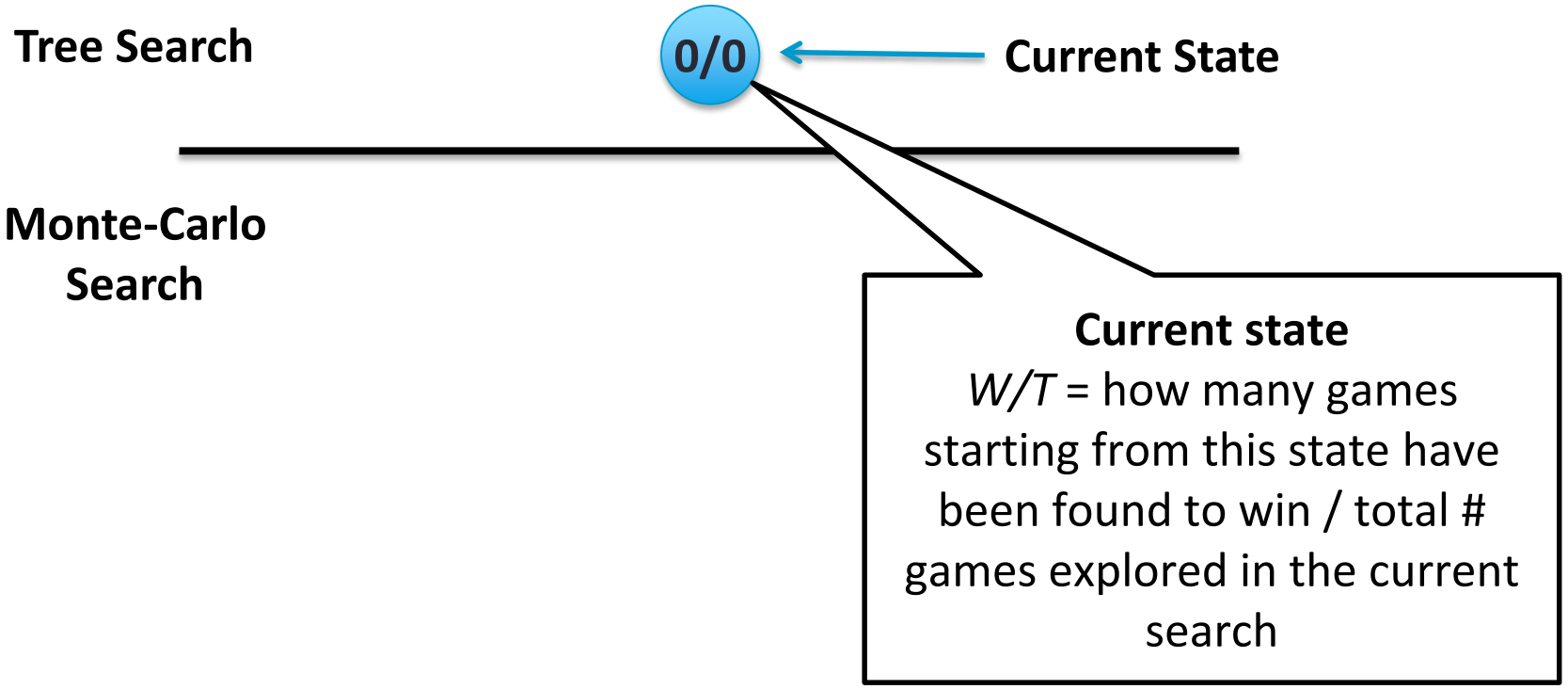
Monte-Carlo:



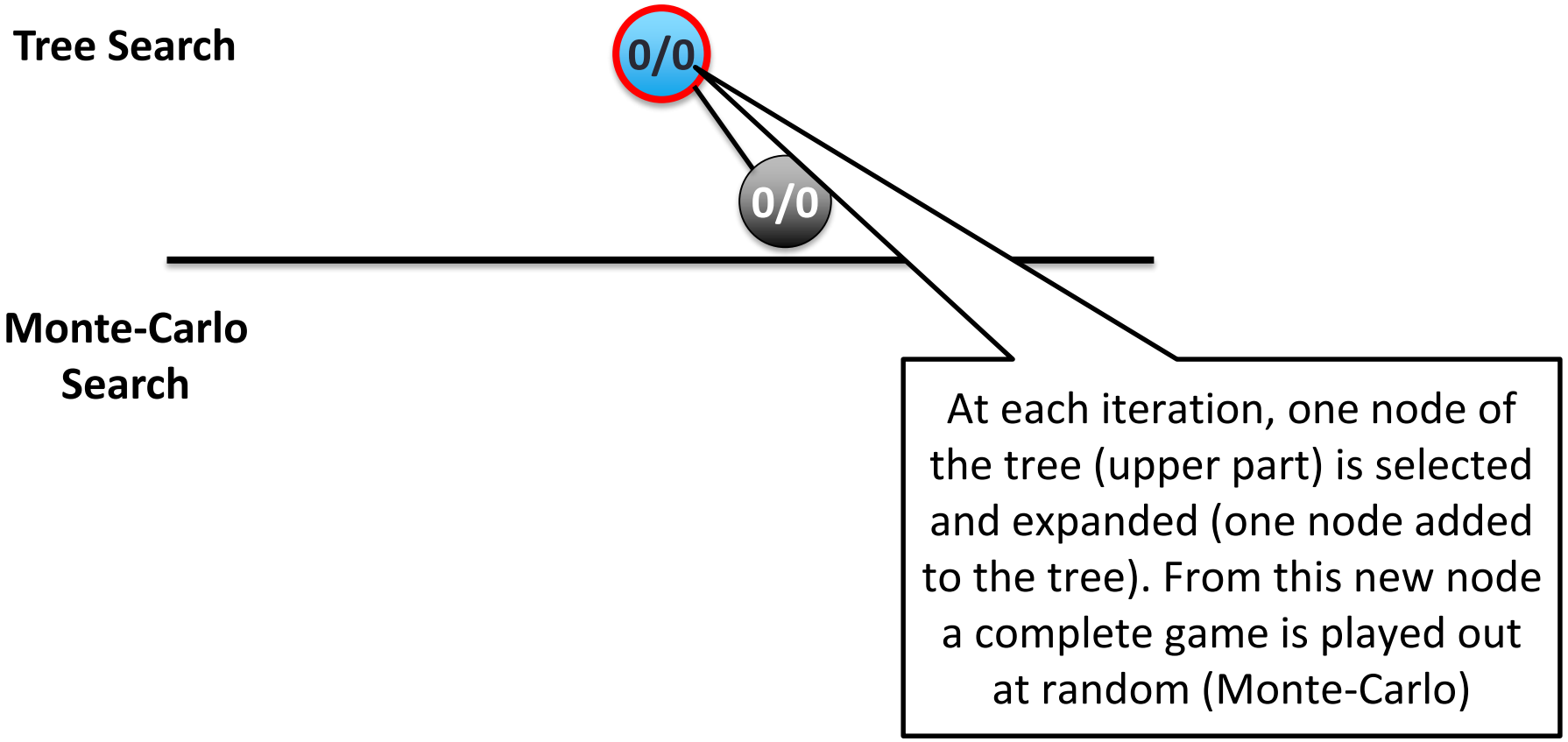
For each possible move at the root node, Monte-Carlo search runs a fixed number *K* of random complete games.
No need for an Eval function!
(but can be used if you want)

Complete Game

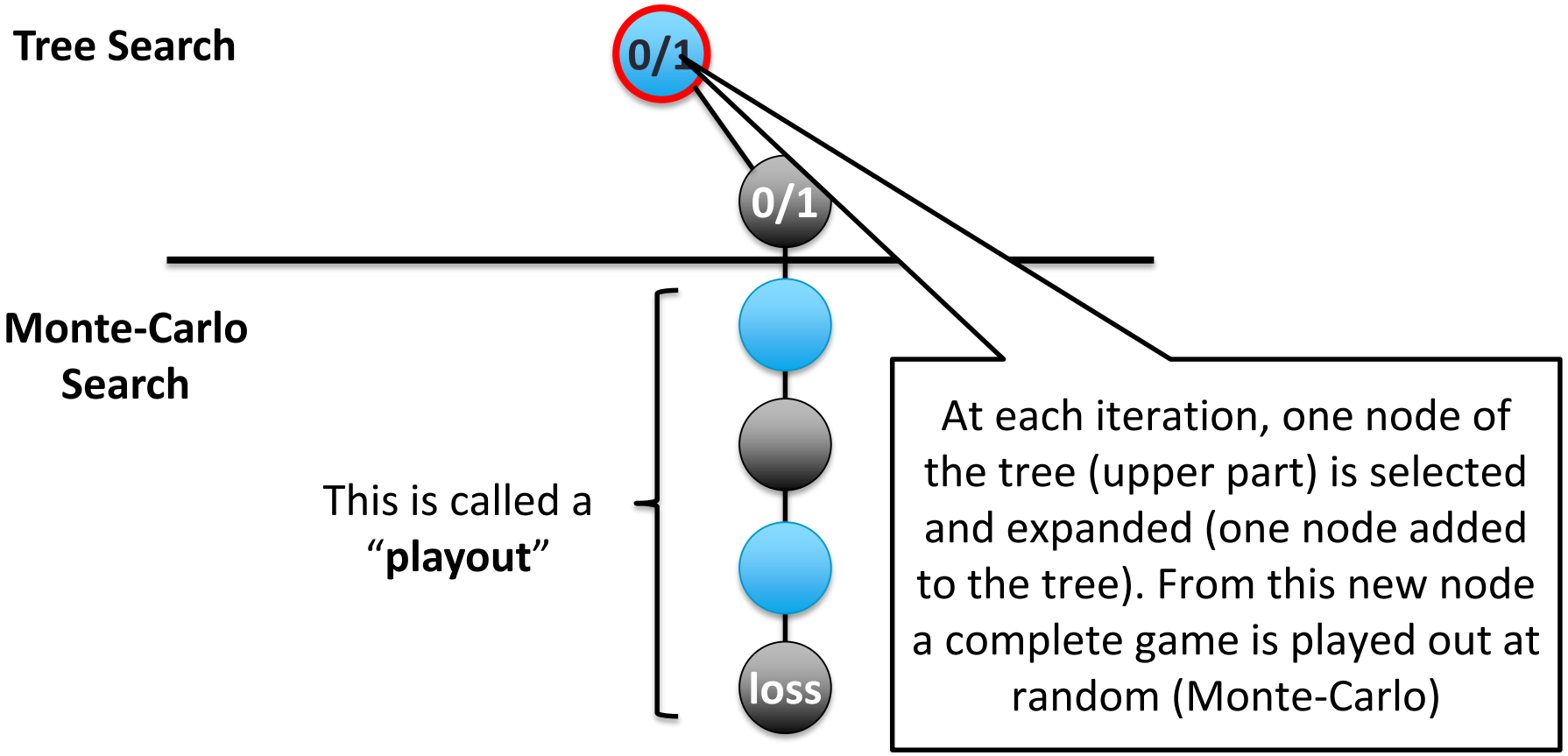
Monte Carlo Tree Search



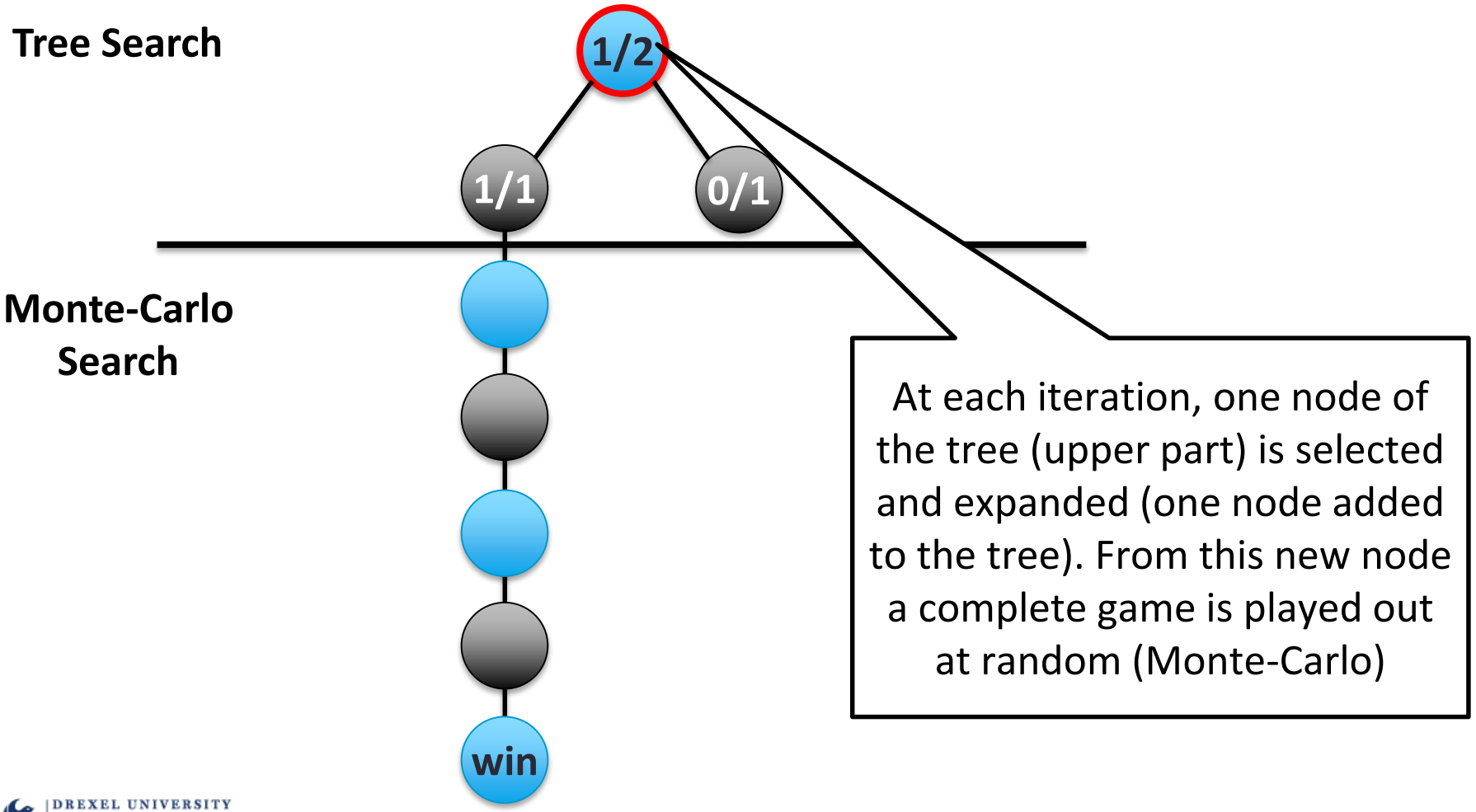
Monte Carlo Tree Search



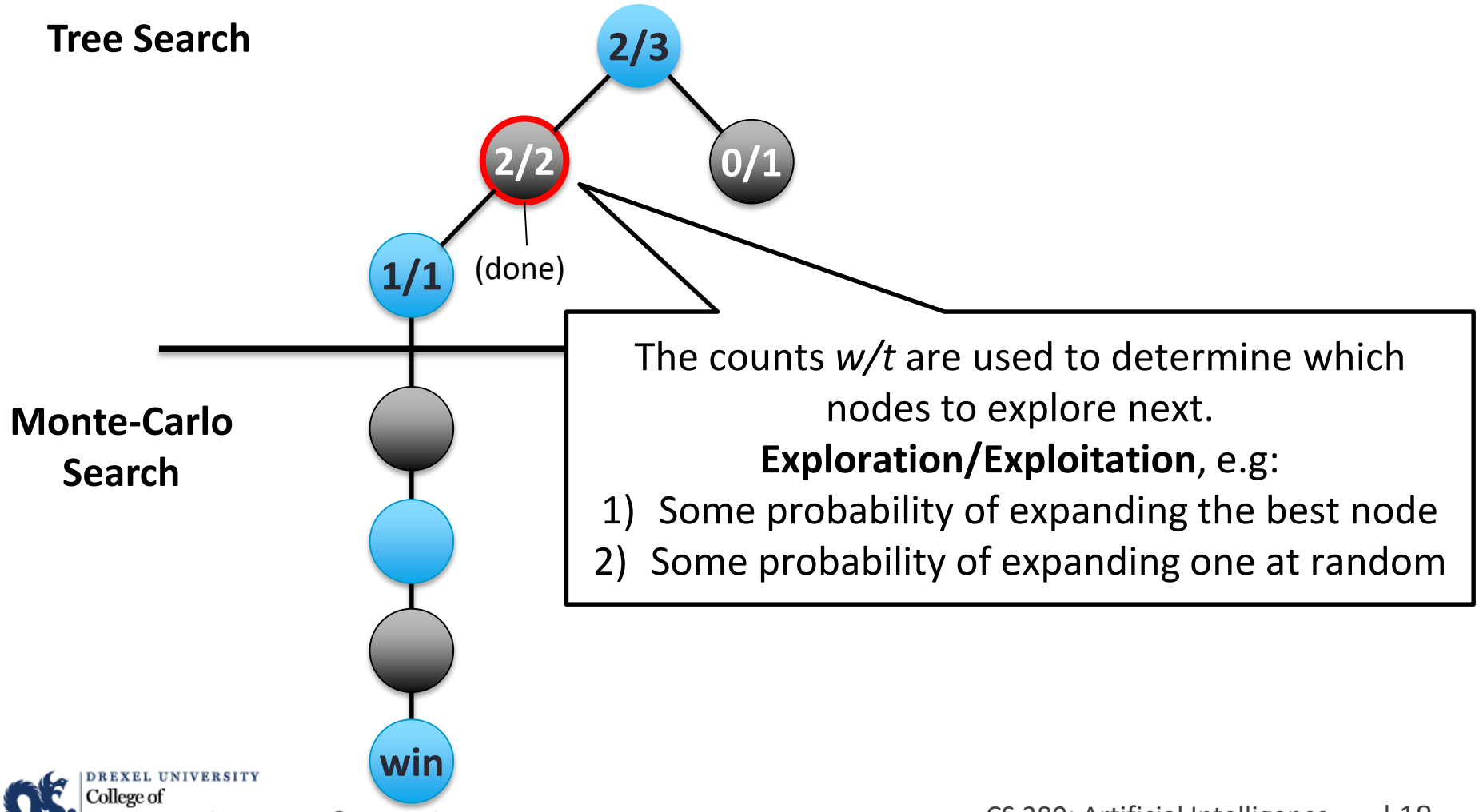
Monte Carlo Tree Search



Monte Carlo Tree Search

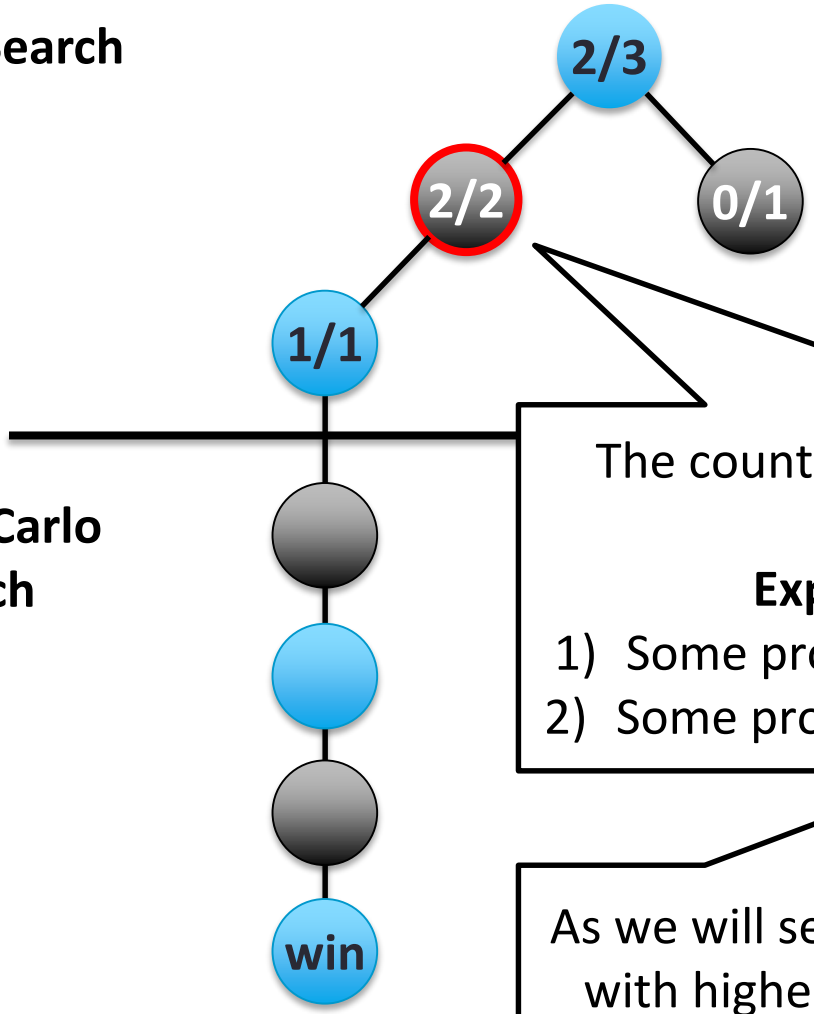


Monte Carlo Tree Search



Monte Carlo Tree Search

Tree Search

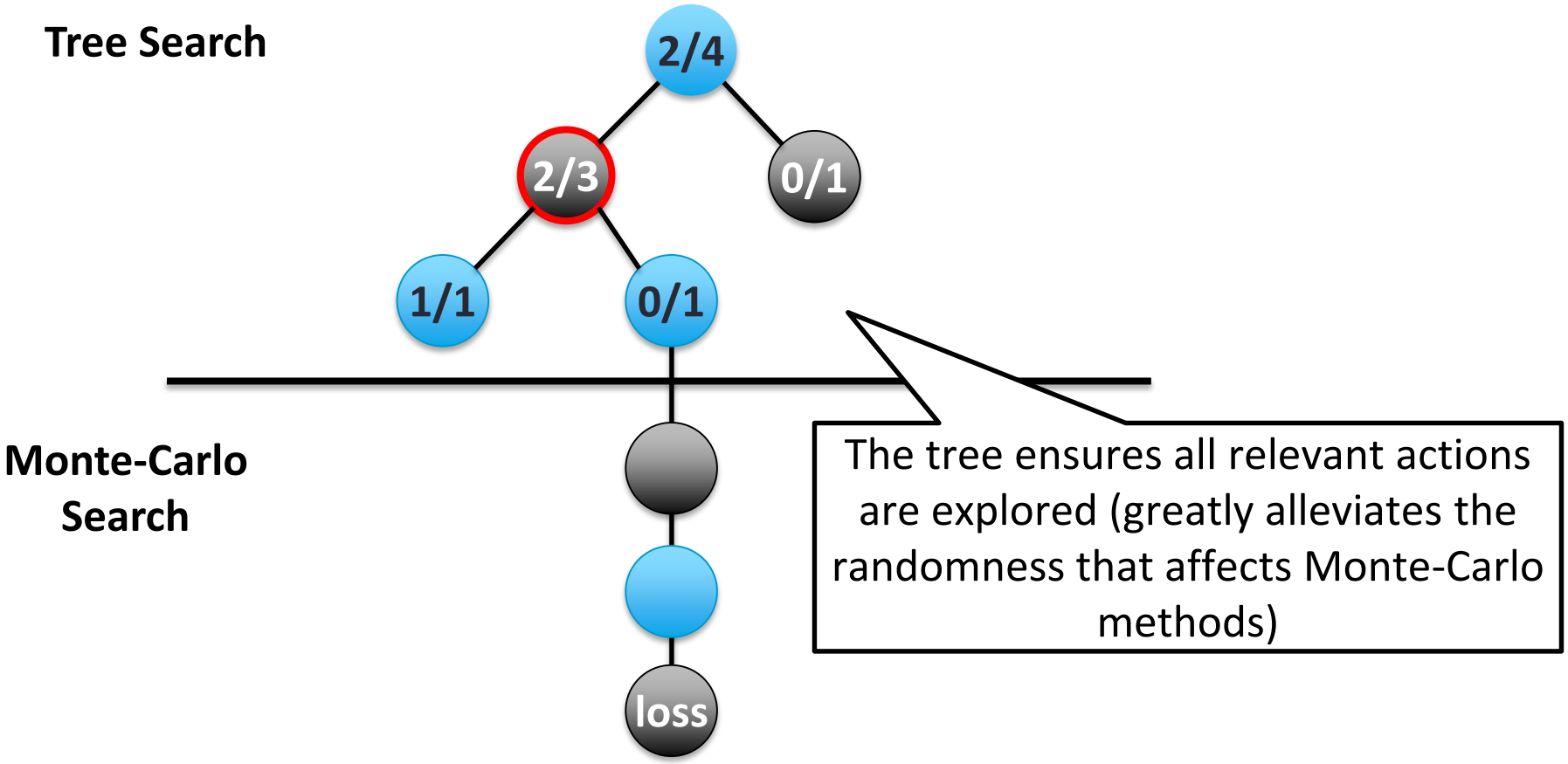


Monte-Carlo Search

The counts w/t are used to determine which nodes to explore next.
Exploration/Exploitation, e.g:
1) Some probability of expanding the best node
2) Some probability of expanding one at random

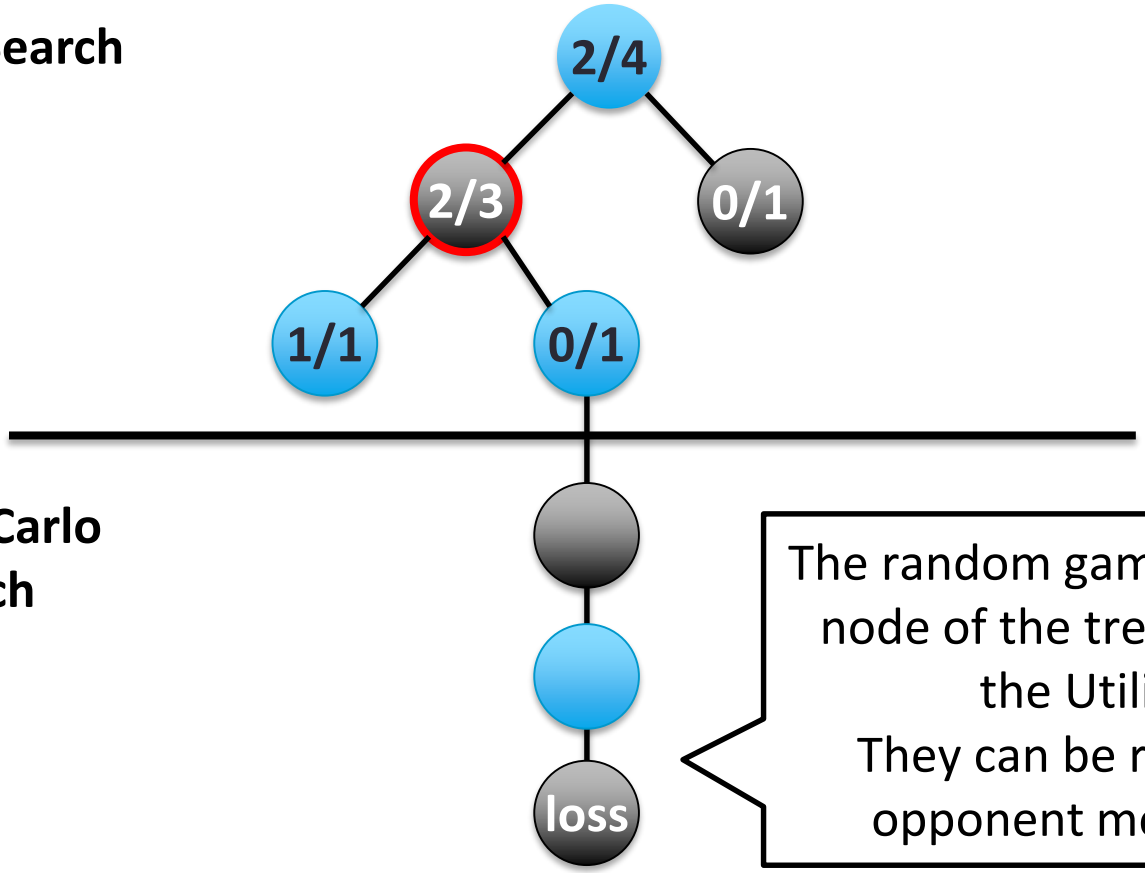
As we will see, we want to expand the best node with higher probability than any of the others

Monte Carlo Tree Search



Monte Carlo Tree Search

Tree Search



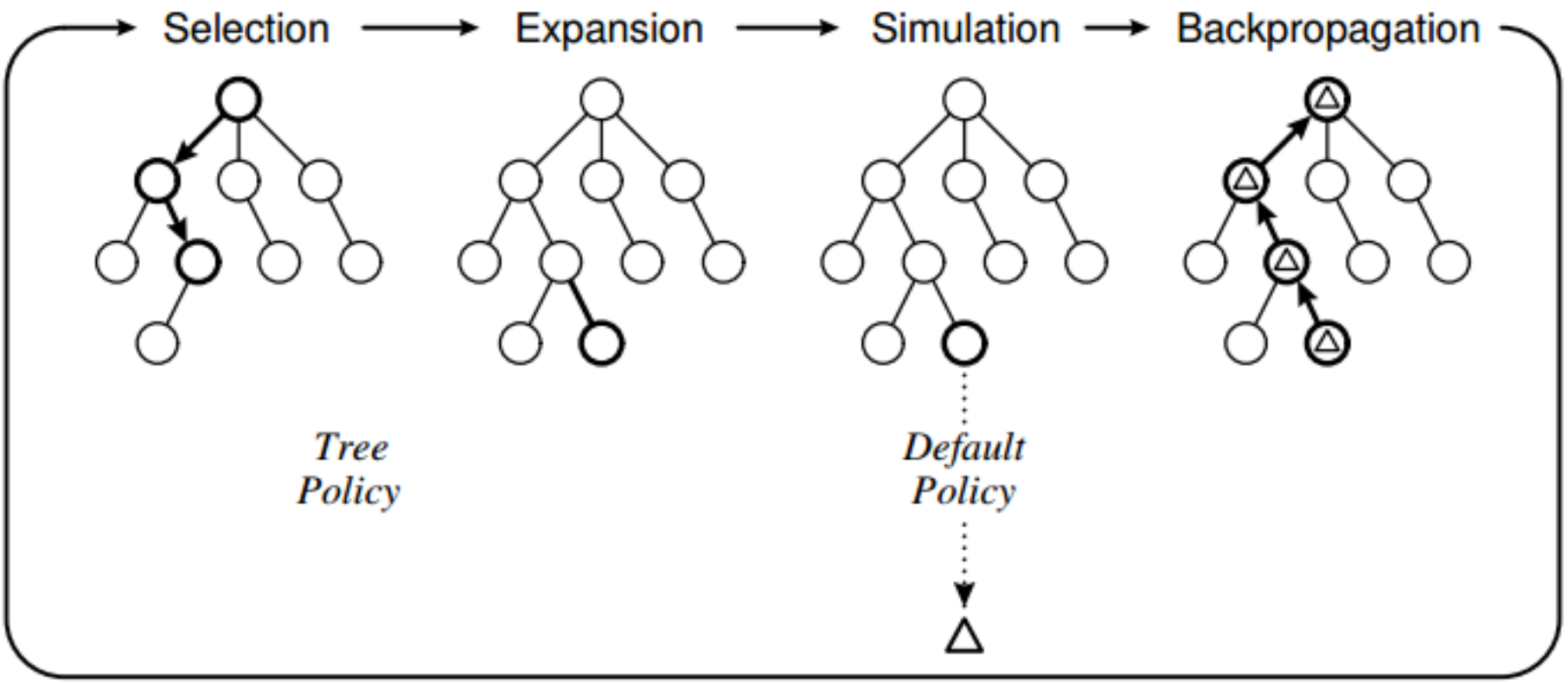
Monte-Carlo Search

The random games played from each node of the tree serve to estimate the Utility function.
They can be random, or use an opponent model (if available)

MCTS Algorithm

```
MCTS(state, player)
  tree = new Node(state, player)
  Repeat until computation budget is exhausted
    node = treePolicy(tree)
    if (node.isTerminal) child = node
                         else child = node.nextChild();
    R = playout(child)
    child.propagateReward(R)
  Return tree.bestChild();
```

Monte Carlo Tree Search



MCTS Algorithm

```
MCTS(state, player)
  tree = new Node(state, player)
  Repeat until computation budget is exhausted
    node = treePolicy(tree)
    if (node.isTerminal) child = node
                         else child = node.nextChild();
    R = playout(child)
    child.propagateReward(R)
  Return tree.bestChild();
```

Monte Carlo Tree Search

- Question: How do we choose the next node to be added to the tree?
  - Start at the root.
  - Descend the tree choosing actions according to the current probability estimates. (Assume a uniform probability distribution for anything you haven't seen before)
  - Add to the tree the first node that you reach that isn't already in it.
- Or we could use something other than just the current probability estimates.
- This looks like the "multi-armed bandit" problem…

Monte Carlo Tree Search

- A “one-armed bandit” (a.k.a. slot machine) …



https://www.amazon.com/Blue-Metal-Money-Plastic-Colourful/dp/B01B1E01A2

Monte Carlo Tree Search

- The “multi-armed bandit” problem…



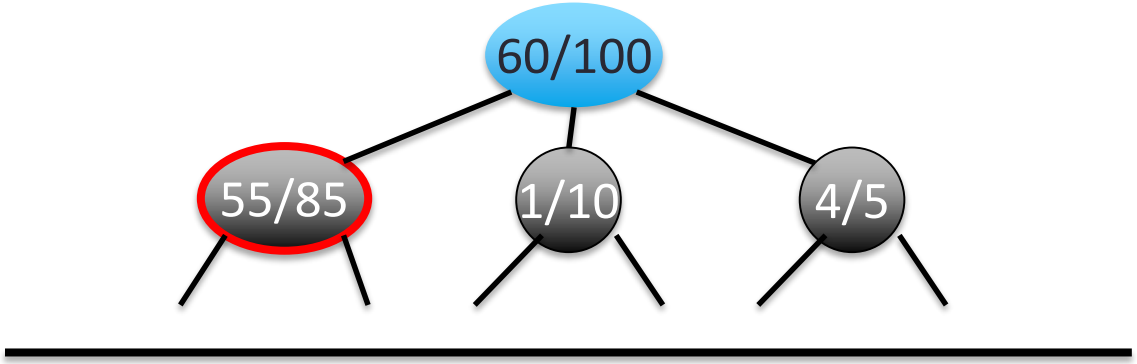
From en.wikipedia.org/wiki/Multi-armed_bandit#/media/File:Las_Vegas_slot_machines.jpg

Tree Policy: ε-greedy

- Given a list of children, which one do we explore in each iteration of MCTS?
- Ideally, we want:
  - To spend more time exploring **good** children (no point wasting lots of time on bad children)
  - But maybe spend a little time exploring **bad** children, just in case they are good (since evaluation is stochastic, it might happen that we were just unlucky, and a child we thought was bad is actually good)
- Simplest idea: ε-greedy
  - With probability 1-ε: choose the current best
  - With probability ε: choose one at random

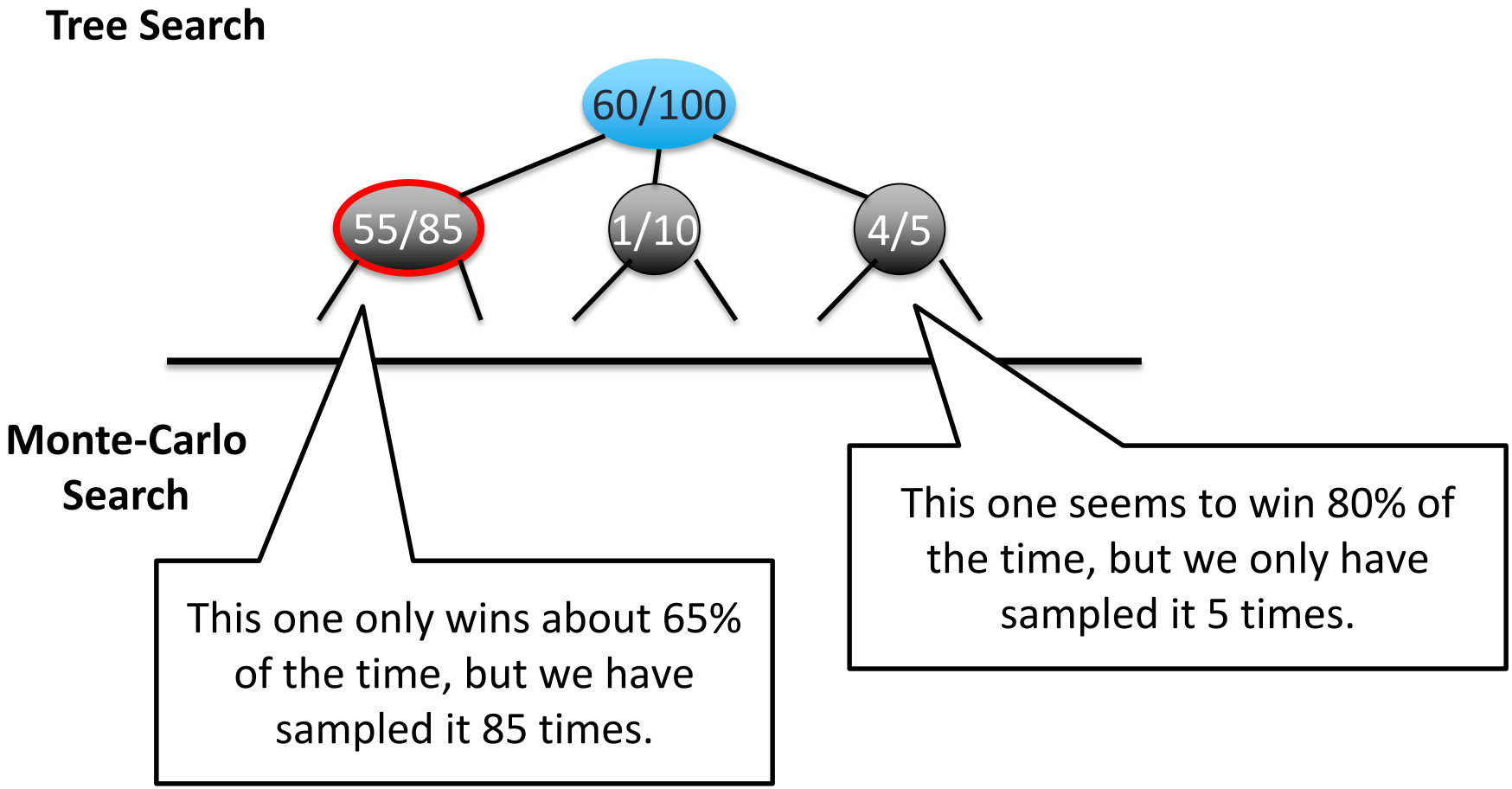
Which is the best child?

Tree Search

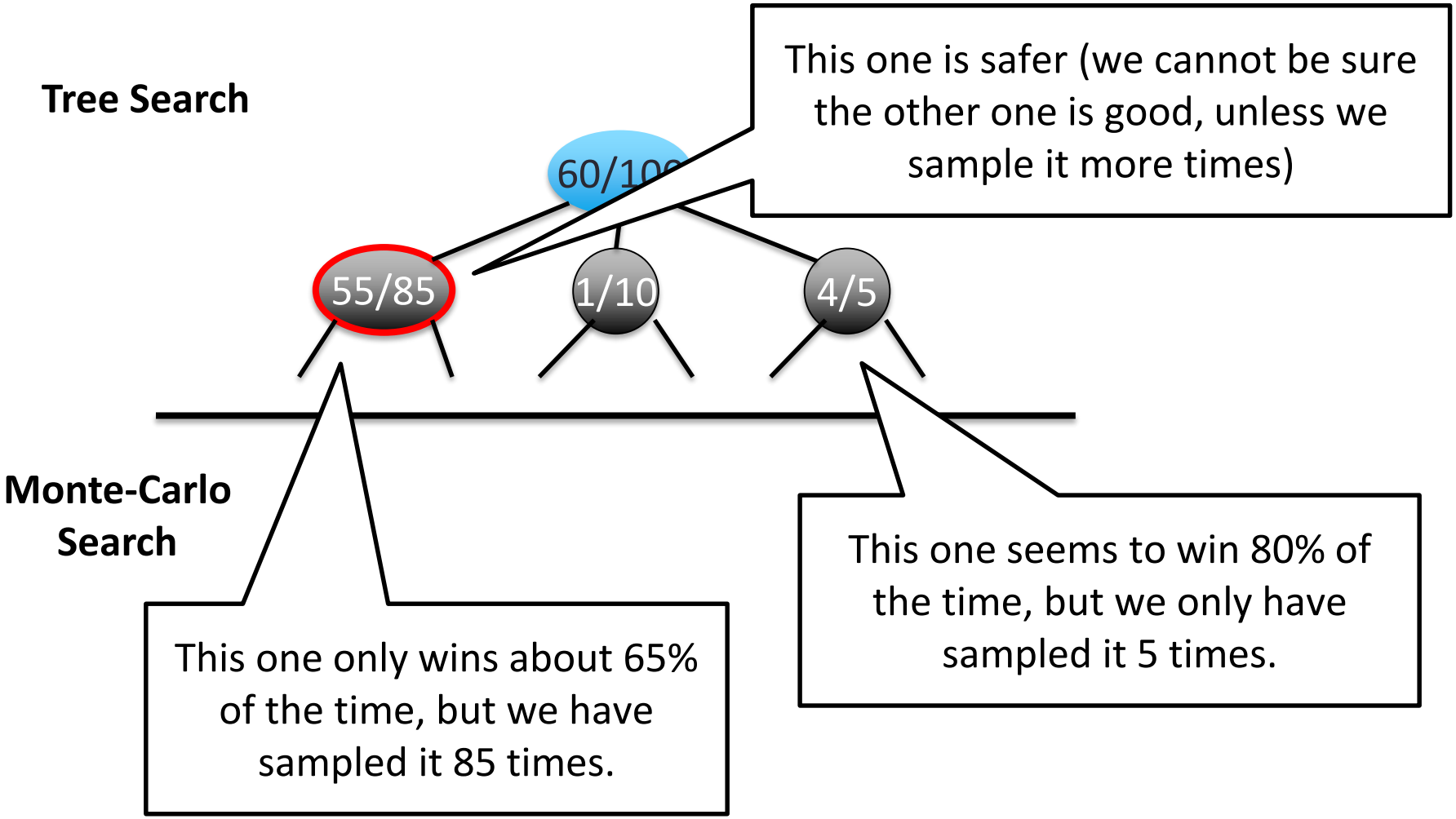


Monte-Carlo Search

Which is the best child?



What is the best child?

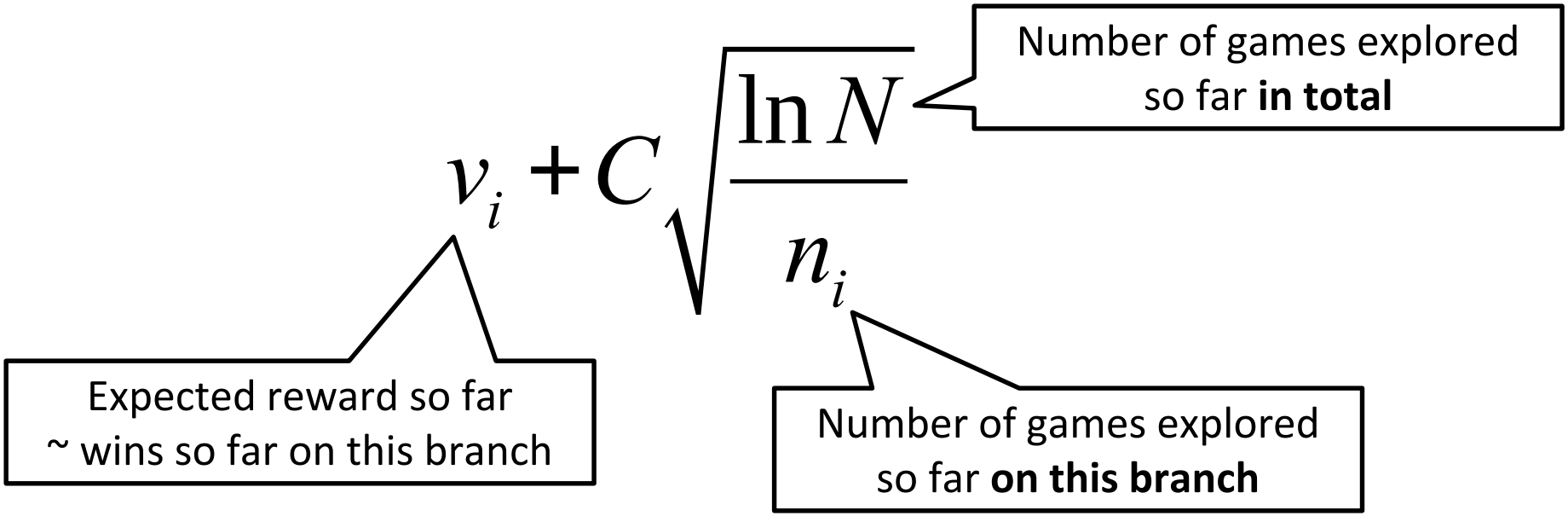


Tree Policy: Can we do Better?

- We just mentioned the ε-greedy policy… but is there a way to do better?
- ε-greedy is robust, but, for example:
  - If there are 3 children:
    - A: 40/100
    - B: 39/100
    - C: 2/100

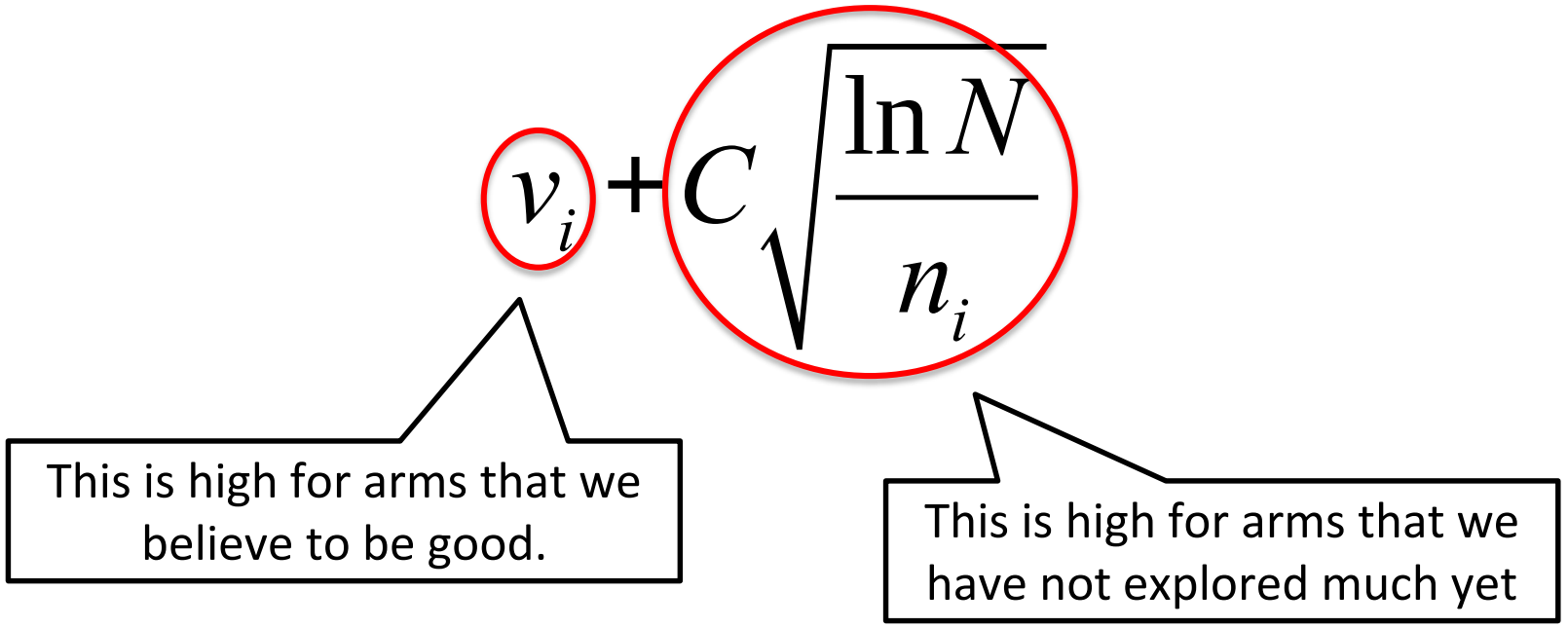
UCB1

- "Upper Confidence Bounds" (version 1)
- Balance between **exploration** and **exploitation**
- UCB value for a given "arm" (or tree branch) *i*



UCB1

- "Upper Confidence Bounds" (version 1)
- Balance between **exploration** and **exploitation**
- UCB value for a given "arm" (or tree branch) *i*



Tree Policy: Can we do Better?

- We just mentioned the ε-greedy policy… but is there a way to do better?
- ε-greedy is robust, but, for example:
  - If there are 3 children:
    - A: 40/100
    - B: 39/100
    - C: 2/100

Almost same values

N = 300

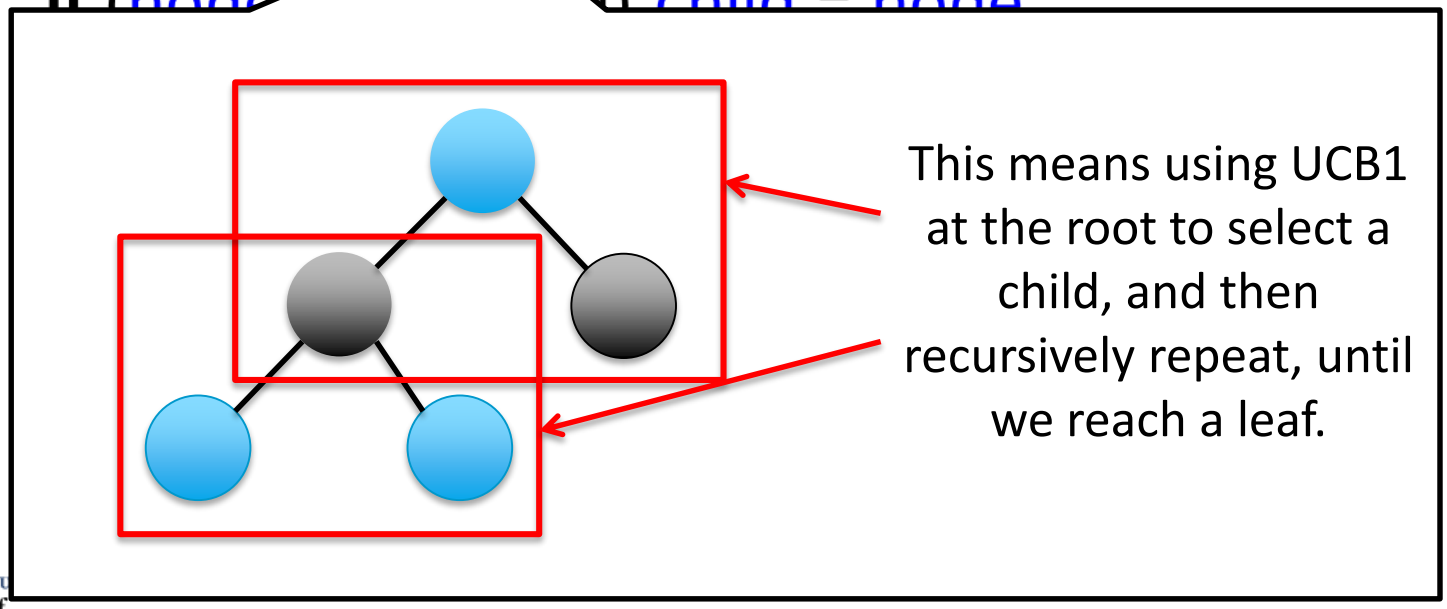
$$v_i + C\sqrt{\frac{\ln N}{n_i}}$$

Upper Confidence Tree Algorithm

```
UCT(state, player)
  tree = new Node(state, player)
  Repeat until computation budget is exhausted
    node = selectNodeViaUCB1(tree)
    if (node.isTerminal) child = node
                         else child = node.nextChild();
    R = playout(child)
    child.propagateReward(R)
  Return tree.bestChild();
```

Upper Confidence Tree Algorithm

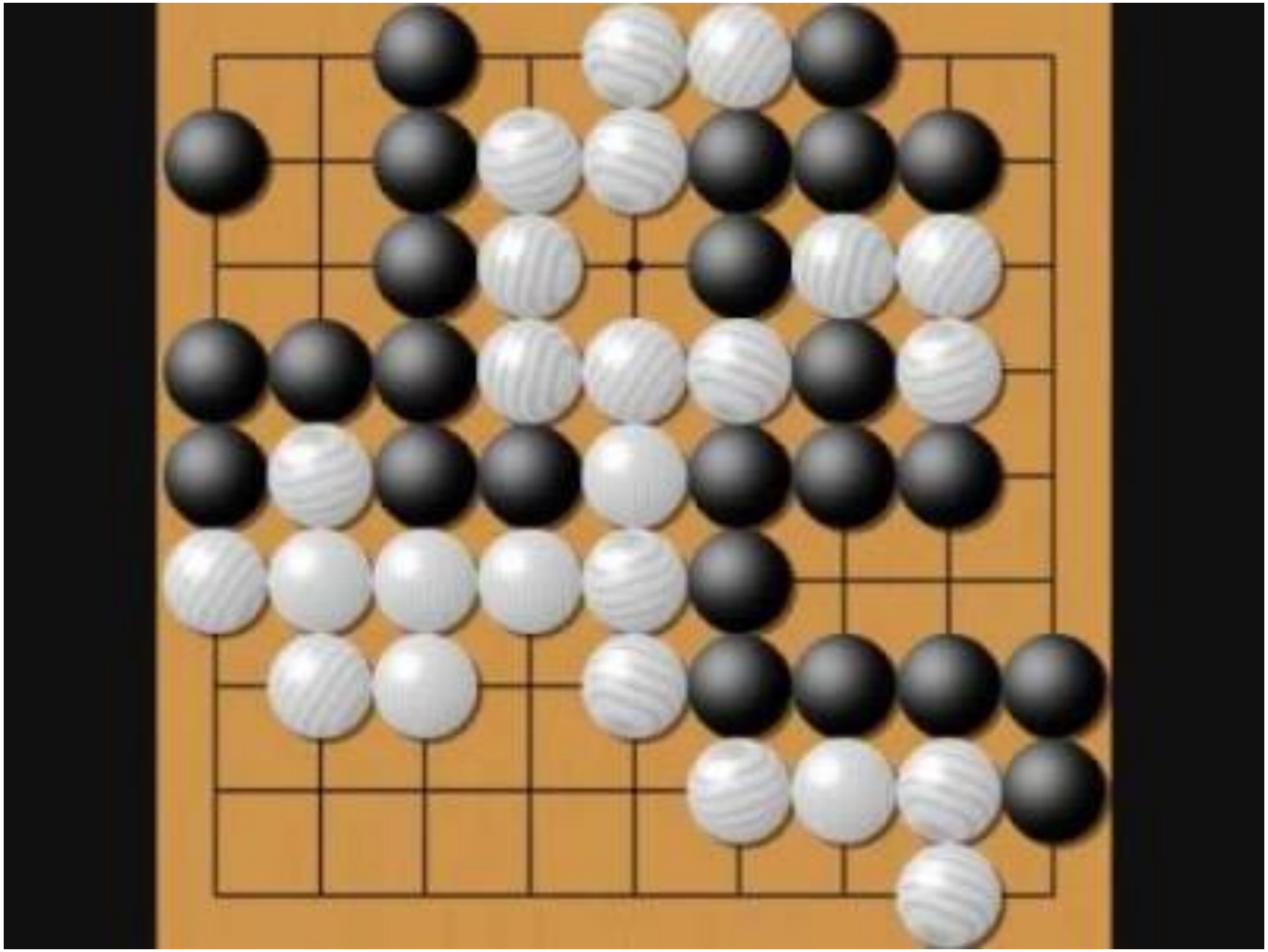
```
UCT(state, player)
  tree = new Node(state, player)
  Repeat until computation budget is exhausted
    node = selectNodeViaUCB1(tree)
    if (node
```



Monte Carlo Tree Search

- After a fixed number of iterations *K* (or after the assigned time is over), MCTS analyzes the tree and picks an action with the highest win ratio
- MCTS algorithms do not explore the whole game tree!
  - They ***sample*** the game tree
  - They spend more time in moves that are more promising
  - They are "***any-time* algorithms**" (can be stopped at any time)
- It can be shown theoretically that when *K* goes to infinity, the values assigned to each action in the MCTS tree converge to those computed by minimax
- MCTS algorithms are the standard algorithms for modern Go playing programs

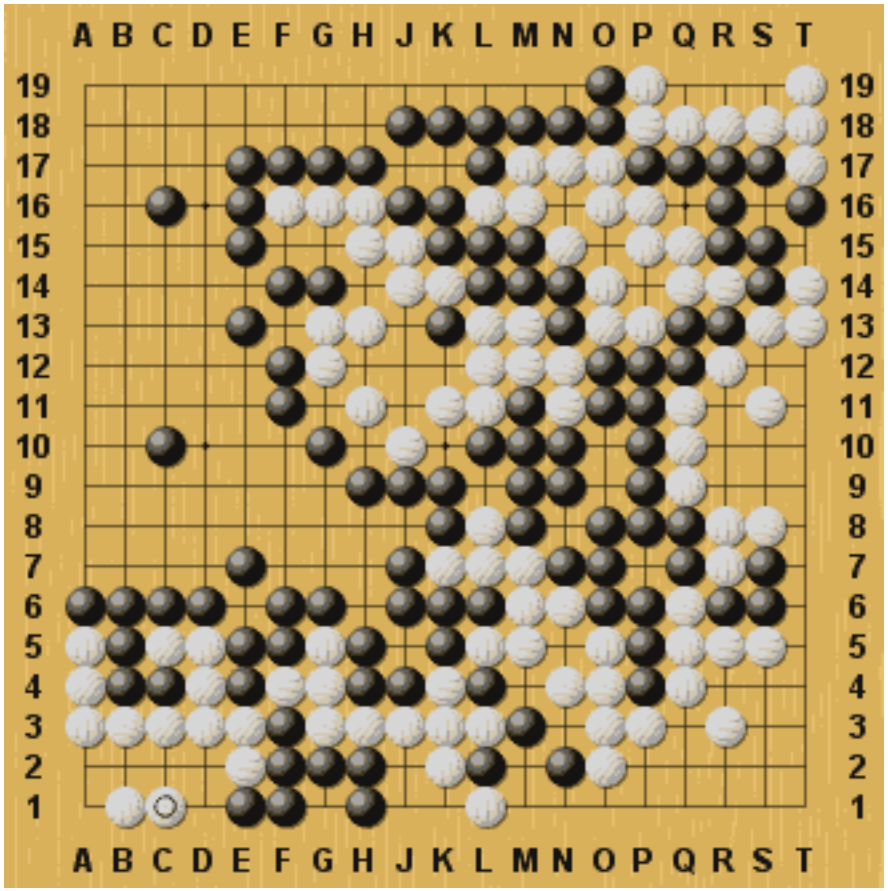
Go



https://www.youtube.com/watch?v=gECcsSeRcNo

Games using MCTS Variants

- Go playing programs:
  - AlphaGo
  - MoGo
  - CrazyStone
  - Valkyria
  - Pachi
  - Fuego
  - The Many Faces of Go
  - Zen
  - …



Go

- Board is 19x19
- Branching factor:
  - Starts at 361
  - Decreases by 1 (sort of) after every move
- Compare Go and Chess:
  - Chess, branching factor around 35:
    - Search at depth 6: 37,515,625 nodes
  - Go, branching factor around 300:
    - Search at depth 6: 729,000,000,000,000 nodes

AlphaGo

- Google's AlphaGo defeated Lee Sedol in 2016, and Ke Jie in May 2017
- How?
  - Integrated **MCTS** with **deep convolutional neural networks**
  - Data set of 30 million positions from the KGS Go Server
  - Trained a collection of neural networks to predict the probability of each move in the dataset and the expected value of a given board
  - Used neural networks to inform MCTS search

AlphaGo

4 Deep Neural Networks trained:

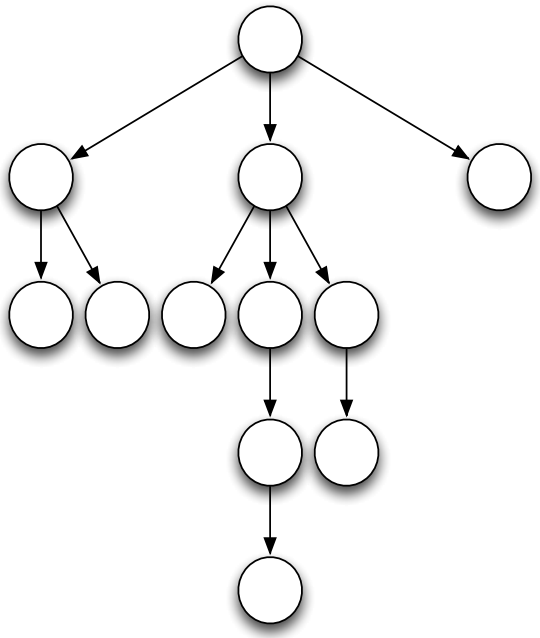
p_σ
- Trained via **Supervised Learning** from 30million positions from the KGS Go server.
- Predicts expert moves with 57% accuracy.

p_π
- Simplification of p_σ (runs faster)
- Predicts expert moves with 24.2% accuracy

p_ρ
- Starts with p_σ and improves it via self-play (reinforcement learning)
- 80% win ratio against p_σ
- 85% win ratio against Pachi (MCTS with 100K rollouts)

v_ρ
- Predicts the winner given a position
- Trained from 30million positions using p_ρ
- Almost as accurate as rollouts with 15K times less CPU.

Integrated into MCTS:



AlphaGo

4 Deep Neural Networks trained:

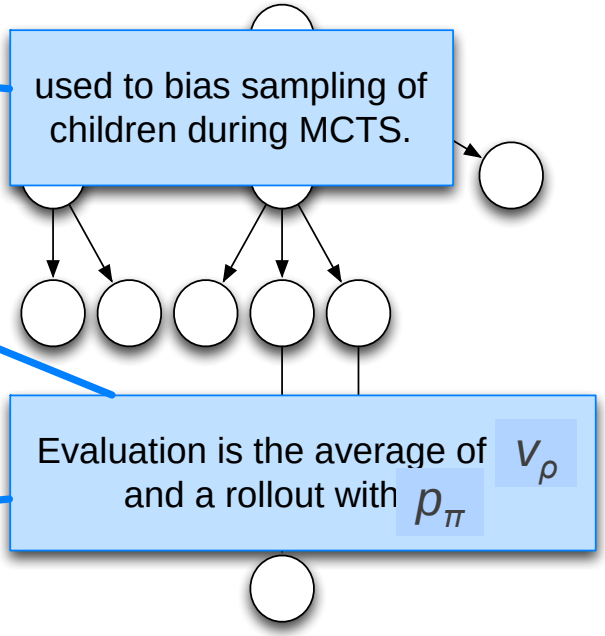
p_σ
- Trained via **Supervised Learning** from 30million positions from the KGS Go server.
- Predicts expert moves with 57% accuracy.

p_π
- Simplification of p_σ (runs faster)
- Predicts expert moves with 24.2% accuracy

p_ρ
- Starts with p_σ and improves it via self-play (reinforcement learning)
- 80% win ratio against p_σ
- 85% win ratio against Pachi (MCTS with 100K rollouts)

v_ρ
- Predicts the winner given a position
- Trained from 30million positions using p_ρ
- Almost as accurate as rollouts with 15K times less CPU.

Integrated into MCTS:



Go → Chess

- **AlphaZero**: AlphaGo-like system for chess
- **Stockfish**: standard best chess-playing engine
- "The program had four hours to play itself many, many times, thereby becoming its own teacher."
- 2017: AlphaZero won the 100-game match vs. Stockfish with 28 wins, 72 draws, and 0 losses (!)
- See:

  https://www.chess.com/news/view/google-s-alphazero-destroys-stockfish-in-100-game-match

Rock Papers Scissors

- "Cheating" robot: https://vimeo.com/42703118
- Winning every time: https://www.youtube.com/watch?v=g_1Ll4fhRso

Adversarial Search Summary

- For when there's more than one agent in the world
- Search on a **Game Tree**
  - Max layers: for our moves
  - Min layers: for our opponent's moves
  - "Average" layers: for chance elements (e.g. dice rolls)
- Algorithms:
  - Minimax
    - with Alpha-Beta pruning
  - Expecti-minimax
  - Monte Carlo Search
  - Monte Carlo Tree Search

CS 380: ARTIFICIAL INTELLIGENCE

Lecture 8: Logical Agents

Summary so far:

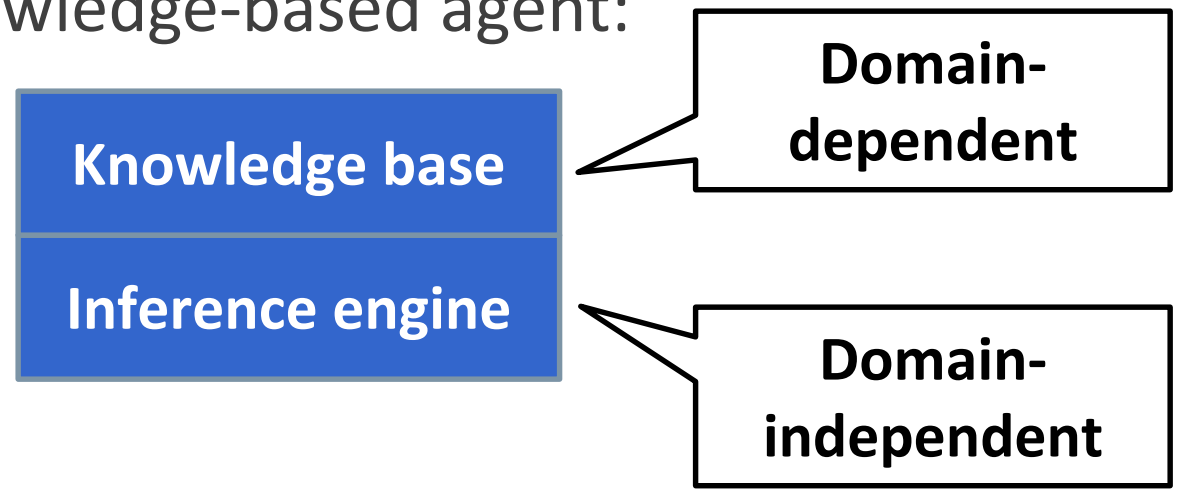
- What is AI?
  - Rational Agents
- Problem solving:
  - Systematic search
    - Uninformed search: DFS, BFS, ID
    - Informed search: heuristics, Greedy Search, A*
  - Local Search: Hill Climbing, Simulated Annealing
  - Adversarial Search: Minimax, alpha-beta, Monte Carlo Tree Search
- **Next: Knowledge Representation + Reasoning**

Knowledge

- Knowledge base:
  - When you think of “knowledge”, what do you think of?

Knowledge

- Knowledge base:
  - For our purposes: A set of **sentences** in a **formal language**
- Knowledge-based agent:



- "Declarative programming"

Declarative Programming

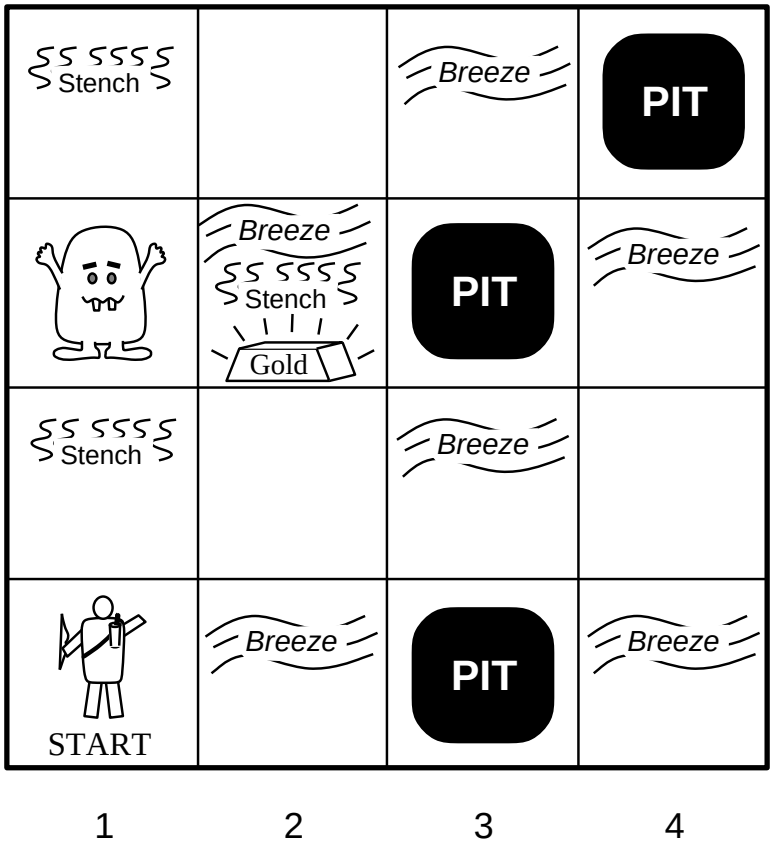
- Tell an agent **WHAT** to do, but not **HOW**.
- Encode all the necessary knowledge, and let the agent *infer* how to achieve the goals.
- For example, instead of programming a specific sorting algorithm, we could tell an agent:
  - Here's a set of numbers.
  - Now I want them sorted, which means that for every adjacent numbers *a* and *b*, they must satisfy *a* <= *b*
  - You (the agent) figure out how ☺
- In the limit, this is really hard

Wumpus World

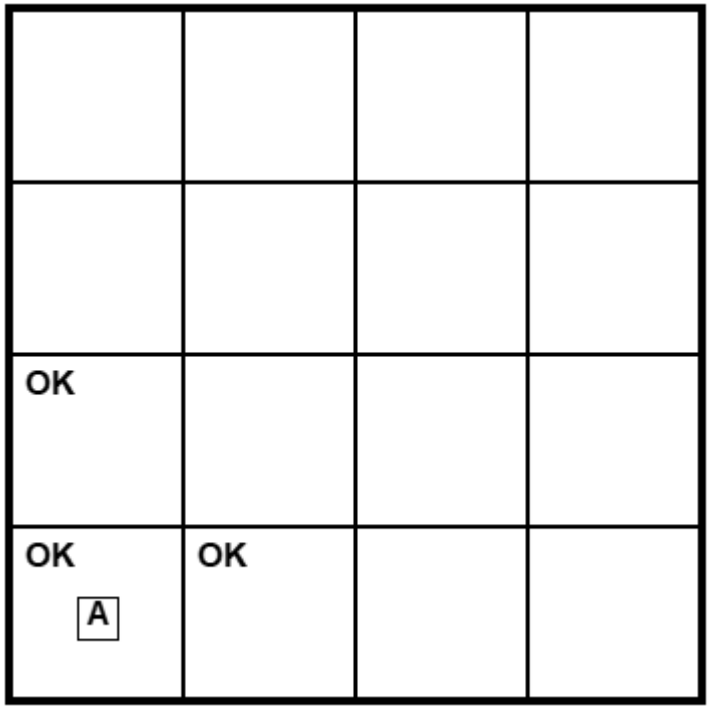
- There's a cave with a terrible "Wumpus" that eats anyone who enters its room ☹
- The Wumpus can be shot with an arrow if you're facing the right direction ☺
- … but you only have 1 arrow ☹
- Some rooms have bottomless pits that trap you ☹
- So why are you poking around Wumpus World? There's gold in one room!! ☺ ☺ ☺

Wumpus World PEAS Description

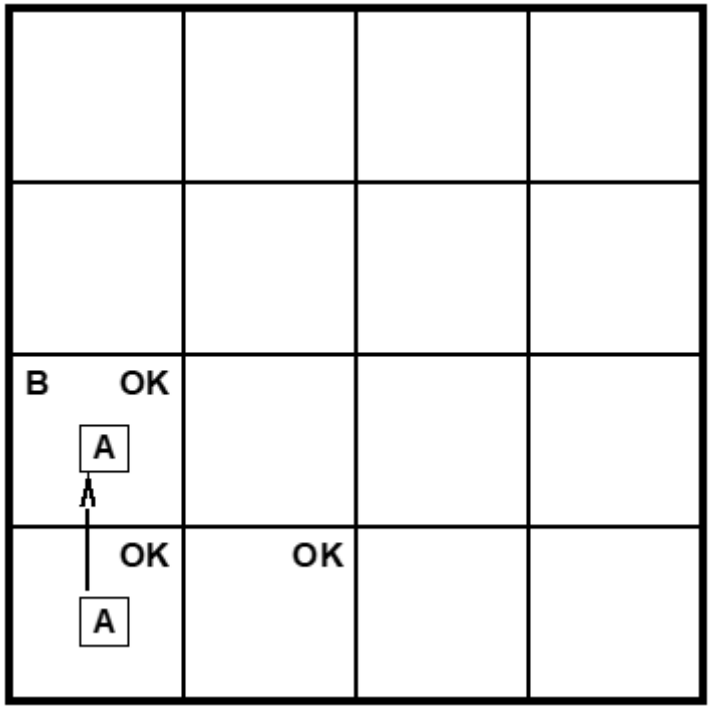
- Performance measure
  - Gold +1000, death -1000,
  - -1 per step, -10 for using arrow
- Environment
  - Squares adjacent to wumpus are smelly
  - Squares adjacent to pit are breezy
  - Glitter iff gold is in the same square
  - Shooting kills wumpus if you are facing it
  - Shooting uses up the only arrow
  - Grabbing picks up gold if in same square
  - Releasing drops gold in same square
- Actuators
  - Left turn, Right turn, Forward, Grab, Release, Shoot
- Sensors - Breeze, Glitter, Smell



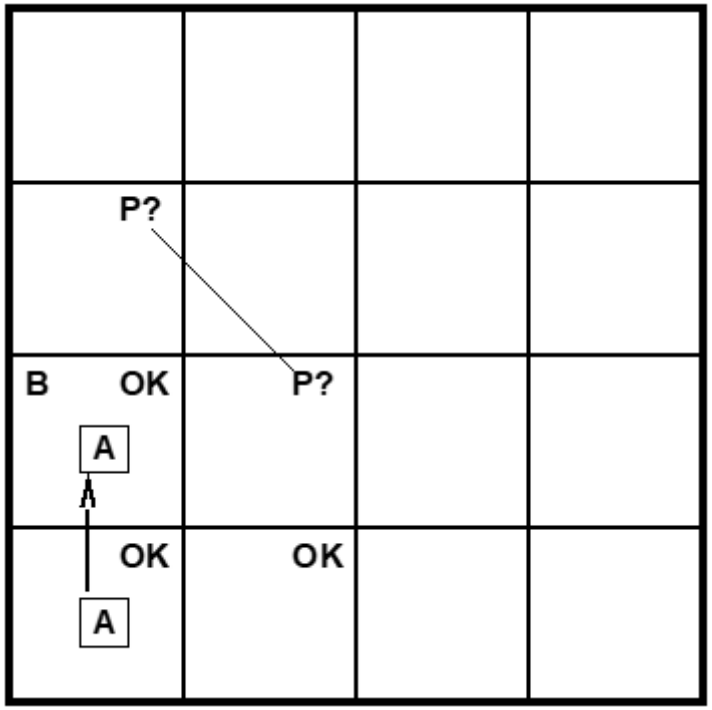
Exploring the Wumpus World



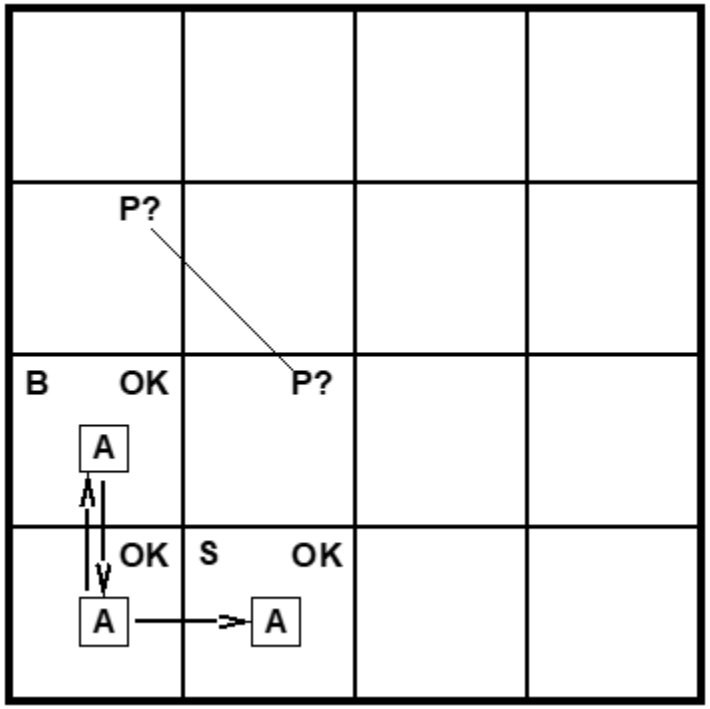
Exploring the Wumpus World



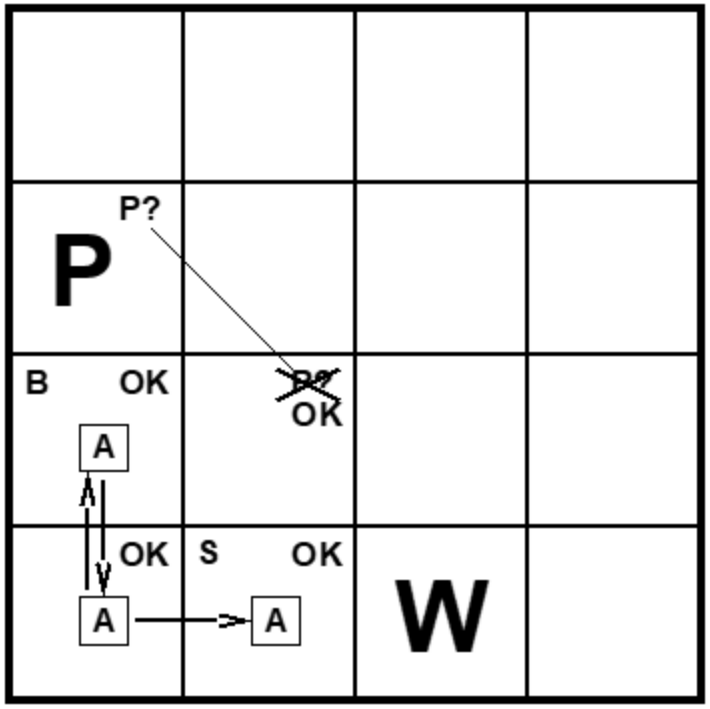
Exploring the Wumpus World



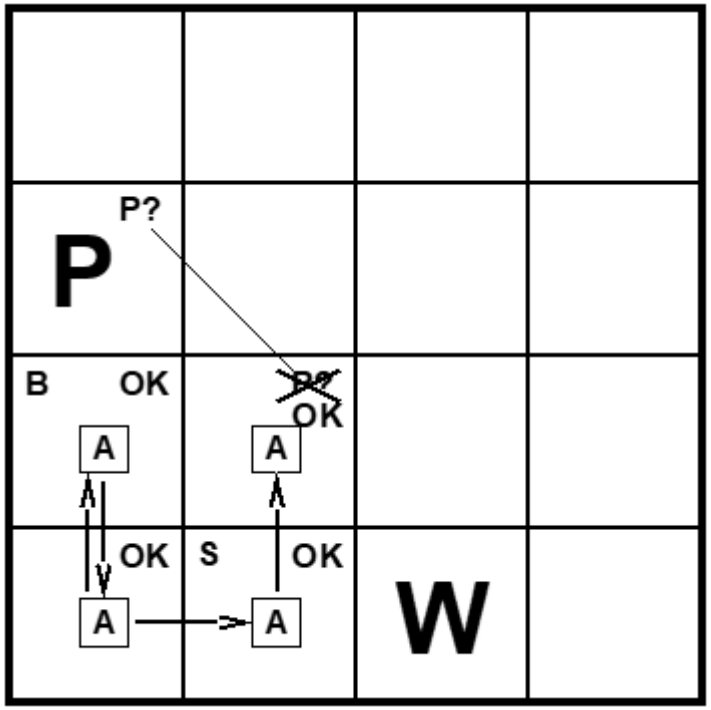
Exploring the Wumpus World



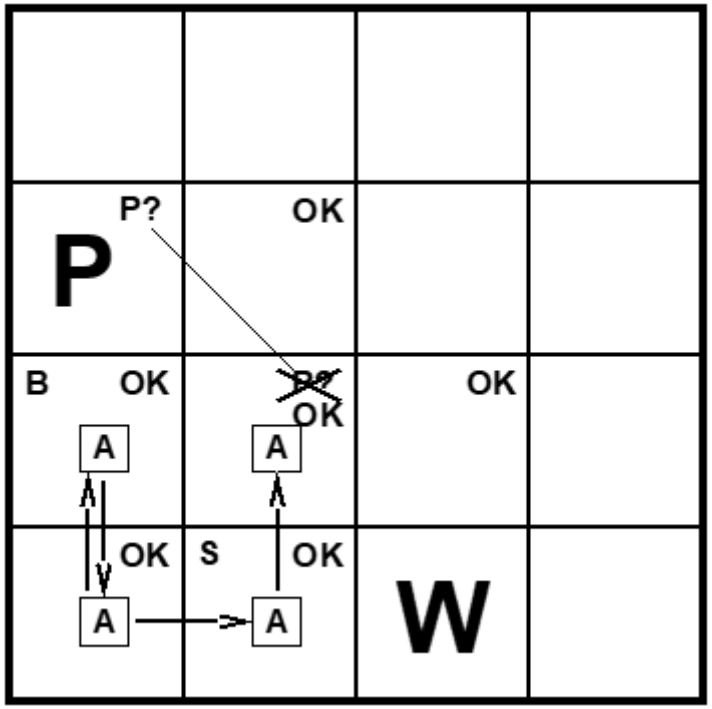
Exploring the Wumpus World



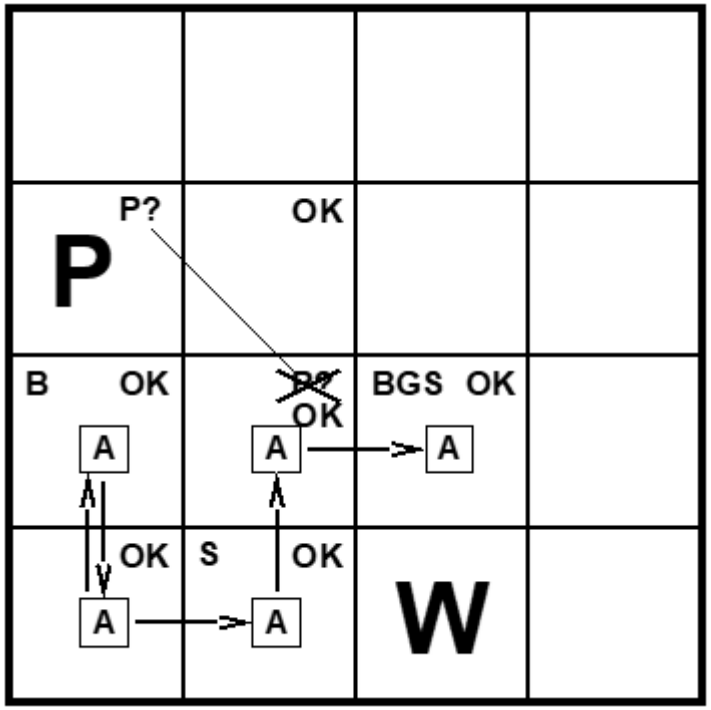
Exploring the Wumpus World



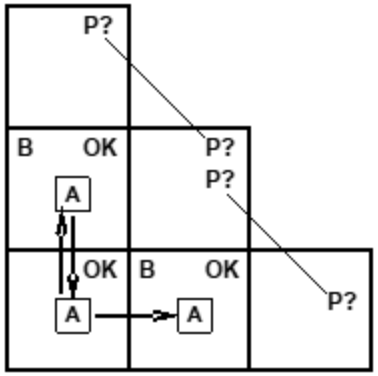
Exploring the Wumpus World



Exploring the Wumpus World

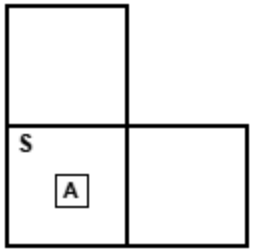


Other tight spots



Breeze in (1,2) and (2,1)
⇒ no safe actions

Assuming pits uniformly distributed,
(2,2) has pit w/ prob 0.86, vs. 0.31



Smell in (1,1)
⇒ cannot move
Can use a strategy of coercion:
shoot straight ahead
wumpus was there ⇒ dead ⇒ safe
wumpus wasn't there ⇒ safe

Logic in general

- Logics are formal languages for representing information such that conclusions can be drawn
- Syntax defines the sentences in the language
- Semantics defines the "meaning" of sentences, i.e., define truth of a sentence in a world
- E.g., the language of arithmetic
  - $x + 2 \geq y$ is a sentence; $x2 + y >$ is not a sentence
  - $x+ 2 \geq y$ is true iff the number $x+ 2$ is no less than the number y
  - $x+ 2 \geq y$ is true in a world where $x = 7, y = 1$
  - $x+ 2 \geq y$ is false in a world where $x = 0, y = 6$

Entailment

- Entailment means that one thing follows from another: $KB \vdash \alpha$
- Knowledge base KB entails sentence α

  If and only if

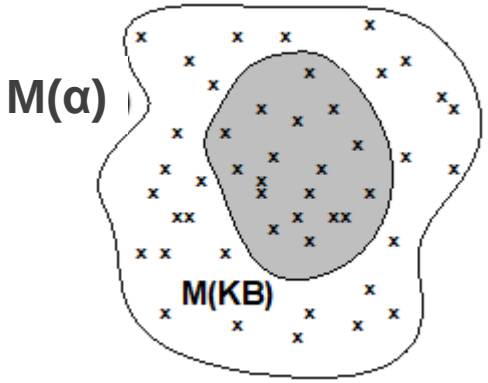
  α is true in all worlds where KB is true
- E.g., the KB containing "the Giants won" and "the Reds won" entails "Either the Giants won or the Reds won"
- E.g., $x + y = 4$ entails $4 = x + y$
- Entailment is a relationship between sentences (i.e., syntax) that is based on semantics.

Models

- Logicians typically think in terms of models, which are formally structured worlds with respect to which truth can be evaluated
- We say m is a model of a sentence α if α is true in m
- M(α) is the set of all models of α
- Then KB ⊨ α if and only if M(KB) ⊆ M(α)

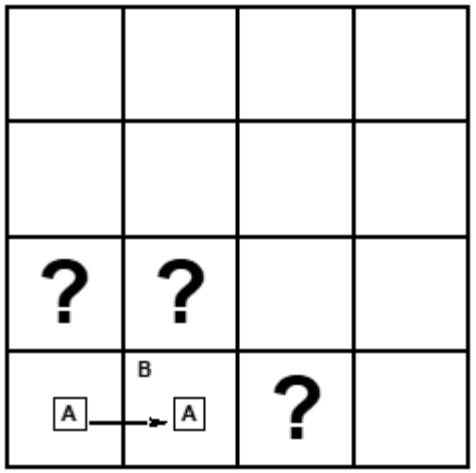
E.g. KB = Giants won and Reds won

α = Giants won

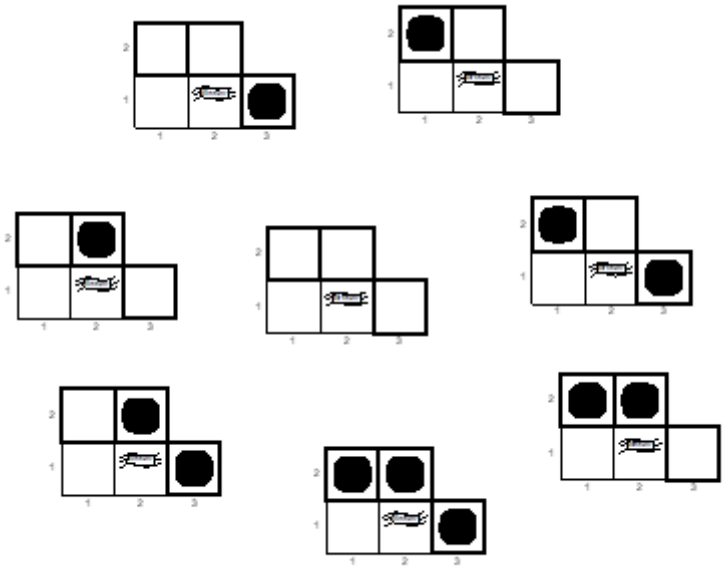


Entailment in the Wumpus World

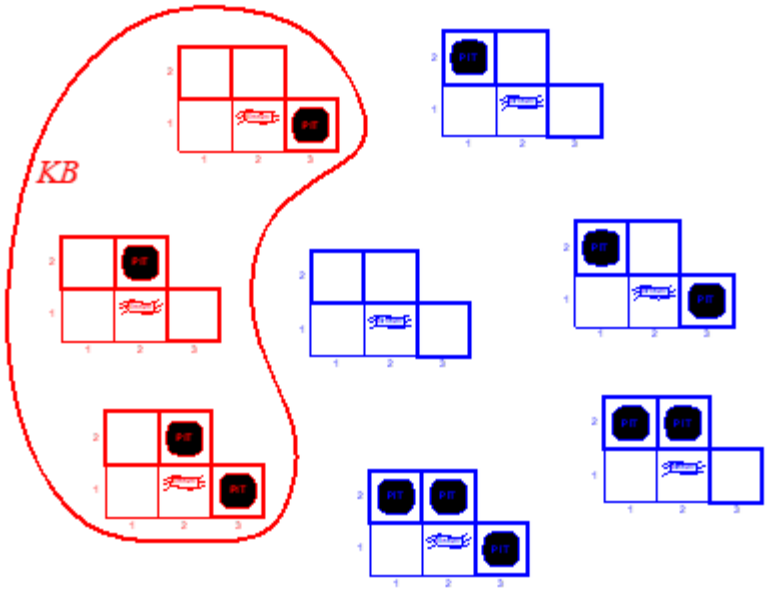
- Situation after detecting nothing in [1,1], moving right, breeze in [2,1]
- Consider possible models for ?s assuming only pits
- 3 Boolean choices => 8 possible models



Wumpus Models

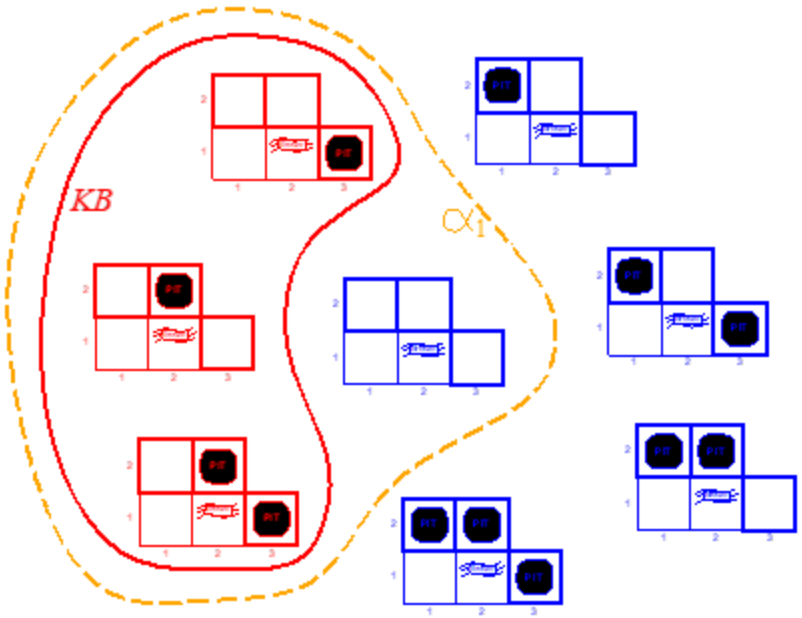


Wumpus Models



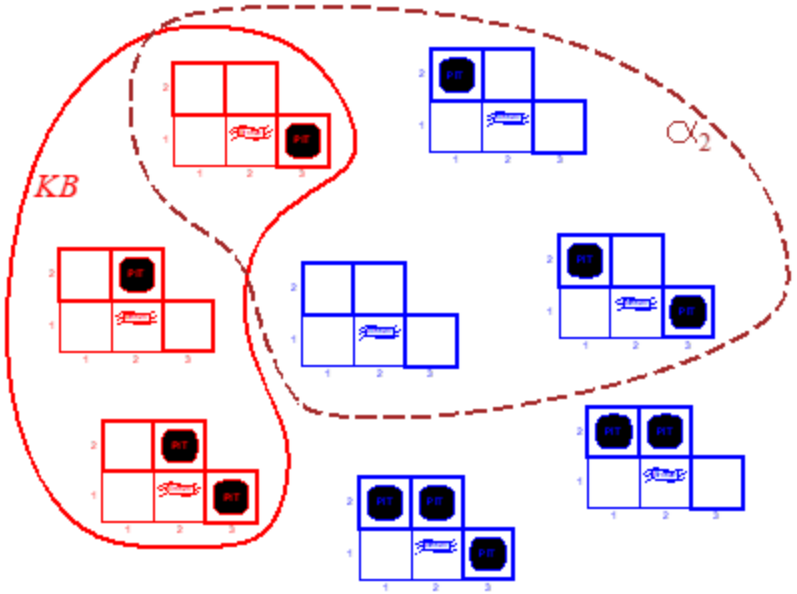
- *KB* = wumpus-world rules + observations

Wumpus Models



- KB = wumpus-world rules + observations
- α_1 = "[1,2] is safe", KB $\models \alpha_1$, proved by model checking

Wumpus Models



- KB = wumpus-world rules + observations
- α_2 = "[2,2] is safe", KB $\not\models \alpha_2$

Propositional Logic: Syntax

- Propositional logic is the simplest logic – illustrates basic ideas
- The proposition symbols P_1,P_2, etc are sentences
- If S is a sentence, ¬S is a sentence (negation)
- If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (conjunction)
- If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (disjunction)
- If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (implication)
- If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (biconditional)

Propositional Logic: Semantics

- Each model specifies true/false for each proposition symbol
- E.g., $P_{1,2}$ $P_{2,2}$ $P_{3,1}$
  false false true

(With these symbols, 8 possible models can be enumerated automatically)

- Rules for evaluating truth with respect to a model m:

$\neg S$	is true iff	S	is false			
$S_1 \wedge S_2$	is true iff	S_1	is true and	S_2	is true	
$S_1 \vee S_2$	is true iff	S_1	is true or	S_2	is true	
$S_1 \Rightarrow S_2$	is true iff	S_1	is false or	S_2	is true	
i.e.,	is false iff	S_1	is true and	S_2	is false	
$S_1 \Leftrightarrow S_2$	is true iff	$S_1 \Rightarrow S_2$	is true and	$S_2 \Rightarrow S_1$	is true	

Simple recursive process evaluates an arbitrary sentence, e.g.,
$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$ = true $\wedge$ (false $\vee$ true) = true $\wedge$ true = true

Truth Tables for Connectives

P	Q	¬P	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Wumpus World Sentences

- Let $P_{i,j}$ be true if there is a pit in $[i,j]$.
- Let $B_{i,j}$ be true if there is a breeze in $[i,j]$.

$\neg P_{1,1}$

$\neg B_{1,1}$

$B_{2,1}$

"Pits cause breezes in adjacent squares"

Wumpus World Sentences

- Let $P_{i,j}$ be true if there is a pit in [i,j].
- Let $B_{i,j}$ be true if there is a breeze in [i,j].

Rules

R_1 $\neg P_{1,1}$

R_2 $\neg B_{1,1}$

R_3 $B_{2,1}$

"Pits cause breezes in adjacent squares"

R_4 $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

R_5 $B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

"A square is breezy if and only if there is an adjacent pit"

These rules together form our KB

Truth Tables for Inference

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	true
false	true	false	false	false	true	false	true	true	true	true	true	true
false	true	false	false	false	true	true	true	true	true	true	true	true
false	true	false	false	true	false	false	true	false	false	true	true	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false	true	true	false	true	false

- Enumerate rows (different assignments to symbols), if KB is true in row, check that α is too
- KB = rules derived from observations, laws of Wumpus
- α = what we're interested in testing/checking

Truth Tables for Inference

	$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
	false	false	false	false	false	false	false	true	true	true	true	false	false
	false	false	false	false	false	false	true	true	true	false	true	false	false
	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
	false	true	false	false	false	false	false	true	true	false	true	true	false
KB	false	true	false	false	false	false	true	true	true	true	true	true	true
KB	false	true	false	false	false	true	false	true	true	true	true	true	true
KB	false	true	false	false	false	true	true	true	true	true	true	true	true
	false	true	false	false	true	false	false	true	false	false	true	true	false
	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
	true	true	true	true	true	true	true	false	true	true	false	true	false

- Enumerate rows (different assignments to symbols), if KB is true in row, check that α is too
- KB = rules derived from observations, laws of Wumpus
- α = what we're interested in testing/checking

Pros and Cons of Propositional Logic

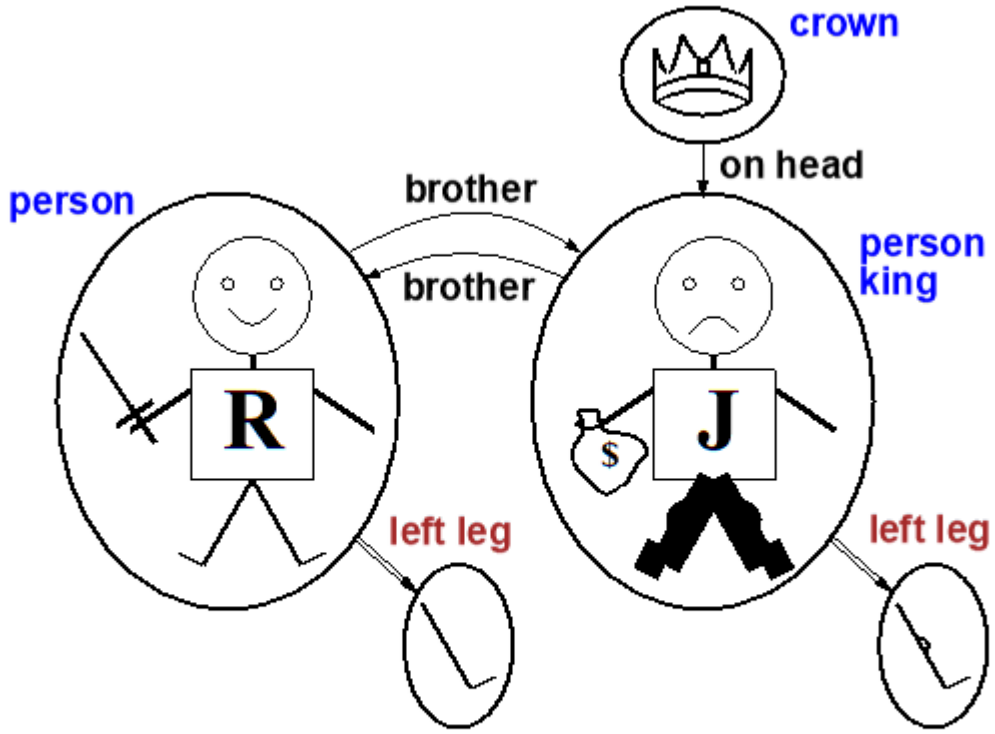
- ☺ Propositional logic is declarative: pieces of syntax correspond to facts
- ☺ Propositional logic allows partial/disjunctive/negated information (unlike most data structures and databases)
- ☺ Propositional logic is compositional: meaning $B_{1,1} \wedge P_{1,2}$ is derived from meaning of $B_{1,1}$ and of $P_{1,2}$
- ☺ Meaning in propositional logic is context-independent (unlike natural language, where meaning depends on context)
- ☹ Propositional logic has very limited expressive power (unlike natural language)

E.g., cannot say "pits cause breezes in adjacent squares" except by writing one sentence for each square

First-order Logic

- Whereas propositional logic assumes world contains facts, first-order logic (like natural language) assumes the world contains
  - Objects: people, houses, numbers, theories, Ronald McDonald, colors, baseball games, wars, centuries…
  - Relations: red, round, bogus, prime, multistoried… , brother of, bigger than, inside, part of, has color, occurred after, owns, comes between…
  - Functions: father of, best friend, third inning of, one more than, end of…

Models for First-order Logic: Example



Syntax of FOL: Basic Elements

- Constants — KingJohn, 2, UCB, …
- Predicates — Brother, >, …
- Functions — Sqrt, LeftLegOf, …
- Variables — *x, y, a, b, …*
- Connectives — ∧ ∨ ¬ ⇒ ⇔
- Equality — =
- Quantifiers — ∀ ∃

Atomic Sentences

- Atomic sentence = *predicate*($term_1$, …, $term_m$)

  or $term_1 = term_2$

- Term = *function*($term_1$, …, $term_m$)

  or *constant* or *variable*

E.g., *Brother(KingJohn, RichartTheLionheart)*

> (Length(LeftLegOf(Richard)), Length(LeftLegOf(KingJohn)))

Complex Sentences

- Complex sentences are made from atomic sentences using connectives

$\neg S$, $S_1 \wedge S_2$, $S_1 \vee S_2$, $S_1 \Rightarrow S_2$, $S_1 \Leftrightarrow S_2$

E.g.

Sibling(KingJohn, Richard) ⇒ Sibling(Richard, KingJohn)

>(1,2) V ≤ (1,2)

>(1,2) Λ ¬> (1,2)

Models for FOL: Lots!

- Entailment in propositional logic can be computed by enumerating models
- We can enumerate the FOL models for a given KB vocabulary:

For each number of domain elements n from 1 to ∞

For each k-ary predicate P_k in the vocabulary

For each possible k-ary relation on n objects

For each constant symbol C in the vocabulary

For each choice of referent for C from n objects…

Computing entailment by enumerating FOL models is not easy!

Universal Quantification

- ∀ *<variables> <sentence>*
- Everyone at Berkeley is smart:
  - ∀*x* *At(x, Berkeley) ⇒ Smart(x)*
- ∀*x* *P* is true in a model *m* iff *P* is true with *x* being each possible object in the model
- Roughly speaking, equivalent to the conjunction of instantiations of P

(At(KingJohn, Berkeley) ⇒ Smart(KingJohn))

∧ (At(Richard, Berkeley) ⇒ Smart(Richard))

∧ (At(Berkeley, Berkeley) ⇒ Smart(Berkeley)) …

A common mistake to avoid

- Typically, $\Rightarrow$ is the main connective with $\forall$
- Common mistake, using $\wedge$ as the main connective with $\forall$:

$$\forall x \;\; At(x, Berkeley) \wedge Smart(x)$$

Means "Everyone is at Berkeley and everyone is smart"

Existential Quantification

- ∃ *<variable> <sentence>*
- Someone at Stanford is smart:
  - *∃x At(x, Stanford) ∧ Smart(x)*
- *∃x P* is true in a model *m* iff *P* is true with *x* being some possible object in the model
- Roughly speaking, equivalent to the disjunction of instantiations of *P*

(At(KingJohn, Stanford) ∧ Smart(KingJohn))

∨ (At(Richard, Stanford) ∧ Smart(Richard))

∨ …

Another common mistake to avoid

- Typically, $\wedge$ is the main connective with $\exists$
- Common mistake: using $\Rightarrow$ as the main connective with $\exists$:

$$\exists x \; At(x, Stanford) \Rightarrow Smart(x)$$

Is true if there is anyone who is not at Stanford!

Properties of quantifiers

- $\forall x\ \forall y$ is the same as $\forall y\ \forall x$
- $\exists x\ \exists y$ is the same as $\exists y\ \exists x$
- $\exists x\ \forall y$ is not the same as $\forall y\ \exists x$
- $\exists x\ \forall y\ \ Loves(x,y)$
  - "There is a person who loves everyone in the world"
- $\forall y\ \exists x\ \ Loves(x,y)$
  - "Everyone in the world is loved by at least one person"
- Quantifier duality: each can be expressed using the other
  - $\forall x\ \ Likes(x, IceCream)\ \ \neg\exists x\ \neg Likes(x, IceCream)$
  - $\exists x\ Likes(x, Broccoli)\ \ \ \ \neg\forall x\ \neg Likes(x, Broccoli)$

Fun with Sentences

- Brothers are siblings

Fun with Sentences

Brothers are siblings

- $\forall x,y\ Brother(x,y) \Rightarrow Sibling(x,y).$

"Sibling" is symmetric

Fun with Sentences

Brothers are siblings

- $\forall x,y\ Brother(x,y) \Rightarrow Sibling(x,y).$

"Sibling" is symmetric

- $\forall x,y\ Sibling(x,y) \Leftrightarrow Sibling(y,x).$

One's mother is one's female parent

Fun with Sentences

Brothers are siblings

- $\forall x,y\ Brother(x,y) \Rightarrow Sibling(x,y).$

"Sibling" is symmetric

- $\forall x,y\ Sibling(x,y) \Leftrightarrow Sibling(y,x).$

One's mother is one's female parent

- $\forall x,y\ Mother(x,y) \Leftrightarrow (Female(x) \wedge (Parent(x,y)).$

A first cousin is a child of a parent's sibling

Fun with Sentences

Brothers are siblings

- $\forall x,y\ Brother(x,y) \Rightarrow Sibling(x,y).$

"Sibling" is symmetric

- $\forall x,y\ Sibling(x,y) \Leftrightarrow Sibling(y,x).$

One's mother is one's female parent

- $\forall x,y\ Mother(x,y) \Leftrightarrow (Female(x) \wedge (Parent(x,y)).$

A first cousin is a child of a parent's sibling

- $\forall x,y\ FirstCousin(x,y) \Leftrightarrow \exists px,py\ \ Parent(px,x) \wedge Sibling(px,py) \wedge Parent(py,y)$

Interacting with FOL KBs

Suppose a wumpus-world is using a FOL KB and perceives a smell and a breeze (but no glitter) at *t = 5.*

Tell(KB, Percept([Smell, Breeze, None],5))

Ask(KB, ∃a Action(a,5))

I.e., does *KB* entail any particular actions at *t = 5*?

Answer: *Yes, {a/Shoot}* ← substitution (binding list)

Given a sentence *S* and a substitution *σ*,

Sσ denotes that result of plugging *σ* into *S*; e.g.,

S = Smarter(x,y)

σ = {x/Hillary, y/Bill}

Sσ = Smarter(Hillary, Bill)

Ask (KB, S) returns some/all σ such that KB⊨Sσ

Deducing Hidden Properties

Properties at locations:

$\forall x,t \;\; At(Agent,x,t) \wedge Smelt(t) \Rightarrow Smelly(x)$

$\forall x,t \;\; At(Agent,x,t) \wedge Breeze(t) \Rightarrow Breezy(x)$

Squares are breezy near a pit:

Diagnostic rule – infer cause from effect

$\forall y \; Breezy(y) \Rightarrow \exists x \; Pit(x) \wedge Adjacent(x,y)$

Causal rule – infer effect from cause

$\forall x,y \; Pit(x) \wedge Adjacent(x,y) \Rightarrow Breezy(y)$

Neither of these is complete – e.g., the causal rule doesn't say whether squares far away from pits can be breezy

Definition for the Breezy predicate:

$$\forall y \; Breezy(y) \;\; \Leftrightarrow [\exists x \; Pit(x) \wedge Adjacent(x,y)]$$

Describing Actions

- "Effect" axiom – describes changes due to action

$\forall s\ AtGold(s) \Rightarrow Holding(Gold, Result(Grab,s))$

- "Frame" axiom – describes non-changes due to action

$\forall s\ HaveArrow(s) \Rightarrow HaveArrow(Result(Grab,s))$

- Frame problem: find an elegant way to handle non-change
  - (a) representation – avoid frame axioms
  - (b) inference – avoid repeated "copy-overs" to keep track of state
- Qualification problem: true descriptions of real actions required endless caveats – what if gold is slippery or nailed down or …
- Ramification problem: real actions have many secondary consequences – what about the dust on the gold, wear and tear on gloves, …

Making Plans

- Initial condition in KB:

  $At(Agent, [1,1], S_0)$

  $At(Gold, [1,2], S_0)$

- Query: $Ask(KB, \exists s\ Holding(Gold,s))$
  - i.e., in what situation will I be holding the gold?
- Answer: $\{s/Result(Grab, Result(Forward, S_0))\}$
  - i.e., go forward and then grab the gold
- This assumes that the agent is interested in plans starting at S_0 and that S_0 is the only situation described in KB

Making Plans: A better way

- Represent plans as action sequences $[a_1, a_2, \ldots, a_n]$

PlanResult(p,s) is the result of executing *p* in *s*

- Then the query $Ask(KB, \exists p \;\; Holding(Gold, PlanResult(p,S_0)))$ has the solution $\{p/[Forward,Grab]\}$
- Definition of *PlanResult* in terms of *Result*:

$\forall s \;\; PlanResult([],s) = s$

$\forall a,p,s \;\; PlanResult([a|p],s) = PlanResult(p,Result(a,s))$

- Planning systems are special-purpose reasoners designed to do this type of inference more efficiently than a general-purpose reasoner

Summary

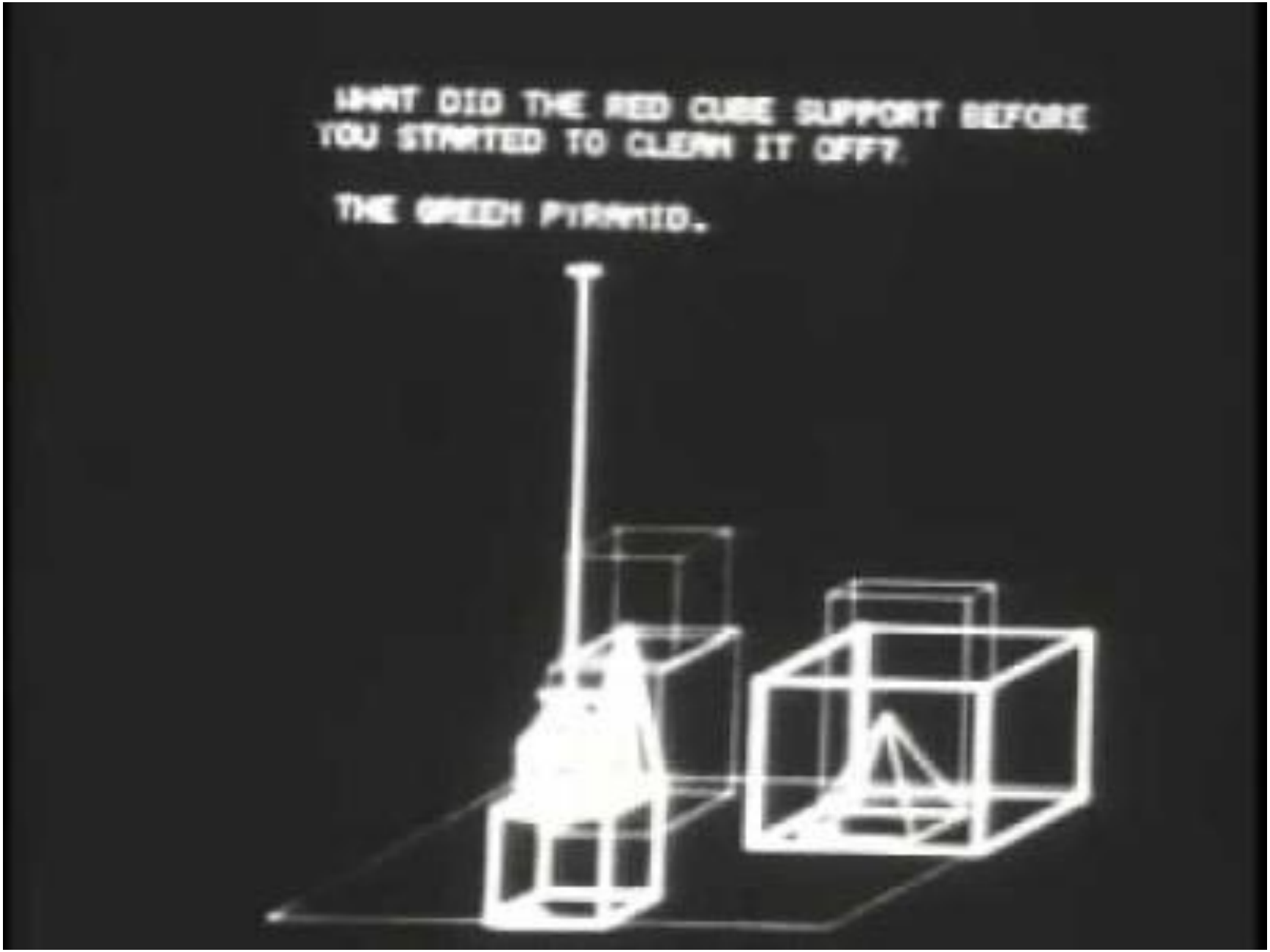
- Predicate logic
  - Basic representation for factual knowledge
- First-order logic
  - Objects and relations are semantic primitives
  - Syntax: constants, functions, predicates, equality, quantifiers
  - Situation calculus:
    - Conventions for describing actions and changes in FOL
    - Can formulate planning as inference on a situation calculus KB
  - Increased expressive power over propositional logic
    - … or at least, sufficient to define Wumpus world

CS 380: ARTIFICIAL INTELLIGENCE

Lecture 9: Natural Language Processing

Natural Language Processing

- Let's start with some classic examples:
  - Eliza (Weizenbaum, 1966)
    - Sample implementation: http://www.masswerk.at/elizabot/
  - SHRDLU (Winograd, 1970)
    - http://hci.stanford.edu/~winograd/shrdlu/
    - https://www.youtube.com/watch?v=QAJz4YKUwqw
    - How did SHRDLU get its name? http://hci.stanford.edu/~winograd/shrdlu/name.html


WHAT DID THE RED CUBE SUPPORT BEFORE YOU STARTED TO CLEAN IT OFF?
THE GREEN PYRAMID.

Natural Language Processing

- SHRDLU is quite impressive for late 1960s!
- So, is NLP solved nowadays? (after ~50 years?)
  - Not at all!
- But SHRDLU worked pretty well — can't we just apply the algorithms in SHRDLU to other domains?
  - SHRDLU worked on a "micro-world", which masks most of the hard problems in NLP (ambiguity, metaphor, non-compositionality, etc.)
  - Efforts to scale SHRDLU to larger domains have consistently failed (e.g., CyC)

Natural Language Processing

- There are many possible perspectives for an analysis of language…
  - Linguistics: e.g., general & specific aspects of language
  - Psychology: e.g., developmental learning
  - Computer Science: e.g., grammar & parsing
  - Sociology: e.g., language across cultures
  - Etc.
- Here we focus on NLP from an **agent** perspective: an agent/human communicating with an agent/human

Communication

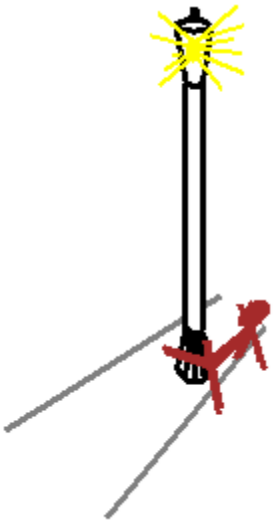
- "Classical" view (pre-1953)
  - Language consists of sentences that are true/false (logic)
- "Modern" view (post-1953)
  - Language is a form of action

Wittgenstein(1953) Philosophical Investigations

Austin (1962) How to Do Things with Words

Searle (1969) Speech Acts

- Why?
  - To change the actions of other agents



Speech Acts



- Speech acts achieve the speaker´s goals:
  - Inform "There's a pit in front of you"
  - Query "Can you see the gold?
  - Command "Pick it up"
  - Promise "I'll share the gold with you"
  - Acknowledge "OK"
- Speech act planning requires knowledge of
  - Situation
  - Semantic and syntactic conventions
  - Hearer's goals, knowledge base, and rationality

Stages of Communication (informing)

- Intention — S wants to inform H that P
- Generation — S selects words W to express P in context C
- Synthesis — S utters words W

- Perception — H perceives W' in context C'
- Analysis — H infers possible meanings $P_1, \ldots P_n$
- Disambiguation — H infers intended meaning P_i
- Incorporation — H incorporates P_i into KB

How could this go wrong?

Stages of Communication (informing)

- Intention — S wants to inform H that P
- Generation — S selects words W to express P in context C
- Synthesis — S utters words W

- Perception — H perceives W' in context C'
- Analysis — H infers possible meanings $P_1, \ldots P_n$
- Disambiguation — H infers intended meaning P_i
- Incorporation — H incorporates P_i into KB

How could this go wrong?

- Insincerity (S doesn't believe P)
- Speech wreck ignition failure (speech recognition failure ☺)
- Ambiguous utterance
- Differing understanding of current context (C != C')

Stages of Communication (informing)

- Intention — S wants to inform H that P
- Generation — S selects words W to express P in context C
- Synthesis — S utters words W

- Perception — H perceives W' in context C'
- Analysis — H infers possible meanings $P_1, \ldots P_n$
- Disambiguation — H infers intended meaning P_i
- Incorporation — H incorporates P_i into KB

Ambiguity examples from NYT Crossword:

They hang around in kitchens = ?

Bare-bones outfit = ?

Device for on-line conversation? = ?

Grammar

- Vervet monkeys, antelopes, etc. use isolated symbols for sentences
  - Restricted set of communicable propositions, no generative capacity

Grammar

- Vervet monkeys, antelopes, etc. use isolated sym[illegible]sentences



Vervet monkeys are known to elicit **predator-specific alarm calls**. Three well-documented vervet monkey alarm calls are those for leopard, martial eagle and python. **Leopard alarm calls are short tonal calls** produced in a series of inhalations and exhalations. **Eagle alarm calls are low pitched grunts** while python alarm calls are high pitched "chutters". Different alarm calls seem to evoke different responses to individuals that heard the alarm calls. However, **the first reaction of a vervet monkey upon hearing an alarm call is to look at the direction of the caller**. Looking at the direction of the caller gives them clues as to why the alarm calls were made and also where the caller is facing reveals the direction of the approaching predator. *[from primatology.net]*

Grammar

- Vervet monkeys, antelopes, etc. use isolated symbols for sentences
  - Restricted set of communicable propositions, no generative capacity

(Chomsky (1957): Syntactic Structures)

- Grammar specifies the compositional structure of complex messages. E.g., speech (linear), text (linear), music (two-dimensional)
- A formal language is a set of strings of terminal symbols
- Each string in the language can be analyzed/generated by the grammar
- The grammar is a set of rewrite rules, e.g.,

  S → NP VP

  Article → the | a | an | …

  Backus-Naur Form (BNF)

  Here S is the sentence symbol, NP and VP are nonterminals

Grammar Types

- Regular: nonterminal → terminal[nonterminal]

  $S \rightarrow aS$

  $S \rightarrow \Lambda$ (Λ = uppercase lambda = empty string)
- Context-free: nonterminal → anything

  $S \rightarrow aSb$
- Context-sensitive: more nonterminals

  $ASB \rightarrow AAaBB$
- Recursively enumerable: no constraints
- Natural languages probably context-free, parsable in real time!

Wumpus Lexicon

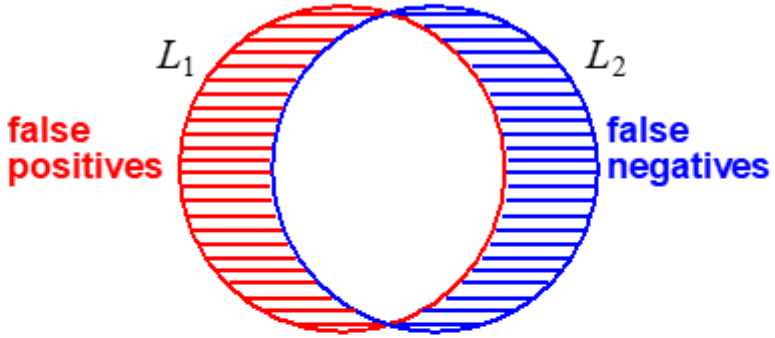
- Noun → stench | breeze | glitter | nothing | Wumpus | pit | pits | gold | east | …
- Verb → is | see | smell | shoot | feel | stinks | go | grab | carry | kill | turn | …
- Adjective → right | left | east | south | back | smelly | …
- Adverb → here | there | nearby | ahead | right | left | east | south | back | …
- Pronoun → me | you | I | it | …
- Name → John | Mary | Boston | UCB | PAJC | …
- Article → the | a | an | …
- Preposition → to | in | on | near | …
- Conjunction → and | or | but | …
- Digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Wumpus Grammar

S	→	NP VP	I + feel a breeze
	|	S Conjunction S	I feel a breeze + and + I smell a wumpus
NP	→	Pronoun	I
	|	Noun	pits
	|	Article Noun	the + wumpus
	|	Digit Digit	3 4
	|	NP PP	the wumpus + to the east
	|	NP RelClause	the wumpus + that is smelly
VP	→	Verb	stinks
	|	VP NP	feel + a breeze
	|	VP Adjective	is + smelly
	|	VP PP	turn + to the east
	|	VP Adverb	go + ahead
PP	→	Preposition NP	to + the east
RelClause	→	that VP	that + is smelly

Grammaticality Judgements

Formal language L_1 may differ from natural language L_2



Adjusting L_1 to agree with L_2 is a learning problem!

* the gold grab the wumpus
* I smell the wumpus the gold
  I give the wumpus the gold
* I donate the wumpus the gold

Intersubjective agreement somewhat reliable, independent of semantics!
Real grammars 10–500 pages, insufficient even for "proper" English

Parse Trees

Exhibit the grammatical structure of a sentence

I shoot the wumpus

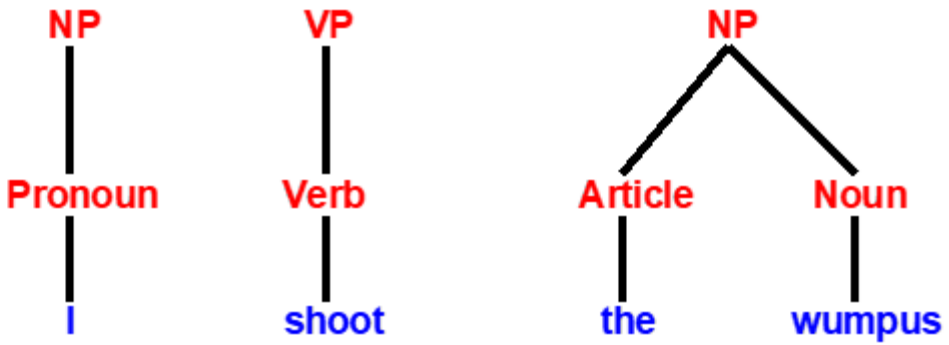
Parse Trees

Exhibit the grammatical structure of a sentence



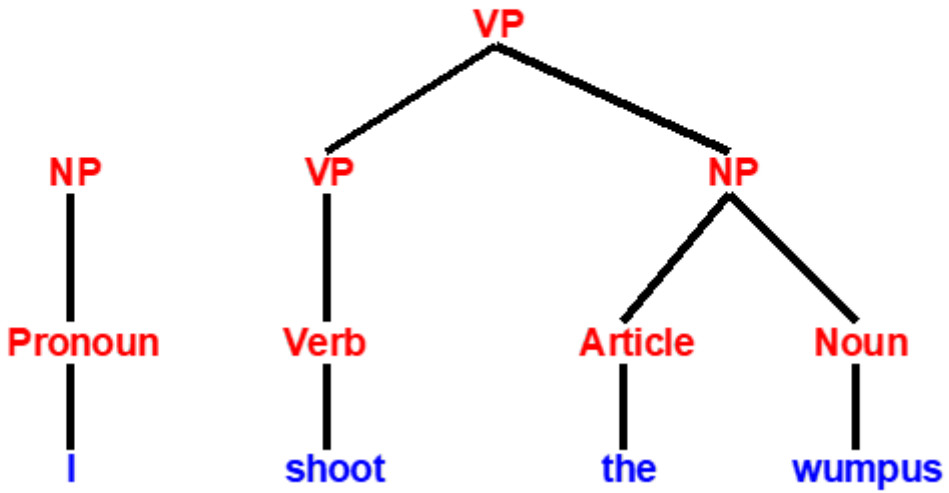
Parse Trees

Exhibit the grammatical structure of a sentence



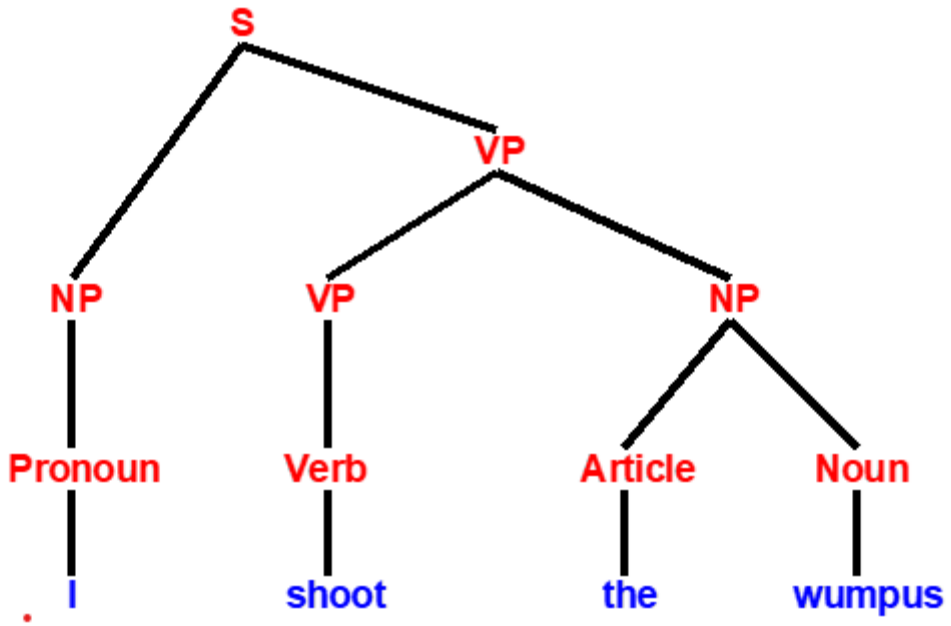
Parse Trees

Exhibit the grammatical structure of a sentence



Parse Trees

Exhibit the grammatical structure of a sentence



Syntax in NLP

- Most view syntactic structure as an essential step towards meaning;
  - "Mary hit John" ≠ "John hit Mary"
  - "And since I was not informed – as a matter of fact, since I did not know that there were excess funds until we, ourselves, in that checkup after the whole thing blew up, and that was, if you'll remember, that was the incident in which the attorney general came to me and told me that he had seen a memo that indicated there were no more funds"

Syntax in NLP

- Most view syntactic structure as an essential step towards meaning;
  - "Mary hit John" ≠ "John hit Mary"
  - "And since I was not informed – as a matter of fact, since I did not know that there were excess funds until we, ourselves, in that checkup after the whole thing blew up, and that was, if you'll remember, that was the incident in which the attorney general came to me and told me that he had seen a memo that indicated there were no more funds"
  - "Wouldn't the sentence 'I want to put a hyphen between the words Fish and And and And and Chips in my Fish-And-Chips sign' have been clearer if quotation marks had been placed before Fish, and between Fish and and, and and and And, and And and and, and and and And, and And and and, and and and Chips, as well as after Chips?

Real Language

- Real human languages provide many problems for NLP:
  - Ambiguity
  - Anaphora
  - Indexicality
  - Vagueness
  - Discourse structure
  - Metonymy
  - Metaphor
  - Noncompositionality

Ambiguity

- Squad helps dog bite victim

Ambiguity

- Squad helps dog bite victim
- Helicopter powered by human flies

Ambiguity

- Squad helps dog bite victim
- Helicopter powered by human flies
- I ate spaghetti with meatballs

Ambiguity

- Squad helps dog bite victim
- Helicopter powered by human flies
- I ate spaghetti with meatballs

  salad

Ambiguity

- Squad helps dog bite victim
- Helicopter powered by human flies
- I ate spaghetti with meatballs

  salad

  abandon

Ambiguity

- Squad helps dog bite victim
- Helicopter powered by human flies
- I ate spaghetti with meatballs

  salad

  abandon

  a fork

Ambiguity

- Squad helps dog bite victim
- Helicopter powered by human flies
- I ate spaghetti with meatballs

  salad

  abandon

  a fork

  a friend

Ambiguity

- Squad helps dog bite victim
- Helicopter powered by human flies
- I ate spaghetti with meatballs

  salad

  abandon

  a fork

  a friend

Ambiguity can be lexical (polysemy), syntactic, semantic, referential

Anaphora

- Using pronouns to refer back to entities already introduced in the text
- After Mary proposed to John, they found a preacher and got married.

Anaphora

- Using pronouns to refer back to entities already introduced in the text
- After Mary proposed to John, they found a preacher and got married.
- For the honeymoon, they went to Hawaii.

Anaphora

- Using pronouns to refer back to entities already introduced in the text
- After Mary proposed to John, they found a preacher and got married.
- For the honeymoon, they went to Hawaii.
- Mary saw a ring through the window and asked John for it.

Anaphora

- Using pronouns to refer back to entities already introduced in the text
- After Mary proposed to John, they found a preacher and got married.
- For the honeymoon, they went to Hawaii.
- Mary saw a ring through the window and asked John for it.
- Mary threw a rock at the window and broke it.

Anaphora

- Using pronouns to refer back to entities already introduced in the text
- After Mary proposed to John, they found a preacher and got married.
- For the honeymoon, they went to Hawaii.
- Mary saw a ring through the window and asked John for it.
- Mary threw a rock at the window and broke it.
- *Compare*: Mary threw a vase at the wall and broke it.

Indexicality

- Indexical sentences refer to utterance situation (place, time, etc)
- I am over here.
- Why did you do that?

Metonymy

- Using one noun phrase to stand for another
- I've read Shakespeare
- Chrysler announced record profits
- The ham sandwich on Table 4 wants another beer.

Metaphor

- “Non-literal” usage of words and phrases, often systematic:
- I’ve tried killing the process, but it won’t die. Its parent keeps it alive.

Noncompositionality

- Meaning of a phrase = combined meaning of its terms?

Noncompositionality

- Basketball shoes

Noncompositionality

- Basketball shoes
- Baby shoes

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

- Red book

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

- Red book
- Red pen

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

- Red book
- Red pen
- Red hair

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

- Red book
- Red pen
- Red hair
- Red herring

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

- Red book
- Red pen
- Red hair
- Red herring

- Small moon

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

- Red book
- Red pen
- Red hair
- Red herring

- Small moon
- Large molecule

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

- Red book
- Red pen
- Red hair
- Red herring

- Small moon
- Large molecule
- Mere child

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

- Red book
- Red pen
- Red hair
- Red herring

- Small moon
- Large molecule
- Mere child
- Alleged murderer

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

- Red book
- Red pen
- Red hair
- Red herring

- Small moon
- Large molecule
- Mere child
- Alleged murderer
- Real leather

Noncompositionality

- Basketball shoes
- Baby shoes
- Alligator shoes
- Designer shoes
- Brake shoes

- Red book
- Red pen
- Red hair
- Red herring

- Small moon
- Large molecule
- Mere child
- Alleged murderer
- Real leather
- Artificial grass

NLP in the Real World



Jeremy Fowler 
@JFowlerESPN

Follow 

I'm not expecting the Steelers to make a move for cornerback help at the deadline barring unforeseen development. Will likely roll with what they have.

12:19 PM - 30 Oct 2018



Steeler 4 Life @jarvis1104 · Oct 30

Replying to @JFowlerESPN @Alex_Kozora

Sadly this tweet can be sent every year at this time and remain true

7 80



John @jrw412 · Oct 30

Replying to @JFowlerESPN

Huge surprise

1 12



jONh @jonhthagreat · Oct 30

not really, as much as i love the steelers they are probably the most boring team when it comes to making deadline deals

1 1 6



my name jason @jbmorton21 · Oct 30

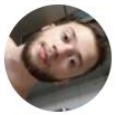
You're not that great at identifying sarcasm

1 9



jONh @jonhthagreat · Oct 30

lol i didnt even think of sarcasm for some reason 😂



Daniel Cabral Nephas @medanielrose · Oct 30

Replying to @JFowlerESPN @SteelTownUsa

Dude, dont u know words can hurt? U dont have to say it just cuz its true



@BurghSportsRT · Oct 30

Replying to @JFowlerESPN

With all that money they got from bell being out .. didn't take the opportunity of what was given



Michael Perrotti @mperrotti22 · Oct 30

Replying to @JFowlerESPN



NLP in the Real World



NLP in the Real World

- Thinking beyond agents, how you do NLP depends on what task you're trying to accomplish
- Examples
  - Part-of-speech classification (noun/verb/etc.)
  - Classification w.r.t. a taxonomy (e.g., Amazon products)
  - Authorship detection (e.g., Founding Fathers)
  - Language detection (which language is this?)
  - Sentiment analysis (is this positive, negative, other?)
  - Concept extraction, summarization (what is this about?)
  - Machine translation (one language to another)
  - Language generation (e.g., Chat-GPT)

NLP in Python

- Let's check out a robust Python library for doing NLP work: spaCy (plus spacytextblob)

```
> python -m pip install spacy spacytextblob
```

- According to their website…
  - spaCy is a free, open-source library for advanced NLP in Python
  - spaCy is **not** a platform, an API, an out-of-the-box chatbot engine, or research software
  - (though they've recently added integration with chatbots, LLMs, etc.)
  - For more info, see https://spacy.io/usage/spacy-101

nlp.py

```
import spacy
from spacytextblob.spacytextblob import SpacyTextBlob

# before running, you'll need the spacy English model and the textblob corpora:
# > python -m spacy download en_core_web_sm
# > python -m textblob.download_corpora

# load the English model
nlp = spacy.load('en_core_web_sm')

# define the text to analyze
text = 'Apple is looking at buying U.K. startup for $1 billion'
print(f'\nText: "{text}"')

# analyze the text to create a spacy augmented "document"
doc = nlp(text)

# print each token along with some of its properties
print(f'\nToken    Lemma    POS    Shape   Alpha  Stop\n' + ('-' * 49))
for token in doc:
    print(f'{token.text:<9} {token.lemma_:<9} {token.pos_:<7} ' +
          f'{token.shape_:<7} {token.is_alpha:<6} {token.is_stop:<6}')

# print the named entities
print(f'\nEntity      Start   End    Label\n' + ('-' * 36))
for ent in doc.ents:
    print(f'{ent.text:<12} {ent.start_char:<7} {ent.end_char:<7} {ent.label_:<8}')
```

```
> python nlp.py

Text: "Apple is looking at buying U.K. startup for $1 billion"

Token    Lemma    POS    Shape   Alpha  Stop
-----------------------------------------------
Apple    Apple    PROPN  Xxxxx   1      0
is       be       AUX    xx      1      1
looking  look     VERB   xxxx    1      0
at       at       ADP    xx      1      1
buying   buy      VERB   xxxx    1      0
U.K.     U.K.     PROPN  X.X.    0      0
startup  startup  NOUN   xxxx    1      0
for      for      ADP    xxx     1      1
$        $        SYM    $       0      0
1        1        NUM    d       0      0
billion  billion  NUM    xxxx    1      0

Entity      Start  End    Label
---------------------------------
Apple       0      5      ORG
U.K.        27     31     GPE
$1 billion  44     54     MONEY
```

Token Information

Token: The original word text

Lemma: The base form of the word

POS: The part-of-speech tag

Shape: The word shape – capitalization, punctuation, digits

Alpha: Is the token alphabetic?

Stop: Is the token a "stop word"?

Named Entities

ORG = Organization

GPE = Geopolitical Organization

MONEY = Currency

nlp.py

```
...

# add spacytextblob to pipeline for sentiment analysis
nlp.add_pipe('spacytextblob')

# run sentiment analysis on sample texts

def analyze_sentiment(text):
    doc = nlp(text)
    print(f'\nText:          "{text}"')
    print(f'Polarity:      {doc._.blob.polarity:.2f}')
    print(f'Subjectivity:   {doc._.blob.subjectivity:.2f}')

analyze_sentiment("It's great that Apple is buying a U.K. startup")
analyze_sentiment("It's not cool that Apple is buying a U.K. startup")
analyze_sentiment("It's ridiculous that Apple is buying a U.K. startup")
```

Text: "It's great that Apple is buying a U.K. startup"
Polarity: 0.80
Subjectivity: 0.75

Text: "It's not cool that Apple is buying a U.K. startup"
Polarity: -0.17
Subjectivity: 0.65

Text: "It's ridiculous that Apple is buying a U.K. startup"
Polarity: -0.33
Subjectivity: 1.00

Polarity – ranges from -1 to 1,
where 1 is positive sentiment,
-1 is negative sentiment,
and 0 is neutral sentiment

Subjectivity — ranges from 0 to 1
where 0 is objective
and 1 is subjective)

NLP in the Real World

- Problem: A perfect, unambiguous parsed understanding of a sentence is often elusive.
- In the face of real-world language & ambiguity, how can we handle the explosion of possible grammatical rules?
  - Allow for partially-formed parse trees
  - Incorporate probabilistic parsing
  - Use modern techniques (e.g., neural networks) that can better handle “imperfect” language

Coming up…

- These ideas lead us naturally into the next topic, Machine Learning
  - How can agents learn what to do and when to do it?
  - Once they know what to do, how can they improve their behavior over time?